Finite Element Method Magnetics

Version 5.0

User's Manual

October 13, 2026

David Meeker
dmeeker@ieee.org

Contents

1 Introduction **7**
- 1.1 Running FEMM from the Command Line . . . 8
- 1.2 Overview . . . 10
- 1.3 Relevant Partial Differential Equations . . . 10
  - 1.3.1 Magnetostatic Problems . . . 10
  - 1.3.2 Time-Harmonic Magnetic Problems . . . 11
  - 1.3.3 Electrostatic Problems . . . 12
  - 1.3.4 Heat Flow Problems . . . 13
  - 1.3.5 Current Flow Problems . . . 14
- 1.4 Boundary Conditions . . . 15
  - 1.4.1 Magnetic and Electrostatic BCs . . . 15
  - 1.4.2 Heat Flow BCs . . . 16
- 1.5 Finite Element Analysis . . . 17

2 Interactive Shell **19**
- 2.1 DXF Import/Export . . . 19
- 2.2 Magnetics Preprocessor . . . 20
  - 2.2.1 Preprocessor Drawing Modes . . . 20
  - 2.2.2 Keyboard and Mouse Commands . . . 21
  - 2.2.3 View Manipulation . . . 21
  - 2.2.4 Grid Manipulation . . . 24
  - 2.2.5 Edit . . . 25
  - 2.2.6 Problem Definition . . . 25
  - 2.2.7 Definition of Properties . . . 27
  - 2.2.8 Exterior Region . . . 39
  - 2.2.9 Analysis Tasks . . . 40
- 2.3 Magnetics Postprocessor . . . 40
  - 2.3.1 Postprocessor modes . . . 41
  - 2.3.2 View and Grid Manipulation . . . 41
  - 2.3.3 Keyboard Commands . . . 42
  - 2.3.4 Mouse Actions . . . 42
  - 2.3.5 Miscellaneous Useful View Commands . . . 43
  - 2.3.6 Contour Plot . . . 43
  - 2.3.7 Density Plot . . . 44
  - 2.3.8 Vector Plots . . . 44

2.3.9 Line Plots . . . 44
2.3.10 Line Integrals . . . 46
2.3.11 Block Integrals . . . 47
2.3.12 Force/Torque Calculation . . . 50
2.3.13 Circuit Results . . . 53
2.4 Electrostatics Preprocessor . . . 53
2.4.1 Problem Definition . . . 54
2.4.2 Definition of Properties . . . 55
2.4.3 Analysis Tasks . . . 60
2.5 Electrostatics Postprocessor . . . 61
2.5.1 Contour Plot . . . 61
2.5.2 Density Plot . . . 61
2.5.3 Vector Plots . . . 62
2.5.4 Line Plots . . . 62
2.5.5 Line Integrals . . . 63
2.5.6 Block Integrals . . . 64
2.5.7 Force/Torque Calculation . . . 65
2.5.8 Conductor Results . . . 66
2.6 Heat Flow Preprocessor . . . 66
2.6.1 Problem Definition . . . 67
2.6.2 Definition of Properties . . . 68
2.6.3 Analysis Tasks . . . 73
2.7 Heat Flow Postprocessor . . . 74
2.7.1 Contour Plot . . . 75
2.7.2 Density Plot . . . 75
2.7.3 Vector Plots . . . 75
2.7.4 Line Plots . . . 75
2.7.5 Line Integrals . . . 76
2.7.6 Block Integrals . . . 77
2.7.7 Conductor Results . . . 78
2.8 Current Flow Preprocessor . . . 78
2.8.1 Problem Definition . . . 78
2.8.2 Definition of Properties . . . 80
2.8.3 Analysis Tasks . . . 85
2.9 Current Flow Postprocessor . . . 86
2.9.1 Contour Plot . . . 86
2.9.2 Density Plot . . . 86
2.9.3 Vector Plots . . . 86
2.9.4 Line Plots . . . 87
2.9.5 Line Integrals . . . 88
2.9.6 Block Integrals . . . 88
2.9.7 Conductor Results . . . 89
2.10 Exporting of Graphics . . . 89

3 Python Scripting **91**
3.1 What is Python Scripting? . . . 91
3.2 Importing Python Modules . . . 91
3.3 Files and Directories . . . 95
3.4 Predefined Constants and Units . . . 96
3.5 Common Scripting Command Set . . . 97
3.6 Magnetics Preprocessor Scripting Command Set . . . 98
3.6.1 Object Add/Remove Commands . . . 98
3.6.2 Geometry Selection Commands . . . 98
3.6.3 Object Labeling Commands . . . 99
3.6.4 Problem Commands . . . 100
3.6.5 Mesh Commands . . . 101
3.6.6 Editing Commands . . . 101
3.6.7 Zoom Commands . . . 102
3.6.8 View Commands . . . 102
3.6.9 Object Properties . . . 103
3.6.10 Miscellaneous . . . 106
3.7 Magnetics Post Processor Scripting Command Set . . . 107
3.7.1 Data Extraction Commands . . . 108
3.7.2 Selection Commands . . . 112
3.7.3 Zoom Commands . . . 112
3.7.4 View Commands . . . 113
3.7.5 Miscellaneous . . . 114
3.8 Electrostatics Preprocessor Scripting Command Set . . . 116
3.8.1 Object Add/Remove Commands . . . 116
3.8.2 Geometry Selection Commands . . . 117
3.8.3 Object Labeling Commands . . . 117
3.8.4 Problem Commands . . . 118
3.8.5 Mesh Commands . . . 119
3.8.6 Editing Commands . . . 119
3.8.7 Zoom Commands . . . 120
3.8.8 View Commands . . . 121
3.8.9 Object Properties . . . 121
3.8.10 Miscellaneous . . . 123
3.9 Electrostatics Post Processor Scripting Command Set . . . 124
3.9.1 Data Extraction Commands . . . 124
3.9.2 Selection Commands . . . 126
3.9.3 Zoom Commands . . . 127
3.9.4 View Commands . . . 127
3.9.5 Miscellaneous . . . 129
3.10 Heat Flow Preprocessor Scripting Command Set . . . 130
3.10.1 Object Add/Remove Commands . . . 130
3.10.2 Geometry Selection Commands . . . 131
3.10.3 Object Labeling Commands . . . 131
3.10.4 Problem Commands . . . 132

3.10.5 Mesh Commands . . . 133
3.10.6 Editing Commands . . . 133
3.10.7 Zoom Commands . . . 134
3.10.8 View Commands . . . 135
3.10.9 Object Properties . . . 135
3.10.10 Miscellaneous . . . 137
3.11 Heat Flow Post Processor Scripting Command Set . . . 139
3.11.1 Data Extraction Commands . . . 139
3.11.2 Selection Commands . . . 140
3.11.3 Zoom Commands . . . 141
3.11.4 View Commands . . . 141
3.11.5 Miscellaneous . . . 143
3.12 Current Flow Preprocessor Scripting Command Set . . . 144
3.12.1 Object Add/Remove Commands . . . 144
3.12.2 Geometry Selection Commands . . . 145
3.12.3 Object Labeling Commands . . . 146
3.12.4 Problem Commands . . . 147
3.12.5 Mesh Commands . . . 147
3.12.6 Editing Commands . . . 147
3.12.7 Zoom Commands . . . 149
3.12.8 View Commands . . . 149
3.12.9 Object Properties . . . 149
3.12.10 Miscellaneous . . . 151
3.13 Current Flow Post Processor Scripting Command Set . . . 153
3.13.1 Data Extraction Commands . . . 153
3.13.2 Selection Commands . . . 155
3.13.3 Zoom Commands . . . 156
3.13.4 View Commands . . . 156
3.13.5 Miscellaneous . . . 158

4 Plotting with matplotlib 160
4.1 What you need . . . 160
4.2 Which plot command to use . . . 161
4.3 Plotting a solution on its own mesh . . . 161
4.4 Other packages . . . 162
4.5 Which `numpy`, which `scipy` . . . 165
4.6 Limitations . . . 166

5 Mathematica Interface 167

6 TCP/IP Interface 168

7 Numerical Methods **170**
7.1 Finite Element Formulation 170
7.2 Linear Solvers 170
7.3 Field Smoothing 172

A Appendix **175**
A.1 Modeling Permanent Magnets 175
A.2 Bulk Lamination Modeling 178
A.3 Open Boundary Problems 181
A.3.1 Truncation of Outer Boundaries 181
A.3.2 Asymptotic Boundary Conditions 181
A.3.3 Kelvin Transformation 182
A.4 Nonlinear Time Harmonic Formulation 185

Chapter 1

Introduction

FEMM is a suite of programs for solving low frequency electromagnetic problems on two-dimensional planar and axisymmetric domains. The program currently addresses linear/nonlinear magnetostatic problems, linear/nonlinear time harmonic magnetic problems, linear electrostatic problems, steady-state and transient heat flow problems, and current flow problems.

FEMM is divided into three parts:

- Interactive shell (`femm_gui`). This program is a Multiple Document Interface pre-processor and a post-processor for the various types of problems solved by FEMM. It contains a CAD-like interface for laying out the geometry of the problem to be solved and for defining material properties and boundary conditions. Autocad DXF files can be imported to facilitate the analysis of existing geometries. Field solutions can be displayed in the form of contour and density plots. The program also allows the user to inspect the field at arbitrary points, as well as evaluate a number of different integrals and plot various quantities of interest along user-defined contours.

- Headless shell (`femm_headless`). This program is a headless version of the preprocessor and postprocessor, meant primarily for use with external scripting interfaces where a GUI is not needed. It can also be run interactively from a terminal, where it presents a Python read-eval-print loop (REPL) for entering commands directly.

  On Windows, a second copy of the same program is built as `femm_headlessw.exe`. It is identical except that it is linked as a GUI-subsystem application, so it never opens a console window. That is the copy the language bindings (`pyfemm`, `octavefemm`, `mathfemm`, `juliafemm`, and the Lua client) launch, so that driving FEMM from another program does not make a console flash up. Use `femm_headless` yourself, from a terminal or a batch script, where you want to see its output and be able to interrupt it with Ctrl+C — neither of which the windowless copy can offer. On Linux and macOS there is no such distinction and only `femm_headless` is built.

- Solvers (`fkn` for magnetics, `belasolv` for electrostatics, `hsolv` for heat flow, and `csolv` for current flow). Each solver takes a set of data files that describe the problem and solves the relevant partial differential equations to obtain values for the desired field throughout the solution domain. The `tangle` mesher is directly integrated into each solver.

The Python scripting language is integrated into both shells. Python can both build and analyze a geometry and evaluate the post-processing results, simplifying the creation of various sorts of "batch" runs.

In addition, all edit boxes in the user interface are parsed by Python, allowing equations or mathematical expressions to be entered into any edit box in lieu of a numerical value. In any edit box in FEMM, a selected piece of text can be evaluated by Python via a selection on the right mouse button menu.

The purpose of this document is to give a brief explanation of the kind of problems solved by FEMM and to provide a fairly detailed documentation of the programs' use.

1.1 Running FEMM from the Command Line

Both the interactive shell (`femm_gui`) and the headless engine (`femm_headless`) accept command-line arguments. A file or script argument may be given as an absolute path or as a path relative to the current directory.

Interactive shell:
```
femm_gui [--port N] [file]
femm_gui -run script.py [args...]
```

- *file* — A file to open on startup. A problem document (`.fem`, `.fee`, `.feh`, or `.fec`) or a solution file (`.ans`, `.res`, `.anh`, or `.anc`) is opened in a window, and a Python script (`.py`) is executed. In each case the window is left open afterward. This is also the action taken when a file is opened through an operating-system file association (*e.g.* by double-clicking it).

- `-run script.py` — Run *script.py* and then exit, instead of leaving the window open. The exit code is `0` on success and `1` if the script raises an error (the error is also shown in a message box). Because the geometry, solution, and any errors are displayed while the script runs, this is a convenient way to develop a script interactively before running the identical `-run` command under `femm_headless`.

- `--port N` — Listen for scripting-interface connections on TCP port *N* rather than the default, 48015.

Headless engine:
```
femm_headless [--port N] [script.py]
femm_headless -run script.py [args...]
femm_headless -i [script.py]
femm_headless --version | --help
```

- *script.py* — Run the script, and then continue listening for scripting-interface connections on the TCP port.

- `-run script.py` — Run the script and then exit (batch mode). The exit code is `0` on success, `1` on a script error, and `130` if the script is interrupted with Ctrl+C.

- `-i`, `--interactive` — Present an interactive Python read-eval-print loop (REPL). If a script is also supplied, it is run first. Starting `femm_headless` with no arguments at a terminal also enters the REPL.

- `--port N` — Listen on TCP port *N* rather than the default, 48015.
- `--version` — Print the version and build number — for example, `FEMM headless engine v5.0.20261013 (build 1108)` — and exit. The version names the release, numbered by its date. The build number names the exact source the program was built from: it goes up with every change, so a test build made between releases has a higher number than the release it follows, and `-dirty` after it marks a build with uncommitted changes. The installer and disk image are named by the same two numbers. This is the quickest way to say which build you are running when reporting a problem, and it answers without starting the Python engine or opening a port, so it still works on a machine where FEMM itself will not start. The same version and build number appear in the GUI under `Help|About`.
- `-h`, `--help` — Print a summary of these options and exit.

An unrecognized option is an error: `femm_headless` names it, prints the usage summary, and exits with code `2` rather than trying to run it as a script.

Passing arguments to a script. Anything after the script name is passed to the script rather than interpreted by FEMM, and appears in the script as `sys.argv`. Following the usual Python convention, `sys.argv[0]` is the script's own path and `sys.argv[1:]` are the arguments that followed it:

```
femm_headless -run sweep.py rotor.fem 250
```

then, inside `sweep.py`,

```
import sys
model = sys.argv[1]
turns = int(sys.argv[2])
```

All arguments arrive as strings, so a numeric one must be converted with `int()` or `float()`. The `argparse` module is available if the script needs options rather than positional arguments.

Naming the script ends FEMM's own argument list, so any switch intended for FEMM must be given *before* the script. In

```
femm_headless --port 48020 -run sweep.py --port 5
```

the first `--port` sets the TCP port and the second is passed to the script. This is the same rule Python itself follows.

Legacy Lua scripts: `femm-lua [-h] [-p PORT] [script.lua [args...]]`

Scripts written for the Lua scripting interface of the original FEMM can be run with the bundled Lua 4 client, `femm-lua`. It launches the interpreter and connects to `femm_gui` — or, with `-h` (equivalently `--headless`), to `femm_headlessw` — over the same TCP interface used by the Python bindings. Given no script argument, `femm-lua` opens an interactive Lua prompt. Arguments given after *script.lua* are passed to it in the global `arg` table, Lua 4's equivalent of `sys.argv`. The available Lua commands are documented in the separate *Lua Scripting Guide* (`femm-lua.pdf`).

A given FEMM serves one client on one TCP port, so two `femm-lua` runs started *at the same time* cannot both use the default port 48015. Give each its own with `-p`:

```
start "" femm-lua -h -p 48016 job1.lua
start "" femm-lua -h -p 48017 job2.lua
```

Runs performed one after another need nothing: each finishes, and its FEMM exits, before the next begins.

Lua support is provided only for compatibility with scripts written for the original FEMM; new scripting should use FEMM's integrated Python interface.

1.2 Overview

The goal of this section is to give the user a brief description of the problems that FEMM solves. This information is not really crucial if you are not particularly interested in the approach that FEMM takes to formulating the problems. You can skip most of *Overview*, but take a look at Section 1.4. This section contains some important pointers about assigning enough boundary conditions to get a solvable problem.

Some familiarity with electromagnetism and Maxwell's equations is assumed, since a review of this material is beyond the scope of this manual. However, the author has found several references that have proved useful in understanding the derivation and solution of Maxwell's equations in various situations. A very good introductory-level text for magnetic and electrostatic problems is Plonus's *Applied electromagnetics* [1]. A good intermediate-level review of Maxwell's equations, as well as a useful analogy of magnetism to similar problems in other disciplines is contained in Hoole's *Computer-aided analysis and design of electromagnetic devices* [2]. For an advanced treatment, the reader has no recourse but to refer to Jackson's *Classical electrodynamics* [3]. For thermal problems, the author has found White's *Heat and mass transfer* [4] and Haberman's *Elementary applied partial differential equations* [5] to be useful in understanding the derivation and solution of steady-state temperature problems.

1.3 Relevant Partial Differential Equations

FEMM addresses some limiting cases of Maxwell's equations. The magnetics problems addressed are those that can be considered as "low frequency problems," in which displacement currents can be ignored. Displacement currents are typically relevant to magnetics problems only at radio frequencies. In a similar vein, the electrostatics solver considers the converse case in which only the electric field is considered and the magnetic field is neglected. FEMM also solves 2D/axisymmetric heat conduction problems. This heat conduction problem is mathematically very similar to the solution of electrostatic problems.

1.3.1 Magnetostatic Problems

Magnetostatic problems are problems in which the fields are time-invariant. In this case, the *field intensity* (H) and *flux density* (B) must obey:

$$\nabla \times H = J \tag{1.1}$$

$$\nabla \cdot B = 0 \tag{1.2}$$

subject to a constitutive relationship between B and H for each material:

$$B = \mu H \tag{1.3}$$

If a material is nonlinear (*e.g.* saturating iron or alnico magnets), the permeability, μ is actually a function of B:

$$\mu = \frac{B}{H(B)} \tag{1.4}$$

FEMM goes about finding a field that satisfies (1.1)-(1.3) via a *magnetic vector potential* approach. Flux density is written in terms of the vector potential, A, as:

$$B = \nabla \times A \tag{1.5}$$

Now, this definition of B always satisfies (1.2). Then, (1.1) can be rewritten as:

$$\nabla \times \left(\frac{1}{\mu(B)} \nabla \times A \right) = J \tag{1.6}$$

For a linear isotropic material (and assuming the Coulomb gauge, $\nabla \cdot A = 0$), eq. (1.6) reduces to:

$$-\frac{1}{\mu} \nabla^2 A = J \tag{1.7}$$

FEMM retains the form of (1.6), so that magnetostatic problems with a nonlinear B-H relationship can be solved.

In the general 3-D case, A is a vector with three components. However, in the 2-D planar and axisymmetric cases, two of these three components are zero, leaving just the component in the "out of the page" direction.

The advantage of using the vector potential formulation is that all the conditions to be satisfied have been combined into a single equation. If A is found, B and H can then be deduced by differentiating A. The form of (1.6), an elliptic partial differential equation, arises in the study of many different types of engineering phenomenon. There are a large number of tools that have been developed over the years to solve this particular problem.

1.3.2 Time-Harmonic Magnetic Problems

If the magnetic field is time-varying, eddy currents can be induced in materials with a non-zero conductivity. Several other Maxwell's equations related to the electric field distribution must also be accommodated. Denoting the *electric field intensity* as E and the *current density* as J, E and J obey the constitutive relationship:

$$J = \sigma E \tag{1.8}$$

The induced electric field then obeys:

$$\nabla \times E = -\frac{\partial B}{\partial t} \tag{1.9}$$

Substituting the vector potential form of B into (1.9) yields:

$$\nabla \times E = -\nabla \times \dot{A} \tag{1.10}$$

In the case of 2-D problems, (1.10) can be integrated to yield:

$$E = -\dot{A} - \nabla V \tag{1.11}$$

and the constitutive relationship, (1.8) employed to yield:

$$J = -\sigma \dot{A} - \sigma \nabla V \tag{1.12}$$

Substituting into (1.6) yields the partial differential equation:

$$\nabla \times \left(\frac{1}{\mu(B)} \nabla \times A \right) = -\sigma \dot{A} + J_{src} - \sigma \nabla V \tag{1.13}$$

where J_{src} represents the applied current sources. The ∇V term is an additional voltage gradient that, in 2-D problems, is constant over a conducting body. FEMM uses this voltage gradient in some harmonic problems to enforce constraints on the current carried by conductive regions.

FEMM considers (1.13) for the case in which the field is oscillating at one fixed frequency. For this case, a *phasor transformation* [2] yields a steady-state equation that is solved for the amplitude and phase of A. This transformation is:

$$A = \mathrm{Re}\left[a\left(\cos \omega t + j \sin \omega t\right)\right] = \mathrm{Re}\left[a e^{j\omega t}\right] \tag{1.14}$$

in which a is a complex number. Substituting into (1.13) and dividing out the complex exponential term yields the equation that FEMM actually solves for harmonic magnetic problems:

$$\nabla \times \left(\frac{1}{\mu_{eff}(B)} \nabla \times a \right) = -j\omega\sigma a + \hat{J}_{src} - \sigma \nabla V \tag{1.15}$$

in which $\hat{J}_{src}$ represents the phasor transform of the applied current sources.

Strictly speaking, the permeability μ should be constant for harmonic problems. However, FEMM retains a nonlinear relationship in the harmonic formulation, allowing the program to approximate the effects of saturation on the phase and amplitude of the fundamental of the field distribution. The form of the BH curve is not exactly the same as in the DC case. Instead, "effective permeability" μ_{eff} is selected to give the correct amplitude of the fundamental component of the waveform under sinusoidal excitation. There are a number of subtleties to the nonlinear time harmonic formulation–this formulation is addressed in more detail in Appendix A.4.

FEMM also allows for the inclusion of complex and frequency-dependent permeability in time harmonic problems. These features allow the program to model materials with thin laminations and approximately model hysteresis effects.

1.3.3 Electrostatic Problems

Electrostatic problems consider the behavior of electric field intensity, E, and electric flux density (alternatively electric displacement), D. There are two conditions that these quantities must obey.

The first condition is the differential form of Gauss' Law, which says that the flux out of any closed volume is equal to the charge contained within the volume:

$$\nabla \cdot D = \rho \tag{1.16}$$

where ρ represents charge density. The second is the static form of Faraday's law of induction:

$$\nabla \times E = 0 \tag{1.17}$$

Displacement and field intensity are also related to one another via the constitutive relationship:

$$D = \varepsilon E \tag{1.18}$$

where ε is the electrical permittivity. Although some electrostatics problems might have a nonlinear constitutive relationship between D and E, the program only considers linear problems.

To simplify the computation of fields which satisfy these conditions, the program employs the electric scalar potential, V, defined by its relation to E as:

$$E = -\nabla V \tag{1.19}$$

Because of the vector identity $\nabla \times \nabla \psi = 0$ for any scalar ψ, Ampere's loop law is automatically satisfied. Substituting into Gauss' Law and applying the constitutive relationship yields the second-order partial differential equation:

$$-\varepsilon \nabla^2 V = \rho \tag{1.20}$$

which applies over regions of homogeneous ε. The program solves (1.20) for voltage V over a user-defined domain with user-defined sources and boundary conditions.

1.3.4 Heat Flow Problems

The heat flow problems addressed by FEMM are heat conduction problems, either steady-state or stepped forward in time from a previous solution. These problems are represented by a temperature gradient, G(analogous to the field intensity, E for electrostatic problems), and heat flux density, F(analogous to electric flux density, D, for electrostatic problems).

The heat flux density must obey Gauss' Law, which says that the heat flux out of any closed volume is equal to the heat generation within the volume. Analogous to the electrostatic problem, this law is represented in differential form as:

$$\nabla \cdot F = q \tag{1.21}$$

where q represents volume heat generation.
Temperature gradient and heat flux density are also related to one another via the constitutive relationship:

$$F = kG \tag{1.22}$$

where k is the thermal conductivity. Thermal conductivity is often a weak function of temperature. FEMM allows for the variation of conductivity as an arbitrary function of temperature.

Ultimately, one is generally interested in discerning the temperature, T, rather than the heat flux density or temperature gradient. Temperature is related to the temperature gradient, G, by:

$$G = -\nabla T \tag{1.23}$$

Substituting (1.23) into Gauss' Law and applying the constitutive relationship yields the second-order partial differential equation:

$$-\nabla \cdot (k \nabla T) = q \tag{1.24}$$

FEMM solves (1.24) for temperature T over a user-defined domain with user-defined heat sources and boundary conditions.

1.3.5 Current Flow Problems

The current flow problems solved by FEMM are essentially quasi-electrostatic problems in which the magnetic field terms in Maxwell's equations can be neglected but in which the displacement current terms (neglected in magnetostatic and eddy current problems) are relevant.

Again restating Maxwell's Equations, the electric and magnetic fields must obey:

$$\nabla \times H = J + \dot{D} \tag{1.25}$$

$$\nabla \cdot B = 0 \tag{1.26}$$

$$\nabla \times E = -\dot{B} \tag{1.27}$$

$$\nabla \cdot D = \rho \tag{1.28}$$

subject to the constitutive relations:

$$J = \sigma E \tag{1.29}$$

$$D = \varepsilon E \tag{1.30}$$

The divergence of (1.25) can be taken to yield:

$$\nabla \cdot (\nabla \times H) = \nabla \cdot J + \nabla \cdot \dot{D} \tag{1.31}$$

By application of a standard vector identity, the left-hand side of (1.31) is zero, leading to:

$$\nabla \cdot J + \nabla \cdot \dot{D} = 0 \tag{1.32}$$

As before, we can assume an electric potential, V, that is related to field intensity, E, by:

$$E = -\nabla V \tag{1.33}$$

Because the flux density, B, is assumed to be negligibly small, (1.26) and (1.27) are suitably satisfied by this choice of potential.

If a phasor transformation is again assumed, wherein differentiation with respect to time is replaced by multiplication by $j\omega$, the definition of voltage can be substituted into (1.32) to yield:

$$-\nabla \cdot ((\sigma + j\omega\varepsilon) \nabla V) = 0 \tag{1.34}$$

If it is assumed that the material properties are piece-wise continuous, things can be simplified slightly to:

$$-(\sigma + j\omega\varepsilon)\nabla^2 V = 0 \tag{1.35}$$

FEMM solves (1.35) to analyze current flow problems.

Eq. (1.35) also applies for the solution of DC current flow problems. At zero frequency, the term associated with electrical permittivity vanishes, leaving:

$$-\sigma\nabla^2 V = 0 \tag{1.36}$$

By simply specifying a zero frequency, this formulation solves DC current flow problems in a consistent fashion.

1.4 Boundary Conditions

Some discussion of boundary conditions is necessary so that the user will be sure to define an adequate number of boundary conditions to guarantee a unique solution.

1.4.1 Magnetic and Electrostatic BCs

Boundary conditions for magnetic and electrostatic problems come in five varieties:

- *Dirichlet*. In this type of boundary condition, the value of potential A or V is explicitly defined on the boundary, *e.g.* $A = 0$. The most common use of Dirichlet-type boundary conditions in magnetic problems is to define $A = 0$ along a boundary to keep magnetic flux from crossing the boundary. In electrostatic problems, Dirichlet conditions are used to fix the voltage of a surface in the problem domain.

- *Neumann*. This boundary condition specifies the normal derivative of potential along the boundary. In magnetic problems, the homogeneous Neumann boundary condition, $\partial A/\partial n = 0$ is defined along a boundary to force flux to pass the boundary at exactly a 90° angle to the boundary. This sort of boundary condition is consistent with an interface with a very highly permeable metal.

- *Robin*. The Robin boundary condition is sort of a mix between Dirichlet and Neumann, prescribing a relationship between the value of A and its normal derivative at the boundary. An example of this boundary condition is:

  $$\frac{\partial A}{\partial n} + cA = 0$$

  This boundary condition is most often used in FEMM to define "impedance boundary conditions" that allow a bounded domain to mimic the behavior of an unbounded region. In the context of heat flow problems, this boundary condition can be interpreted as a convection boundary condition. In heat flow problems, radiation boundary conditions are linearized about the solution from the last iteration. The linearized form of the radiation boundary condition is also a Robin boundary condition.

- *Periodic* A periodic boundary condition joins two boundaries together. In this type of boundary condition, the boundary values on corresponding points of the two boundaries are set equal to one another.

- *Antiperiodic* The antiperiodic boundary condition also joins together two boundaries. However, the boundary values are made to be of equal magnitude but opposite sign.

If no boundary conditions are explicitly defined, each boundary defaults to a homogeneous Neumann boundary condition. However, a non-derivative boundary condition must be defined somewhere (or the potential must be defined at one reference point in the domain) so that the problem has a unique solution.

For axisymmetric magnetic problems, $A = 0$ is enforced on the line $r = 0$. In this case, a valid solution can be obtained without explicitly defining any boundary conditions, as long as part of the boundary of the problem lies along $r = 0$. This is not the case for electrostatic problems, however. For electrostatic problems, it is valid to have a solution with a non-zero potential along $r = 0$.

1.4.2 Heat Flow BCs

There are six types of boundary conditions for heat flow problems:

- *Fixed Temperature* The temperature along the boundary is set to a prescribed value.

- *Heat Flux* The heat flux, f, across a boundary is prescribed. This boundary condition can be represented mathematically as:

$$k\frac{\partial T}{\partial n} + f = 0 \tag{1.37}$$

  where n represents the direction normal to the boundary.

- *Convection* Convection occurs if the boundary is cooled by a fluid flow. This boundary condition can be represented as:

$$k\frac{\partial T}{\partial n} + h\left(T - T_o\right) = 0 \tag{1.38}$$

  where h is the "heat transfer coefficient" and T_o is the ambient cooling fluid temperature.

- *Radiation* Heat flux via radiation can be described mathematically as:

$$k\frac{\partial T}{\partial n} + \beta k_{sb}\left(T^4 - T_o^4\right) = 0 \tag{1.39}$$

  where β is the emissivity of the surface (a dimensionless value between 0 and 1) and k_{sb} is the Stefan-Boltzmann constant.

- *Periodic* A periodic boundary condition joins two boundaries together. In this type of boundary condition, the boundary values on corresponding points of the two boundaries are set equal to one another.

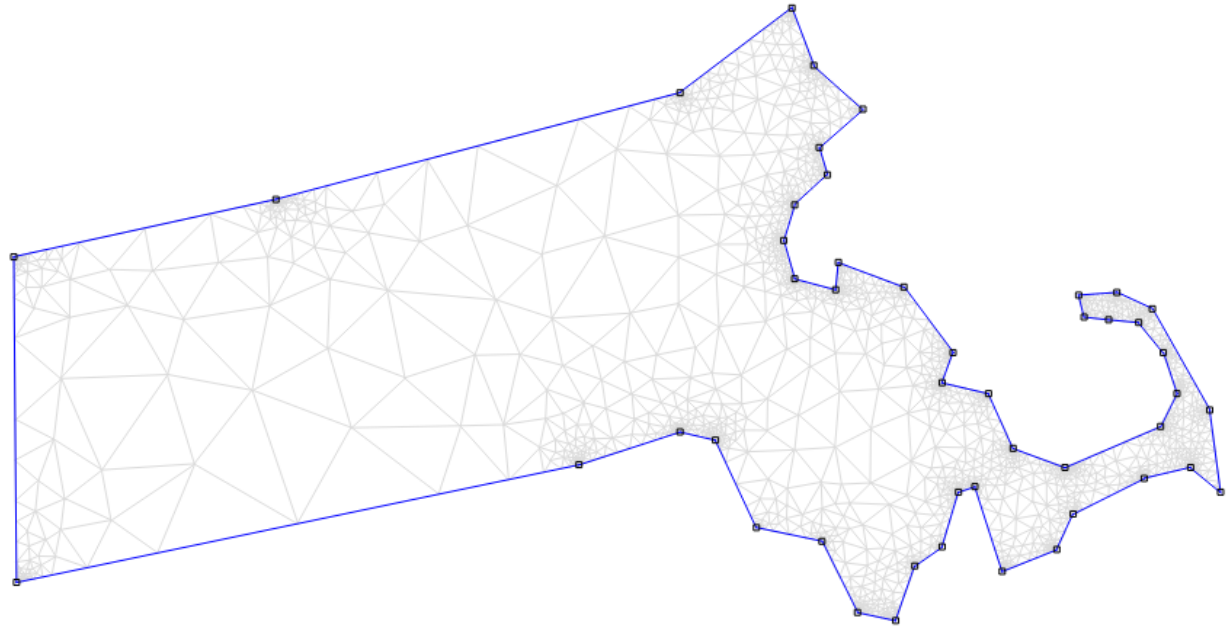


Figure 1.1: Triangulation of Massachusetts

- *Antiperiodic* The antiperiodic boundary condition also joins together two boundaries. However, the boundary values are made to be of equal magnitude but opposite sign.

If no boundary conditions are explicitly defined, each boundary defaults to an insulated condition (*i.e.* no heat flux across the boundary). However, a non-derivative boundary condition must be defined somewhere (or the temperature must be defined at one reference point in the domain) so that the problem has a unique solution.

1.5 Finite Element Analysis

Although the differential equations of interest appear relatively compact, it is typically very difficult to get closed-form solutions for all but the simplest geometries. This is where finite element analysis comes in. The idea of finite elements is to break the problem down into a large number of regions, each with a simple geometry (*e.g.* triangles). For example, Figure 1.1 shows a map of Massachusetts broken down into triangles. Over these simple regions, the "true" solution for the desired potential is approximated by a very simple function. If enough small regions are used, the approximate potential closely matches the exact solution.

The advantage of breaking the domain down into a number of small elements is that the problem becomes transformed from a small but difficult to solve problem into a big but relatively easy to solve problem. Through the process of discretization, a linear algebra problem is formed with perhaps tens of thousands of unknowns. However, algorithms exist that allow the resulting linear algebra problem to be solved, usually in a short amount of time.

Specifically, FEMM discretizes the problem domain using triangular elements. Over each element, the solution is approximated by a linear interpolation of the values of potential at the

three vertices of the triangle. The linear algebra problem is formed by minimizing a measure of the error between the exact differential equation and the approximate differential equation as written in terms of the linear trial functions.

Chapter 2

Interactive Shell

The FEMM Interactive Shell is currently broken into six major sections:

- Magnetics Preprocessor
- Electrostatics Preprocessor
- Heat Flow Preprocessor
- Magnetics Postprocessor
- Electrostatics Postprocessor
- Heat Flow Postprocessor

This section of the manual explains the functionality of each section in detail.

2.1 DXF Import/Export

A common aspect of all preprocessor modes is DXF Import/Export. For interfacing with CAD programs and other finite element packages, femm supports the import and export of the AutoCAD dxf file format. Specifically, the dxf interpreter in femm was written to the dxf revision 13 standards. Only 2D dxf files can be imported in a meaningful way.

To import a dxf file, select `Import DXF` off of the `File` menu. A dialog will appear after the file is seleted asking for a tolerance. This tolerance is the maximum distance between two points at which the program considers two points to be the same. The default value is usually sufficient. For some files, however, the tolerance needs to be increased (*i.e.* made a larger number) to import the file correctly. FEMM does not understand all the possible tags that can be included in a dxf file; instead, it simply strips out the commands involved with drawing lines, circles, and arcs. All other information is simply ignored.

Generally, dxf import is a useful feature. It allows the user to draw an initial geometry using their favorite CAD package. Once the geometry is laid out, the geometry can be imported into femm and detailed for materials properties and boundary conditions.

Do not despair if femm takes a while to import dxf files (especially large dxf files). The reason that femm can take a long time to import dxf files is that a lot of consistency checking must be

performed to turn the dxf file into a valid finite element geometry. For example, large dxf files might take up to a minute or two to import.

The current femm geometry can be exported in dxf format by selecting the `Export DXF` option off of the `File` menu in any preprocessor window. The dxf files generated from femm can then be imported into CAD programs to aid in the mechanical detailing of a finalized design, or imported into other finite element or boundary element programs.

2.2 Magnetics Preprocessor

The preprocessor is used for drawing the problems geometry, defining materials, and defining boundary conditions. A new instance of the preprocessor can be created by selecting `File|New` off of the main menu and then selecting "Magnetics Problem" from the list of problem types which then appears.

Drawing a valid geometry usually consists of four (though not necessarily sequential) tasks:

- Drawing the endpoints of the lines and arc segments that make up a drawing.
- Connecting the endpoints with either line segments or arc segments
- Adding "Block Label" markers into each section of the model to define material properties and mesh sizing for each section.
- Specifying boundary conditions on the outer edges of the geometry.

This section will describe exactly how one goes about performing these tasks and creating a problem that can be solved.

2.2.1 Preprocessor Drawing Modes

The key to using the preprocessor is that the preprocessor is always in one of five modes: the *Point* mode, the *Segment* mode, *Arc Segment* mode, the *Block* mode, or the *Group* mode. The first four of these modes correspond to the four types of entities that define the problems geometry: nodes that define all corners in the solution geometry, line segments and arc segments that connect the nodes to form boundaries and interfaces, and block labels that denote what material properties and mesh size are associated with each solution region. When the preprocessor is in a one of the first four drawing modes, editing operations take place only upon the selected type of entity. The fifth mode, the group mode, is meant to glue different objects together into parts so that entire parts can be manipulated more easily.

One can switch between drawing modes by clicking the appropriate button on the Drawing Mode potion of the toolbar. This section of the toolbar is pictured in Figure 2.1. The buttons



Figure 2.1: Drawing Mode toolbar buttons.

correspond to Point, Line Segment, Arc Segment, Block Label, and Group modes respectively. The default drawing mode when the program begins is the Point mode.

2.2.2 Keyboard and Mouse Commands

Although most of the tasks that need to be performed are available via the toolbar, some important functions are invoked only through the use of "hot keys". A summary of these keys and their associated functions is contained in Table 2.1.

Likewise, specific functions are associated with mouse button input. The user employs the mouse to create new object, select obects that have already been created, and inquire about object properties. Table 2.2 is a summary of the mouse button click actions.

Selection is a toggle throughout: right-clicking an object that is already selected unselects it, and a right double-click selects or unselects the object it reports on as well as describing it. A right click also abandons a segment or arc that is part-way drawn. In Group mode, objects that have been left in group 0 are not selected by a right click, since group 0 means "no group" rather than a group of its own.

Two mouse actions work the same way in every mode and are not listed in the table: dragging with the middle button pans the view, and the wheel zooms in and out about the mouse pointer.

2.2.3 View Manipulation

Generally, the user needs to size or move the view of the problem geometry displayed on the screen. Most of the view manipulation commands are available via buttons on the preprocessor toolbar. The functionality of can generally also be accessed via the 'View Manipulation Keys' listed in Table 2.1. The View Manipulation toolbar buttons are pictured in Figure 2.2. The meaning of the



Figure 2.2: View Manipulation toolbar buttons.

View Manipulation toobar buttons are:

- The arrows on the toolbar correspond to moving the view in the direction of the arrow approximately $1/2$ of the current screen width.
- The button showing a magnifying glass over a page scales the screen to the smallest possible view that displays the entire problem geometry.
- The "+" and "-" buttons zoom the current view in and out, respectively.
- The button showing a magnifying glass over a dashed rectangle allows the view to be zoomed in on a user-specified part of the screen. To use this tool, first push the toolbar button. Then,

Point Mode Keys	
Key	Function
Space	Edit the properties of selected point(s)
Tab	Display dialog for the numerical entry of coordinates for a new point
Escape	Unselect all points
Delete	Delete selected points

Line/Arc Segment Mode Keys	
Key	Function
Space	Edit the properties of selected segment(s)
Escape	Unselect all segments and line starting points
Delete	Delete selected segment(s)

Block Label Mode Keys	
Key	Function
Space	Edit the properties of selected block labels(s)
Tab	Display dialog for the numerical entry of coordinates for a new label
Escape	Unselect all block labels
Delete	Delete selected block label(s)

Group Mode Keys	
Key	Function
Space	Edit group assignment of the selected objects
Escape	Unselect all
Delete	Delete selected block label(s)

Mesh Sizing Keys (any mode)	
Key	Function
F3	Halve the mesh size on every block label
F4	Double the mesh size on every block label

View Manipulation Keys	
Key	Function
Left Arrow	Pan left
Right Arrow	Pan right
Up Arrow	Pan up
Down Arrow	Pan down
Page Up	Zoom in
Page Down	Zoom out
Home	Zoom “natural”

Table 2.1: Magnetics Preprocessor hot keys

Point Mode	
Action	Function
Left Button Click	Create a new point at the current mouse pointer location
Right Button Click	Select or unselect the nearest point
Right Button DblClick	Report the nearest point's coordinates, nodal property and group

Segment Mode	
Action	Function
Left Button Click	Pick the nearest point; a second click completes the segment
Right Button Click	Select or unselect the nearest segment
Right Button DblClick	Report the nearest segment's length and properties

Arc Segment Mode	
Action	Function
Left Button Click	As Segment mode; the second click opens the arc's dialog
Right Button Click	Select or unselect the nearest arc
Right Button DblClick	Report the nearest arc's angle, radius, center and properties

Block Label Mode	
Action	Function
Left Button Click	Create a new block label at the current mouse pointer location
Right Button Click	Select or unselect the nearest block label
Right Button DblClick	Report the nearest label's material, circuit, mesh size and group

Group Mode	
Action	Function
Right Button Click	Select or unselect the group of the nearest object

Table 2.2: Magnetics Preprocessor Mouse button actions

move the mouse pointer to one of the desired corners of the "new" view. Press and hold the left mouse button. Drag the mouse pointer to the opposite diagonal corner of the desired "new" view. Last, release the left mouse button. The view will zoom in to a window that best fits the user's desired window.

Some infrequently used view commands are also available, but only as options off of the `Zoom` selection of the main menu. This menu contains all of the manipulations available from the toolbar buttons, plus the options `Keyboard`, `Status Bar`, and `Toolbar`.

The `Keyboard` selection allows the user to zoom in to a window in which the window's corners are explicitly specified by the user via keyboard entry of the corners' coordinates. When this selection is chosen, a dialog pops up prompting for the locations of the window corners. Enter the desired window coordinates and hit "OK". The view will then zoom to the smallest possible

window that bounds the desired window corners. Typically, this view manipulation is only done as a new drawing is begun, to initially size the view window to convenient boundaries.

The `Status Bar` selection can be used to hide or show the one-line status bar at the bottom of the femm window. Generally, it is desirable for the toolbar to be displayed, since the current location of the mouse pointer is displayed on the status line.

The `Toolbar` selection can be used to hide or show the toolbar buttons. The toolbar is not fundamentally necessary to running femm, because any selection on the toolbar is also available via selections off of the main menu. If more space on the screen is desired, this option can be chosen to hide the toolbar. Selecting it a second time will show the toolbar again. It may be useful to note that the toolbar can be undocked from the main screen and made to "float" at a user-defined location on screen. This is done by pushing the left mouse button down on an area of the toolbar that is not actually a button, and then dragging the toolbar to its desired location. The toolbar can be docked again by moving it back to its original position.

2.2.4 Grid Manipulation

To aid in drawing your geometry, a useful tool is the Grid. When the grid is on, a grid of light blue pixels will be displayed on the screen. The spacing between grid points can be specified by the user, and the mouse pointer can be made to "snap" to the closest grid point.

The easiest way to manipulate the grid is through the used of the Grid Manipulation toolbar buttons. These buttons are pictured in Figure 2.3. The top button in Figure 2.3 shows and hides



Figure 2.3: Grid Manipulation toolbar buttons.

the grid. The default is that the button is pushed in, showing the current grid. The second button, with an icon of an arrow pointing to a grid point, is the "snap to grid" button. When this button is pushed in, the location of the mouse pointer is rounded to the nearest grid point location. By default, the "snap to grid" button is not pressed. The bottom button brings up the Grid Properties dialog. This dialog is shown in Figure 2.4.

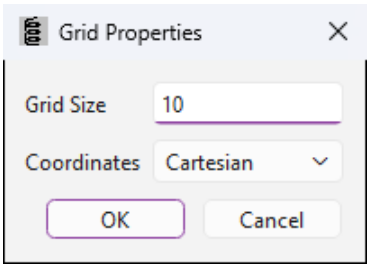


Figure 2.4: Grid Properties dialog.

The Grid Properties dialog has an edit box for the user to enter the desired grid sizing. When the box appears, the number in this edit box is the current grid size. The dialog also contains a drop

list that allows the user to select between Cartesian and Polar coordinates. If Cartesian is selected, points are specified by their (x,y) coordinates for a planar problem, or by their (r,z) coordinates for an axisymmetric problem. If Polar is selected, points are specified by an angle and a radial distance from the origin. The default is Cartesian coordinates.

2.2.5 Edit

Several useful tasks can be performed via the `Edit` menu off of the main menu.

Perhaps the most frequently used is the `Undo` command. Choosing this selection undoes the last addition or deletion that the user has made to the model's geometry.

For selecting many objects quickly, the `Select Group` command is useful. This command allows the user to select objects of the current type located in an arbitrary rectangular box. When this command is selected, move the mouse pointer to one corner of the region that is to be selected. Press and hold the left mouse button. Then, drag the mouse pointer to the opposite diagonal corner of the region. A red box will appear, outlining the region to be selected. When the desired region has been specified, release the left mouse button. All objects of the current type completely contained within the box will become selected.

Any objects that are currently selected can be moved, copied, or pasted. To move or copy selected objects, simply choose the corresponding selection off of the main menu's `Edit` menu. A dialog will appear prompting for an amount of displacement or rotation.

2.2.6 Problem Definition

The definition of problem type is specified by choosing the `Problem` selection off of the main menu. Selecting this option brings up the Problem Definition dialog, shown in Figure 2.5

The dialog's entries, in the order they appear:

- `Frequency (Hz)`. For a magnetostatic problem, the user should choose a frequency of zero. If the frequency is non-zero, the program will perform a harmonic analysis, in which all field quantities are oscillating at this prescribed frequency. The default frequency is zero.

- `Length Units`. This box identifies what unit is associated with the dimensions prescribed in the model's geometry. Currently, the program supports inches, millimeters, centimeters, meters, mils, and μmeters.

- `Problem Type`. This drop list allows the user to choose from a 2-D planar problem (the `Planar` selection), or an axisymmetric problem (the `Axisymmetric` selection).

- `Precision`. The number in this edit box specifies the stopping criteria for the linear solver. The linear algebra problem could be represented by:

  $$Mx = b$$

  where M is a square matrix, b is a vector, and x is a vector of unknowns to be determined. The solver precision value determines the maximum allowable value for $||b - Mx||/||b||$. The default value is 10^{-8}.

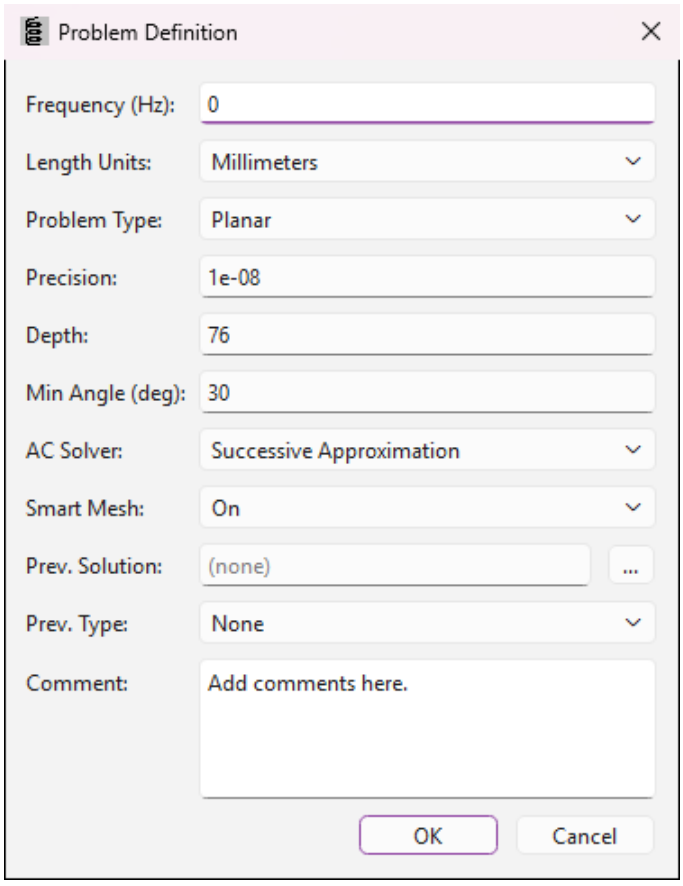


Figure 2.5: Problem Definition dialog.

- `Depth`. If a Planar problem is selected, this edit box becomes enabled. This value is the length of the geometry in the "into the page" direction. This value is used for scaling integral results in the post processor (*e.g.* force, inductance, etc.) to the appropriate length. The units of the Depth selection are the same as the selected length units. For files imported from version 3.2, the Depth is chosen so that the depth equals 1 meter, since in version 3.2, all results from planar problems are reported per meter.

- `Min Angle (deg)`. The entry in this box is used as a constraint to the mesh generator, `tangle`. The mesh generator adds points to the mesh to ensure that no angles smaller than the specified angle occur. The edit box will accept values between 1 and 33.8 degrees.

- `AC Solver`. Selects the iteration used for a *nonlinear harmonic* problem: `Successive Approximation` or `Newton`. Newton generally converges in fewer passes on strongly saturated problems, at more work per pass. The selection has no effect on linear or magnetostatic problems.

- `Smart Mesh`. When `On`, the mesh generator derives a characteristic element size from the average length of the model's line segments and applies it at the ends of those segments, refining the mesh toward corners. Endpoints belonging only to arcs are left alone, so a smooth surface such as a rotor bore is not refined merely for being curved. When `Off`,

element sizes come only from the mesh size specified on each block label, together with any per-segment maximum side length.

- `Prev. Solution`. The name of a previously computed solution (`.ans`) file, used together with `Prev. Type` below. The button to the right of the box opens a file selection dialog. If the box is left empty, `Prev. Type` is forced to `None`.

- `Prev. Type`. What the previous solution is used for. `None` ignores it and solves the problem from scratch. `Incremental` treats the previous solution as an operating point and solves a small-signal problem about it, using the incremental permeability at that point – the way to obtain, for example, an incremental inductance, or the response to a small AC excitation superimposed on a DC bias. `Frozen` holds the permeability at the values the previous solution reached, rather than re-evaluating it from the present field.

- `Comment`. Optionally, the user can enter in a few lines of text that give a brief description of the problem that is being solved. This is useful if the user is running several small variations on a given geometry. The comment can then be used to identify the relevant features for a particular geometry.

2.2.7 Definition of Properties

To make a solvable problem definition, the user must identify boundary conditions, block materials properties, and so on. The different types of properties defined for a given problem are defined via the `Properties` selection off of the main menu.

When the `Properties` selection is chosen, a drop menu appears that has selections for Materials, Boundary, Point, and Circuits. When any one of these selections is chosen, the dialog pictured in Figure 2.6 appears. This dialog is the manager for a particular type of property, and its title changes to match the type chosen from the menu. All currently defined properties of that type are listed in the box on the left. At the beginning of a new model definition, the list will be empty, since no properties have yet been defined. Pushing the `Add` button allows the user to define a new property. The `Delete` button removes the property currently selected in the list, and `Modify` opens the selected property for viewing and editing — as does double-clicking the property itself. `Close` dismisses the dialog. Specifics for defining the various property types are addressed in the following subsections.

In general, a number of these edit boxes prompt the user for both real and imaginary components for entered values. If the problem you are defining is magnetostatic (zero frequency), enter the desired value in the box for the real component, and leave a zero in the box for the imaginary component. The reason that femm uses this formalism is to obtain a relatively smooth transition from static to time-harmonic problems. Consider the definition of the *Phasor transformation* in Eq. 1.14. The phasor transformation assumes that all field values oscillate in time at a frequence of ω. The phasor transformation takes the cosine part of the field value and represents it as the real part of a complex number. The imaginary part represents the magnitude of the sine component, 90° out of phase. Note what happens as the frequency goes to zero:

$$\lim_{\omega \to 0} \left(a_{re} \cos \omega t - a_{im} \sin \omega t \right) = a_{re} \tag{2.1}$$

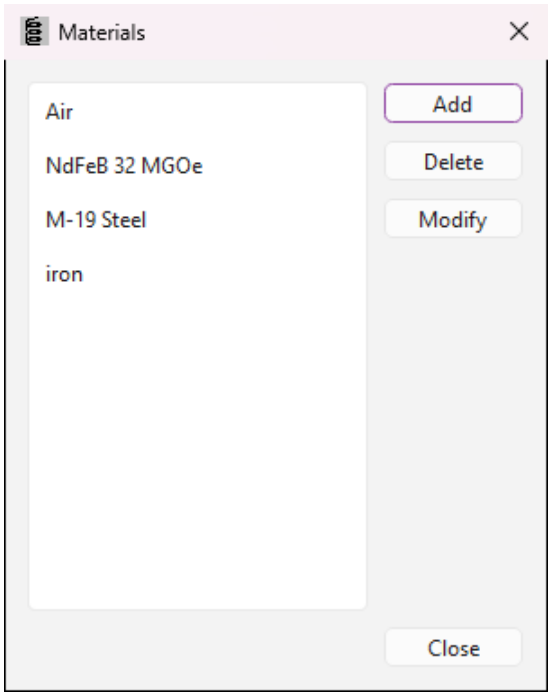


Figure 2.6: Materials dialog

Therefore, the magnetostatic ($\omega = 0$) values are just described by the real part the specified complex number.

Point Properties

If a new point property is added or an existing point property modified, the `Nodal Property` dialog box appears. This dialog box is pictured in Figure 2.7

The first selection is the `Name` edit box. The default name is "New Point Property," but this name should be changed to something that describes the property that you are defining.

The two radio buttons below the name choose what sort of point property is being defined: a specified vector potential, A, at a given point, or a prescribed point current, J, at a given point. The two definitions are mutually exclusive. Selecting one enables its group of boxes and zeroes and disables the other, so a point property carries either a potential or a current, never both.

The `Specified Vector Potential, Wb/m` box holds A, which can be defined as a complex value, if desired. Typically, A needs to be defined as some particular value (usually zero) at some point in the solution domain for problems with derivative boundary conditions on all sides. This is the typical use of defining a point vector potential.

The `Point Current, Amps` box holds J. The value of J can be defined as complex, if desired.

Boundary Properties

The `Boundary Condition` dialog box is used to specify the properties of line segments or arc segments that are to be boundaries of the solution domain. When a new boundary property is

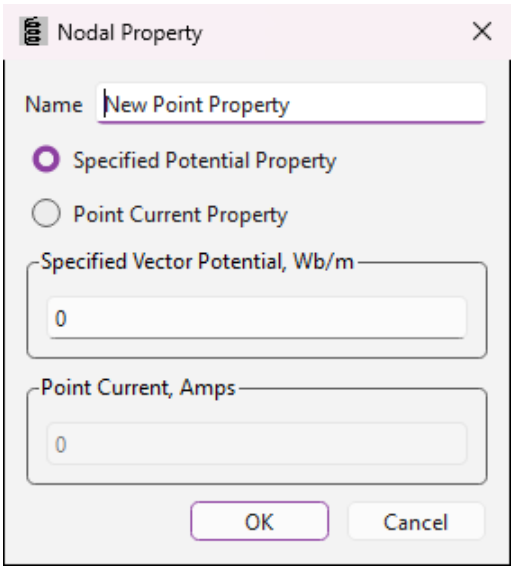


Figure 2.7: Nodal Property dialog.

added or an existing property modified, the `Boundary Condition` dialog pictured in Figure 2.8 appears.

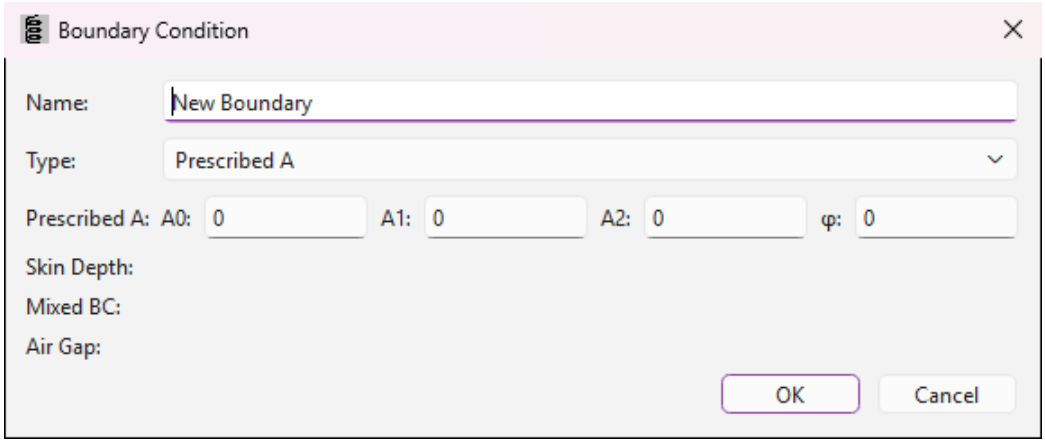


Figure 2.8: Boundary Condition dialog.

The first selection in the dialog is the `Name` of the property. The default name is "New Boundary," but you should change this name to something more descriptive of the boundary that is being defined.

The next selection is the `Type` drop list. This specifies the boundary condition type. The boxes below the drop list hold the parameters belonging to the selected type; the rows for the other types stay empty. Currently, FEMM supports the following types of boundaries:

- `Prescribed A` With this type of boundary condition, the vector potential, A, is prescribed along a given boundary. This boundary condition can be used to prescribe the flux passing normal to a boundary, since the normal flux is equal to the tangential derivative of A along

the boundary. The form for A along the boundary is specified via the parameters A_0, A_1, A_2 and ϕ in the `Prescribed A` row. If the problem is planar, the parameters correspond to the formula:

$$A = (A_0 + A_1 x + A_2 y)e^{j\phi} \tag{2.2}$$

If the problem type is axisymmetric, the parameters correspond to:

$$A = (A_0 + A_1 r + A_2 z)e^{j\phi} \tag{2.3}$$

- `Small Skin Depth` This boundary condition denotes an interface with a material subject to eddy currents at high enough frequencies such that the skin depth in the material is very small. A good discussion of the derivation of this type of boundary condition is contained in [2]. The result is a Robin boundary condition with complex coefficients of the form:

$$\frac{\partial A}{\partial n} + \left(\frac{1+j}{\delta}\right) A = 0 \tag{2.4}$$

where the n denotes the direction of the outward normal to the boundary and δ denotes the skin depth of the material at the frequency of interest. The skin depth, δ is defined as:

$$\delta = \sqrt{\frac{2}{\omega \mu_r \mu_o \sigma}} \tag{2.5}$$

where μ_r and σ are the relative permeability and conductivity of the thin skin depth eddy current material. These parameters are defined by specifying μ and σ in the `Skin Depth` row of the dialog. At zero frequency, this boundary condition degenerates to $\partial A/\partial n = 0$ (because skin depth goes to infinity).

- `Mixed` This denotes a boundary condition of the form:

$$\left(\frac{1}{\mu_r \mu_o}\right) \frac{\partial A}{\partial n} + c_o A + c_1 = 0 \tag{2.6}$$

The parameters for this class of boundary condition are specified in the `Mixed BC` row of the dialog. By the choice of coefficients, this boundary condition can either be a Robin or a Neumann boundary condition. There are two main uses of this boundary condition:

1. By carefully selecting the c_0 coefficient and specifying $c_1 = 0$, this boundary condition can be applied to the outer boundary of your geometry to approximate an up-bounded solution region. For more information on open boundary problems, refer to Appendix A.3.
2. The Mixed boundary condition can used to set the field intensity, H, that flows parallel to a boundary. This is done by setting c_0 to zero, and c_1 to the desired value of H in units of Amp/Meter. Note that this boundary condition can also be used to prescribe $\partial A/\partial n = 0$ at the boundary. However, this is unnecessary–the 1st order triangle finite elements give a $\partial A/\partial n = 0$ boundary condition by default.

- `Strategic Dual Image` This is sort of an "experimental" boundary condition that I have found useful for my own purposes from time to time. This boundary condition mimics an "open" boundary by solving the problem twice: once with a homogeneous Dirichlet boundary condition on the SDI boundary, and once with a homogeneous Neumann condition on the SDI boundary. The results from each run are then averaged to get the open boundary result. This boundary condition should only be applied to the outer boundary of a circular domain in 2-D planar problems. Through a method-of-images argument, it can be shown that this approach yields the correct open-boundary result for problems with no iron (*i.e* just currents or linear magnets with unit permeability in the solution region).

- `Periodic` This type of boundary condition is applied to either two segments or two arcs to force the magnetic vector potential to be identical along each boundary. This sort of boundary is useful in exploiting the symmetry inherent in some problems to reduce the size of the domain which must be modeled. The domain merely needs to be periodic, as opposed to obeying more restrictive $A = 0$ or $\partial A/\partial n = 0$ line of symmetry conditions. Another useful application of periodic boundary conditions is for the modeling of "open boundary" problems, as discussed in Appendix A.3.3. Often, a periodic boundary is made up of several different line or arc segments. A different periodic condition must be defined for each section of the boundary, since each periodic BC can only be applied to a line or arc and a corresponding line or arc on the remote periodic boundary.

- `Antiperiodic` The antiperiodic boundary condition is applied in a similar way as the periodic boundary condition, but its effect is to force two boundaries to be the negative of one another. This type of boundary is also typically used to reduce the domain which must be modeled, *e.g.* so that an electric machine might be modeled for the purposes of a finite element analysis with just one pole.

- `Periodic Air Gap` and `Anti-Periodic Air Gap` These define an *air gap element*, applied to a pair of concentric arcs bounding an annulus of air — typically the air gap of a rotating machine. Instead of meshing the annulus, the program couples the field on the inner arc directly to the field on the outer arc, which means the two sides can be rotated with respect to each other without the mesh changing at all. The `Inner` and `Outer` boxes in the `Air Gap` row give the angles, in degrees, through which the inner and outer boundaries are rotated, so stepping a machine through a series of rotor positions is a matter of changing `Inner` between runs. Choose `Periodic Air Gap` when the modeled sector repeats identically around the circle and `Anti-Periodic Air Gap` when it repeats with alternating sign, on the same basis as the ordinary `Periodic` and `Antiperiodic` conditions above.

Materials Properties

The `Material Properties` dialog box is used to specify the properties to be associated with block labels. The properties specified in this dialog have to do with the material that the block is composed of, as well as some attributes about how the material is put together (laminated). When a new material property is added or an existing property modified, the `Material Properties` dialog pictured in Figure 2.9 appears.

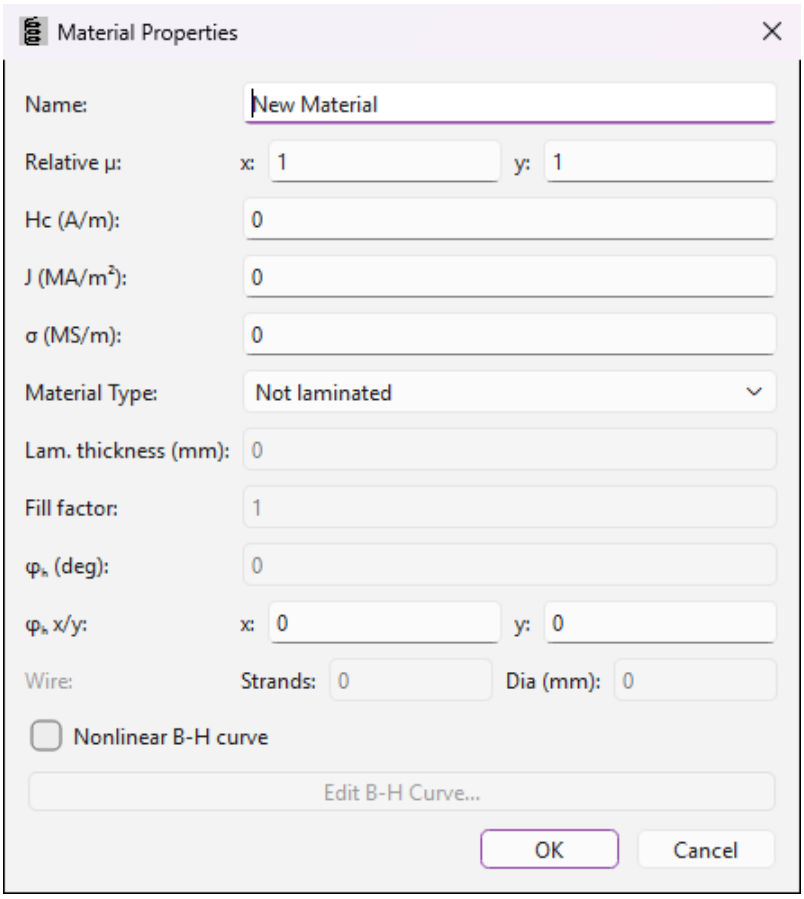


Figure 2.9: Material Properties dialog.

As with Point and Boundary properties, the first step is to choose a descriptive name for the material that is being described. Enter it in the `Name` edit box in lieu of "New Material."

Next decide whether the material will have a linear or nonlinear B-H curve, using the `Nonlinear B-H curve` check box near the bottom of the dialog. This choice governs which of the boxes above it are enabled: a linear material is described by its permeability, a nonlinear one by a B-H curve, and the dialog disables whichever set does not apply.

With `Nonlinear B-H curve` left unchecked, the `Relative` μ boxes are enabled. FEMM allows you to specify different relative permeabilities in the vertical and horizontal directions (μ_x in the `x` box for the horizontal direction, and μ_y in the `y` box for the vertical direction).

Below them is the ϕ_h `x/y` pair, ϕ_{hx} and ϕ_{hy}, which denote the hysteresis lag angle corresponding to each direction, to be used in cases in which linear material properties have been specified. A simple, but surprisingly effective, model for hysteresis in harmonic problems is to assume that hysteresis creates a constant phase lag between B and H that is independent of frequency. This is exactly the same as assuming that hysteresis loop has an elliptical shape. Since the hysteresis loop is not exactly elliptical, the perceived hysteresis angle will vary somewhat for different amplitudes of excitation. The hysteresis angle is typically not a parameter that appears on manufacturer's data sheets; you have to identify it yourself from a frequency sweep on a toroidal coil with a core composed of the material of interest. For most laminated steels, the hysteresis angle lies between 0°

and 20° [6]. This same reference also has a very good discussion of the derivation and application of the fixed phase lag model of hysteresis.

If `Nonlinear B-H curve` is checked instead, the `Relative` μ and ϕ_h `x/y` boxes are disabled and the `Edit B-H Curve...` button becomes enabled. To enter in points on your B-H curve, hit that button. When the button is pushed a dialog appears that allows you to enter in B-H data (see Figure 2.10). The information to be entered in these dialogs is usually obtained by picking

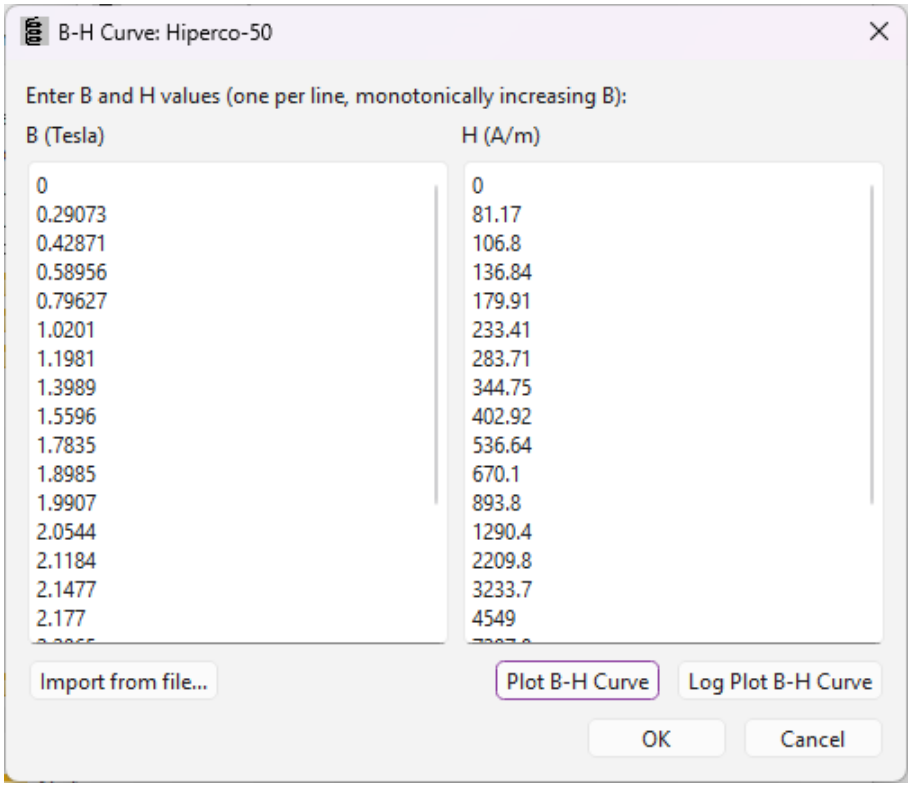


Figure 2.10: B-H data entry dialog.

points off of manufacturer's data sheets. For obvious reasons, you must enter the same number of points in the "B" (flux density) column as in the "H" (field intensity) column. To define a nonlinear material, you must enter *at least* three points, and you should enter ten or fifteen to get a good fit. The points must be in order of increasing B; a point that does not advance B is discarded, so a curve that appears to have lost points is usually one with a repeated or out-of-order B value.

Rather than typing the points in, the `Import from file...` button reads them from a text file with two numbers per line, B first, separated by spaces, tabs or commas. Lines beginning with `#` or `%` are ignored, so a file's header does not have to be stripped out first.

After you are done entering in your B-H data points, it is a good idea to view the B-H curve to see that it looks like it is "supposed" to. This is done by pressing the `Plot B-H Curve` button or the `Log Plot B-H Curve` button on the B-H data dialog. The first point of a B-H curve is usually at $H = 0$, which has no place on a logarithmic axis, so the log plot leaves it out. You should see a B-H curve that looks something like the curve pictured in Figure 2.11. The boxes in the plot represent the locations of the entered B-H points, and the line represents a cubic spline derived from the entered data. Since FEMM interpolates between your B-H points using cubic splines, it

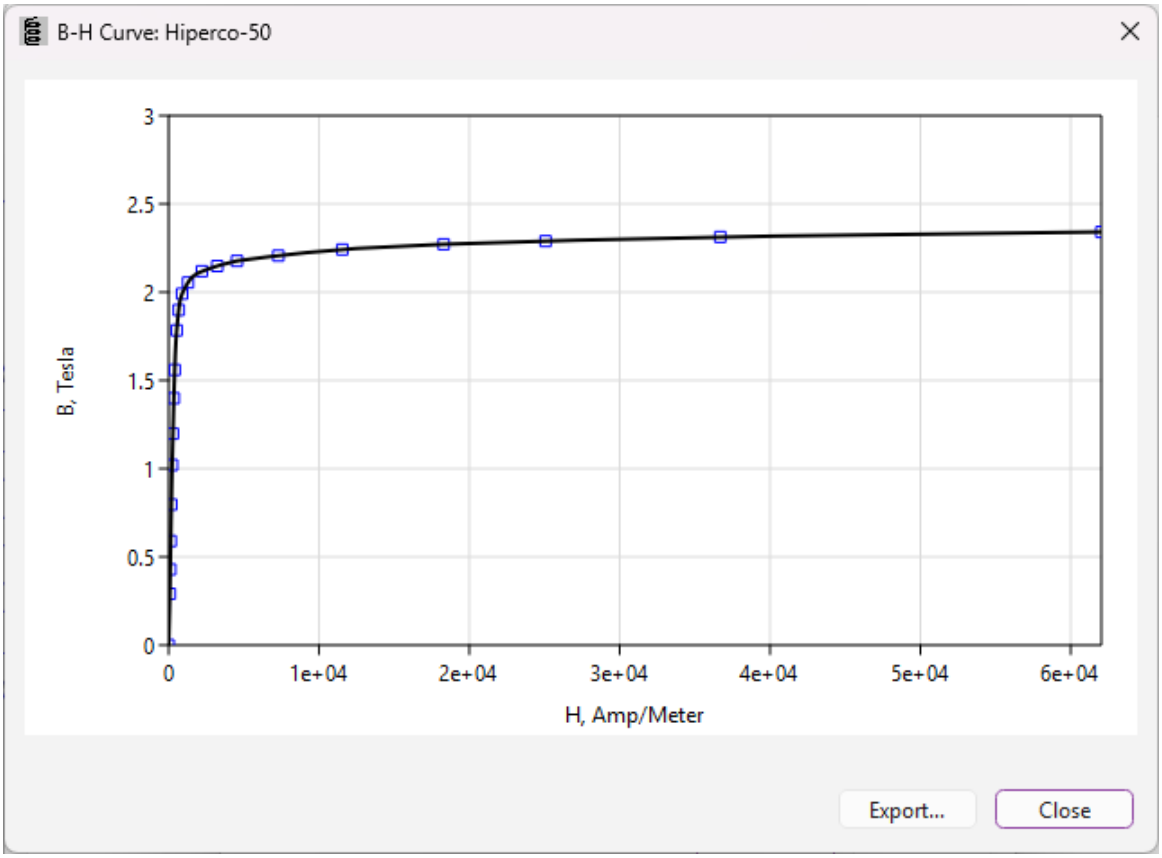


Figure 2.11: Sample B-H curve plot.

is possible to get a bad curve if you haven't entered an adequate number of points. "Weird" B-H curves can result if you have entered too few points around relatively sudden changes in the B-H curve. Femm "takes care of" bad B-H data (*i.e.* B-H data that would result in a cubic spline fit that is not single-valued) by repeatedly smoothing the B-H data using a three-point moving average filter until a fit is obtained that is single-valued. This approach is robust in the sense that it always yields a single-valued curve, but the result might be a poor match to the original B-H data. Adding more data points in the sections of the curve where the curvature is high helps to eliminate the need for smoothing.

It may also be important to note that FEMM extrapolates linearly off the end of your B-H curve if the program encounters flux density/field intensity levels that are out of the range of the values that you have entered. This extrapolation may make the material look more permeable than it "really" is at high flux densities. You have to be careful to enter enough B-H values to get an accurate solution in highly saturated structures so that the program is interpolating between your entered data points, rather than extrapolating.

The ϕ_h `(deg)` box, which is enabled only when `Nonlinear B-H curve` is checked, holds a parameter ϕ_{hmax}. For nonlinear problems, the hysteresis lag is assumed to be proportional to the effective permeability. At the highest effective permeability, the hysteresis angle is assumed to

reach its maximal value of ϕ_{hmax}. This idea can be represented by the formula:

$$\phi_h(B) = \left(\frac{\mu_{eff}(B)}{\mu_{eff,max}}\right)\phi_{hmax} \tag{2.7}$$

The `Hc (A/m)` edit box holds the coercivity H_c. If the material is a permanent magnet, you should enter the magnet's coercivity here in units of Amperes per meter. The box is disabled, and forced to zero, for the laminated and wire material types, which are soft-magnetic by construction. There are some subtleties involved in defining permanent magnet properties (especially nonlinear permanent magnets). Please refer to the Appendix A.1 for a more thorough discussion of the modeling of permanent magnets in FEMM.

The `J (MA/m`^{2}`)` edit box represents J, the source current density in the block, in megamperes per square meter. The "source current density" denotes the current in the block at DC. At frequencies other than DC in a region with non-zero conductivity, eddy currents will be induced which will change the total current density so that it is no longer equal to the source current density. Use "circuit properties" to impose a value for the total current carried in a region with eddy currents. Source current density can be complex valued, if desired.

The σ `(MS/m)` edit box denotes the electrical conductivity of the material in the block. This value is generally only used in time-harmonic (eddy current) problems. The units for conductivity are 10^6 Seymens/Meter (equivalent to $10^6(\Omega * \text{Meters})^{-1}$). For reference, copper at room temperature has a conductivity of 58 MS/m; a good silicon steel for motor laminations might have a conductivity of as low as 2 MS/m. Commodity-grade transformer laminations are more like 9 MS/m. You should note that conductivity generally has a strong dependence upon temperature, so you should choose your values of conductivity keeping this caveat in mind.

The `Material Type` drop list describes how the material is put together. If the material is laminated, this list is used to denote the direction in which the material is laminated (`Laminated in-plane`, `Laminated x-direction` or `Laminated y-direction`). If the material is meant to represent a bulk wound coil, this drop list specifies the sort of wire from which the coil is constructed (`Magnet wire`, `Plain stranded wire`, `Litz wire`, `Square wire`, `10% CCA` or `15% CCA`). The two CCA types are copper-clad aluminum wire with 10% or 15% of the cross-section copper; their proximity and skin effects follow their own curve fits rather than those for solid copper wire, and the conductivity entered for the material should be that of the clad wire as a whole. The default entry, `Not laminated`, denotes a solid material. Selecting a wire type resets the material's magnetic and source properties – μ, H_c, J, and any B-H curve – since those are implied by the wire's geometry and fill rather than specified directly.

The laminated selections in this list are illustrated in Figure 2.12, which labels them by the axis the laminations run parallel to: `Laminated x-direction` is the "parallel to X" case, and `Laminated y-direction` the "parallel to Y" case. Currently, the laminations are constrained to run along a particular axis.

If some sort of laminated construction is selected in the drop list, the `Lam. thickness (mm)` and `Fill factor` edit boxes become enabled. The lamination thickness, fill factor, and lamination orientation parameters are used to implement a bulk model of laminated material. The result of this model is that one can account for laminations with hysteresis and eddy currents in harmonic problems. For magnetostatic problems, one can approximate the effects of nonlinear laminations without the necessity of modeling the individual laminations separately. This bulk lamination model is discussed in more detail in the Appendix (Section A.2).

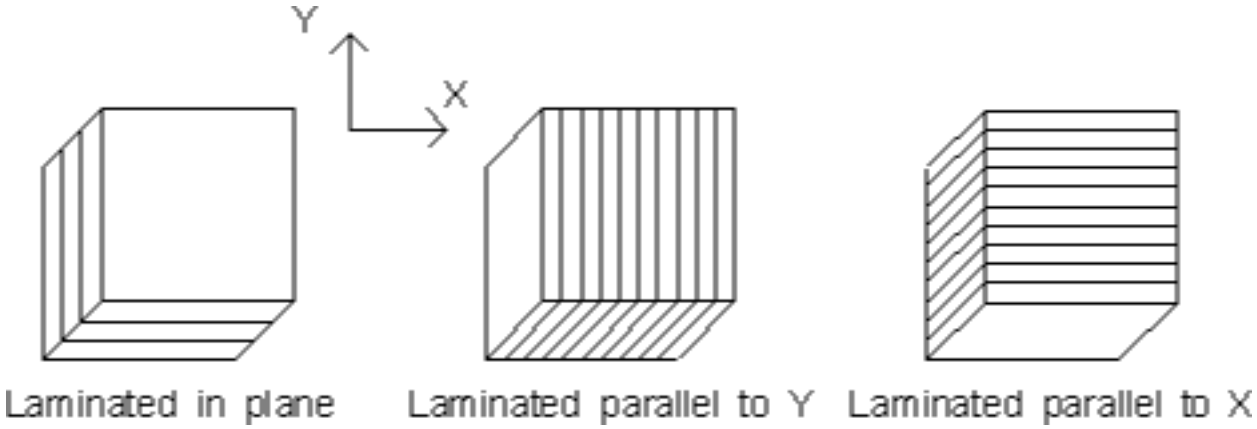


Figure 2.12: Different lamination orientation options.

The `Lam. thickness (mm)` edit box represents the thickness of the laminations used for this type of material. Enter the thickness of *just the iron part* (not the iron plus the insulation), in millimeters. The box is disabled for the material types that are not laminated.

Associated with the lamination thickness edit box is the `Fill factor` edit box. This is the fraction of the core that is filled with iron. For example, if you had a lamination in which the iron was 12.8 mils thick, and the insulation bewteen laminations was 1.2 mils thick, the fill factor would be:

$$\text{Fill Factor} = \frac{12.8}{1.2 + 12.8} = 0.914$$

If a wire type is selected, the `Wire` row's `Strands` and `Dia (mm)` edit boxes become enabled. If the `Magnet wire` or `Square wire` types are selected, it is understood that there can only be one strand, so the `Strands` box is set to 1 and disabled. The wire's diameter (or width) is then entered in the `Dia (mm)` edit box. For stranded, Litz and CCA wire, one enters the number of strands and the strand diameter. Currently, only builds with a single strand gauge are supported.

If a wire type is specified, the material property can be applied to a "bulk" coil region each individual turn need not be modeled. In DC problems, the results will automatically be adjusted for the implied fill factor. For AC problems, the the fill factor is taken into account, and AC proximity and skin effect losses are taken into account via effective complex permeability and conductivity that are automatically computed for the wound region.

Material Library

Since one kind of material might be needed in several different models, FEMM has a built-in library of Material Properties definitions. The user can access and maintain this library through the `Properties | Material Library` selection off of the main menu. When this option is selected, the `Material Library` dialog pictured in Figure 2.13 appears. This dialog allow the user to exchange Block Property definitions between the current model and the materials library via a drag-and-drop interface.

A number of different options are available via a mouse button right-click when the cursor is located on top of a material or folder. Materials can be edited by double-clicking on the desired material.

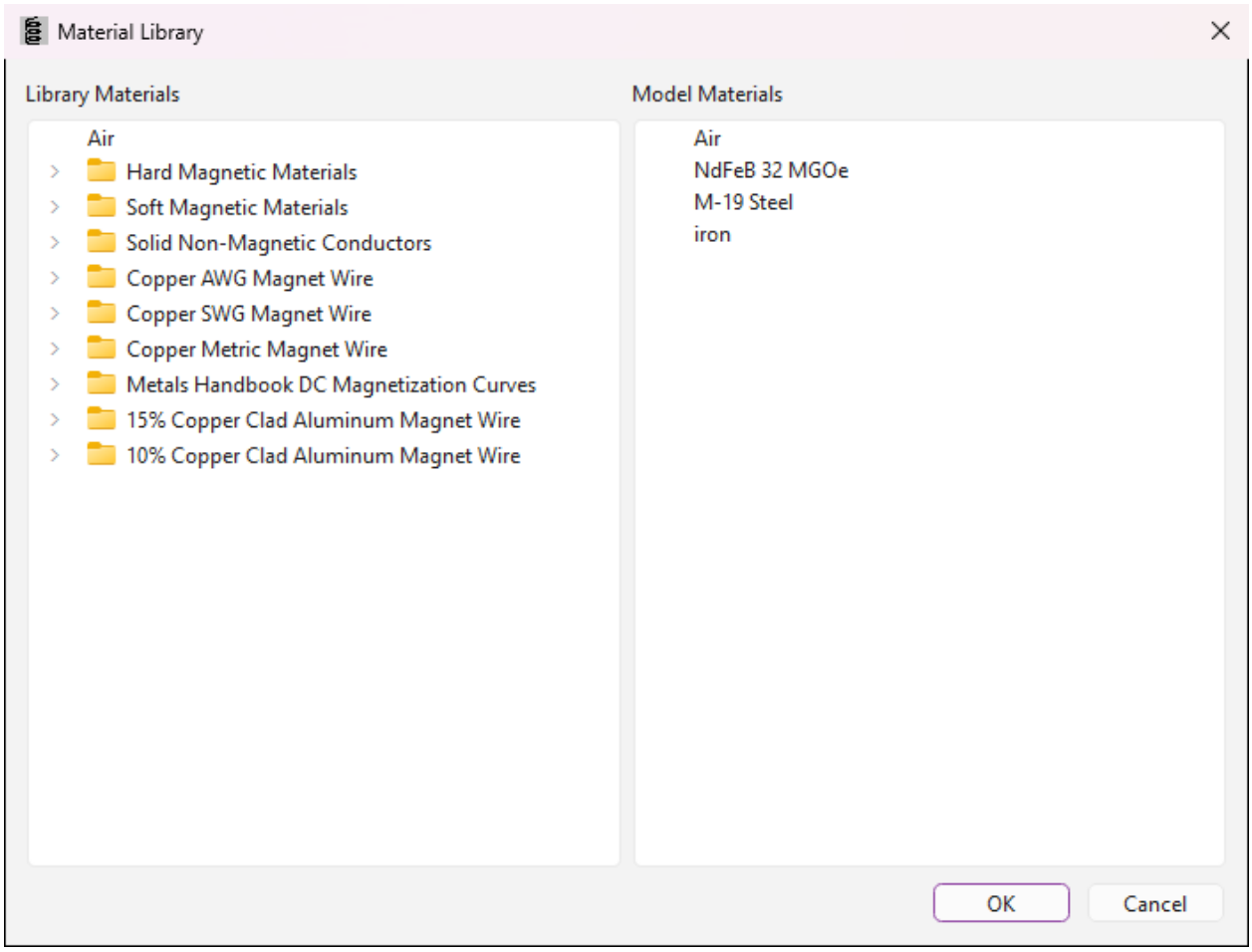


Figure 2.13: Material Library dialog.

Material from other material libraries or models can be imported by selecting the "Import Materials" option from the right-button menu that appears when the pointer is over the root-level folder of either the Library or Model materials lists.

FEMM ships a read-only copy of each materials library beside its executable files (`matlib.dat` for magnetics, and `statlib.dat`, `heatlib.dat` and `condlib.dat` for the other three problem types). Until you change something, that shipped copy is the one you are using.

The first time you press OK in the Material Library dialog, your own copy is written to your personal FEMM directory, and from then on that is the one FEMM reads and writes:

Windows	`%APPDATA%\FEMM\`
macOS	`~/Library/Application Support/FEMM/`
Linux, FreeBSD	`~/.local/share/FEMM/`

Your copy belongs to you rather than to the installation, which means your materials survive upgrading or reinstalling FEMM, are not shared with other users of the same machine, and work even where FEMM has been installed somewhere you cannot write — inside a signed application bundle on macOS, or under `/usr` on Linux. **Deleting your copy restores the materials FEMM was shipped with**, since FEMM then goes back to reading the shipped one.

Because your copy is yours, it does not automatically gain materials added to later releases of FEMM. If a new version ships a library that differs from the one your copy was made from, FEMM offers you the new one the next time you open the Material Library dialog. Accepting replaces your library with the new one; declining keeps yours, and you are not asked again until a later version changes it once more. You can always import individual materials from another library instead — see "Import Materials" above — which keeps your own edits.

Alongside each library FEMM keeps a small `.seed` file recording which shipped version your copy was made from. It is what makes the offer above possible; it is not a backup, and deleting it simply means you may be offered an update you have already seen.

Circuit Properties

The purpose of the circuit properties is to allow the user to apply constraints on the current flowing in one or more blocks. Circuits can be defined as either "parallel" or "series" connected.

If "parallel" is selected, the current is split between all regions marked with that circuit property on the basis of impedance (current is split such that the voltage drop is the same across all sections connected in parallel). Only solid conductors can be connected in parallel.

If "series" is selected, the specified current is applied to each block labeled with that circuit property. In addition, blocks that are labeled with a series circuit property can also be assigned a number of turns, such that the region is treated as a stranded conductor in which the total current is the series circuit current times the number of turns in the region. The number of turns for a region is prescribed as a block label property for the region of interest. All stranded coils must be defined as series-connected (because each turn is connected together with the other turns in series). Note that the number of turns assigned to a block label can be either a positive or a negative number. The sign on the number of turns indicated the direction of current flow associated with a positive-valued circuit current.

For magnetostatic problems, one could alternatively apply a source current density over the conductor of interest and achieve similar results. For eddy current problems, however, the "circuit" properties are much more useful–they allow the user to define the current directly, and they allow the user to assign a particular connectivity to various regions of the geometry. This information is used to obtain impedance, flux linkage, etc., in a relatively painless way in the postprocessor.

By applying circuit properties, one can also enforce connectivity in eddy current problems. By default, all objects in eddy current problems are "shorted together at infinity"–that is, there is nothing to stop induced currents from returning in other sections of the domain that might not be intended to be physically connected. By applying a parallel-connected circuit with a zero net current density constraint to each physical "part" in the geometry, the connectivity of each part is enforced and all is forced to be conserved inside the part of interest.

The dialog for entering circuit properties is pictured in Figure 2.14.

The `Name` box names the circuit, so that block labels can be assigned to it and so that the postprocessor can report results for it. `Current (A)` is the current carried by the circuit, and can be entered as a complex value for harmonic problems.

The `Type` radio buttons make the series or parallel choice described above. Because a number of turns is meaningful only for a series connection, the `Turns` entry in the block label's own dialog is disabled whenever the label belongs to a parallel circuit.

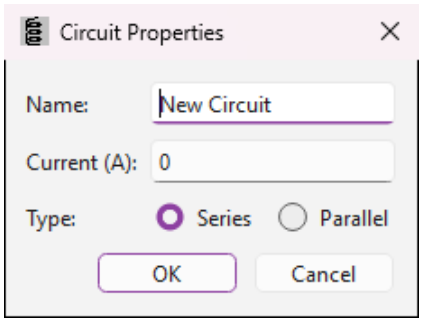


Figure 2.14: Circuit Properties dialog

2.2.8 Exterior Region

One often desires to solve problems on an unbounded domain. Appendix A.3.3 describes an easy-to-implement conformal mapping method for representing an unbounded domain in a 2D planar finite element analysis. Essentially, one models two disks–one represents the solution region of interest and contains all of the items of interest, around which one desires to determine the magnetic field. The second disk represents the region exterior to the first disk. If periodic boundary conditions are employed to link the edges of the two disks, it can be shown (see Appendix A.3.3) that the result is exactly equivalent to solving for the fields in an unbounded domain.

One would also like to apply the same approach to model unbounded axisymmetric problems, as well as unbounded planar problems. Unfortunately, the Kelvin Transformation is a bit more complicated for axisymmetric problems. In this case, the permeability of the external region has to vary based on distance from the center of the external region and the outer radius of the external region. The approach is described in detail in [22]. FEMM automatically implements the variation of permeability in the exterior region, but a bit more information must be collected to perform the permeability grading required in the external region. This is where the "External Region" parameters come in–these are the parameters that the program needs to define the permeabilities of elements in the external region for "unbounded" axisymmetric problems.

Specifically, there are three parametes that are collected in the dialog that appears when the user selects the External Region properties. These are:

- `Center of Exterior Region` The location along the z-axis of the axisymmetric problem where the center of the block representing the external region is located.
- `Radius of Exterior Region` Radius of the sphere representing the exterior region.
- `Radius of Interior Region` Radious of the spehre representing the interior region (*i.e.* the region in which the items of interest are located).

To finish defining the axisymmetric external region, the `Block located in an external region` check box must be selected in any block labels that are located in the region that is desired to be the axisymmetric external region.

2.2.9 Analysis Tasks

Meshing the model, analyzing the model, and viewing the results are most easily performed by the toolbar buttons pictured in Figure 2.15

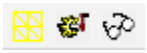


Figure 2.15: Toolbar buttons for starting analysis tasks.

The first of these buttons (with the "yellow mesh" icon) runs the mesh generator. The solver actually automatically calls the mesh generator to make sure that the mesh is up to date, so you never *have* to call the mesher from within femm. However, it is almost always important to get a look at the mesh and see that it "looks right." When the mesh generation button is pressed, the mesher is called. After the geometry is triangulated, the finite element mesh is loaded into memory and displayed underneath the defined nodes, segments, and block labels as a set of yellow lines.

If you have a very large model, just keeping all of the mesh information in core can take up a significant amount of memory. If you are about to analyze a very large problem, it might be a good idea to choose the `Mesh | Purge Mesh` option off of the main menu. When this option is selected, the mesh is removed from memory, and the memory that it occupied is freed for other uses.

The second button, with the "hand-crank" icon, executes the solver, `fkn`. While fkn runs, a `Solver` window displays status information to the user. fkn requires no user interaction while it is running, although the window's `Abort` button will stop it. When fkn is finished analyzing your problem, the window closes. The time that fkn requires is highly dependent on the problem being solved. Solution times can range from less than a second to several minutes, depending upon the size and complexity of the problem. Generally, linear magnetostatic problems take the least amount of time. Harmonic problems take slightly more time, because the answer is in terms of complex numbers. The complex numbers effectively double the number of unknowns as compared to a magnetostatic problem with the same mesh. The slowest problems to analyze are nonlinear time-harmonic problems, since multiple successive approximation iterations must be used to converge on the final solution. However, nonlinear problems rarely take more than about a dozen. Later iterations in nonlinear problems are usually are quite fast compared to the first iteration or two because the later iterations can be initialized with an approximate solution that is very close to the "actual" solution.

For users who have a technical interest in what is "under the hood" in fkn, some details are provided in the Appendix (Section 7).

The "big magnifying glass" icon is used to display the results in a postprocessing window once the analysis is finished. A detailed description of the magnetics postprocessor addressed in Section 2.3.

2.3 Magnetics Postprocessor

The magnetics postprocessing functionality of FEMM is used to view solutions generated by the `fkern` solver. A magnetics postprocessor window can be opened either by loading some previously run analyses via `File|Open` on the main menu, or by pressing the "big magnifying glass" icon from

within a preprocessor window to view a newly generated solution. Magnetics postprocessor data files stored on disk have the `.ans` extension.

2.3.1 Postprocessor modes

Similar to the preprocessor, the postprocessor always operates in one of three modes, depending upon the task to be performed. These modes are:

- *Point Values Mode* In this mode, the user can click on various points in the solution region. Local field values are then listed in the `FEMM Output` window.
- *Contour Mode* This mode allows the user to define arbitrary contours in the solution region. Once a contour is defined, plots of field quantities can be produced along the contour, and various line integrals can be evaluated along the contour.
- *Block Mode* The Block Mode lets the user define a subdomain in the solution region. Once the block has been defined, a variety of area and volume integrals can be taken over the defined subdomain. Integrals include stored energy (inductance), various kinds of losses, total current in the block, and so on.

The current postprocessor mode is controlled via the Analysis Mode toolbar buttons, shown in Figure 2.16. The buttons denote, respectively, Point Values mode, Contour Mode, and Block



Figure 2.16: Analysis Mode toolbar buttons

Mode. The depressed button denotes the current mode. The default mode when postprocessor starts is Point Values mode.

2.3.2 View and Grid Manipulation

The aspects of the current view and of the current grid are regulated via the use of toolbar buttons. The view is manipulated by the following toolbar buttons: and the grid settings are manipulated by



Figure 2.17: View Manipulation toolbar buttons.



Figure 2.18: Grid Manipulation toolbar buttons.

these grid manipulation toolbar buttons: The grid and view manipulation work in exactly the same fashion as these same features in the preprocessor. Refer to Section 2.2.4 for a detailed description of grid manipulation, and to Section 2.2.3 for view manipulation.

2.3.3 Keyboard Commands

Unlike the preprocessor, postprocessor is not very dependent on keyboard commands.

In the Point Values mode, there is only one relevant keypress. In this mode, the Tab key allows the user to enter the coordinates of a specific point. After the coordinates of a point are entered, the field values at that point are displayed in the `FEMM Output` window.

In the Contours, there are four relevant keys. Pressing the Escape key wipes out any current contour or block definition. Pressing the Delete removes the last point added to the current contour or block edge. Pressing the Shift key allows the user to turn the last segment in the prescribed contour from a straight line into an arc. A dialog pops up after the key is pressed that prompts for attributes of the the desired arc. Last, pressing the Tab keys allows the user to numerically enter the coordinates of a point to be included in the current contour.

In the Block mode, the Escape and Delete keys have the same definition as in Contours mode. In the Block mode, the Tab key does not do anything, since all points on the contour must also be Points defining the model's geometry.

2.3.4 Mouse Actions

In contrast, the operation of the postprocessor is very dependent upon input from the mouse.

In the Point Values mode, a Left Button Click is used to display field values at the current mouse location. If Snap to Grid is enabled, values are displayed at the closes grid point instead.

In the Contours mode, mouse clicks are used to define the contour. A Left Button Click adds the closest Point in the model's geometry. Via a Right Button Click, the current mouse pointer location is added to the contour. A contour appears as a red line on the screen.

Blocks are defined in Block mode in a fashion very similar to the way in which contours are defined. A block is defined by drawing a contour around the region of interest. The contour appears as a green line on the postprocessor screen. When the ends of the contour meet, the block is defined. All elements enclosed by the contour (all elements that form the block) then turn green in the postprocessor window.

A Left Button Click attempts to add the nearest Point in the input geometry to the Block's contour. However, a block can only be defined along line and arc segments from the input geometry. Each node on the boundary of the block must be selected in order to form the Block boundary. In Block mode, the Right mouse button has no function.

2.3.5 Miscellaneous Useful View Commands

The postprocessor's `View` menu collects the display options. `Density Plot`, `Contour Plot` and `Vector Plot` are described in the sections that follow. The first two are check items that switch their plot on and off directly; the toolbar buttons of the same name open the dialogs that configure them. The remaining entries are:

- `Show Mesh` Draws the finite element mesh over the solution. It is off when a solution is first loaded.
- `Smoothing` By default, a smoothing algorithm is applied to the flux density solution. Because first-order triangles are used as trial functions for vector potential, the resulting flux density and field intensity distributions are piece-wise constant in each element. The smoothing algorithm uses a nearest neighbor interpolation to obtain linear B and H distributions over each element. The smoothed solution generally looks better on the screen, and somewhat increases the accuracy of B and H near the vertices of each element. This item toggles smoothing for the solution in the active window. The `Edit|Preferences` dialog sets the value that newly opened solutions start with, and the `mo_smooth` scripting command does the same job from a script.
- `Show Points` Especially when making graphics for reports, presentations, etc, it may be desirable to hide the small boxes on the screen that denote input node points. The `Show Points` option allows the user to toggle whether or not the input points are shown. As with smoothing, `Edit|Preferences` sets the starting value.
- `Show Block Names` Toggles the display of the block labels' names. It is off when a solution is first loaded.
- `Circuit Properties` Opens the circuit results dialog described in the *Circuit Results* section below. In the scalar physics the entry is titled `Conductor Properties` instead.
- `Problem Info` Reports the problem type, frequency, depth and length units of the solution, along with the number of nodes and elements in its mesh.
- `Plot B-H Curves` Asks which nonlinear material to plot and how many points to sample (the count starts at the `Edit|Preferences` value), then draws that material's B-H curve as the solver used it. For a DC problem this is the curve you entered. For a harmonic problem it is the effective curve: the fundamental of B under sinusoidal H, with the hysteresis lag, lamination eddy currents and fill factor folded in. It is complex, so it is drawn as Re[B] and Im[B] against $|H|$. The plot window's `Export...` button writes the points to a file.

2.3.6 Contour Plot

One of the most useful ways to get a subjective feel for a magnetics finite element solution is by plotting the "flux lines." These are the streamlines along which flux flows in the finite element geometry. Where flux lines are close together, the flux density is high. In FEMM's vector potential formulation, flux lines are simply plots of the level contours of the vector potential, A, in planar problems, or level contours of $2\pi rA$ in axisymmetric problems.



Figure 2.19: Graph Mode toolbar buttons.

For harmonic problems, the contours are a little more subtle–A has both real and imaginary components. In this case, postprocessor allows the user to plot contours of either the real or the imaginary part of A. Real contours appear black, and Imaginary contours appear as grey.

By default, a set of 19 flux lines are plotted when a solution is initially loaded into postprocessor. The number and type of flux lines to be plotted can be altered using the Contours Plot icon in the Graph Mode section of the toolbar (see Figure 2.19. The Contour Plot icon is the icon with the black contours. When this button is pressed, a dialog pops up, allowing the choice of the number of contours and which contours to plot (either real, imaginary, or none). The default of 19 is itself a preference, set under `Edit|Preferences`.

In the contour plot dialog, a check box is also present titled "Show stress tensor mask". If this box is checked, the contour lines associated with the last Weighted Stress Tensor integration are also displayed, by default as orange flux lines.

2.3.7 Density Plot

Density plots are also a useful way to get a quick feel for the flux density in various parts of the model. A density plot can be displayed by pressing the middle button in the Graph Mode section of the toolbar (see Figure 2.19). A dialog the pops up that allows the user to turn density plotting on. The user can choose to plot flux density, field intensity, or current density. If the solution is to a harmonic problem, the user can choose to plot either the magnitude or just the real or imaginary part of these quantities.

The flux density at each point is classified into one of twenty contours distributed evenly between either the minimum and maximum flux densities or user-specified bounds.

2.3.8 Vector Plots

A good way of getting a feel for the direction and magnitude of the field is with plots of the field vectors. With this type of plot arrows are plotted such that the direction of the arrow indicates the direction of the field and the size of the arrow indicates the magnitude of the field. The presence and appearance of this type of plot can be controlled by pressing the "arrows" icon pictured in Figure 2.19.

2.3.9 Line Plots

When postprocessor is in Contours Mode, various field values of interest can be plotted along the defined contour. A plot of a field value defined contour is performed by pressing the "graphed function" icon in the Plot, Integration, and Circuit Results group of toolbar buttons, shown in Figure 2.20. When this button is pressed, the `X-Y Plot` dialog (see Figure 2.21) appears with a drop list containing the types of line plots available. Choose the desired type of plot and press "OK." The dialog also has an edit box for the number of points to be sampled along the contour.



Figure 2.20: Line Plot, Integration, and Circuit Results toolbar buttons

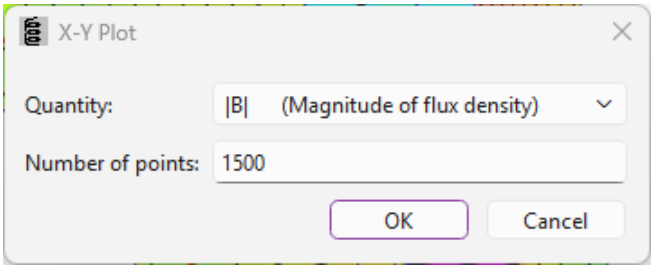


Figure 2.21: X-Y Plot dialog.

After "OK" is pressed, the program computes the desired values along the defined contour. These values are then displayed automatically in a plot window.

To save the plotted values, press the `Export...` button on the plot window. A file selection dialog appears and prompts for the name of a file to which the data is written. The data is written as comma-separated values (`.csv`), with a header row naming the columns: position along the contour, followed by one column for each curve on the plot.

Currently, the type of line plots supported are:

- Vector potential along the contour;
- Magnitude of the flux density along the contour;
- Component of flux density normal to the contour;
- Component of flux density tangential to the contour;
- Magnitude of the field intensity along the contour;
- Component of field intensity normal to the contour;
- Component of field intensity tangential to the contour;
- Eddy current density along the contour;
- Total current density along the contour.

In all of these plots, the direction of the normal is understood to be as shown in Figure 2.22. The tangential direction is understood to be the direction in which the contour was defined.

In certain cases, the quantity to be plotted can be ambiguous. This can occur, for example, if a plot of the tangential field intensity is requested on a contour running along an interface between air and a piece of iron. In this case, there is a discontinuity in the tangential field intensity, and the value of this quantity is different on each side of the interface. The postprocessor resolves the conflict by always evaluating the plots at a differentially small distance to the "normal" side of the line. Therefore, by defining the same contour but reversing the order in which the points are specified, plots of the quantity of interest on each side of a boundary can be obtained.

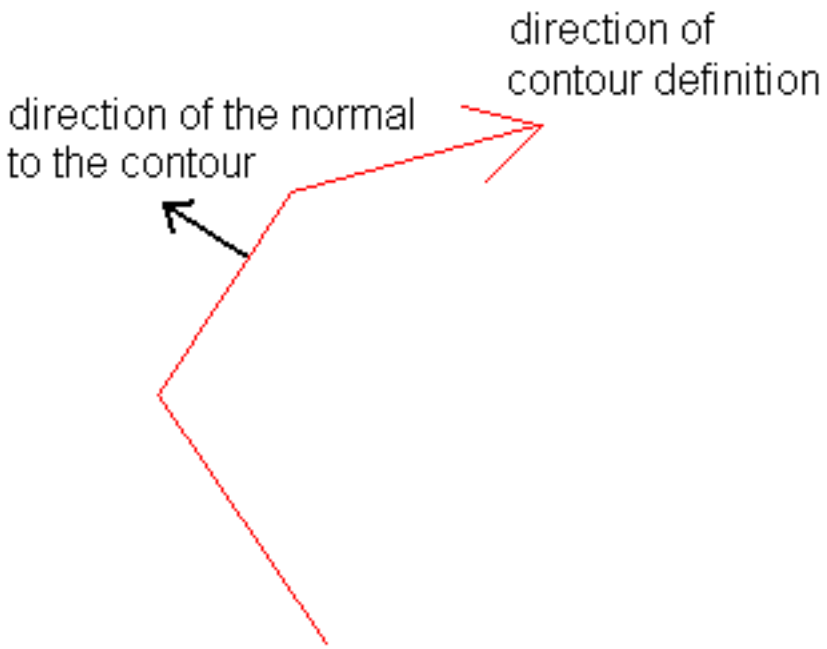


Figure 2.22: When in doubt plots and integrals taken on this side of a contour.

2.3.10 Line Integrals

Once a contour has been specified in Contours mode, Line Integrals can be performed along the specified contour. How the integral is evaluated depends on which one is chosen. `B.n` and `H.t` follow from the vector potential at the contour's endpoints, and `Contour Length` is a sum of segment lengths, so all three are exact. The stress tensor integrals and `(B.n)^2` instead sample the field at 400 points along each segment of the contour and sum by the midpoint rule.

To perform an integration, press the "integral" icon on the toolbar (as shown in Figure 2.20). A small dialog will appear with a drop list. Choose the desired integral from the drop list and press `OK`. The amount of time required to perform the integral will be virtually instantaneous for some types of integrals; however, some types may require several seconds to evaluate. When the evaluation of the integral is completed, the answer appears on the screen in a pop-up box.

One "tip" that may aid with the definition of contours of integration is that a curved integration contour can be defined in a fairly painless fashion by hitting the Shift key. Hitting the Shift key tells the program to turn the last segment in the defined integration contour into an arc segment. A dialog then pops up that prompts for the desired attributes of the arc.

The line integrals currently supported are:

- `B.n`. This integral returns the total flux passing normal to the contour. This integral is useful for determining the total flux in a bulk flux path. This result might then be compared to predictions from a simpler magnetic circuit model, for example.

- `H.t`. The integral of the tangential field intensity along the contour yields the magnetomotive force drop between the endpoints of the contour. Again, this integral is useful for comparison to or validation of magnetic circuit models.

- `Contour Length`. This integral returns the length of the defined contour in meters.
- `Force from stress tensor`. This integral totals the force produced on the contour derived from Maxwell's stress tensor. Deriving meaningful force results requires some care in the choice of integration path; refer to Section 2.3.12 for a detailed discussion of force and torque calculation.
- `Torque from stress tensor`. This selection integrates the torque about the point (0,0) inferred from Maxwell's stress tensor. Again, some guidelines must be followed to get accurate torque results (see Section 2.3.12).
- `B.n^2`. This selection evaluates the integral of the square of the normal flux along the line. This integral is not so commonly used, but it has been useful in the past for some specialized purposes, like determining the RMS amplitude of a periodic flux distribution.

2.3.11 Block Integrals

Once a closed contour has been specified in Block mode and the block appears highlighted in green, Block Integrals can be performed over the specified area. These integrals are performed by analytically integrating the specified kernel over each element in the defined region, and summing the results for all elements.

To perform an integration, press the "integral" icon on the toolbar (as shown in Figure 2.20). A small dialog will appear with a drop list. Choose the desired integral from the drop list and press `OK`. Generally, volume integrals take several seconds to evaluate, especially on dense meshes. Be patient. When the evaluation of the integral is completed, the answer appears on the screen in a pop-up box.

The block integrals currently supported are:

- `A.J` This integral is the evaluation of $\int A \cdot J dV$ that is usually used to evaluate inductance for linear problems. Generally, the self-inductance of a coil is:

$$L_{self} = \frac{\int A \cdot J dV}{i^2} \tag{2.8}$$

  where i is the current flowing through the coil.

- `A` This integral, the evaluation of $\int A dV$, can be used to evaluate mutual inductances between coils. Similar to the formula for self inductance, mutual inductance is:

$$L_{mutual} = \frac{\int A_1 \cdot J_2 dV_2}{i_1 i_2} \tag{2.9}$$

  where A_1 is the component of A produced by the first coil, J_2 is the current in the second coil, and i_1 and i_2 are the current in the first and second coils, respectively. dV_2 is meant to denote that the integral is taken over the volume of the second coil. We can rearrange (2.9) into a somewhat simpler form by noting that $n_2 * i_2 = J_2 * a_2$. That is, the total amps times

turns for the second coil equals the current density in the second coil times the second coil's cross-section area. Substituting for J_2 in (2.9) yields:

$$L_{mutual} = \frac{n_2}{i_1 a_2}\left(\int_{J_{2+}} A_1 dV_2 - \int_{J_{2-}} A_1 dV_2\right) \tag{2.10}$$

where the first bracketed term in (2.10) is the contribution from the turns of coil 2 that are pointed out of the page and the second term is the contribution from the turns of coil 2 that are pointed into the page. To evaluate mutual inductance with FEMM, one substitutes values into (2.10). First, run the model with only "coil 1" turned on. Then, integrate A over the volume in which the second coil lies (although the second coil is not turned on). For planar problems, you will typically have to make two separate integrations–one over the region where the turns in "coil 2" are pointed out of the page (*i.e.* that part of the coil in which a positive current results in current flowing in the out-of-the-page direction), and one over the region in which the turns in "coil 2" are pointed into the page. Add these two results together to get the total $A_1 dV_2$ integral. Lastly, multiply the integral result times $n_2/(i_1 a_2)$ to get mutual inductance.

- `Magnetic field energy` This selection calculates the energy stored in the magnetic field in the specified region. This integral can be used as an alternate method of getting inductance for problems that are linear (at least not heavily saturated). Denoting E as the energy stored in the magnetic field, inductance can be obtained by solving the formula:

$$E = \frac{Li^2}{2} \tag{2.11}$$

In the case of nonlinear materials, the energy is computed via:

$$W = \int \left(\int_0^B H(B')dB'\right) dV \tag{2.12}$$

to take proper account of the energy under nonlinear conditions.

- `Magnetic field coenergy` For linear problems, coenergy is numerically the same as energy. For nonlinear problems, coenergy is defined as:

$$W_c = \int \left(\int_0^H B(H')dH'\right) dV \tag{2.13}$$

Coenergy can be used in an alternative method of force and torque computation. To compute force via coenergy, currents are held constant, and the position of the object upon which the force is desired is perturbed slightly. The force can then be estimated by:

$$F = \frac{W_c(p+\delta) - W_c(p)}{\delta} \tag{2.14}$$

where p denotes the initial position, $p+\delta$ denotes the perturbed position, and δ is the magnitude of the perturbation. The component of force determined in this way acts along the direction of the perturbation–one has to perform two such operations to get both horizontal and vertical components of the force.

- `Hysteresis, Laminated eddy, or Proximity effect`. This selection is typically used to obtain the core losses produced in laminated iron sections in harmonic problems. It also accounts for the proximity effect losses of a region defined as a bulk wound coil.

- `Resistive losses` This selection integrates the i^2R losses due to currents flowing in the "z" direction (or θ direction, if you are evaluating an axisymmetric problem).

- `Block cross-section area`

- `Total losses` This selection totals the losses from all possible loss mechanisms that might apply over the given block. This is especially useful for finding losses in a region that might enclose several different types of materials with different loss mechanisms.

- `Lorentz force (JxB)` Lorentz force is the force produced by a magnetic field acting upon a current:

$$F_{Lorentz} = \int J \times B \, dV \tag{2.15}$$

  Many devices (*e.g.* voice coil actuators) produce forces in a fashion that can be evaluated using this integral.

- `Lorentz torque (rxJxB)` This selection computes the torque about $(0,0)$ resulting from Lorentz forces.

- `Integral of B over block` This integral can be useful in computing Lorentz forces. Since Lorentz force is $J \times B$, the force that would be produced if a coil were placed in a certain part of the solution domain can be inferred by integrating B, and then scaling times an arbitrarily chosen current density to get force.

- `Total current` This integral returns the total specified currents in the given block.

- `Block Volume` For axisymmetric problems, this selection returns the volume swept out by the selected block.

- `Force via Weighted Stress Tensor` The Weighted Stress Tensor block is a volume integral version of Maxwell's stress tensor that automatically picks a collection of paths for the integration that yield "good" force results. This approach is similar to the weighted stress tensor approach described in [7] and/or [8].

  To compute the force on a region or set of regions, the user selects the blocks upon which force result is desired and selects the `Force via Weighted Stress Tensor` integral. The program then computes the weighting function by solving an additional Laplace equation over the air surrounding the blocks upon which the force is to be computed. It may take a few seconds to compute the weighting function–progress is indicated by a progress bar that is displayed while the weighting function is being computed. The stress tensor is then evaluated as a volume integration, and the results are displayed. The results are typically more accurate than the Maxwell Stress Tensor line integral, since in some sense, all possible contours have been averaged to yield the Weighted Stress Tensor force result.

If the user is interested in the contours along which the integral was performed, the "stress tensor mask" box can be checked in the contour plot dialog. A set of orange (by default) lines is then displayed.

- `Torque via Weighted Stress Tensor` This integral is the torque version of the `Force via Weighted Stress Tensor` integral. Instead of force, torque about (0,0) is computed using the same weighting function approach.

- `R^2 (i.e. Moment of Inertia/density)` This integral is used to determine the moment of inertial of the selected blocks. To obtain moment of inertia, the result of this integral must be multiplied by the density of the selected region. For 2D planar problems, the moment of inertia about the z-axis (*i.e.* about x=0,y=0) is computed. For axisymmetric problems, the integral is computed about the r=0 axis.

- `Total Loss Density` This selection reports the `Total losses` of the selected blocks divided by their volume, in Watts per cubic meter. It is a convenience for comparing regions of different size, since the total loss of a large block will exceed that of a small one even when the small one is running hotter.

2.3.12 Force/Torque Calculation

Ultimately, the estimation of magnetically produced forces and torques is often the goal of a finite element analysis. This section discusses some of the different methods of deducing forces and torques using FEMM.

Lorentz Force/Torque

If one is attempting to compute the force on a collection of currents in a region containing *only* materials with a unit relative permeability, the volume integral of Lorentz force is always the method to employ. Lorentz results tend to be very accurate. However, they are only applicable to the forces on conductors with unit permeability (*e.g.* coils in a voice coil actuator).

Weighted Stress Tensor Volume Integral

This volume integral greatly simplifies the computation of forces and torques, as compared to evaluating forces via the stress tensor line integral or by differentiation of coenergy. Merely select the blocks upon which force or torque are to be computed and evaluate the integral. No particular “art” is required in getting good force or torque results (as opposed to the Stress tensor line integral), although results tend to be more accurate with finer meshing around the region upon which the force or torque is to be computed.

One limitation of the Weighted Stress Tensor integral is that the regions upon which the force is being computed must be entirely surrounded by air and/or abutting a boundary. The program checks this and refuses a selection that abuts a region which is not free space, rather than returning a wrong answer. In cases in which the desired region abuts a non-air region, force results may be deduced from differentiation of coenergy–see (2.14).

Maxwell Stress Tensor Line Integral

The indiscriminate use of Maxwell's Stress Tensor can result in bad predictions of forces and torques. The goal of this section is to explain how to set up problems and properly choose integration paths so that good estimations of force and torque might be obtained via stress tensor methods. Generally, you should not use the Stress Tensor line integral to compute forces and torques if it can be avoided (*i.e.* use the volume integral version instead).

Maxwell's stress tensor prescribes a force per unit area produced by the magnetic field on a surface. The differential force produced is:

$$dF = \tfrac{1}{2}\left(H(B\cdot n)+B(H\cdot n)-(H\cdot B)n\right) \tag{2.16}$$

where n denotes the direction normal to the surface at the point of interest. The net force on an object is obtained by creating a surface totally enclosing the object of interest and integrating the magnetic stress over that surface.

While an integration of (2.16) theoretically gives the magnetic force on an object, numerical problems arise when trying to evaluate this integral on a finite element mesh made of first-order triangles. Though the solution for vector potential A is relatively accurate, the distributions of B and H are an order less accurate, since these quantities are obtained by differentiating the trial functions for A. That is, A is described by a linear function over each element, but B and H are piece-wise constant over each element. Errors in B and H can be particularly large in elements in which the exact solution for B and H changes rapidly–these areas are just not well approximated by a piece-wise approximation. Specifically, large errors can arise in the tangential components of B and H in elements adjacent to boundaries between materials of different permeabilities. The worst errors arise on this sort of interface at corners, where the exact solution for B is nearly a singularity.

If the stress tensor integral is evaluated on the interface between two different materials, the results will be particularly erroneous. However, the stress tensor has the property that, for an exact solution, the same result is obtained regardless of the path of integration, as long as that path encircles the body of interest and passes only through air (or at least, every point in the contour is in a region with a constant permeability). This implies that the stress tensor can be evaluated over a contour a few elements away from the surface of an object–where the solution for B and H is much more accurate. Much more accurate force results will be obtained by integrating along the contour a few elements removed from any boundary or interface. The above discussion is the rationale for the first guideline for obtaining forces via stress tensor:

> Never integrate the stress tensor along an interface between materials. Always define the integration contour as a closed path around the object of interest with the contour displaced several elements (at least two elements) away from any interfaces or boundaries.

As an example of a properly defined contour, consider Figure 2.23. This figure represents a horseshoe magnet acting on a block of steel. The magnet is the assembly at the bottom: a steel yoke with a permanent-magnet leg standing at each end, the two legs magnetized in opposite senses so that one presents a north pole and the other a south pole across the gap. The block is the single rectangle above them. The objective is to obtain the magnetic forces acting upon the block. The

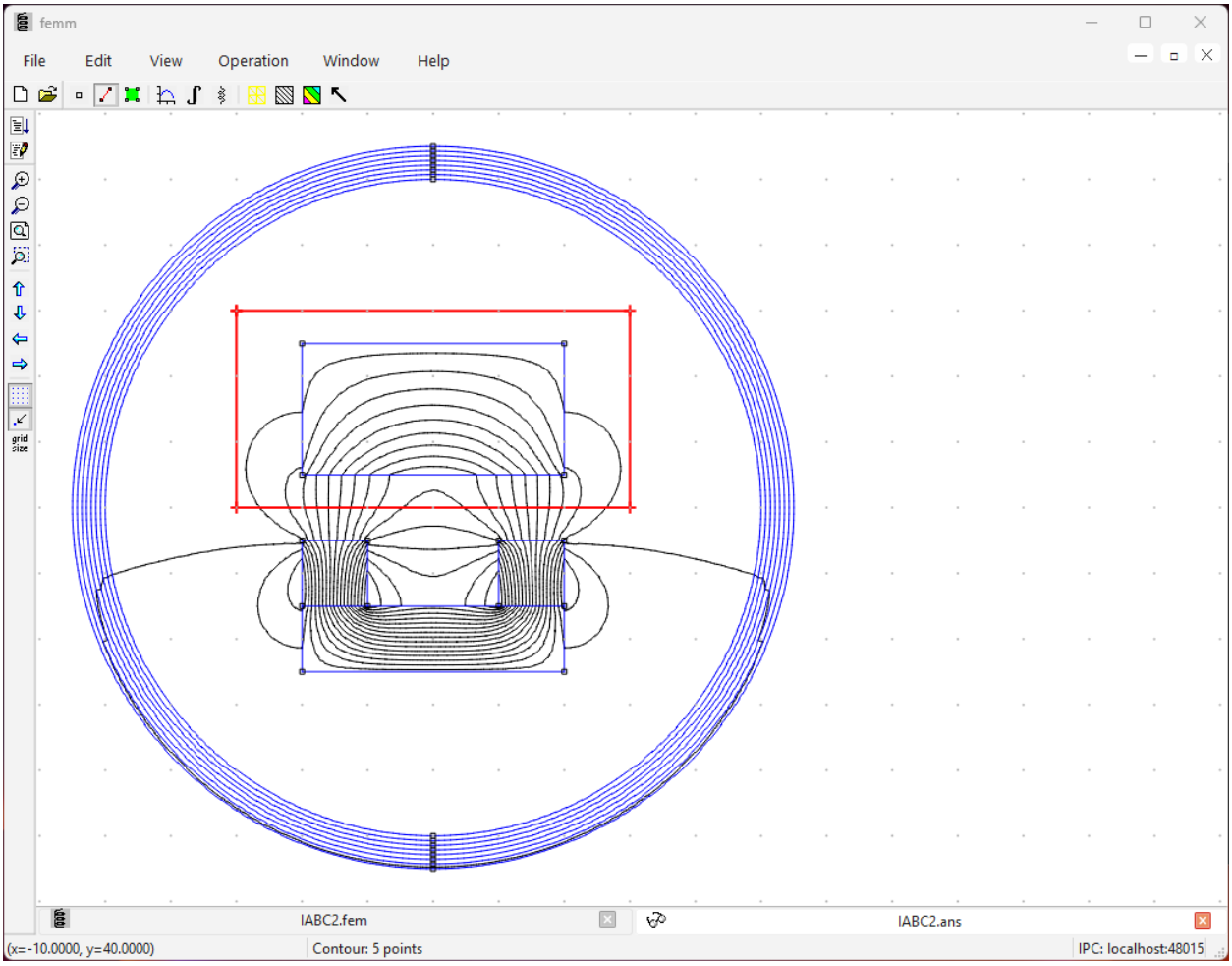


Figure 2.23: Properly defined contour for integration of Maxwell's Stress Tensor

concentric arcs around the outside of the model are a layered open-boundary approximation, not part of the device.

The red line in the figure represents the contour defined for the integration. The contour was defined running clockwise around the block, so that the normal to the contour points outward. Always define your contour in a clockwise direction to get the correct sign. Note that the contour is well removed from the surface of the block, and the contour only passes through air. To aid in the definition of a closed contour, grid and the "snap to grid" were turned on, and the corners of the contour are grid points that were specified by right mouse button clicks.

The second rule of getting good force results is:

Always use as fine a mesh as possible in problems where force results are desired.

Even though an integration path has been chosen properly (away from boundaries and interfaces), some significant error can still arise if a coarse mesh is used. Note that (2.16) is composed of B^2 terms – this means that stress tensor is one order worse in accuracy than B. The only way to get that accuracy back is to use a fine mesh density. A good way to proceed in finding a mesh that is "dense enough" is to solve the problem on progressively finer meshes, evaluating the force on each

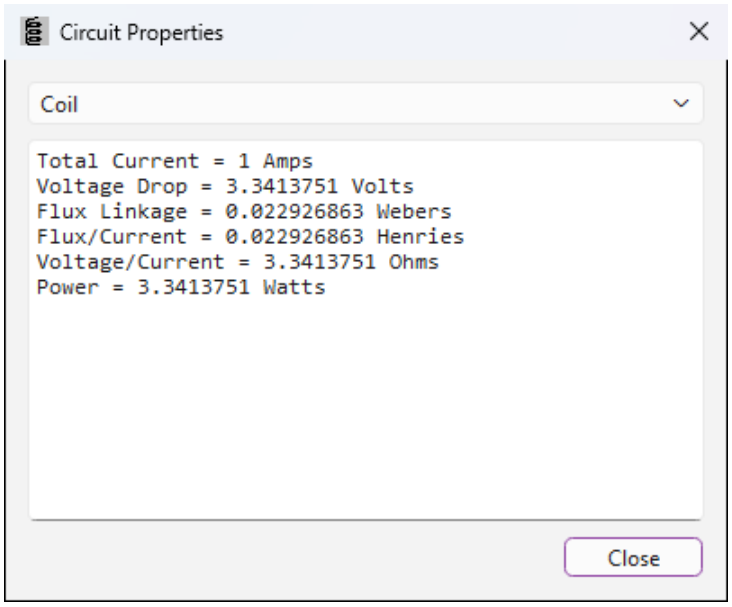


Figure 2.24: Circuit results dialog.

mesh. By comparing the results from different mesh densities, you can get and idea of the level of accuracy (by looking at what digits in the answer that change between various mesh densities). You then pick the smallest mesh density that gives convergence to the desired digit of accuracy.

For torque computations, all the same rules apply as for force computations (*i.e.* define integration contours away from boundaries and interfaces, and use a dense mesh).

2.3.13 Circuit Results

If "circuit" properties are used to specify the excitation, a number of useful properties relative to the circuit are automatically available. To view the circuit results, either press the "Circuit Props" button on the toolbar pictured in Figure 2.20 or select `View|Circuit Properties` off of the postprocessor main menu. A dialog, as pictured in Figure 2.24 will appear. There is a drop list on the dialog, from which the user selects the circuit for which results are desired.

2.4 Electrostatics Preprocessor

The preprocessor is used for drawing the problems geometry, defining materials, and defining boundary conditions. The process of construction of electrostatics problems is mechanically nearly identical to the construction of magnetics problems–refer to Sections 2.2.1 through 2.2.5 for an overview of the FEMM editing and problem creation commands. This section considers those parts of problem definition that are unique to electrostatics problems.

2.4.1 Problem Definition

The definition of problem type is specified by choosing the `Problem` selection off of the main menu. Selecting this option brings up the Problem Definition dialog, shown in Figure 2.25.

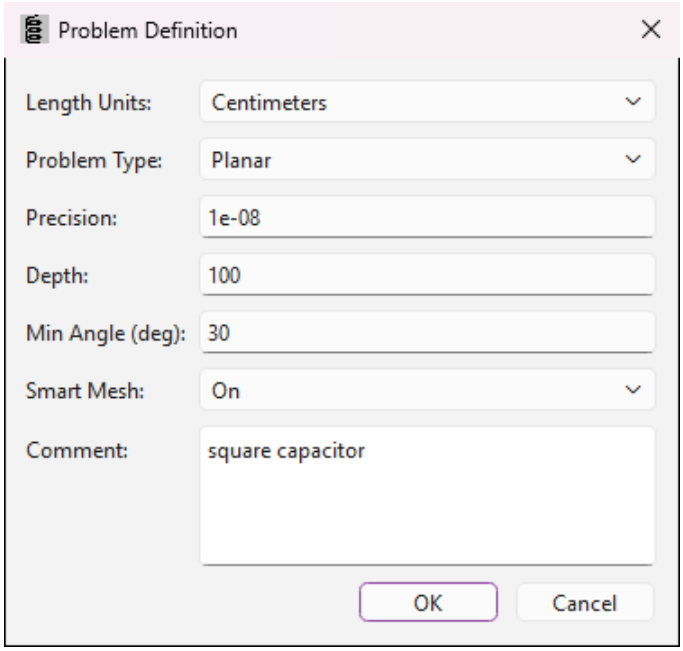


Figure 2.25: Problem Definition dialog.

The dialog's entries, in the order they appear:

- `Length Units`. This box identifies what unit is associated with the dimensions prescribed in the model's geometry. Currently, the program supports inches, millimeters, centimeters, meters, mils, and μmeters.

- `Problem Type`. This drop box allows the user to choose from a 2-D planar problem (the `Planar` selection), or an axisymmetric problem (the `Axisymmetric` selection).

- `Precision`. The number in this edit box specifies the stopping criteria for the linear solver. The linear algebra problem could be represented by:

$$Mx = b$$

  where M is a square matrix, b is a vector, and x is a vector of unknowns to be determined. The solver precision value determines the maximum allowable value for $||b - Mx||/||b||$. The default value is 10^{-8}.

- `Depth`. If a Planar problem is selected, this edit box becomes enabled. This value is the length of the geometry in the "into the page" direction. This value is used for scaling integral results in the post processor (e.g. force, inductance, etc.) to the appropriate length. The units of the Depth selection are the same as the selected length units.

- Min Angle (deg). The entry in this box is used as a constraint sent to the mesh generator, tangle. The mesh generator adds points to the mesh to ensure that no angles smaller than the specified angle occur. If the minimum angle is 20.7 degrees or smaller, the triangulation algorithm is theoretically guaranteed to terminate (assuming infinite precision arithmetic – the mesh generator may fail to terminate if you run out of precision). In practice, the algorithm often succeeds for minimum angles up to 33.8 degrees. For highly refined meshes, however, it may be necessary to reduce the minimum angle to well below 20 to avoid problems associated with insufficient floating-point precision. The edit box will accept values between 1 and 33.8 degrees.

- Smart Mesh. When On, the mesh generator derives a characteristic element size from the average length of the model's line segments and applies it at the ends of those segments, refining the mesh toward corners. Endpoints belonging only to arcs are left alone, so a smooth surface is not refined merely for being curved. When Off, element sizes come only from the mesh size specified on each block label, together with any per-segment maximum side length.

- Comment. The user can enter in a few lines of text that give a brief description of the problem that is being solved. This is useful if the user is running several small variations on a given geometry. The comment can then be used to identify the relevant features for a particular geometry.

2.4.2 Definition of Properties

To make a solvable problem definition, the user must identify boundary conditions, block materials properties, and so on. The different types of properties defined for a given problem are defined via the Properties selection off of the main menu.

When the Properties selection is chosen, a drop menu appears that has selections for Materials, Boundary, Point, and Conductors. When any one of these selections is chosen, the dialog pictured in Figure 2.26 appears.

This dialog is the manager for a particular type of property, and its title changes to match the type chosen from the menu. All currently defined properties of that type are listed in the box on the left. At the beginning of a new model definition, the list will be empty, since no properties have yet been defined. Pushing the Add button allows the user to define a new property. The Delete button removes the property currently selected in the list, and Modify opens the selected property for viewing and editing — as does double-clicking the property itself. Close dismisses the dialog. Specifics for defining the various property types are addressed in the following subsections.

Point Properties

If a new point property is added or an existing point property modified, the Nodal Property dialog box appears. This dialog box is pictured in Figure 2.27.

The first selection is the Name edit box. The default name is "New Point Property," but this name should be changed to something that describes the property that you are defining.

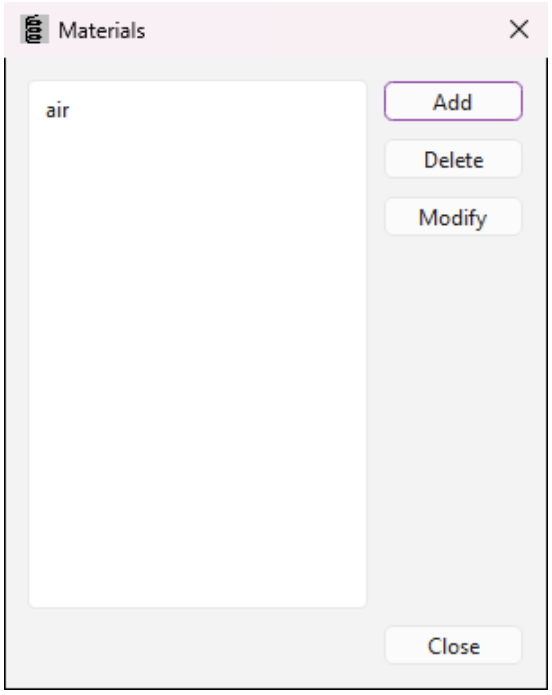


Figure 2.26: Materials dialog.

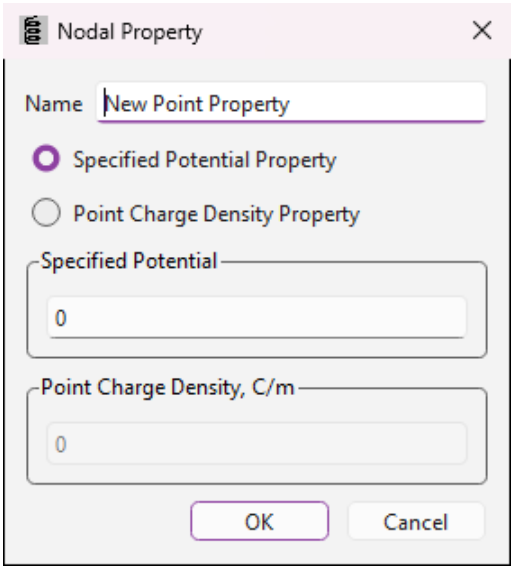


Figure 2.27: Nodal Property dialog.

Next are edit boxes for defining the voltage at a given point, or prescribing a point charge density at a given point. The type of point property is chosen via the radio buttons, and the value is entered in the enabled edit box.

Boundary Properties

The `Boundary Properties` dialog box is used to specify the properties of line segments or arc segments that are to be boundaries of the solution domain. When a new boundary property is added or an existing property modified, the `Boundary Property` dialog pictured in Figure 2.28 appears.

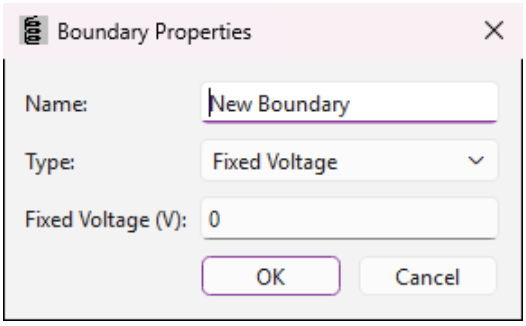


Figure 2.28: Boundary Properties dialog.

The first selection in the dialog is the `Name` of the property. The default name is "New Boundary," but you should change this name to something more descriptive of the boundary that is being defined.

The next selection is the `Type` drop list. This specifies the boundary condition type. The dialog shows only the parameters that belong to the selected type. Currently, FEMM electrostatics problems support the following types of boundaries:

`Fixed Voltage` With this type of boundary condition, potential V is set to a prescribed along a given boundary

`Mixed` This denotes a boundary condition of the form:

$$\varepsilon_r \varepsilon_o \frac{\partial V}{\partial n} + c_o V + c_1 = 0 \tag{2.17}$$

The parameters for this class of boundary condition are specified in the `Mixed BC` boxes in the dialog. By the choice of coefficients, this boundary condition can either be a Robin or a Neumann boundary condition. By carefully selecting the c_0 coefficient and specifying $c_1 = 0$, this boundary condition can be applied to the outer boundary of your geometry to approximate an unbounded solution region. For more information on open boundary problems, refer to the Appendix.

`Surface Charge Density` This selection is used to apply distributions of line charge to segments or arc segments in the problem. Unlike all of the other boundary conditions, this BC type is often used on internal boundaries between materials or on isolated segments. Typically, other BCs are only used on exterior boundaries.

`Periodic` This type of boundary condition is applied to either two segments or two arcs to force the potential to be identical along each boundary. This sort of boundary is useful in exploiting the symmetry inherent in some problems to reduce the size of the domain which must be modeled. The

domain merely needs to be periodic, as opposed to obeying more restrictive $V = 0$ or $\partial V/\partial n = 0$ line of symmetry conditions. Another useful application of periodic boundary conditions is for the modeling of "open boundary" problems, as discussed in Appendix A.3.3. Often, a periodic boundary is made up of several different line or arc segments. A different periodic condition must be defined for each section of the boundary, since each periodic BC can only be applied to a line or arc and a corresponding line or arc on the remote periodic boundary.

`Antiperiodic` The antiperiodic boundary condition is applied in a similar way as the periodic boundary condition, but its effect is to force two boundaries to be the negative of one another. This type of boundary is also typically used to reduce the domain which must be modeled, e.g. so that an electric machine might be modeled for the purposes of a finite element analysis with just one pole.

Materials Properties

The `Material Properties` dialog box is used to specify the properties to be associated with block labels. The properties specified in this dialog have to do with the material of which the block is composed. When a new material property is added or an existing property modified, the `Material Properties` dialog pictured in Figure 2.29 appears.

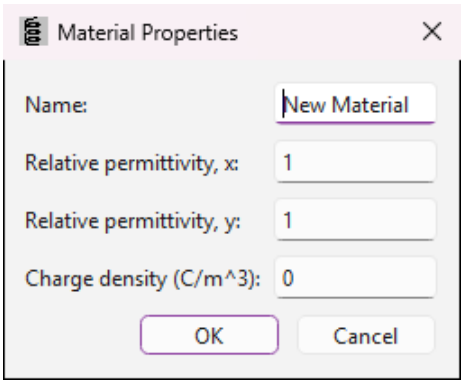


Figure 2.29: Material Properties dialog.

As with Point and Boundary properties, the first step is to choose a descriptive name for the material that is being described. Enter it in the `Name` edit box in lieu of "New Material."

Next, permittivity for the material needs to be specified. FEMM allows you to specify different relative permittivities in the vertical and horizontal directions: ε_x, in the `Relative permittivity, x` box, applies in the x- or horizontal direction, and ε_y, in the `Relative permittivity, y` box, in the y- or vertical direction.

A volume charge density (ρ) can also be prescribed, in C/m^3, in the `Charge density` box.

Material Library

Since one kind of material might be needed in several different models, FEMM has a built-in library of electrostatic Material Properties definitions. The user can access and maintain this library through the `Properties | Material Library` selection off of the main menu. When this option is selected, the `Material Library` dialog pictured in Figure 2.30 appears.

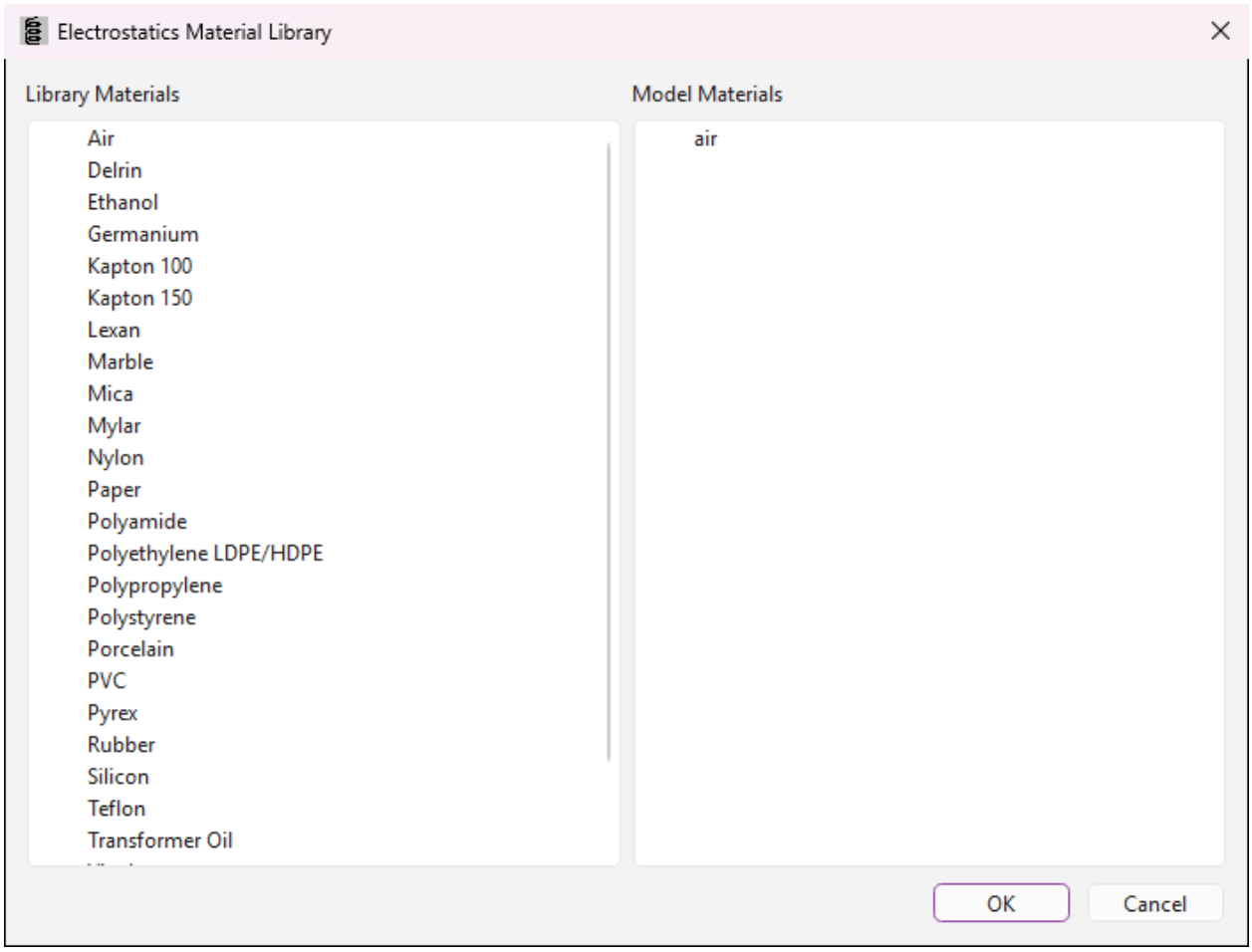


Figure 2.30: Electrostatics Material Library dialog

This dialog allow the user to exchange Block Property definitions between the current model and the materials library via a drag-and-drop interface.

A number of different options are available via a mouse button right-click when the cursor is located on top of a material or folder. Materials can be edited by double-clicking on the desired material.

Material from other material libraries or models can be imported by selecting the "Import Materials" option from the right-button menu that appears when the pointer is over the root-level folder of either the Library or Model materials lists.

The materials library should be located in the same directory as the femm executable files, under the filename `statlib.dat`. If you move the materials library, femm will not be able to find it.

Conductor Properties

The purpose of the conductor properties is mainly to allow the user to apply constraints on the total amount of charge carried on a conductor. Alternatively, conductors with a fixed voltage can be defined, and the program will compute the total charge carried on the conductor during the

solution process.

For fixed voltages, one could alternatively apply a `Fixed Voltage` boundary condition. However, applying a fixed voltage as a conductor allows the user to group together several physically disjoint surfaces into one conductor upon which the total net charge is automatically computed.

The dialog for entering conductor properties is pictured in Figure 2.31.

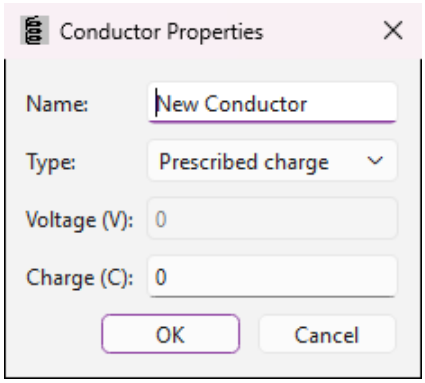


Figure 2.31: Conductor Properties dialog.

The `Name` box names the conductor, so that it can be assigned to segments and arcs and reported on in the postprocessor. The `Type` drop list chooses which of the two quantities is prescribed: `Prescribed charge` or `Prescribed voltage`. Whichever is selected, its edit box is enabled and the other is greyed out — the greyed quantity is the one the program computes for you during the solution.

2.4.3 Analysis Tasks

Meshing the model, analyzing the model, and viewing the results are most easily performed by the toolbar buttons pictured in Figure 2.32.



Figure 2.32: Toolbar buttons for starting analysis tasks.

The first of these buttons (with the "yellow mesh" icon) runs the mesh generator. The solver actually automatically calls the mesh generator to make sure that the mesh is up to date, so you never have to call the mesher from within femm. However, it is almost always important to get a look at the mesh and see that it "looks right." When the mesh generation button is pressed, the mesher is called. After the geometry is triangulated, the finite element mesh is loaded into memory and displayed underneath the defined nodes, segments, and block labels as a set of yellow lines.

If you have a very large model, just keeping all of the mesh information in core can take up a significant amount of memory. If you are about to analyze a very large problem, it might be a good idea to choose the `Mesh | Purge Mesh` option off of the main menu. When this option is selected, the mesh is removed from memory, and the memory that it occupied is freed for other uses.

The second button, with the "hand-crank" icon, executes the solver, `belasolv`. The mesh is brought up to date first: the `tangle` mesher is built into the solver, so meshing is the solver's own

first step rather than a separate program. While belasolv runs, a `Solver` window displays status information to the user. belasolv requires no user interaction while it is running, although the window's `Abort` button will stop it. When belasolv is finished analyzing your problem, the window closes. The time that belasolv requires is highly dependent on the problem being solved. Solution times are typically on the order of 1 to 10 seconds, depending upon the size and complexity of the problem and the speed of the machine analyzing the problem.

The "big magnifying glass" icon is used to run the postprocessor once the analysis is finished.

2.5 Electrostatics Postprocessor

The electrostatics postprocessing functionality of FEMM is used to view solutions generated by the `belasolv` solver. An electrostatics postprocessor window can be opened either by loading some previously run analyses via `File|Open` on the main menu, or by pressing the "big magnifying glass" icon from within a preprocessor window to view a newly generated solution. Electrostatics postprocessor data files stored on disk have the `.res` extension.

Operation of the electrostatics postprocessor (*i.e.* modes, view manipulation) is very similar to that of the magnetics postprocessor. Refer to Sections 2.3.1 through 2.3.5 for this information.

2.5.1 Contour Plot

One of the most useful ways to get a subjective feel for a solution is by plotting the eqipotentials of voltage. By default, a set of 19 equipotential lines are plotted when a solution is initially loaded into the postprocessor. The number and type of equipotential lines to be plotted can be altered using the Contours Plot icon in the Graph Mode section of the toolbar (see Figure 2.33). The Contour Plot icon is the icon with the black contours.



Figure 2.33: Graph Mode toolbar buttons.

When this button is pressed, a dialog pops up, allowing the choice of the number of contours.

In the contour plot dialog, a check box is also present titled "Show stress tensor mask". If this box is checked, the contour lines associated with the last Weighted Stress Tensor integration are also displayed, by default as orange flux lines.

2.5.2 Density Plot

Density plots are also a useful way to get a quick feel for the field in various parts of the model. By default, a density plot of voltage is displayed when the postprocessor first starts; which quantity is plotted, or whether a density plot appears at all, is set under `Edit|Preferences`. The plot can also be changed at any time by pressing the middle button in the Graph Mode section of the toolbar (see Figure 2.33). A dialog then pops up that allows the user to turn density plotting on.

The user can select between density plots of Voltage (V) or the magnitude of Electric Field Intensity (E) or Electric Flux Density (D). The field at each point is classified into one of twenty

contours distributed evenly between either the minimum and maximum values or user-specified bounds.

2.5.3 Vector Plots

A good way of getting a feel for the direction and magnitude of the field is with plots of the field vectors. With this type of plot arrows are plotted such that the direction of the arrow indicates the direction of the field and the size of the arrow indicates the magnitude of the field. The presence and appearance of this type of plot can be controlled by pressing the "arrows" icon pictured in Figure 2.33.

2.5.4 Line Plots

When the postprocessor is in Contours Mode, various field values of interest can be plotted along the defined contour. A plot of a field value defined contour is performed by pressing the "graphed function" icon in the Plot, Integration and Conductor Results group of toolbar buttons, shown in Figure 2.34.



Figure 2.34: Line Plot, Integration, and Conductor Results toolbar buttons.

When this button is pressed, the `X-Y Plot` dialog (see Figure 2.35) appears with a drop list containing the types of line plots available. Choose the desired type of plot and press "OK." The dialog also has an edit box for the number of points to be sampled along the contour.

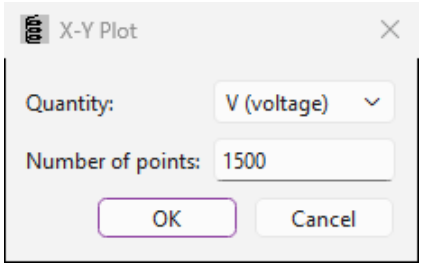


Figure 2.35: X-Y Plot dialog.

After "OK" is pressed, the program computes the desired values along the defined contour. These values are then displayed automatically in a plot window.

To save the plotted values, press the `Export...` button on the plot window. A file selection dialog appears and prompts for the name of a file to which the data is written. The data is written as comma-separated values (`.csv`), with a header row naming the columns: position along the contour, followed by one column for each curve on the plot.

Currently, the type of line plots supported are: Potential along the contour; Magnitude of the flux density along the contour; Component of flux density normal to the contour; Component of flux density tangential to the contour; Magnitude of the field intensity along the contour; Component of field intensity normal to the contour; Component of field intensity tangential to the contour;

In all of these plots, the direction of the normal is understood to be as shown in Figure 2.36. The tangential direction is understood to be the direction in which the contour was defined.

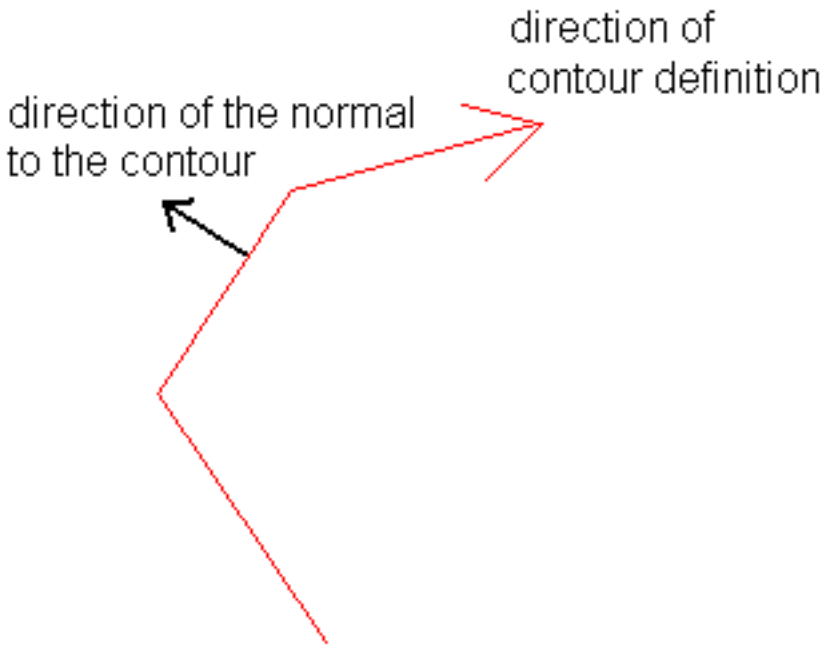


Figure 2.36: When in doubt plots and integrals taken on this side of a contour.

In certain cases, the quantity to be plotted can be ambiguous. This can occur, for example, if a plot of the tangential field intensity is requested on a contour running along an interface between two materials of differing permittivity. In this case, there is a discontinuity in the tangential field intensity, and the value of this quantity is different on each side of the interface. The postprocessor resolves the conflict by always evaluating the plots at a differentially small distance to the "normal" side of the line. Therefore, by defining the same contour but reversing the order in which the points are specified, plots of the quantity of interest on each side of a boundary can be obtained.

2.5.5 Line Integrals

Once a contour has been specified in Contours mode, Line Integrals can be performed along the specified contour. How the integral is evaluated depends on which one is chosen. `E.t` follows from the voltage at the contour's endpoints, and `Contour Length` is a sum of segment lengths, so both are exact. `D.n` and the stress tensor integrals instead sample the field along each segment of the contour and sum by the midpoint rule.

To perform an integration, press the "integral" icon on the toolbar (as shown in Figure 2.34). A small dialog will appear with a drop list. Choose the desired integral from the drop list and press `OK`. The amount of time required to perform the integral will be virtually instantaneous for some types of integrals; however, some types may require several seconds to evaluate. When the evaluation of the integral is completed, the answer appears on the screen in a pop-up box.

The line integrals currently supported are:

- `E.t`. This integral returns the voltage drop along the defined contour.

- `D.n`. This integral returns the total electric flux passing through a volume defined by extruding or sweeping the defined contour. If this integral is performed over a closed contour, the resulting quantity is equal to the charge contained inside the contour.

- `Contour Length`. This integral returns the length of the defined contour in meters, as well as the area of the extruded or swept volume associated with the defined contour.

- `Force from stress tensor`. This integral totals the force produced on the contour derived from Maxwell's stress tensor. Deriving meaningful force results requires some care in the choice of integration path; refer to Section 2.5.7 for a detailed discussion of force and torque calculation.

- `Torque from stress tensor`. This selection integrates the torque about the point (0,0) inferred from Maxwell's stress tensor. Again, some guidelines must be followed to get accurate torque results.

2.5.6 Block Integrals

To select the regions over which a block integral is to be performed, left-click with the mouse in the desired region. The selected region will appear highlighted in green. For some block integrals (*i.e.* weighted stress tensor force and torque), one desires to select a conductor composed of lines and points, rather than a block. In this case, the desired conductor can be selected by clicking on it with the right mouse button. The selected conductor will appear in red.

To perform an integration, press the "integral" icon on the toolbar (as shown in Figure 2.34), and a dialog will appear with a drop list. Choose the desired integral from the drop list and press `OK`. The integral is then performed by analytically integrating the specified kernel over each element in the defined region, and summing the results for all elements. Volume integrals may take several seconds to evaluate, especially on dense meshes. Be patient. When the evaluation of the integral is completed, the answer appears on the screen in a pop-up box.

The block integrals currently supported are:

- `Stored energy`. This selection calculates the energy stored in the electric field in the specified region by integrating $(\frac{1}{2}D \cdot E)$ over the selected area.

- `Block cross-section area`

- `Block volume`

- `Average D over block`

- `Average E over block`

- `Force via Weighted Stress Tensor`. The Weighted Stress Tensor block integrals automatically compute a weighting function over the finite element mesh that allows all possible air elements to contribute to the stress tensor integration. This approach is similar to the weighted stress tensor approach described in [7] and/or [8]. To compute the force on a region or set of regions, the user selects the blocks upon which force result is desired and

selects the `Force via Weighted Stress Tensor` integral. The program then computes the weighting function by solving an additional Laplace equation over the air surrounding the blocks upon which the force is to be computed. It may take a few seconds to compute the weighting function–progress is indicated by a progress bar that is displayed while the weighting function is being computed. The stress tensor is then evaluated as a volume integration, and the results are displayed. The results are typically more accurate than the Maxwell Stress Tensor line integral, since in some sense, all possible contours have been averaged to yield the Weighted Stress Tensor force result.

If the user is interested in the contours along which the integral was performed, the "stress tensor mask" box can be checked in the contour plot dialog. A set of orange (by default) lines is then displayed.

- `Torque via Weighted Stress Tensor`. This integral is the torque version of the `Force via Weighted Stress Tensor` integral. Instead of force, torque about (0,0) is computed using the same weighting function approach.

2.5.7 Force/Torque Calculation

Often, the calculation of electrostatic forces and torques is a goal of finite element analysis. This section discusses some of the different methods of deducing electrostatic forces and torques.

Weighted Stress Tensor Volume Integral

This volume integral greatly simplifies the computation of forces and torques. Merely select the blocks or conductors upon which force or torque are to be computed and evaluate the integral. No particular “art” is required in getting good force or torque results (as opposed to the Stress tensor line integral), although results tend to be more accurate with finer meshing around the region upon which the force or torque is to be computed.

One limitation of the Weighted Stress Tensor integral is that the regions upon which the force is being computed must be entirely surrounded by air and/or abutting a boundary. In cases in which the desired region abuts a non-air region, force results may be deduced from differentiation of stored electric field energy.

Maxwell Stress Tensor Line Integral

Generally, you should not use the Stress Tensor line integral to compute forces and torques if it can be avoided (i.e. use the volume integral version instead). The indiscriminate use of Maxwell’s Stress Tensor can result in bad predictions of forces and torques.

Maxwell’s stress tensor prescribes a force per unit area produced by the electric field on a surface. The differential force produced is:

$$dF = \frac{1}{2}\left(D\left(E\cdot n\right)+E\left(D\cdot n\right)-\left(D\cdot E\right)n\right) \tag{2.18}$$

where n denotes the direction normal to the surface at the point of interest. The net force on an object is obtained by creating a surface totally enclosing the object of interest and integrating the stress over that surface.

For best results, never integrate the stress tensor along an interface between materials. Always define the integration contour as a closed path around the object of interest with the contour displaced several elements (at least two elements) away from any interfaces or boundaries.

Always use as fine a mesh as possible in problems where force results are desired. A good way to proceed in finding a mesh that is "dense enough" is to solve the problem on progressively finer meshes, evaluating the force on each mesh. By comparing the results from different mesh densities, you can get and idea of the level of accuracy (by looking at what digits in the answer that change between various mesh densities). You then pick the smallest mesh density that gives convergence to the desired digit of accuracy.

For torque computations, all the same rules apply as for force computations (i.e. define integration contours away from boundaries and interfaces, and use a dense mesh).

2.5.8 Conductor Results

If conductor properties are used to specify the excitation, a useful byproduct is ready access to the voltage and charge on the conductor. To view the conductor results, either press the "Conductor Props" toolbar button pictured in Figure 2.34 or select `View|Conductor Properties` off of the postprocessor main menu. A dialog, as pictured in Figure 2.37 will appear. There is a drop list on the dialog, from which the user selects the conductor for which results are desired. When a conductor is selected, the voltage and charge associated with that conductor are displayed.

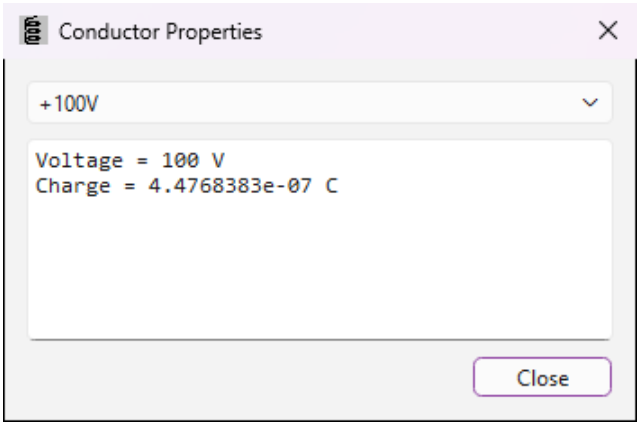


Figure 2.37: Conductor results dialog.

2.6 Heat Flow Preprocessor

The preprocessor is used for drawing the problems geometry, defining materials, and defining boundary conditions. The process of construction of heat flow problems is mechanically nearly identical to the construction of magnetics problems–refer to Sections 2.2.1 through 2.2.5 for an overview of the FEMM editing and problem creation commands. This section considers those parts of problem definition that are unique to heat flow problems.

2.6.1 Problem Definition

The definition of problem type is specified by choosing the `Problem` selection off of the main menu. Selecting this option brings up the Problem Definition dialog, shown in Figure 2.38.

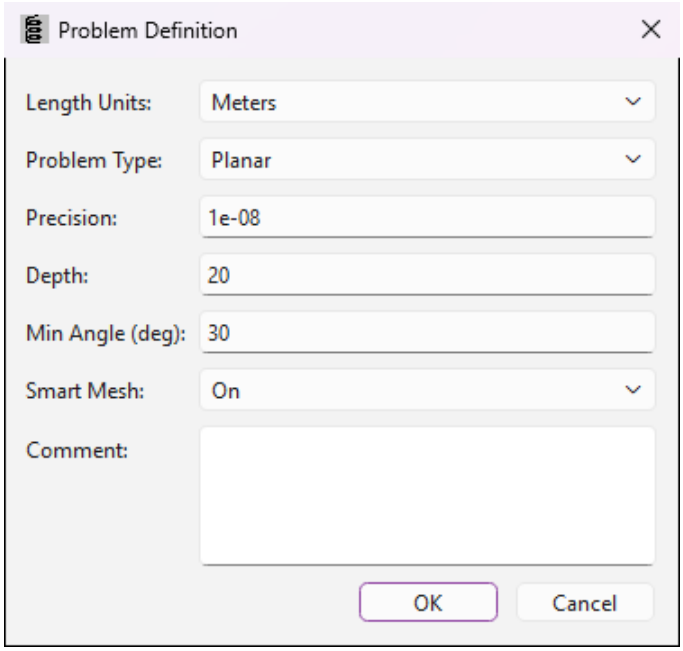


Figure 2.38: Problem Definition dialog.

The dialog's entries, in the order they appear:

- `Length Units`. This box identifies what unit is associated with the dimensions prescribed in the model's geometry. Currently, the program supports inches, millimeters, centimeters, meters, mils, and μmeters.

- `Problem Type`. This drop box allows the user to choose from a 2-D planar problem (the `Planar` selection), or an axisymmetric problem (the `Axisymmetric` selection).

- `Precision`. The number in this edit box specifies the stopping criteria for the linear solver. The linear algebra problem could be represented by:

$$Mx = b$$

  where M is a square matrix, b is a vector, and x is a vector of unknowns to be determined. The solver precision value determines the maximum allowable value for $||b - Mx||/||b||$. The default value is 10^{-8}.

- `Depth`. If a Planar problem is selected, this edit box becomes enabled. This value is the length of the geometry in the "into the page" direction. This value is used for scaling integral results in the post processor (e.g. force, inductance, etc.) to the appropriate length. The units of the Depth selection are the same as the selected length units.

- Min Angle (deg). The entry in this box is used as a constraint sent to the mesh generator, tangle. The mesh generator adds points to the mesh to ensure that no angles smaller than the specified angle occur. If the minimum angle is 20.7 degrees or smaller, the triangulation algorithm is theoretically guaranteed to terminate (assuming infinite precision arithmetic – the mesh generator may fail to terminate if you run out of precision). In practice, the algorithm often succeeds for minimum angles up to 33.8 degrees. For highly refined meshes, however, it may be necessary to reduce the minimum angle to well below 20 to avoid problems associated with insufficient floating-point precision. The edit box will accept values between 1 and 33.8 degrees.

- Smart Mesh. When On, the mesh generator derives a characteristic element size from the average length of the model's line segments and applies it at the ends of those segments, refining the mesh toward corners. Endpoints belonging only to arcs are left alone, so a smooth surface is not refined merely for being curved. When Off, element sizes come only from the mesh size specified on each block label, together with any per-segment maximum side length.

- Comment. The user can enter in a few lines of text that give a brief description of the problem that is being solved. This is useful if the user is running several small variations on a given geometry. The comment can then be used to identify the relevant features for a particular geometry.

2.6.2 Definition of Properties

To make a solvable problem definition, the user must identify boundary conditions, block materials properties, and so on. The different types of properties defined for a given problem are defined via the Properties selection off of the main menu.

When the Properties selection is chosen, a drop menu appears that has selections for Materials, Boundary, Point, and Conductors. When any one of these selections is chosen, the dialog pictured in Figure 2.39 appears.

This dialog is the manager for a particular type of property, and its title changes to match the type chosen from the menu. All currently defined properties of that type are listed in the box on the left. At the beginning of a new model definition, the list will be empty, since no properties have yet been defined. Pushing the Add button allows the user to define a new property. The Delete button removes the property currently selected in the list, and Modify opens the selected property for viewing and editing — as does double-clicking the property itself. Close dismisses the dialog. Specifics for defining the various property types are addressed in the following subsections.

Point Properties

If a new point property is added or an existing point property modified, the Nodal Property dialog box appears. This dialog box is pictured in Figure 2.40.

The first selection is the Name edit box. The default name is New Point Property, but this name should be changed to something that describes the property that you are defining.

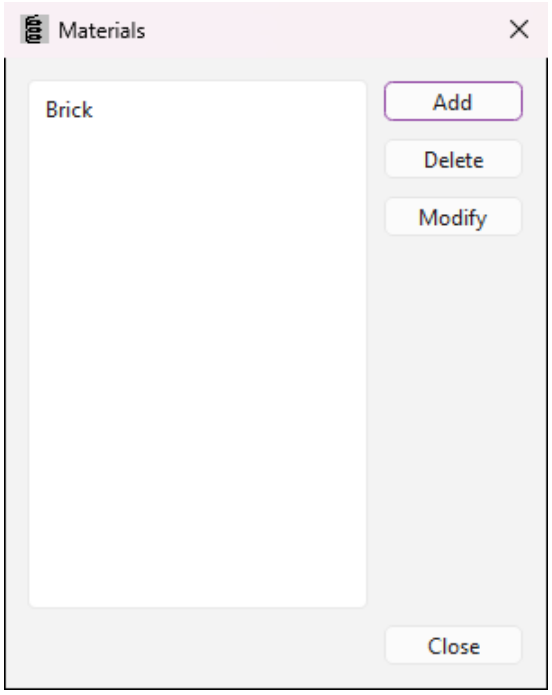


Figure 2.39: Materials dialog.

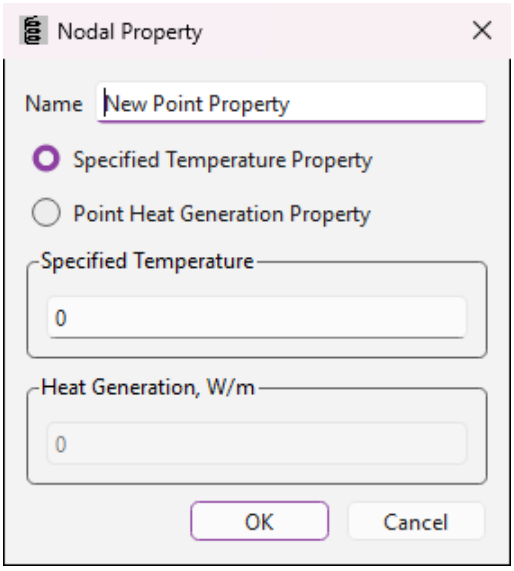


Figure 2.40: Nodal Property dialog.

Next are edit boxes for defining the temperature at a given point, or prescribing a heat generation at a given point. The type of point property is chosen via the radio buttons, and the value is entered in the enabled edit box.

Boundary Properties

The `Boundary Properties` dialog box is used to specify the properties of line segments or arc segments that are to be boundaries of the solution domain. When a new boundary property is added or an existing property modified, the `Boundary Property` dialog pictured in Figure 2.41 appears.

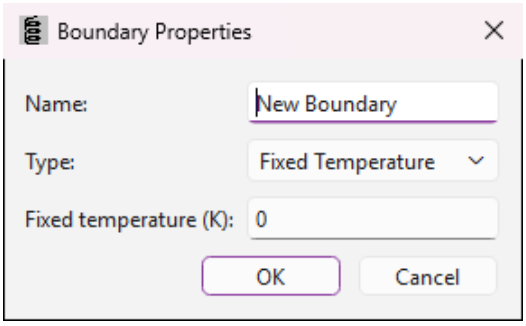


Figure 2.41: Boundary Properties dialog.

The first selection in the dialog is the `Name` of the property. The default name is `New Boundary`, but you should change this name to something more descriptive of the boundary that is being defined.

The next selection is the `Type` drop list. This specifies the boundary condition type, and the dialog shows only the parameters that belong to the selected type. Currently, FEMM heat flow problems support the following types of boundaries: Fixed Temperature, Heat Flux, Convection, Radiation, Periodic, and Anti-Periodic. These boundary conditions are described in detail in Section 1.4.

Materials Properties

The `Material Properties` dialog box is used to specify the properties to be associated with block labels. The properties specified in this dialog have to do with the material of which the block is composed. When a new material property is added or an existing property modified, the `Material Properties` dialog pictured in Figure 2.42 appears.

As with Point and Boundary properties, the first step is to choose a descriptive name for the material that is being described. Enter it in the `Name` edit box in lieu of `New Material`.

Next, the thermal conductivity for the material needs to be specified. The `T-k Curve` drop list chooses between a conductivity that is constant and one that varies with temperature.

With `Constant Thermal Conductivity` selected, FEMM allows you to specify different conductivities in the vertical and horizontal directions: k_x, in the `Thermal conductivity, x` box, applies in the x- or horizontal direction, and k_y, in the `Thermal conductivity, y` box, in the y- or vertical direction. Both are in W/m-K.

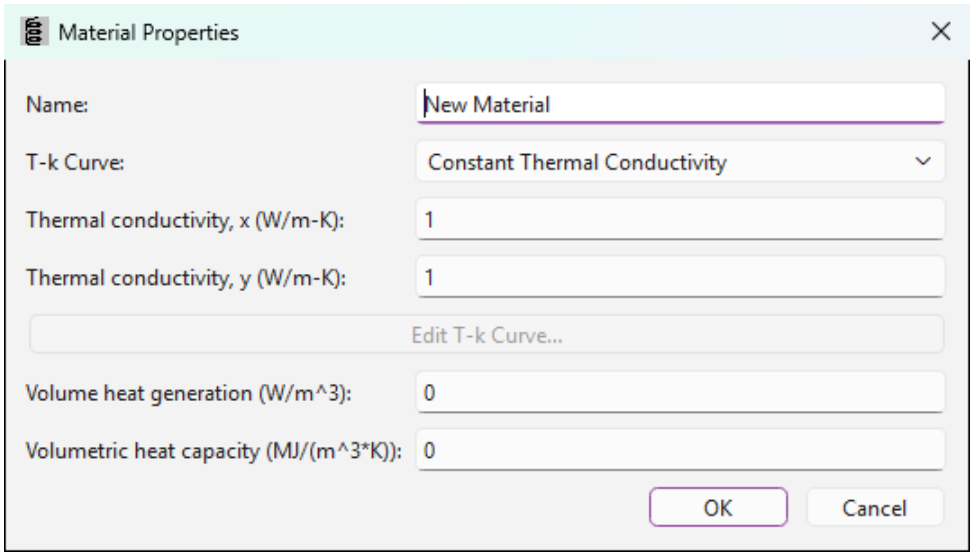


Figure 2.42: Material Properties dialog.

If `Thermal Conductivity Depends on Temperature` is selected instead, those two boxes are disabled and the `Edit T-k Curve...` button becomes enabled. Press the button to enter temperature-conductivity pairs, one per line, in the dialog pictured in Figure 2.43 — the same arrangement in which B-H data is entered for a nonlinear magnetic material. The points need not be entered in order; they are sorted by temperature. The program interpolates linearly between the entered points, and if it must extrapolate off the end of the defined curve, conductivity takes the value of the nearest defined T-k point. A temperature-dependent material is isotropic: the interpolated conductivity applies in both directions.

The `Import from file...` button reads the points from a two-column text file. The program first asks which column is which and what units the file is in; temperatures in Celsius, Fahrenheit or Rankine and conductivities in Btu/(ft*hr*F) or kcal/(m*hr*K) are converted to Kelvin and W/(m*K) as they are read. `Plot T-k Curve` draws the interpolated curve through the entered points, so a bad point is easy to spot.

A volume heat generation can also be prescribed, in W/m^3, in the `Volume heat generation` box.

The `Volumetric heat capacity` box, in MJ/(m^3·K), is used only by transient analyses — that is, by problems that step forward in time from a previous solution, which are set up through the `hi_probdef` scripting command's previous solution and time step arguments. The entry has no effect on a steady-state solution.

Material Library

Since one kind of material might be needed in several different models, FEMM has a built-in library of thermal Material Properties definitions. The user can access and maintain this library through the `Properties | Material Library` selection off of the main menu. When this option is selected, the `Material Library` dialog pictured in Figure 2.44 appears.

This dialog allow the user to exchange Block Property definitions between the current model

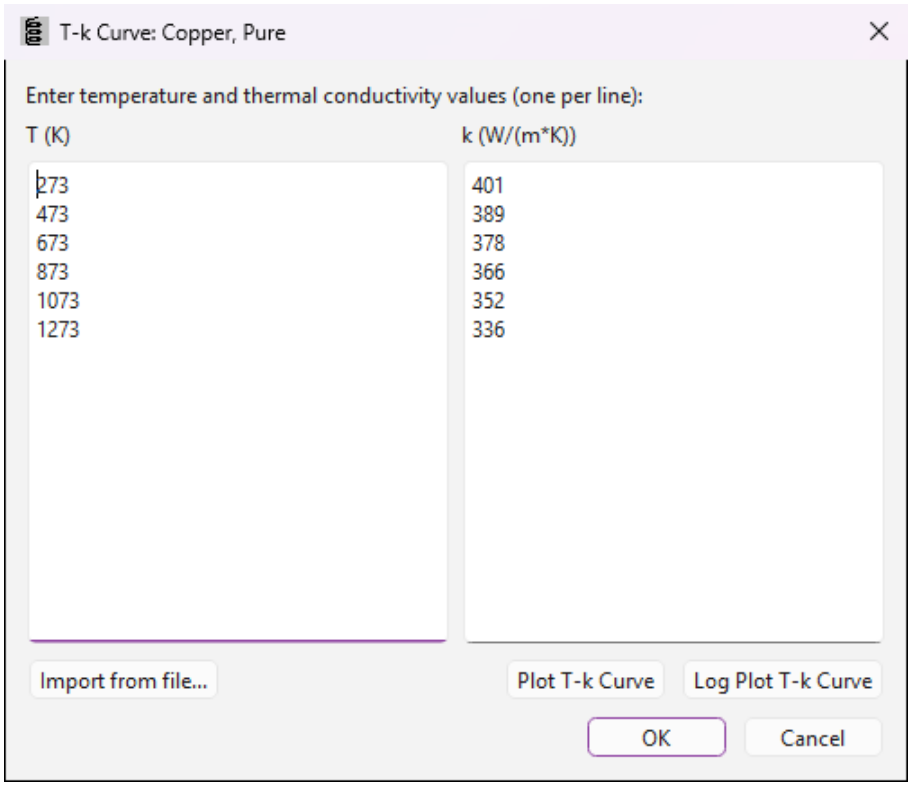


Figure 2.43: Thermal conductivity curve dialog.

and the materials library via a drag-and-drop interface.

A number of different options are available via a mouse button right-click when the cursor is located on top of a material or folder. Materials can be edited by double-clicking on the desired material.

Material from other material libraries or models can be imported by selecting the "Import Materials" option from the right-button menu that appears when the pointer is over the root-level folder of either the Library or Model materials lists.

The materials library should be located in the same directory as the FEMM executable files, under the filename `heatlib.dat`. If you move the materials library, FEMM will not be able to find it.

Conductor Properties

The purpose of the conductor properties is mainly to allow the user to apply constraints on the total amount of heat flowing in and out of a surface. Alternatively, conductors with a fixed temperature can be defined, and the program will compute the total heat flow through the during the solution process.

For fixed temperatures, one could alternatively apply a `Fixed Temperature` boundary condition. However, applying a fixed temperature as a conductor allows the user to group together several physically disjoint surfaces into one conductor upon which the total heat flux is automatically computed.

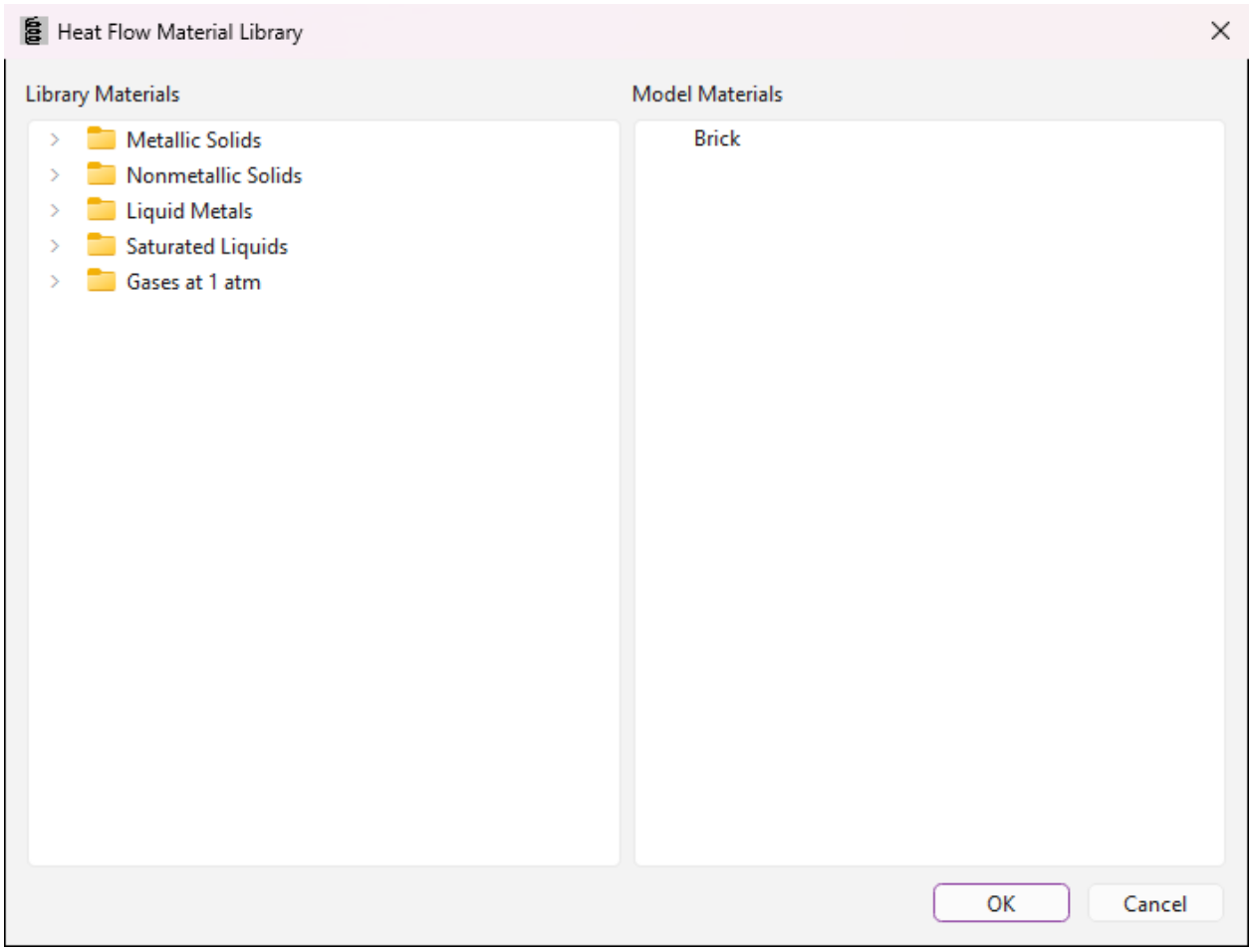


Figure 2.44: Electrostatics Material Library dialog

The dialog for entering conductor properties is pictured in Figure 2.45.

The `Name` box names the conductor, so that it can be assigned to segments and arcs and reported on in the postprocessor. The `Type` drop list chooses which of the two quantities is prescribed: `Prescribed heat flux` or `Prescribed temperature`. Whichever is selected, its edit box is enabled and the other is greyed out — the greyed quantity is the one the program computes for you during the solution.

2.6.3 Analysis Tasks

Meshing the model, analyzing the model, and viewing the results are most easily performed by the toolbar buttons pictured in Figure 2.46.

The first of these buttons (with the "yellow mesh" icon) runs the mesh generator. The solver actually automatically calls the mesh generator to make sure that the mesh is up to date, so you never have to call the mesher from within FEMM. However, it is almost always important to get a look at the mesh and see that it "looks right." When the mesh generation button is pressed, the mesher is called. After the geometry is triangulated, the finite element mesh is loaded into memory and displayed underneath the defined nodes, segments, and block labels as a set of yellow lines.

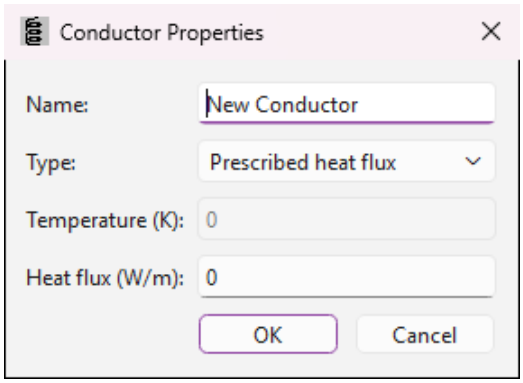


Figure 2.45: Conductor Properties dialog.



Figure 2.46: Toolbar buttons for starting analysis tasks.

If you have a very large model, just keeping all of the mesh information in core can take up a significant amount of memory. If you are about to analyze a very large problem, it might be a good idea to choose the `Mesh | Purge Mesh` option off of the main menu. When this option is selected, the mesh is removed from memory, and the memory that it occupied is freed for other uses.

The second button, with the "hand-crank" icon, executes the solver, `hsolv`. The mesh is brought up to date first: the `tangle` mesher is built into the solver, so meshing is the solver's own first step rather than a separate program. While hsolv runs, a `Solver` window displays status information to the user. hsolv requires no user interaction while it is running, although the window's `Abort` button will stop it. When hsolv is finished analyzing your problem, the window closes. The time that hsolv requires is highly dependent on the problem being solved. Solution times are typically on the order of 1 to 10 seconds, depending upon the size and complexity of the problem and the speed of the machine analyzing the problem.

The "big magnifying glass" icon is used to run the postprocessor once the analysis is finished.

2.7 Heat Flow Postprocessor

The heat flow postprocessing functionality of FEMM is used to view solutions generated by the `hsolv` solver. A heat flow postprocessor window can be opened either by loading some previously run analyses via `File|Open` on the main menu, or by pressing the "big magnifying glass" icon from within a preprocessor window to view a newly generated solution. Heat flow postprocessor data files stored on disk have the `.anh` extension.

Operation of the heat flow postprocessor (*i.e.* modes, view manipulation) is very similar to that of the magnetics postprocessor. Refer to Sections 2.3.1 through 2.3.5 for this information.

2.7.1 Contour Plot

One of the most useful ways to get a subjective feel for a solution is by plotting the eqipotentials of temperature. The number and type of equipotential lines to be plotted can be altered using the Contours Plot icon in the Graph Mode section of the toolbar (see Figure 2.47). The Contour Plot icon is the icon with the black contours.



Figure 2.47: Graph Mode toolbar buttons.

When this button is pressed, a dialog pops up, allowing the choice of the number of contours.

2.7.2 Density Plot

Density plots are also a useful way to get a quick feel for the temperature, flux density, etc., in various parts of the model. By default, a density plot denoting temperature is displayed when the postprocessor first starts. (This behavior can be changed via Edit|Preferences on the main menu). However, the plot can be displayed by pressing the "spectrum" button in the Graph Mode section of the toolbar (see Figure 2.47). A dialog the pops up that allows the user to turn density plotting on.

The user can select between density plots of temperature or the magnitude of temperature gradient or heat flux density. The solution at each point is classified into one of twenty contours distributed evenly between either the minimum and maximum flux densities or user-specified bounds.

2.7.3 Vector Plots

A good way of getting a feel for the direction and magnitude of the field is with plots of the field vectors. With this type of plot arrows are plotted such that the direction of the arrow indicates the direction of the field and the size of the arrow indicates the magnitude of the field. The presence and appearance of this type of plot can be controlled by pressing the "arrows" icon pictured in Figure 2.47.

2.7.4 Line Plots

When the postprocessor is in Contours Mode, various field values of interest can be plotted along the defined contour. A plot of a field value defined contour is performed by pressing the "graphed function" icon in the Plot, Integration and Conductor Results group of toolbar buttons, shown in Figure 2.48.



Figure 2.48: Line Plot, Integration, and Conductor Results toolbar buttons.

When this button is pressed, the `X-Y Plot` dialog (see Figure 2.49) appears with a drop list containing the types of line plots available. Choose the desired type of plot and press "OK." The dialog also has an edit box for the number of points to be sampled along the contour.

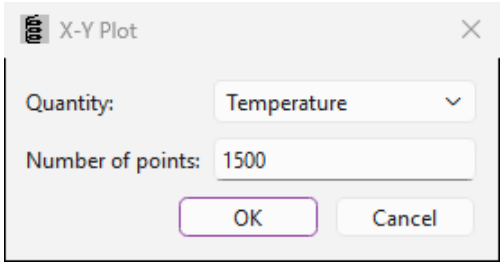


Figure 2.49: X-Y Plot dialog.

After "OK" is pressed, the program computes the desired values along the defined contour. When the computation is finished, a window will appear with a plot of the selected quantity.

To save the plotted values, press the `Export...` button on the plot window. A file selection dialog appears and prompts for the name of a file to which the data is written. The data is written as comma-separated values (`.csv`), with a header row naming the columns: position along the contour, followed by one column for each curve on the plot.

Currently, the type of line plots supported are: Temperature along the contour; Magnitude of the heat flux density along the contour; Component of heat flux density normal to the contour; Component of heat flux density tangential to the contour; Magnitude of the temperature gradient along the contour; Component of temperature gradient normal to the contour; Component of temperature gradient tangential to the contour;

In all of these plots, the direction of the normal is understood to be as shown in Figure 2.50. The tangential direction is understood to be the direction in which the contour was defined.

In certain cases, the quantity to be plotted can be ambiguous. This can occur, for example, if a plot of the tangential field intensity is requested on a contour running along an interface between two materials of differing permittivity. In this case, there is a discontinuity in the tangential field intensity, and the value of this quantity is different on each side of the interface. The postprocessor resolves the conflict by always evaluating the plots at a differentially small distance to the "normal" side of the line. Therefore, by defining the same contour but reversing the order in which the points are specified, plots of the quantity of interest on each side of a boundary can be obtained.

2.7.5 Line Integrals

Once a contour has been specified in Contours mode, Line Integrals can be performed along the specified contour. These integrals are performed by evaluating a large number of points at evenly spaced along the contour and integrating using a simple trapezoidal-type integration scheme.

To perform an integration, press the "integral" icon on the toolbar (as shown in Figure 2.48). A small dialog will appear with a drop list. Choose the desired integral from the drop list and press `OK`. The amount of time required to perform the integral will be virtually instantaneous for some types of integrals; however, some types may require several seconds to evaluate. When the evaluation of the integral is completed, the answer appears on the screen in a pop-up box.

The line integrals currently supported are:

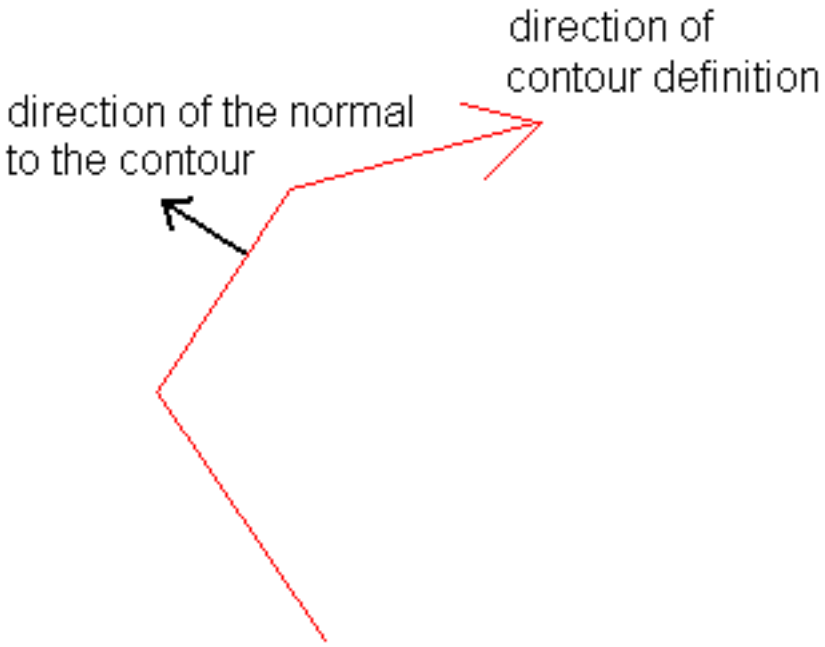


Figure 2.50: When in doubt plots and integrals taken on this side of a contour.

- `Temperature Difference (G.t)`. This integral returns the temperature difference between the ends of the contour
- `Heat Flux (F.n)`. This integral returns the total heat flux passing through a volume defined by extruding or sweeping the defined contour.
- `Contour length & area`. The length of the contour, and the area formed by extruding the contour.
- `Average temperature`. The average temperature along the line.

2.7.6 Block Integrals

To select the regions over which a block integral is to be performed, left-click with the mouse in the desired region. The selected region will appear highlighted in green.

To perform an integration, press the "integral" icon on the toolbar (as shown in Figure 2.48), and a dialog will appear with a drop list. Choose the desired integral from the drop list and press `OK`. The integral is then performed by analytically integrating the specified kernel over each element in the defined region, and summing the results for all elements. Volume integrals may take several seconds to evaluate, especially on dense meshes. Be patient. When the evaluation of the integral is completed, the answer appears on the screen in a pop-up box.

The block integrals currently supported are:

- `Average temperature over volume`
- `Block cross-section area`

- `Block volume`
- `Average F over volume`
- `Average G over volume`

2.7.7 Conductor Results

If conductor properties are used to specify the excitation, a useful byproduct is ready access to the temperature of and heat flux through the conductor. To view the conductor results, either press the "Conductor Props" toolbar button pictured in Figure 2.48 or select `View|Conductor Properties` off of the postprocessor main menu. A dialog, as pictured in Figure 2.51 will appear. There is a drop list on the dialog, from which the user selects the conductor for which results are desired. When a conductor is selected, the temperature and heat flux associated with that conductor are displayed.

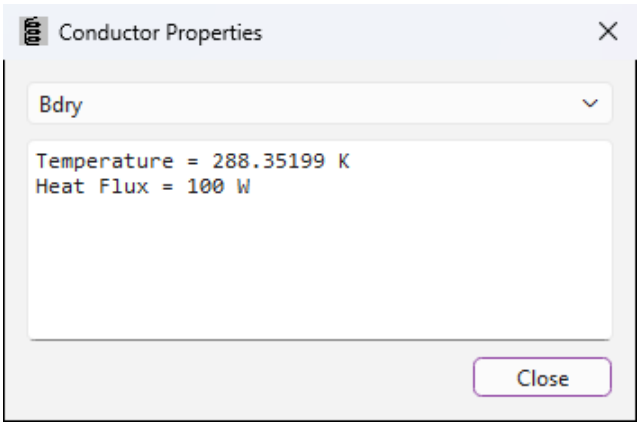


Figure 2.51: Conductor results dialog.

2.8 Current Flow Preprocessor

The preprocessor is used for drawing the problems geometry, defining materials, and defining boundary conditions. The process of construction of current flow problems is mechanically nearly identical to the construction of magnetics problems–refer to Sections 2.2.1 through 2.2.5 for an overview of the FEMM editing and problem creation commands. This section considers those parts of problem definition that are unique to current flow problems.

2.8.1 Problem Definition

The definition of problem type is specified by choosing the `Problem` selection off of the main menu. Selecting this option brings up the Problem Definition dialog, shown in Figure 2.52.

The dialog's entries, in the order they appear:

- `Frequency (Hz)` denotes the frequency at which the problem is to be analyzed.

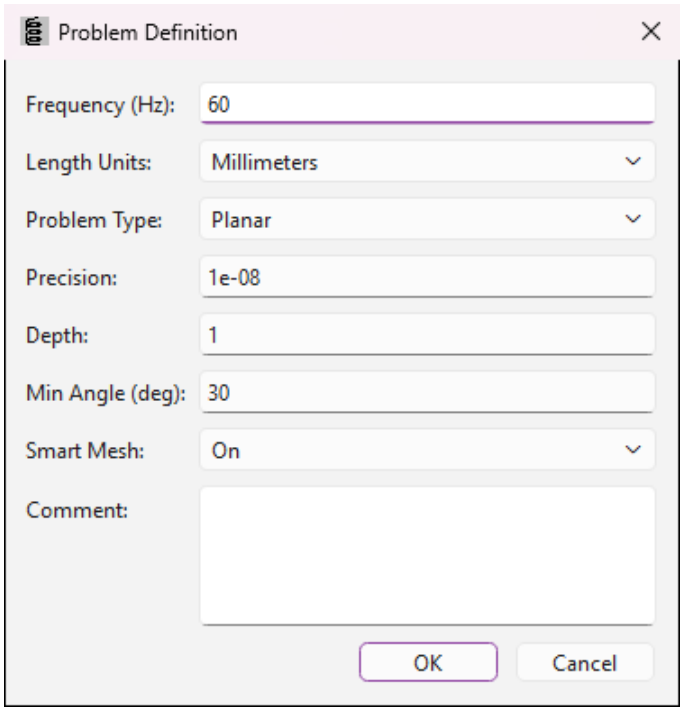


Figure 2.52: Problem Definition dialog.

- `Length Units`. This box identifies what unit is associated with the dimensions prescribed in the model's geometry. Currently, the program supports inches, millimeters, centimeters, meters, mils, and μmeters.

- `Problem Type`. This drop box allows the user to choose from a 2-D planar problem (the `Planar` selection), or an axisymmetric problem (the `Axisymmetric` selection).

- `Precision`. The number in this edit box specifies the stopping criteria for the linear solver. The linear algebra problem could be represented by:

$$Mx = b$$

  where M is a square matrix, b is a vector, and x is a vector of unknowns to be determined. The solver precision value determines the maximum allowable value for $||b - Mx||/||b||$. The default value is 10^{-8}.

- `Depth`. If a Planar problem is selected, this edit box becomes enabled. This value is the length of the geometry in the "into the page" direction. This value is used for scaling integral results in the post processor (e.g. force, inductance, etc.) to the appropriate length. The units of the Depth selection are the same as the selected length units.

- `Min Angle (deg)`. The entry in this box is used as a constraint sent to the mesh generator, `tangle`. The mesh generator adds points to the mesh to ensure that no angles smaller than the specified angle occur. If the minimum angle is 20.7 degrees or smaller, the triangulation

algorithm is theoretically guaranteed to terminate (assuming infinite precision arithmetic – the mesh generator may fail to terminate if you run out of precision). In practice, the algorithm often succeeds for minimum angles up to 33.8 degrees. For highly refined meshes, however, it may be necessary to reduce the minimum angle to well below 20 to avoid problems associated with insufficient floating-point precision. The edit box will accept values between 1 and 33.8 degrees.

- `Smart Mesh`. When `On`, the mesh generator derives a characteristic element size from the average length of the model's line segments and applies it at the ends of those segments, refining the mesh toward corners. Endpoints belonging only to arcs are left alone, so a smooth surface is not refined merely for being curved. When `Off`, element sizes come only from the mesh size specified on each block label, together with any per-segment maximum side length.

- `Comment`. The user can enter in a few lines of text that give a brief description of the problem that is being solved. This is useful if the user is running several small variations on a given geometry. The comment can then be used to identify the relevant features for a particular geometry.

2.8.2 Definition of Properties

To make a solvable problem definition, the user must identify boundary conditions, block materials properties, and so on. The different types of properties defined for a given problem are defined via the `Properties` selection off of the main menu.

When the `Properties` selection is chosen, a drop menu appears that has selections for Materials, Boundary, Point, and Conductors. When any one of these selections is chosen, the dialog pictured in Figure 2.53 appears.

This dialog is the manager for a particular type of property, and its title changes to match the type chosen from the menu. All currently defined properties of that type are listed in the box on the left. At the beginning of a new model definition, the list will be empty, since no properties have yet been defined. Pushing the `Add` button allows the user to define a new property. The `Delete` button removes the property currently selected in the list, and `Modify` opens the selected property for viewing and editing — as does double-clicking the property itself. `Close` dismisses the dialog. Specifics for defining the various property types are addressed in the following subsections.

Point Properties

If a new point property is added or an existing point property modified, the `Nodal Property` dialog box appears. This dialog box is pictured in Figure 2.54.

The first selection is the `Name` edit box. The default name is `New Point Property`, but this name should be changed to something that describes the property that you are defining.

Next are edit boxes for defining the voltage at a given point, or prescribing a current generation at a given point. The type of point property is chosen via the radio buttons, and the value is entered in the enabled edit box.

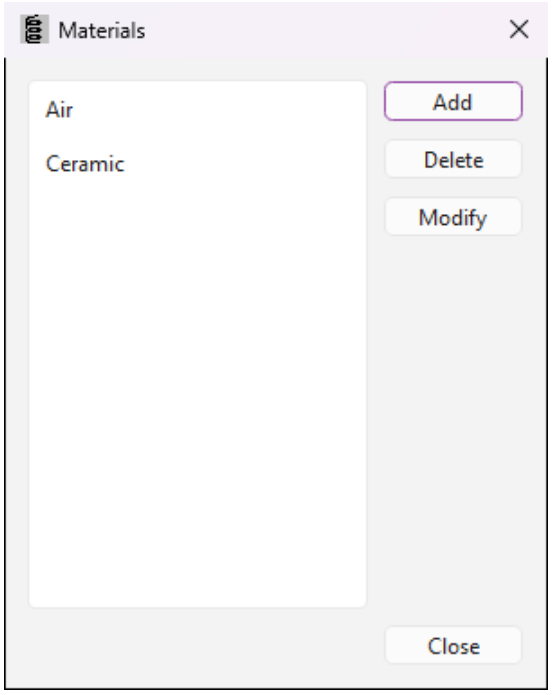


Figure 2.53: Materials dialog.

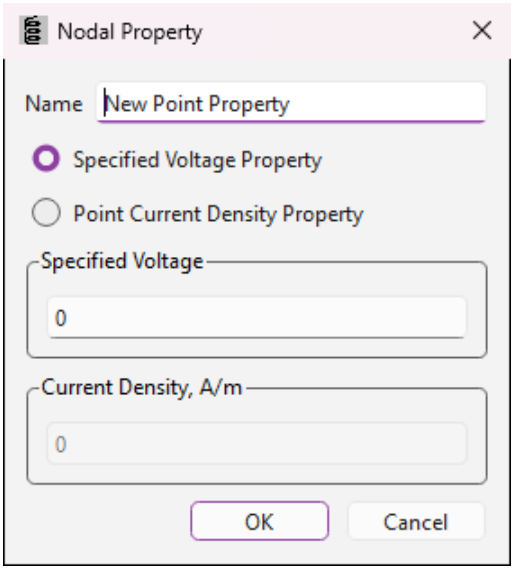


Figure 2.54: Nodal Property dialog.

Boundary Properties

The `Boundary Properties` dialog box is used to specify the properties of line segments or arc segments that are to be boundaries of the solution domain. When a new boundary property is added or an existing property modified, the `Boundary Property` dialog pictured in Figure 2.55 appears.

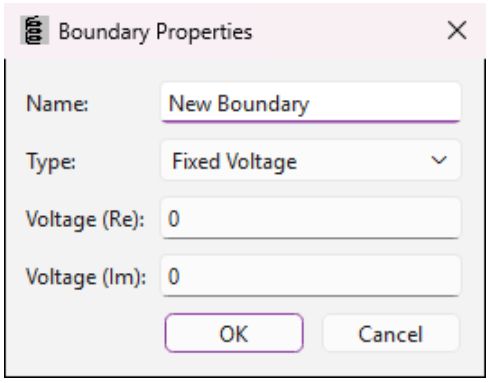


Figure 2.55: Boundary Properties dialog.

The first selection in the dialog is the `Name` of the property. The default name is `New Boundary`, but you should change this name to something more descriptive of the boundary that is being defined.

The next selection is the `Type` drop list. This specifies the boundary condition type, and the dialog shows only the parameters that belong to the selected type. Currently, FEMM supports the following types of boundaries: Fixed Voltage, Mixed BC, Surface Current Density, Periodic, and Anti-Periodic. These boundary conditions are described in detail in Section 1.4.

Materials Properties

The `Material Properties` dialog box is used to specify the properties to be associated with block labels. The properties specified in this dialog have to do with the material of which the block is composed. When a new material property is added or an existing property modified, the `Material Properties` dialog pictured in Figure 2.56 appears.

As with Point and Boundary properties, the first step is to choose a descriptive name for the material that is being described. Enter it in the `Name` edit box in lieu of `New Material`.

Next, electrical conductivity for the material needs to be specified. FEMM allows you to specify different electrical conductivities in the vertical and horizontal directions: σ_x, in the `Conductivity, σx` box, applies in the x- or horizontal direction, and σ_y, in the `Conductivity, σy` box, in the y- or vertical direction. Both are in S/m.

The next pair of boxes represents the relative electrical permittivity for the material. Similar to the electrical conductivity, ε_x represents permittivity in the x- or horizontal direction, and ε_y for the y- or vertical direction. If the material is a lossy dielectric, this value is considered to be the amplitude of complex permittivity.

A common way of describing lossy dielectrics is via the "loss tangent". Losses can be considered as resulting from a complex-valued electrical permittivity. If the complex-valued permittivity

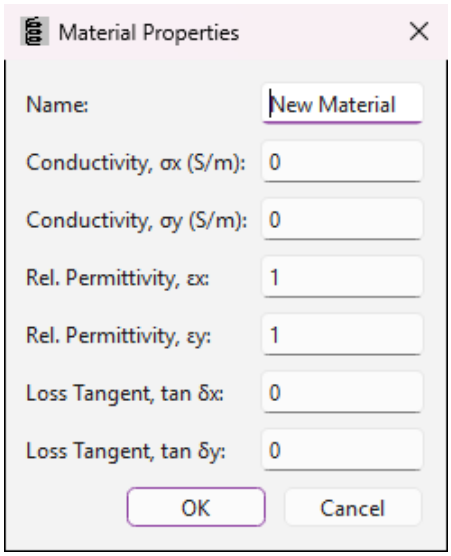


Figure 2.56: Material Properties dialog.

is defined as:

$$\varepsilon = |\varepsilon| \left(\cos\phi - j\sin\phi\right) \tag{2.19}$$

The loss tangent is then defined as:

$$\text{loss tangent} = \frac{\sin\phi}{\cos\phi} \tag{2.20}$$

The loss tangent is entered separately for each direction, in the `Loss Tangent, tan` δx and `Loss Tangent, tan` δy boxes.

For material that are also conductive, FEMM combines the defined conductivity, permittivity, and loss tangent to obtain the complex-valued effective electrical conductivities:

$$\begin{aligned} \sigma_{x,eff} &= \sigma_x + j\omega\varepsilon_o\varepsilon_x e^{-j\phi} \\ \sigma_{y,eff} &= \sigma_y + j\omega\varepsilon_o\varepsilon_y e^{-j\phi} \end{aligned} \tag{2.21}$$

which takes into account resistive losses and addition dielectric losses due to the definition of a non-zero loss tangent.

Material Library

Since one kind of material might be needed in several different models, FEMM has a built-in library of current flow Material Properties definitions. The user can access and maintain this library through the `Properties | Material Library` selection off of the main menu. When this option is selected, the `Current Flow Material Library` dialog pictured in Figure 2.57 appears.

This dialog allows the user to exchange material definitions between the current model and the materials library via a drag-and-drop interface.

A number of different options are available via a mouse button right-click when the cursor is located on top of a material or folder. Materials can be edited by double-clicking on the desired material.

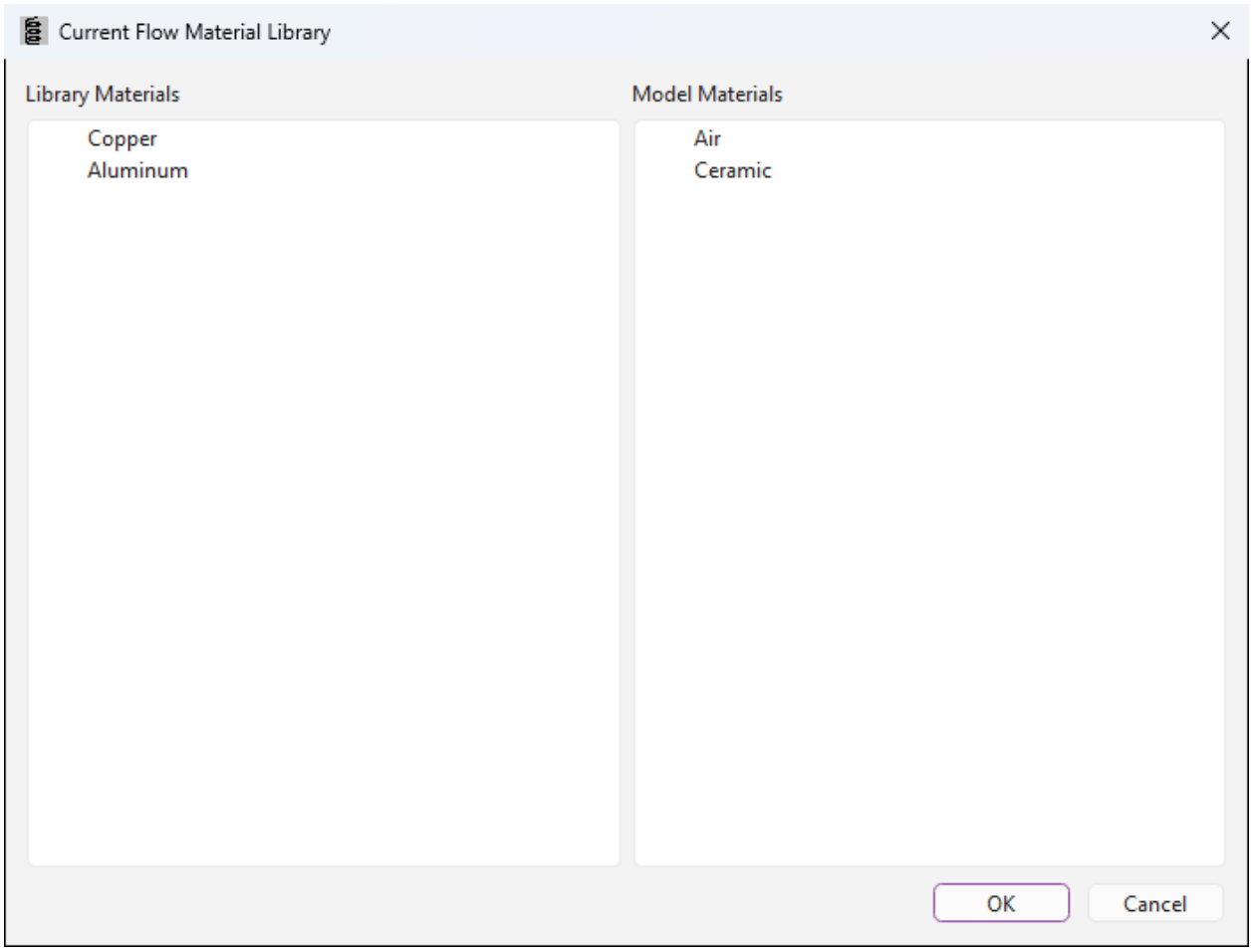


Figure 2.57: Current Flow Material Library dialog

Material from other material libraries or models can be imported by selecting the "Import Materials" option from the right-button menu that appears when the pointer is over the root-level folder of either the Library or Model materials lists.

The materials library should be located in the same directory as the FEMM executable files, under the filename `condlib.dat`. If you move the materials library, FEMM will not be able to find it.

Conductor Properties

The purpose of the conductor properties is mainly to allow the user to apply constraints on the total amount of current flowing in and out of a surface. Alternatively, conductors with a fixed voltage can be defined, and the program will compute the total current flow through the during the solution process.

For fixed voltages, one could alternatively apply a `Fixed Voltage` boundary condition. However, applying a fixed voltage as a conductor allows the user to group together several physically disjoint surfaces into one conductor upon which the total current flow is automatically computed.

The dialog for entering conductor properties is pictured in Figure 2.58.

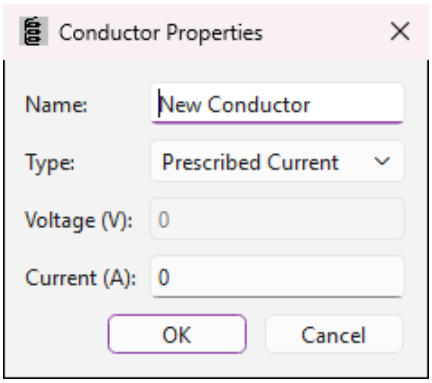


Figure 2.58: Conductor Properties dialog.

The `Name` box names the conductor, so that it can be assigned to segments and arcs and reported on in the postprocessor. The `Type` drop list chooses which of the two quantities is prescribed: `Prescribed Current` or `Prescribed Voltage`. Whichever is selected, its edit box is enabled and the other is greyed out — the greyed quantity is the one the program computes for you during the solution.

2.8.3 Analysis Tasks

Meshing the model, analyzing the model, and viewing the results are most easily performed by the toolbar buttons pictured in Figure 2.59.

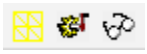


Figure 2.59: Toolbar buttons for starting analysis tasks.

The first of these buttons (with the "yellow mesh" icon) runs the mesh generator. The solver actually automatically calls the mesh generator to make sure that the mesh is up to date, so you never have to call the mesher from within FEMM. However, it is almost always important to get a look at the mesh and see that it "looks right." When the mesh generation button is pressed, the mesher is called. After the geometry is triangulated, the finite element mesh is loaded into memory and displayed underneath the defined nodes, segments, and block labels as a set of yellow lines.

If you have a very large model, just keeping all of the mesh information in core can take up a significant amount of memory. If you are about to analyze a very large problem, it might be a good idea to choose the `Mesh | Purge Mesh` option off of the main menu. When this option is selected, the mesh is removed from memory, and the memory that it occupied is freed for other uses.

The second button, with the "hand-crank" icon, executes the solver, `csolv`. The mesh is brought up to date first: the `tangle` mesher is built into the solver, so meshing is the solver's own first step rather than a separate program. While csolv runs, a `Solver` window displays status information to the user. csolv requires no user interaction while it is running, although the window's `Abort` button will stop it. When csolv is finished analyzing your problem, the window closes. The time that csolv requires is highly dependent on the problem being solved. Solution

times are typically on the order of 1 to 10 seconds, depending upon the size and complexity of the problem and the speed of the machine analyzing the problem.

The "big magnifying glass" icon is used to run the postprocessor once the analysis is finished.

2.9 Current Flow Postprocessor

The current flow postprocessing functionality of FEMM is used to view solutions generated by the `csolv` solver. A current flow postprocessor window can be opened either by loading some previously run analyses via `File|Open` on the main menu, or by pressing the "big magnifying glass" icon from within a preprocessor window to view a newly generated solution. Current flow postprocessor data files stored on disk have the `.anc` extension.

Operation of the current flow postprocessor (*i.e.* modes, view manipulation) is very similar to that of the magnetics postprocessor. Refer to Sections 2.3.1 through 2.3.5 for this information.

2.9.1 Contour Plot

One of the most useful ways to get a subjective feel for a solution is by plotting the eqipotentials of voltage. The number and type of equipotential lines to be plotted can be altered using the Contours Plot icon in the Graph Mode section of the toolbar (see Figure 2.60). The Contour Plot icon is the icon with the black contours.



Figure 2.60: Graph Mode toolbar buttons.

When this button is pressed, a dialog pops up, allowing the choice of the number of contours.

2.9.2 Density Plot

Density plots are also a useful way to get a quick feel for the voltage, current density, etc., in various parts of the model. By default, a density plot denoting voltage is displayed when the postprocessor first starts. (This behavior can be changed via Edit|Preferences on the main menu). However, the plot can be displayed by pressing the "spectrum" button in the Graph Mode section of the toolbar (see Figure 2.60). A dialog the pops up that allows the user to turn density plotting on.

The user can select between density plots of voltage or the magnitude of voltage gradient or current density. The solution at each point is classified into one of twenty contours distributed evenly between either the minimum and maximum densities or user-specified bounds.

2.9.3 Vector Plots

A good way of getting a feel for the direction and magnitude of the field is with plots of the field vectors. With this type of plot arrows are plotted such that the direction of the arrow indicates the direction of the field and the size of the arrow indicates the magnitude of the field. The presence and appearance of this type of plot can be controlled by pressing the "arrows" icon pictured in Figure 2.60.

2.9.4 Line Plots

When the postprocessor is in Contours Mode, various field values of interest can be plotted along the defined contour. A plot of a field value defined contour is performed by pressing the "graphed function" icon in the Plot, Integration and Conductor Results group of toolbar buttons, shown in Figure 2.61.



Figure 2.61: Line Plot, Integration, and Conductor Results toolbar buttons.

When this button is pressed, the `X-Y Plot` dialog (see Figure 2.62) appears with a drop list containing the types of line plots available. Choose the desired type of plot and press "OK." The dialog also has an edit box for the number of points to be sampled along the contour.

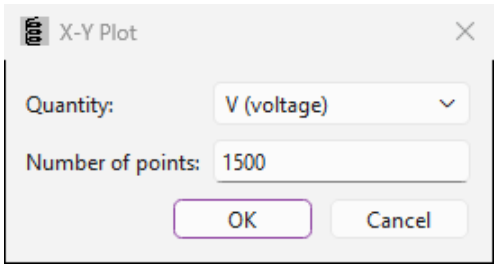


Figure 2.62: X-Y Plot dialog.

After "OK" is pressed, the program computes the desired values along the defined contour. When computation of the values is finished, a plot window will appear with a graph of the selected quantity.

To save the plotted values, press the `Export...` button on the plot window. A file selection dialog appears and prompts for the name of a file to which the data is written. The data is written as comma-separated values (`.csv`), with a header row naming the columns: position along the contour, followed by one column for each curve on the plot.

Currently, the type of line plots supported are:

- `V` Voltage
- `|J|` Magnitude of current density
- `J.n` Normal current density
- `J.t` Tangential current density
- `|E|` Magnitude of electric field intensity
- `E.n` Normal electric field intensity
- `E.t` Tangential electric field intensity
- `|Jc|` Magnitude of conduction current density

- `Jc.n` Normal conduction current density
- `Jc.t` Tangential conduction current density
- `|Jd|` Magnitude of displacment current density
- `Jd.n` Normal displacement current density
- `Jd.t` Tangential displacement current density

2.9.5 Line Integrals

Once a contour has been specified in Contours mode, Line Integrals can be performed along the specified contour. These integrals are performed by evaluating a large number of points at evenly spaced along the contour and integrating using a simple trapezoidal-type integration scheme.

To perform an integration, press the "integral" icon on the toolbar (as shown in Figure 2.61). A small dialog will appear with a drop list. Choose the desired integral from the drop list and press `OK`. The amount of time required to perform the integral will be virtually instantaneous for some types of integrals; however, some types may require several seconds to evaluate. When the evaluation of the integral is completed, the answer appears on the screen in a pop-up box.

The line integrals currently supported are:

- `Voltage Difference (E.t)`. This integral returns the voltage difference between the ends of the contour
- `Current Flow (J.n)`. This integral returns the total current passing through a volume defined by extruding or sweeping the defined contour.
- `Contour length & area`. The length of the contour, and the area formed by extruding the contour.
- `Average Voltage`. The average voltage along the line.

2.9.6 Block Integrals

To select the regions over which a block integral is to be performed, left-click with the mouse in the desired region. The selected region will appear highlighted in green.

To perform an integration, press the "integral" icon on the toolbar (as shown in Figure 2.61), and a dialog will appear with a drop list. Choose the desired integral from the drop list and press `OK`. The integral is then performed by analytically integrating the specified kernel over each element in the defined region, and summing the results for all elements. Volume integrals may take several seconds to evaluate, especially on dense meshes. Be patient. When the evaluation of the integral is completed, the answer appears on the screen in a pop-up box.

The block integrals currently supported are:

- `Real Power`
- `Reactive Power`

- Apparent Power
- Time-Average Stored Energy
- Block cross-section area
- Block volume

2.9.7 Conductor Results

If conductor properties are used to specify the excitation, a useful byproduct is ready access to the voltage of and current through the conductor. To view the conductor results, either press the "Conductor Props" toolbar button pictured in Figure 2.61 or select View|Conductor Properties off of the postprocessor main menu. A dialog, as pictured in Figure 2.63 will appear. There is a drop list on the dialog, from which the user selects the conductor for which results are desired. When a conductor is selected, the voltage and current associated with that conductor are displayed.

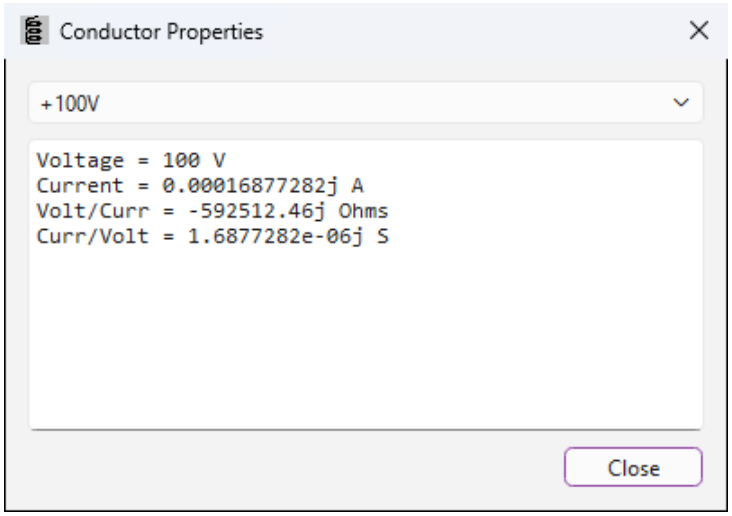


Figure 2.63: Conductor results dialog.

2.10 Exporting of Graphics

Ultimately, you may want to export graphics from FEMM for inclusion in reports and so on.

The easiest way is the Copy as Bitmap selection off of the main menu's Edit list. This copies whatever is currently displayed in the active window to the clipboard as a bitmap, which can then be pasted directly into most applications.

To write the same picture straight to a file instead, use the mi_savebitmap and mo_savebitmap scripting commands (and their ei_, hi_ and ci_ equivalents), described in the Python Scripting chapter. Because a script can set up the view before saving, this is also the convenient way to produce the same figure from several models, or the same model at several rotor positions.

FEMM does not produce vector output. Earlier versions could copy a Windows Enhanced Metafile, and the `savemetafile` scripting commands are still accepted so that old scripts do not fail, but they do nothing.

Chapter 3

Python Scripting

3.1 What is Python Scripting?

The Python scripting language is used to add scripting/batch processing facilities to FEMM. The Interactive Shell can run Python scripts through the `Open Python Script` selection on the Files menu, or Python commands can be entered in directly to the Python Console Window.

FEMM uses MicroPython, an embedded implementation of Python 3. Documentation about programming in Python can be found at `https://docs.micropython.org`. Because the scripting files are text, they can be edited with any text editor (*e.g.* notepad).

In addition to the standard Python command set described in [9], a number of FEMM-specific functions have been added for manipulating files in both the pre- and post-processor. These commands are described in the following sections.

All three ways of building a string work, and can be mixed freely:

```
turns, current = 250, 1.5 + 0.2j
print("%d turns at %g A" % (turns, current))
print("{} turns at {:g} A".format(turns, current))
print(f"{turns} turns at {current:g} A")
```

Each prints `250 turns at 1.5+0.2j A`. Complex values are accepted by `%g`, `%e` and `%f` and print the way desktop Python prints them, which is a FEMM addition — desktop Python raises instead, having no particular need to format a complex number. Since so many FEMM results are complex, that seemed the wrong trade to inherit.

3.2 Importing Python Modules

FEMM scripts can import user-written Python modules using the standard `import` statement. Modules are searched for in three locations, in this order:

1. The current working directory (set automatically when a script is opened via the toolbar, or manually via `setcurrentdirectory()`).

2. A `python/` folder located next to the FEMM executables. This is where FEMM's own bundled modules live.

3. A `python/` folder in your personal FEMM directory:

Windows	`%APPDATA%\FEMM\python`
macOS	`~/Library/Application Support/FEMM/python`
Linux, FreeBSD	`~/.local/share/FEMM/python`

   (on Linux and FreeBSD, `$XDG_DATA_HOME/FEMM/python` if you have set that variable). The folder is not created for you; make it if you want to use it.

Put your own modules in the third location. It is the only one of the three that belongs to you: the `python/` folder beside the executables holds the modules FEMM ships, and installing a new version of FEMM replaces its entire contents, so anything you add there is lost on the next upgrade. On a system where FEMM has been installed for all users it may not be writable at all.

The search order means FEMM's own modules take precedence. A file of your own named, say, `csv.py` will not override the `csv` that FEMM provides — which is deliberate, since a shadowed standard module produces errors that look like faults in FEMM itself. Choose a name that is not already taken.

For example, creating `myutils.py` in your personal `python/` folder lets any FEMM script use it with `import myutils`.

Additional pure-Python modules compatible with MicroPython can be obtained from the MicroPython standard library repository at `https://github.com/micropython/micropython-lib`. Download the desired `.py` file and place it in your personal `python/` folder.

Note: only pure-Python modules can be added this way; CPython C-extension modules are not compatible with MicroPython.

Modules that are built in

These need no installation — just `import`:

- `numpy`, `scipy` — arrays and linear algebra, provided by `ulab`. Much, but not all, of the NumPy API; `import numpy as np` works as it would in desktop Python.
- `math`, `cmath` — real and complex mathematics.
- `os`, `os.path` — files and directories (see the next section).
- `sys` — `argv`, `path`, `modules`, `platform`, `version`.
- `time` — `time()`, `localtime()`, `gmtime()`, `mktime()`, `sleep()`, and the `ticks_ms`/`ticks_diff` pair for timing. Seconds are counted from 1970 and `time()` returns a float, as in desktop Python. `localtime()` and `mktime()` work in the machine's own timezone and `gmtime()` in UTC, again as in desktop Python. `sleep()` keeps the user interface responsive, so a delay in a script does not freeze the window.

- `datetime` — `datetime`, `date`, `time`, `timedelta` and `timezone`, with `strftime()`, `strptime()`, `isoformat()`, `fromisoformat()` and `isocalendar()`. Two things it does not have: a timezone database (build fixed offsets with `timezone(timedelta(hours=2))`), and format specifiers inside f-strings — write `d.strftime("%Y-%m-%d")` rather than `f"{d:%Y-%m-%d}"`, which this interpreter cannot dispatch. Month and day names are always English.

- `json`, `re` — reading and writing JSON, and regular expressions. `re` covers the usual syntax including counted repetition (`\d{4}`), and provides `findall`, `finditer`, `split`, `sub`, `subn`, `fullmatch` and `escape`. Three things it does not have: the flags (`re.IGNORECASE` and friends), lookahead and lookbehind, and named groups. All three fail immediately rather than quietly doing the wrong thing, so a pattern that compiles behaves as it reads.

- `csv` — reading and writing comma-separated files, with `reader`, `writer`, `DictReader`, `DictWriter`, `Sniffer` and the dialect registry. Writing a CSV file is worth one note: open it with `newline=""`, as in

  ```
  import csv
  f = open("results.csv", "w", newline="")
  w = csv.writer(f)
  w.writerow(["n", "x", "y"])
  w.writerow([1, 0.5, 1.0])
  f.close()
  ```

  Without `newline=""` every line ends with an extra carriage return, and Excel then shows a blank row between each pair of records. This is not a FEMM quirk — desktop Python behaves the same way on Windows, and for the same reason. A file being read benefits from it too: it is what lets a quoted field contain a line break.

- `random` — `random()`, `uniform()`, `randint()`, `choice()`, `seed()`. Seeded from the clock at startup, so results differ from run to run unless `seed()` is called.

- `binascii`, `hashlib` — hex and base64 encoding, SHA-256.

- `glob` — filenames by pattern, which is how a batch job finds its models:

  ```
  import glob
  for path in sorted(glob.glob("models/*.fem")):
      open(path)
      mi_analyze()
  ```

  `*`, `?` and `[abc]` work in any part of the path, and `glob.glob("**/*.fem", recursive=True)` walks subfolders too. Results come back unsorted, as in desktop Python — sort them if the order matters.

- `os`, `shutil`, `tempfile` — files and directories. `shutil` adds `copyfile`, `copy`, `copytree`, `move` and `rmtree` on top of what `os` offers, which is what a script moving result files around wants.

- `itertools`, `functools`, `operator`, `copy` — looping, sorting and grouping helpers. `itertools.groupby` turns a flat sweep into per-parameter runs, `operator.itemgetter` sorts a table of results by a column, and `copy.deepcopy` duplicates a parameter dictionary without sharing its contents.
- `contextlib`, `traceback`, `types` — `with`-statement helpers, formatted exception text, and the type objects.
- `logging`, `textwrap` — progress messages with levels and timestamps, and paragraph formatting for reports.
- `pathlib`, `stat`, `string`, `pprint`, `bisect`, `heapq`, `base64`, `pickle`, `abc`, `inspect`, `argparse` — the rest of the everyday standard library. Note that `string` here is the character constants and `translate()`; `Template` and `Formatter` are not included.
- `sys` — as above, and since 2026-08 also `stdout`, `stderr` and `stdin`, so `print(msg, file=sys.stderr)` and `sys.stdout.write(...)` work. `print` also accepts `sep` and `end`; note that its default separator is a TAB, which is FEMM's convention rather than desktop Python's space.
- `collections`, `struct`, `array`, `io`, `errno`, `gc` — the remaining general-purpose pieces.
- `matplotlib`, `xlsxwriter`, `zipfile`, `pandas` — publication-quality plots, Excel workbooks, zip archives, and data frames. These four are different from everything above: they are not part of FEMM but are forwarded to a Python installation on the machine, so they need one (see the *Plotting with matplotlib* chapter, which also covers reaching any other Python package the same way).

Memory

There is no fixed script heap and no size limit of FEMM's own: an array is bounded only by the memory the machine will give it, so `numpy.zeros(30000000)` (240 MB) is an ordinary allocation. Memory is reclaimed automatically and returned to the operating system once it has been unused for a while; nothing needs to be freed by hand.

Two details, for scripts that ask:

- `gc.mem_free()` reports the free space *inside* the memory FEMM is currently holding, not the memory available on the machine. A small number there does not mean the next allocation will fail — FEMM simply asks the system for more.
- `gc.threshold()` is accepted and ignored. It configured a collection policy that no longer exists.

`gc.collect()` still forces a collection if a script wants one at a particular moment, but it is rarely needed. If the goal is specifically to hand a large block back to the operating system — after finishing with a several-hundred-megabyte array, say — call it a few times rather than once; a single collection does not always release everything.

A file object that is dropped without being closed is closed for you when the script ends, whether it ended normally or with an error. What is not promised is that it happens any sooner: a

script that opens many files in a loop and drops them can run out of handles before it finishes. Use `with open(...) as f:`, which closes at the end of the block, or call `f.close()`.

MicroPython's versions of these are subsets. The differences are usually small but occasionally surprising: `re` has no module-level `split()`, for instance — use `re.compile(pattern).split(s)`. The MicroPython library documentation describes each module as implemented.

3.3 Files and Directories

The `os` module provides the file and directory operations a script usually needs, with the spelling Python programmers expect:

- `os.getcwd()`, `os.chdir(path)` — read and set the working directory. `getcwd()` and `setcurrentdirectory()` are the same two calls under FEMM's own names.
- `os.listdir([path])` — the names in a directory, as a list.
- `os.mkdir(path)`, `os.makedirs(path)` — create one directory, or a directory and any missing parents.
- `os.remove(path)` (`os.unlink`), `os.rmdir(path)`, `os.rename(old, new)`.
- `os.stat(path)` — a ten-element tuple, as in MicroPython: mode, inode, device, links, uid, gid, size, and the access, modification and creation times. Element 6 is the size in bytes.
- `os.path.exists`, `isdir`, `isfile`, `getsize`, `abspath`, `join`, `split`, `splitext`, `dirname`, `basename` — the usual path helpers.

A failed operation raises `OSError`, so a script can guard one in the normal way:

```
import os
if not os.path.exists(outdir):
    os.makedirs(outdir)
for name in os.listdir(indir):
    if os.path.splitext(name)[1] == ".fem":
        open(os.path.join(indir, name))
        mi_analyze()
```

Paths behave the same way here as in every other FEMM command: a backslash is accepted and normalized to a forward slash, and results come back with forward slashes on all platforms. Accented and non-Latin directory names work — `os`, the FEMM commands, Python's own `open()` and `import` all agree about which file a path names, and a module kept in such a directory can be imported like any other.

Files opened in text mode behave as they do in desktop Python: a `\n` is written as the platform's line ending and read back as `\n`, so text written by a script looks right to other Windows programs. Files opened in binary mode (`"rb"`, `"wb"`) are exact, byte for byte, with no translation at all.

Note that `open()` does double duty. Called with a single argument naming a `.fem`, `.fee`, `.feh` or `.fec` file it opens that *document*, exactly as `opendocument()` does; called any other way it is Python's file `open()`. Reach for `opendocument()` when the intent should be unambiguous.

Writing results as a table

A parameter sweep usually ends with a table of numbers destined for a spreadsheet or a plotting program. There is no `csv` module, but for a purely numeric table `numpy` writes and reads one directly:

```
import numpy as np

data = np.array([[0.0, 1.50, 0.021],
                 [0.5, 1.25, 0.019]])
np.savetxt("sweep.csv", data, delimiter=",", header="angle,By,torque")

back = np.loadtxt("sweep.csv", delimiter=",")
```

`savetxt` accepts `delimiter` and `header` (the header line is written with a leading `#`); it does not accept a `fmt` argument, and writes fifteen decimal places.

When the columns are not all numbers — a material name beside a force, say — or when the header must not begin with a `#`, write the lines directly. It is a two-line loop, and the formatting is then exactly what you asked for:

```
f = open("sweep.csv", "w")
f.write("angle,By,torque\n")
for angle, by, torque in results:
    f.write("%g,%g,%g\n" % (angle, by, torque))
f.close()
```

Reading such a file back is the same in reverse, splitting each line on the comma and converting the pieces. Be aware that splitting on commas is only safe for data of this kind: a general CSV file may quote a field that itself contains a comma or a line break, and unpicking that correctly takes a real parser. For files your own scripts wrote, splitting is exactly right.

3.4 Predefined Constants and Units

A number of constants are predefined and can be used in any script. Because every edit box in the user interface is evaluated by Python, they can equally well be typed into an edit box in place of a number.

- `pi`, `Pi`, `PI` — π.
- `uo`, `muo` — the magnetic permeability of free space, $4\pi \times 10^{-7}$.
- `eo` — the electrical permittivity of free space, $8.8541878188 \times 10^{-12}$.

A set of unit names is predefined as well. Each is equal to the value of one of that unit in the SI unit that FEMM expects for the quantity, so multiplying by one converts a number written in convenient units into the units the program wants. Entering `1000*Oersted` in a field intensity box, for example, is the same as entering `79577.47`.

- **Length**, in meters: `meter`, `inch`, `cm`, `mm`, `um` (`micron`), and `mil`.
- **Flux density**, in Tesla: `Tesla`, `mTesla`, `Gauss`, `kGauss`.
- **Field intensity**, in Amp/Meter: `AmpMeter`, `kAmpMeter`, `Oersted`, `kOersted`.

Plural and capitalized spellings are accepted throughout, so `inch`, `inches`, `Inch` and `Inches` all mean the same thing.

Finally, `AWG(n)` returns the diameter in millimeters of American Wire Gauge size `n`.

3.5 Common Scripting Command Set

A number of FEMM-specific scripting commands exist that are not associated with any particular problem type.

- `clearconsole()` Clears the output window of the Python console.
- `newdocument(doctype)` Creates a new preprocessor document and opens up a new preprocessor window. Specify `doctype` to be `0` for a magnetics problem, `1` for an electrostatics problem, `2` for a heat flow problem, or `3` for a current flow problem. An alternative syntax for this command is `create(doctype)`
- `hideconsole()` Hides the floating Python console window.
- `messagebox("message")` displays the `"message"` string to the screen in a pop-up message box.
- `open("filename")` Opens a document specified by `filename`.
- `print()` This is standard Python "print" command directed to the output of the Python console window. Any number of comma-separated items can be printed at once via the print command.
- `quit()` Close all documents and exit the Interactive Shell at the end of the currently executing Python script.
- `showconsole()` Displays the floating Python console window.
- `smartmesh(state)` Calling with a state of 0 turns off "smart mesh" functionality for the present session; calling with a state of 1 turns "smarth meshing" on. The `smartmesh` function applies across all problem types. To set smart meshing on or off on a file-by-file basis, use `mi_smartmesh`,`ei_smartmesh`, `hi_smartmesh`, and `ci_smartmesh`, depending on problem type. Note that calling the global `smartmesh` with a value of -1 sets the program to defer to the file-by-file setting rather than forcing all smart meshing on or off.

3.6 Magnetics Preprocessor Scripting Command Set

A number of different commands are available in the preprocessor. Command names carry exactly one underscore, the one that follows the two-letter prefix. Original FEMM additionally accepted a longer spelling that broke the remainder of each name up with further underscores; that spelling is not supported.

3.6.1 Object Add/Remove Commands

- `mi_addnode(x,y)` Add a new node at x,y
- `mi_addsegment(x1,y1,x2,y2)` Add a new line segment from node closest to (x1,y1) to node closest to (x2,y2)
- `mi_addblocklabel(x,y)` Add a new block label at (x,y)
- `mi_addarc(x1,y1,x2,y2,angle,maxseg)` Add a new arc segment from the nearest node to (x1,y1) to the nearest node to (x2,y2) with angle 'angle' divided into 'maxseg' segments.
- `mi_drawline(x1,y1,x2,y2)` Adds nodes at (x1,y1) and (x2,y2) and adds a line segment between them. Unlike `mi_addsegment`, which connects the nearest EXISTING nodes, this creates the nodes it needs.
- `mi_drawarc(x1,y1,x2,y2,angle,maxseg)` Adds nodes at (x1,y1) and (x2,y2) and adds an arc segment of the specified angle and discretization between them.
- `mi_drawrectangle(x1,y1,x2,y2)` Adds nodes at the corners of the rectangle with opposite corners (x1,y1) and (x2,y2), then adds the four segments connecting them.
- `mi_deleteselected` Delete all selected objects.
- `mi_deleteselectednodes` Delete selected nodes.
- `mi_deleteselectedlabels` Delete selected block labels.
- `mi_deleteselectedsegments` Delete selected segments.
- `mi_deleteselectedarcsegments` Delete selects arcs.

3.6.2 Geometry Selection Commands

- `mi_clearselected()` Clear all selected nodes, blocks, segments and arc segments.
- `mi_selectsegment(x,y)` Select the line segment closest to (x,y)
- `mi_selectnode(x,y)` Select the node closest to (x,y). Returns the coordinates of the selected node.
- `mi_selectlabel(x,y)` Select the label closet to (x,y). Returns the coordinates of the selected label.

- `mi_selectarcsegment(x,y)` Select the arc segment closest to (x,y)
- `mi_selectgroup(n)` Select the nth group of nodes, segments, arc segments and blocklabels. This function will clear all previously selected elements and leave the editmode in 4 (group)
- `mi_selectcircle(x,y,R,editmode)` selects objects within a circle of radius R centered at (x,y). If only x, y, and R paramters are given, the current edit mode is used. If the editmode parameter is used, 0 denotes nodes, 1 denotes segments, 2 denotes block labels, 3 denotes arcs, and 4 specifies that all entity types are to be selected.
- `mi_selectrectangle(x1,y1,x2,y2,editmode)` selects objects within a rectangle defined by points (x1,y1) and (x2,y2). If no editmode parameter is supplied, the current edit mode is used. If the editmode parameter is used, 0 denotes nodes, 1 denotes segments, 2 denotes block labels, 3 denotes arcs, and 4 specifies that all entity types are to be selected.

3.6.3 Object Labeling Commands

- `mi_setnodeprop("propname",groupno)` Set the selected nodes to have the nodal property `mi_"propname"` and group number `groupno`.
- `mi_setblockprop("blockname", automesh, meshsize, "incircuit", magdirection, group, turns)` Set the selected block labels to have the properties:
  - Block property `"blockname"`.
  - `automesh`: 0 = mesher defers to mesh size constraint defined in `meshsize`, 1 = mesher automatically chooses the mesh density.
  - `meshsize`: size constraint on the mesh in the block marked by this label.
  - Block is a member of the circuit named `"incircuit"`
  - The magnetization is directed along an angle in measured in degrees denoted by the parameter `magdirection`. Alternatively, `magdirection` can be a string containing a formula that prescribes the magnetization direction as a function of element position. In this formula `theta` and `R` denotes the angle in degrees of a line connecting the center each element with the origin and the length of this line, respectively; `x` and `y` denote the x- and y-position of the center of the each element. For axisymmetric problems, `r` and `z` should be used in place of `x` and `y`.
  - A member of group number `group`
  - The number of turns associated with this label is denoted by `turns`.
- `mi_setsegmentprop("propname", elementsize, automesh, hide, group)` Set the select segments to have:
  - Boundary property `"propname"`
  - Local element size along segment no greater than `elementsize`
  - `automesh`: 0 = mesher defers to the element constraint defined by `elementsize`, 1 = mesher automatically chooses mesh size along the selected segments

  - hide: 0 = not hidden in post-processor, 1 == hidden in post processor
  - A member of group number group

- mi_setarcsegmentprop(maxsegdeg, "propname", hide, group) Set the selected arc segments to:
  - Meshed with elements that span at most maxsegdeg degrees per element
  - Boundary property "propname"
  - hide: 0 = not hidden in post-processor, 1 == hidden in post processor
  - A member of group number group

- mi_setgroup(n) Set the group associated of the selected items to n

3.6.4 Problem Commands

- mi_probdef(frequency,units,type,precision,(depth),(minangle),(acsolver) changes the problem definition. Set frequency to the desired frequency in Hertz. The units parameter specifies the units used for measuring length in the problem domain. Valid "units" entries are "inches", "millimeters", "centimeters", "mils", "meters, and "micrometers". Set the parameter problemtype to "planar" for a 2-D planar problem, or to "axi" for an axisymmetric problem. The precision parameter dictates the precision required by the solver. For example, entering 1E-8 requires the RMS of the residual to be less than 10^{-8}. A fifth parameter, representing the depth of the problem in the into-the-page direction for 2-D planar problems, can also also be specified. A sixth parameter represents the minimum angle constraint sent to the mesh generator. A seventh parameter specifies the solver type to be used for AC problems.

- mi_analyze(flag) runs fkern to solve the problem. The flag parameter controls whether the fkern window is visible or minimized. For a visible window, either specify no value for flag or specify 0. For a minimized window, flag should be set to 1.

- mi_loadsolution() loads and displays the solution corresponding to the current geometry.

- mi_setfocus("documentname") Switches the magnetics input file upon which scripting commands are to act. If more than one magnetics input file is being edited at a time, this command can be used to switch between files so that the mutiple files can be operated upon programmatically via Python. documentname should contain the name of the desired document as it appears on the window's title bar.

- mi_saveas("filename") saves the file with name "filename". Note if you use a path you must use two backslashes *e.g.* "c:\\temp\\myfemmfile.fem"

3.6.5 Mesh Commands

- `mi_createmesh()` runs the mesher to create a mesh. Note that this is not a necessary precursor of performing an analysis, as `mi_analyze()` will make sure the mesh is up to date before running an analysis. The number of elements in the mesh is returned as a result.
- `mi_showmesh()` shows the mesh.
- `mi_purgemesh()` clears the mesh out of both the screen and memory.

3.6.6 Editing Commands

- `mi_copyrotate(bx, by, angle, copies, (editaction) )`
  - `bx, by` – base point for rotation
  - `angle` – angle by which the selected objects are incrementally shifted to make each copy. `angle` is measured in degrees.
  - `copies` – number of copies to be produced from the selected objects.
- `mi_copytranslate(dx, dy, copies, (editaction))`
  - `dx,dy` – distance by which the selected objects are incrementally shifted.
  - `copies` – number of copies to be produced from the selected objects.
  - `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group
- `mi_createradius(x, y, r)` turns a corner located at `(x,y)` into a curve of radius `r`.
- `mi_moverotate(bx,by,shiftangle (editaction))`
  - `bx, by` – base point for rotation
  - `shiftangle` – angle in degrees by which the selected objects are rotated.
  - `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group
- `mi_movetranslate(dx,dy,(editaction))`
  - `dx,dy` – distance by which the selected objects are shifted.
  - `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group
- `mi_scale(bx,by,scalefactor,(editaction))`
  - `bx, by` – base point for scaling
  - `scalefactor` – a multiplier that determines how much the selected objects are scaled
  - `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group
- `mi_mirror(x1,y1,x2,y2,(editaction))` mirror the selected objects about a line passing through the points `(x1,y1)` and `(x2,y2)`. Valid `editaction` entries are 0 for nodes, 1 for lines (segments), 2 for block labels, 3 for arc segments, and 4 for groups.

- mi_seteditmode(editmode) Sets the current editmode to:
  - "nodes" - nodes
  - "segments" - line segments
  - "arcsegments" - arc segments
  - "blocks" - block labels
  - "group" - selected group

  This command will affect all subsequent uses of the other editing commands, if they are used WITHOUT the editaction parameter.

3.6.7 Zoom Commands

- mi_zoomnatural() zooms to a "natural" view with sensible extents.
- mi_zoomout() zooms out by a factor of 50%.
- mi_zoomin() zoom in by a factor of 200%.
- mi_zoom(x1,y1,x2,y2) Set the display area to be from the bottom left corner specified by (x1,y1) to the top right corner specified by (x2,y2).

3.6.8 View Commands

- mi_showgrid() Show the grid points.
- mi_hidegrid() Hide the grid points.
- mi_gridsnap("flag") Setting flag to "on" turns on snap to grid, setting flag to "off" turns off snap to grid.
- mi_setgrid(density,"type") Change the grid spacing. The density parameter specifies the space between grid points, and the type parameter is set to "cart" for cartesian coordinates or "polar" for polar coordinates.
- mi_refreshview() Redraws the current view.
- mi_minimize() minimizes the active magnetics input view.
- mi_maximize() maximizes the active magnetics input view.
- mi_restore() restores the active magnetics input view from a minimized or maximized state.
- mi_resize(width,height) resizes the active magnetics input window client area to width $\times$ height.

3.6.9 Object Properties

- `mi_getmaterial("materialname")` fetches the material specified by `materialname` from the materials library.
- `mi_addmaterial("materialname", mu_x, mu_y, H_c, J, Cduct, Lam_d, Phi_hmax, lam_fill, LamType, Phi_hx, Phi_hy,NStrands,WireD)` adds a new material with called `"materialname"` with the material properties:
  - `mu_x` Relative permeability in the x- or r-direction.
  - `mu_y` Relative permeability in the y- or z-direction.
  - `H_c` Permanent magnet coercivity in Amps/Meter.
  - `J` Real Applied source current density in Amps/mm^2.
  - `Cduct` Electrical conductivity of the material in MS/m.
  - `Lam_d` Lamination thickness in millimeters.
  - `Phi_hmax` Hysteresis lag angle in degrees, used for nonlinear BH curves.
  - `Lam_fill` Fraction of the volume occupied per lamination that is actually filled with iron (Note that this parameter defaults to 1 the `femme` preprocessor dialog box because, by default, iron completely fills the volume)
  - `Lamtype` Set to
    - 0 – Not laminated or laminated in plane
    - 1 – laminated x or r
    - 2 – laminated y or z
    - 3 – Magnet wire
    - 4 – Plain stranded wire
    - 5 – Litz wire
    - 6 – Square wire
    - 7 – 10% copper-clad aluminum (CCA) wire
    - 8 – 15% copper-clad aluminum (CCA) wire
  - `Phi_hx` Hysteresis lag in degrees in the x-direction for linear problems.
  - `Phi_hy` Hysteresis lag in degrees in the y-direction for linear problems.
  - `NStrands` Number of strands in the wire build. Should be 1 for Magnet or Square wire.
  - `WireD` Diameter of each wire constituent strand in millimeters.

  Note that not all properties need be defined–properties that aren't defined are assigned default values.
- `mi_addbhpoint("blockname",b,h)` Adds a B-H data point the the material specified by the string `"blockname"`. The point to be added has a flux density of `b` in units of Teslas and a field intensity of `h` in units of Amps/Meter.

- `mi_clearbhpoints("blockname")` Clears all B-H data points associatied with the material specified by `"blockname"`.
- `mi_addpointprop("pointpropname",a,j)` adds a new point property of name `"pointpropname"` with either a specified potential `a` in units Webers/Meter or a point current `j` in units of Amps. Set the unused parameter pairs to 0.
- `mi_addboundprop("propname", A0, A1, A2, Phi, Mu, Sig, c0, c1, BdryFormat, ia, oa)`
  adds a new boundary property with name `"propname"`
  - For a "Prescribed A" type boundary condition, set the `A0, A1, A2` and `Phi` parameters as required. Set all other parameters to zero.
  - For a "Small Skin Depth" type boundary condtion, set the `Mu` to the desired relative permeability and `Sig` to the desired conductivity in MS/m. Set `BdryFormat` to 1 and all other parameters to zero.
  - To obtain a "Mixed" type boundary condition, set `C1` and `C0` as required and `BdryFormat` to 2. Set all other parameters to zero.
  - For a "Strategic dual image" boundary, set `BdryFormat` to 3 and set all other parameters to zero.
  - For a "Periodic" boundary condition, set `BdryFormat` to 4 and set all other parameters to zero.
  - For an "Anti-Perodic" boundary condition, set `BdryFormat` to 5 set all other parameters to zero.
  - For a "Periodic Air Gap", set BdryFormat to 6. Parameters `ia` and `oa`specify the inner boundary angle and outer boundary angle, respectively.
  - For an "Anti-periodic Air Gap", set BdryFormat to 7. The same `ia` and `oa` parameters also apply here."
- `mi_addcircprop("circuitname", i, circuittype)`
  adds a new circuit property with name `"circuitname"` with a prescribed current, `i`. The `circuittype` parameter is 0 for a parallel-connected circuit and 1 for a series-connected circuit.
- `mi_deletematerial("materialname")` deletes the material named `"materialname"`.
- `mi_deleteboundprop("propname")` deletes the boundary property named `"propname"`.
- `mi_deletecircuit("circuitname")` deletes the circuit named `circuitname`.
- `mi_deletepointprop("pointpropname")` deletes the point property named `"pointpropname"`
- `mi_modifymaterial("BlockName",propnum,value)` This function allows for modification of a material's properties without redefining the entire material (*e.g.* so that current can be modified from run to run). The material to be modified is specified by `"BlockName"`.

The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	BlockName	Name of the material
1	μ_x	x (or r) direction relative permeability
2	μ_y	y (or z) direction relative permeability
3	H_c	Coercivity, Amps/Meter
4	J_r	Source current density, MA/m^2
5	σ	Electrical conductivity, MS/m
6	d_{lam}	Lamination thickness, mm
7	ϕ_{hmax}	Hysteresis lag angle for nonlinear problems, degrees
8	LamFill	Iron fill fraction
9	LamType	0 = None/In plane, 1 = parallel to x, 2=parallel to y
10	ϕ_{hx}	Hysteresis lag in x-direction for linear problems, degrees
11	ϕ_{hy}	Hysteresis lag in y-direction for linear problems, degrees
12	NStrands	Number of strands in wire build
13	WireD	Strand diameter, mm

- `mi_modifyboundprop("BdryName",propnum,value)` This function allows for modification of a boundary property. The BC to be modified is specified by `"BdryName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	BdryName	Name of boundary property
1	A_0	Prescribed A parameter
2	A_1	Prescribed A parameter
3	A_2	Prescribed A parameter
4	ϕ	Prescribed A phase
5	μ	Small skin depth relative permeability
6	σ	Small skin depth conductivity, MS/m
7	c_0	Mixed BC parameter
8	c_1	Mixed BC parameter
9	BdryFormat	Type of boundary condition: 0 = Prescribed A 1 = Small skin depth 2 = Mixed 3 = Strategic Dual Image 4 = Periodic 5 = Antiperiodic 6 = Periodic Air Gap 7 = Antiperiodic Air Gap
10	*ia*	Inner boundary angle for air gap element
11	*oa*	Outer boundary angle for air gap element

- `mi_modifypointprop("PointName",propnum,value)` This function allows for modification of a point property. The point property to be modified is specified by `"PointName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

`propnum`	Symbol	Description
0	`PointName`	Name of the point property
1	*A*	Nodal potential, Weber/Meter
2	*J*	Nodal current, Amps

- `mi_modifycircprop("CircName",propnum,value)` This function allows for modification of a circuit property. The circuit property to be modified is specified by `"CircName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

`propnum`	Symbol	Description
0	`CircName`	Name of the circuit property
1	*i*	Total current
2	`CircType`	0 = Parallel, 1 = Series

3.6.10 Miscellaneous

- `mi_savebitmap("filename")` saves a bitmapped screenshot of the current view to the file specified by `"filename"`. As with `mi_saveas`, a backslash in the path must be doubled.

- `mi_savemetafile("filename")` is retained for script compatibility but does nothing: femm-qt has no metafile output. Use `mi_savebitmap` to save a picture of the current view.

- `mi_refreshview()` Redraws the current view.

- `mi_close()` Closes current magnetics preprocessor document and destroys magnetics preprocessor window.

- `mi_shownames(flag)` This function allow the user to display or hide the block label names on screen. To hide the block label names, `flag` should be 0. To display the names, the parameter should be set to 1.

- `mi_readdxf("filename")` This function imports a dxf file specified by `"filename"`.

- `mi_savedxf("filename")` This function saves geometry informationin a dxf file specified by `"filename"`.

- `mi_defineouterspace(Zo,Ro,Ri)` defines an axisymmetric external region to be used in conjuction with the Kelvin Transformation method of modeling unbounded problems. The `Zo` parameter is the z-location of the origin of the outer region, the `Ro` parameter is the radius of the outer region, and the `Ri` parameter is the radius of the inner region (*i.e.* the region

of interest). In the exterior region, the permeability varies as a function of distance from the origin of the external region. These parameters are necessary to define the permeability variation in the external region.

- `mi_attachouterspace()` marks all selected block labels as members of the external region used for modeling unbounded axisymmetric problems via the Kelvin Transformation.
- `mi_detachouterspace()` undefines all selected block labels as members of the external region used for modeling unbounded axisymmetric problems via the Kelvin Transformation.
- `mi_attachdefault()` marks the selected block label as the default block label. This block label is applied to any region that has not been explicitly labeled.
- `mi_detachdefault()` undefines the default attribute for the selected block labels.
- `mi_makeABC(n,R,x,y,bc)` creates a series of circular shells that emulate the impedance of an unbounded domain (i.e. an Improvised Asymptotic Boundary Condition). The `n` parameter contains the number of shells to be used (should be between 1 and 12), `R` is the radius of the solution domain, and `(x,y)` denotes the center of the solution domain. The `bc` parameter should be specified as 0 for a Dirichlet outer edge or 1 for a Neumann outer edge. If the function is called without all the parameters, the function makes up reasonable values for the missing parameters.
- `mi_setprevious(filename,prevtype)` defines the previous solution to be used as the basis for an AC incremental permeability or frozen permeability solution. The `prevtype` field is an integer that specifies whether the solution is to be incremental permeability (`1`) or frozen permeability (`2`). The filename should include the `.ans` extension, *e.g.* `mi_setprevious("mymodel.ans",`
- `mi_smartmesh(state)` Turns on or off smart meshing for the current document. Set `state` to `1` to enable smart meshing or `0` to disable it.
- `mi_setcomment("comment")` Sets the comment/description string associated with the current problem.
- `mi_getboundingbox()` Returns four values: `xmin, xmax, ymin, ymax`, the bounding box of the current geometry.
- `mi_gettitle()` Returns the file path of the current document as a string.

3.7 Magnetics Post Processor Scripting Command Set

There are a number of scripting commands designed to operate in the postprocessor. As with the preprocessor commands, each name carries exactly one underscore, the one after the two-letter prefix.

3.7.1 Data Extraction Commands

- `mo_lineintegral(type)` Calculate the line integral for the defined contour

`type`	name	values 1	values 2	values 3	values 4
0	B.n	total B.n	avg B.n	-	-
1	H.t	total H.t	avg H.t	-	-
2	Contour length	surface area	-	-	
3	Stress Tensor Force	DC r/x force	DC y/z force	2× r/x force	2× y/z force
4	Stress Tensor Torque	DC torque	2× torque	-	-
5	(B.n)^2	total (B.n)^2	avg (B.n)^2	-	-

  Returns typically two (possibly complex) values as results. For force and torque results, the $2\times$ results are only relevant for problems where $\omega \neq 0$. The $1\times$ results are only relevant for incremental permeability AC problems. The $1\times$ results represent the force and torque interactions between the steady-state and the incremental AC solution.

- `mo_blockintegral(type)` Calculate a block integral for the selected blocks

Type	Definition
0	$A \cdot J$
1	A
2	Magnetic field energy
3	Hysteresis and/or lamination losses
4	Resistive losses
5	Block cross-section area
6	Total losses
7	Total current
8	Integral of B_x (or B_r) over block
9	Integral of B_y (or B_z) over block
10	Block volume
11	x (or r) part of steady-state Lorentz force
12	y (or z) part of steady-state Lorentz force
13	x (or r) part of $2\times$ Lorentz force
14	y (or z) part of $2\times$ Lorentz force
15	Steady-state Lorentz torque
16	$2\times$ component of Lorentz torque
17	Magnetic field coenergy
18	x (or r) part of steady-state weighted stress tensor force
19	y (or z) part of steady-state weighted stress tensor force
20	x (or r) part of $2\times$ weighted stress tensor force
21	y (or z) part of $2\times$ weighted stress tensor force
22	Steady-state weighted stress tensor torque
23	$2\times$ component of weighted stress tensor torque
24	R^2 (*i.e.* moment of inertia / density)
25	x (or r) part of $1\times$ weighted stress tensor force
26	y (or z) part of $1\times$ weighted stress tensor force
27	$1\times$ component of weighted stress tensor torque
28	x (or r) part of $1\times$ Lorentz force
29	y (or z) part of $1\times$ Lorentz force
30	$1\times$ component of Lorentz torque

This function returns one (possibly complex) value, *e.g.*: `volume = mo_blockintegral(10)`

- `mo_getpointvalues(X,Y)` Get the values associated with the point at x,y RETURN values in order

Symbol	Definition
A	vector potential A or flux ϕ
B1	flux density B_x if planar, B_r if axisymmetric
B2	flux density B_y if planar, B_z if axisymmetric
Sig	electrical conductivity σ
E	stored energy density
H1	field intensity H_x if planar, H_r if axisymmetric
H2	field intensity H_y if planar, H_z if axisymmetric
Je	eddy current density
Js	source current density
Mu1	relative permeability μ_x if planar, μ_r if axisymmetric
Mu2	relative permeability μ_y if planar, μ_z if axisymmetric
Pe	Power density dissipated through ohmic losses
Ph	Power density dissipated by hysteresis
ff	Fill factor: lamination fill for laminated materials, winding fill for wire materials, otherwise -1

Example: To catch all values at (0.01,0) use

```
A, B1, B2, Sig, E, H1, H2, Je, Js, Mu1, Mu2, Pe, Ph, ff = mo_getpointvalues(0.01,0)
```

For magnetostatic problems, all imaginary quantities are zero.

- `mo_makeplot(PlotType,NumPoints,Filename,FileFormat)` Allows scripted access to the X-Y plot routines. If only `PlotType` or only `PlotType` and `NumPoints` are specified, the command is interpreted as a request to plot the requested plot type to the screen. If, in addition, the `Filename` parameter is specified, the sampled values are written to that file instead of being plotted, in exactly the format the `Export...` button on the plot window produces: comma-separated values with a header row naming the columns, then one row per point holding the position along the contour followed by one column for each plotted quantity. A fourth `FileFormat` argument is accepted and ignored; there is only one data format, and no metafile output. Valid entries for `PlotType` are:

`PlotType`	Definition
0	Potential
1	$\|B\|$
2	$B \cdot n$
3	$B \cdot t$
4	$\|H\|$
5	$H \cdot n$
6	$H \cdot t$
7	J_{eddy}
8	$J_{source} + J_{eddy}$

For example, if one wanted to plot $B \cdot n$ to the screen with 200 points evaluated to make the graph, the command would be:

```
mo_makeplot(2,200)
```

To write the data to disk instead of plotting it, the command would be of the form:

```
mo_makeplot(2,200,"c:\\temp\myfile.txt")
```

- `mo_getprobleminfo()` Returns info on problem description. Returns four values:

Return value	Definition
1	problem type
2	frequency in Hz
3	depth assumed for planar problems in meters
4	length unit used to draw the problem in meters

- `mo_getcircuitproperties("circuit")` Used primarily to obtain impedance information associated with circuit properties. Properties are returned for the circuit property named `"circuit"`. Three values are returned by the function. In order, these results are:
  - `current` Current carried by the circuit
  - `volts` Voltage drop across the circuit
  - `flux_re` Circuit's flux linkage
- `mo_getgapb("BdryName",angle)` Computes the radial and tangential flux density on the centerline of the specified air gap element name (`BdryName`) at the specified `angle`. The angle is specified in degrees."
- `mo_getgapa("BdryName",angle)` Computes the magnetic vector potential on the centerline of the specified air gap element name (`BdryName`) at the specified `angle`. The angle is specified in degrees."
- `mo_gapintegral("BdryName",inttype)` Computes an integral specifiedy by `inttype` over an air gap element specified by `BdryName`. Values for `inttype` in are:

0	DC Torque
1	DC Force
2	Stored Energy
3	2X Torque
4	2X Force
5	Interaction Torque
6	Interaction Force

Torque and energy integrals return one result; force integrals return two results for the x- and y- direction force, respectively.

- `mo_getgapharmonics("BdryName",n)` Returns the `acc`, `acs`,`brc`,`brs`,`btc`, and `bts`. These quantities represent the components of vector potential, radial flux density, and tangential flux density on the centerline of the specified air gap element at the specified angle. For

the nth harmonic, vector potential, radial flux density, and tangential flux density can be represented explicitly as functions of angle via:

$$\begin{aligned} A &= acc\cos n\theta + acs\sin n\theta \\ B_r &= brc\cos n\theta + brs\sin n\theta \\ B_\theta &= btc\cos n\theta + bts\sin n\theta \end{aligned}$$

The angle is specified in degrees. If the function is called with just the `BdryName`, the function returns the highest harmonic order available.

3.7.2 Selection Commands

- `mo_seteditmode(mode)` Sets the mode of the postprocessor to point, contour, or area mode. Valid entries for `mode` are `"point"`, `"contour"`, and `"area"`.
- `mo_selectblock(x,y)` Select the block that contains point (x,y).
- `mo_groupselectblock(n)` Selects all of the blocks that are labeled by block labels that are members of group `n`. If no number is specified (*i.e.* `mo_groupselectblock()`), all blocks are selected.
- `mo_addcontour(x,y)` Adds a contour point at (x,y). If this is the first point then it starts a contour, if there are existing points the contour runs from the previous point to this point. The `mo_addcontour` command has the same functionality as a right-button-click contour point addition when the program is running in interactive mode.
- `mo_bendcontour(angle,anglestep)` Replaces the straight line formed by the last two points in the contour by an arc that spans `angle` degrees. The arc is actually composed of many straight lines, each of which is constrained to span no more than `anglestep` degrees. The `angle` parameter can take on values from -180 to 180 degrees. The `anglestep` parameter must be greater than zero. If there are less than two points defined in the contour, this command is ignored.
- `mo_selectpoint(x,y)` Adds a contour point at the closest input point to (x,y). If the selected point and a previous selected points lie at the ends of an arcsegment, a contour is added that traces along the arcsegment. The `mo_selectpoint` command has the same functionality as the left-button-click contour point selection when the program is running in interactive mode.
- `mo_clearcontour()` Clear a prevously defined contour
- `mo_clearblock()` Clear block selection

3.7.3 Zoom Commands

- `mo_zoomnatural()` Zoom to the natural boundaries of the geometry.
- `mo_zoomin()` Zoom in one level.

- `mo_zoomout()` Zoom out one level.
- `mo_zoom(x1,y1,x2,y2)` Zoom to the window defined by lower left corner (x1,y1) and upper right corner (x2,y2).

3.7.4 View Commands

- `mo_showmesh()` Show the mesh.
- `mo_hidemesh()` Hide the mesh.
- `mo_showpoints()` Show the node points from the input geometry.
- `mo_hidepoints()` Hide the node points from the input geometry.
- `mo_smooth("flag")` This function controls whether or not smoothing is applied to the B and H fields, which are naturally piece-wise constant over each element. Setting `flag` equal to `"on"` turns on smoothing, and setting `flag` to `"off"` turns off smoothing.
- `mo_setweightingscheme(n)` Changes the weighting method used for Weighted Stress Tensor force and torque integrals. Type 0, the default, is recommended: it generates a line of elements in the middle of the air gap between components, minimizing error. Type 1 weights all elements equally and uses all elements in the gap. Types 2, 3 and 4 select alternative element weightings and mask transition profiles.
- `mo_showgrid()` Show the grid points.
- `mo_hidegrid()` Hide the grid points.
- `mo_gridsnap("flag")` Setting `flag` to "on" turns on snap to grid, setting `flag` to `"off"` turns off snap to grid.
- `mo_setgrid(density,"type")` Change the grid spacing. The `density` parameter specifies the space between grid points, and the `type` parameter is set to `"cart"` for cartesian coordinates or `"polar"` for polar coordinates.
- `mo_hidedensityplot()` hides the flux density plot.
- `mo_showdensityplot(legend,gscale,upper_B,lower_B,type)` Shows the flux density plot with options:
  - `legend` Set to `0` to hide the plot legend or `1` to show the plot legend.
  - `gscale` Set to `0` for a colour density plot or `1` for a grey scale density plot.
  - `upper_B` Sets the upper display limit for the density plot.
  - `lower_B` Sets the lower display limit for the density plot.

  - `type` Type of density plot to display. Valid entries are `"bmag"`, `"breal"`, `"bimag"`, and`"logb"` for magnitude, real component, imaginary component, and log magnitude of flux density (B), respectively; `"hmag"`, `"hreal"`, and `"himag"` for magnitude, real component, and imaginary component of field intensity (H); and `"jmag"`, `"jreal"`, and `"jimag"` for magnitude, real component, and imaginary component of current density (J).

  if `legend` is set to `-1` all parameters are ignored and default values are used *e.g.*:
  `mo_showdensityplot(-1)`

- `mo_hidecontourplot()` Hides the contour plot.

- `mo_showcontourplot(numcontours,lower_A,upper_A,type)` shows the A contour plot with options:

  - `numcontours` Number of A equipotential lines to be plotted.
  - `upper_A` Upper limit for A contours.
  - `lower_A` Lower limit for A contours.
  - `type` Choice of `"real"`, `"imag"`, or `"both"` to show either the real, imaginary of both real and imaginary components of A.

  If `numcontours` is `-1` all parameters are ignored and default values are used, *e.g.*:
  `mo_showcontourplot(-1)`

- `mo_showvectorplot(type,scalefactor)` controls the display of vectors denoting the field strength and direction. The parameters taken are the `type` of plot, which should be set to 0 for no vector plot, 1 for the real part of flux density B; 2 for the real part of field intensity H; 3 for the imaginary part of B; 4 for the imaginary part of H; 5 for both the real and imaginary parts of B; and 6 for both the real and imaginary parts of H. The `scalefactor` determines the relative length of the vectors. If the scale is set to 1, the length of the vectors are chosen so that the highest flux density corresponds to a vector that is the same length as the current grid size setting.

- `mo_minimize` minimizes the active magnetics output view.

- `mo_maximize` maximizes the active magnetics output view.

- `mo_restore` restores the active magnetics output view from a minimized or maximized state.

- `mo_resize(width,height)` resizes the active magnetics output window client area to width $\times$ height.

3.7.5 Miscellaneous

- `mo_close()` Closes the current post-processor instance.

- `mo_refreshview()` Redraws the current view.

- `mo_reload()` Reloads the solution from disk.
- `mo_gettitle()` Returns the file path of the current output document as a string.
- `mo_setfocus("documentname")` Switches the magnetics output file upon which post-processor commands are to act. If more than one magnetics output file is open at a time, this command selects between them. `documentname` is the name of the desired document as it appears on the window's title bar; the `.ans` extension may be omitted.
- `mo_savebitmap("filename")` saves a bitmapped screen shot of the current view to the file specified by `"filename"`. A backslash in the path must be doubled, as in any Python string (*e.g.* `"c:\\temp\\myfemmfile.png"`); a file name containing spaces needs nothing special.
- `mo_savemetafile("filename")` is retained for script compatibility but does nothing: femm-qt has no metafile output. Use `mo_savebitmap` to save a picture of the current view.
- `mo_shownames(flag)` This function allow the user to display or hide the block label names on screen. To hide the block label names, `flag` should be 0. To display the names, the parameter should be set to 1.
- `mo_numnodes()` Returns the number of nodes in the in focus magnetics output mesh.
- `mo_numelements()` Returns the number of elements in the in focus magnets output mesh.
- `mo_getnode(n)` Returns the (x,y) or (r,z) position of the nth mesh node.
- `mo_getelement(n)` Returns the following properties for the `n`th element:
  1. Index of first element node
  2. Index of second element node
  3. Index of third element node
  4. x (or r) coordinate of the element centroid
  5. y (or z) coordinate of the element centroid
  6. element area using the length unit defined for the problem
  7. group number associated with the element

  About the bulk getters. The `*_getall...` functions return the whole mesh in one call instead of one item at a time. What you get back depends on where the script runs. Called from a language binding over the IPC connection, the values arrive as a single flat array and the binding wrapper reshapes them into the matrix described below. Called from a script running inside FEMM itself, the function returns that matrix directly, as a numpy array — there is nothing to reshape. Either way the values, their order and their units are the same. Mesh node indices are **1-based**, matching `*_getnode` and `*_getelement`. (Note that the element records inside a solution file are 0-based; the scripting interface adds one on the way out.) Quantities that can have a phase — magnetics and current flow at a nonzero frequency — come back complex; everything else comes back real.

- `mo_getallnodes()` Returns the coordinates of *all* mesh nodes as an $N \times 2$ matrix, one row per node: x_k, y_k. Approximately $50\times$ faster than calling `mo_getnode` in a loop, because all data is obtained in a single call.
- `mo_getallelements()` Returns data for *all* mesh elements as a flat array of $N \times 7$ values. Each group of 7 contains the same fields as `mo_getelement`: three node indices, centroid coordinates, area, and group number. The binding wrappers reshape into an $N \times 7$ matrix.
- `mo_getallb()` Returns B_1, B_2 for *all* element centroids as a flat array of $N \times 2$ values (complex for AC problems). Binding wrappers reshape into an $N \times 2$ matrix.
- `mo_getalla()` Returns the vector potential A at *all* element centroids as a flat array of N values (complex for AC problems).
- `mo_getallgapb("BdryName" [,npts])` Returns B_r, B_θ at `npts` equally spaced angles around the specified air gap element (default 360) as a flat array of `npts`$\times 2$ values. Binding wrappers reshape into an `npts`$\times 2$ matrix.
- `mo_getallgapa("BdryName" [,npts])` Returns the vector potential A at `npts` equally spaced angles around the specified air gap element (default 360) as a flat array of `npts` values.
- `mo_getallgapharmonics("BdryName")` Returns the harmonic coefficients for *all* harmonics of the specified air gap element as a flat array of `nn`$\times 6$ values: `acc, acs, brc, brs, btc, bts` per harmonic. Binding wrappers reshape into an `nn`$\times 6$ matrix.

3.8 Electrostatics Preprocessor Scripting Command Set

A number of different commands are available in the preprocessor. Command names carry exactly one underscore, the one that follows the two-letter prefix. Original FEMM additionally accepted a longer spelling that broke the remainder of each name up with further underscores; that spelling is not supported.

3.8.1 Object Add/Remove Commands

- `ei_addnode(x,y)` Add a new node at x,y
- `ei_addsegment(x1,y1,x2,y2)` Add a new line segment from node closest to (x1,y1) to node closest to (x2,y2)
- `ei_addblocklabel(x,y)` Add a new block label at (x,y)
- `ei_addarc(x1,y1,x2,y2,angle,maxseg)` Add a new arc segment from the nearest node to (x1,y1) to the nearest node to (x2,y2) with angle 'angle' divided into 'maxseg' segments.
- `ei_drawline(x1,y1,x2,y2)` Adds nodes at (x1,y1) and (x2,y2) and adds a line segment between them. Unlike `ei_addsegment`, which connects the nearest EXISTING nodes, this creates the nodes it needs.

- `ei_drawarc(x1,y1,x2,y2,angle,maxseg)` Adds nodes at (x1,y1) and (x2,y2) and adds an arc segment of the specified angle and discretization between them.
- `ei_drawrectangle(x1,y1,x2,y2)` Adds nodes at the corners of the rectangle with opposite corners (x1,y1) and (x2,y2), then adds the four segments connecting them.
- `ei_deleteselected` Delete all selected objects.
- `ei_deleteselectednodes` Delete selected nodes.
- `ei_deleteselectedlabels` Delete selected block labels.
- `ei_deleteselectedsegments` Delete selected segments.
- `ei_deleteselectedarcsegments` Delete selects arcs.

3.8.2 Geometry Selection Commands

- `ei_clearselected()` Clear all selected nodes, blocks, segments and arc segments.
- `ei_selectsegment(x,y)` Select the line segment closest to (x,y)
- `ei_selectnode(x,y)` Select the node closest to (x,y). Returns the coordinates of the selected node.
- `ei_selectlabel(x,y)` Select the label closet to (x,y). Returns the coordinates of the selected label.
- `ei_selectarcsegment(x,y)` Select the arc segment closest to (x,y)
- `ei_selectgroup(n)` Select the nth group of nodes, segments, arc segments and block labels. This function will clear all previously selected elements and leave the edit mode in 4 (group)
- `ei_selectcircle(x,y,R,editmode)` selects objects within a circle of radius R centered at (x,y). If only x, y, and R paramters are given, the current edit mode is used. If the editmode parameter is used, 0 denotes nodes, 1 denotes segments, 2 denotes block labels, 3 denotes arcs, and 4 specifies that all entity types are to be selected.
- `ei_selectrectangle(x1,y1,x2,y2,editmode)` selects objects within a rectangle defined by points (x1,y1) and (x2,y2). If no editmode parameter is supplied, the current edit mode is used. If the editmode parameter is used, 0 denotes nodes, 1 denotes segments, 2 denotes block labels, 3 denotes arcs, and 4 specifies that all entity types are to be selected.

3.8.3 Object Labeling Commands

- `ei_setnodeprop("propname",groupno, "inconductor")` Set the selected nodes to have the nodal property `"propname"` and group number `groupno`. The `"inconductor"` string specifies which conductor the node belongs to. If the node doesn't belong to a named conductor, this parameter can be set to `"<None>"`.

- ei_setblockprop("blockname", automesh, meshsize, group) Set the selected block labels to have the properties:

  Block property "blockname".

  automesh: 0 = mesher defers to mesh size constraint defined in meshsize, 1 = mesher automatically chooses the mesh density.

  meshsize: size constraint on the mesh in the block marked by this label.

  A member of group number group

- ei_setsegmentprop("propname", elementsize, automesh, hide, group, "inconductor",) Set the select segments to have:

  Boundary property "propname"

  Local element size along segment no greater than elementsize

  automesh: 0 = mesher defers to the element constraint defined by elementsize, 1 = mesher automatically chooses mesh size along the selected segments

  hide: 0 = not hidden in post-processor, 1 == hidden in post processor

  A member of group number group

  A member of the conductor specified by the string "inconductor". If the segment is not part of a conductor, this parameter can be specified as "<None>".

- ei_setarcsegmentprop(maxsegdeg, "propname", hide, group, "inconductor") Set the selected arc segments to:

  Meshed with elements that span at most maxsegdeg degrees per element

  Boundary property "propname"

  hide: 0 = not hidden in post-processor, 1 == hidden in post processor

  A member of group number group

  A member of the conductor specified by the string "inconductor". If the segment is not part of a conductor, this parameter can be specified as "<None>".

- ei_setgroup(n) Set the group associated of the selected items to n

3.8.4 Problem Commands

- ei_probdef(units,type,precision,(depth),(minangle)) changes the problem definition. The units parameter specifies the units used for measuring length in the problem domain. Valid "units" entries are "inches", "millimeters", "centimeters", "mils", "meters, and "micrometers". Set problemtype to "planar" for a 2-D planar problem, or to "axi" for an axisymmetric problem. The precision parameter dictates the precision required by the solver. For example, entering 1.E-8 requires the RMS of the residual to be less than 10^{-8}. A fourth parameter, representing the depth of the problem in the into-the-page direction for 2-D planar problems, can also be specified for planar problems. A sixth parameter represents the minimum angle constraint sent to the mesh generator.

- `ei_analyze(flag)` runs `belasolv` to solve the problem. The `flag` parameter controls whether the Belasolve window is visible or minimized. For a visible window, either specify no value for `flag` or specify 0. For a minimized window, `flag` should be set to 1.
- `ei_loadsolution()` loads and displays the solution corresponding to the current geometry.
- `ei_setfocus("documentname")` Switches the electrostatics input file upon which scripting commands are to act. If more than one electrostatics input file is being edited at a time, this command can be used to switch between files so that the mutiple files can be operated upon programmatically via Python. `documentname` should contain the name of the desired document as it appears on the window's title bar.
- `ei_saveas("filename")` saves the file with name `"filename"`. Note if you use a path you must use two backslashes *e.g.* `c:\\temp\\myfemmfile.fee`

3.8.5 Mesh Commands

- `ei_createmesh()` runs the mesher to create a mesh. Note that this is not a necessary precursor of performing an analysis, as `ei_analyze()` will make sure the mesh is up to date before running an analysis. The number of elements in the mesh is pushed back as a return value.
- `ei_showmesh()` toggles the flag that shows or hides the mesh.
- `ei_purgemesh()` clears the mesh out of both the screen and memory.

3.8.6 Editing Commands

- `ei_copyrotate(bx, by, angle, copies, (editaction) )`

  `bx, by` – base point for rotation

  `angle` – angle by which the selected objects are incrementally shifted to make each copy. `angle` is measured in degrees.

  `copies` – number of copies to be produced from the selected objects.

  `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group
- `ei_copytranslate(dx, dy, copies, (editaction))`

  `dx,dy` – distance by which the selected objects are incrementally shifted.

  `copies` – number of copies to be produced from the selected objects.

  `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group
- `ei_createradius(x, y, r)` turns a corner located at `(x,y)` into a curve of radius `r`.

- `ei_moverotate(bx,by,shiftangle (editaction))`

  `bx, by` – base point for rotation

  `shiftangle` – angle in degrees by which the selected objects are rotated.

  `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- `ei_movetranslate(dx,dy,(editaction))`

  `dx,dy` – distance by which the selected objects are shifted.

  `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- `ei_scale(bx,by,scalefactor,(editaction))`

  `bx, by` – base point for scaling

  `scalefactor` – a multiplier that determines how much the selected objects are scaled

  `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- `ei_mirror(x1,y1,x2,y2,(editaction))` mirror the selected objects about a line passing through the points `(x1,y1)` and `(x2,y2)`. Valid `editaction` entries are 0 for nodes, 1 for lines (segments), 2 for block labels, 3 for arc segments, and 4 for groups.

- `ei_seteditmode(editmode)` Sets the current editmode to:

  `"nodes"` – nodes

  `"segments"` - line segments

  `"arcsegments"` - arc segments

  `"blocks"` - block labels

  `"group"` - selected group

  This command will affect all subsequent uses of the other editing commands, if they are used WITHOUT the `editaction` parameter.

3.8.7 Zoom Commands

- `ei_zoomnatural()` zooms to a "natural" view with sensible extents.

- `ei_zoomout()` zooms out by a factor of 50%.

- `ei_zoomin()` zoom in by a factor of 200%.

- `ei_zoom(x1,y1,x2,y2)` Set the display area to be from the bottom left corner specified by `(x1,y1)` to the top right corner specified by `(x2,y2)`.

3.8.8 View Commands

- `ei_showgrid()` Show the grid points.
- `ei_hidegrid()` Hide the grid points.
- `ei_gridsnap("flag")` Setting flag to ”on” turns on snap to grid, setting flag to "off"turns off snap to grid.
- `ei_setgrid(density,"type")` Change the grid spacing. The density parameter specifies the space between grid points, and the type parameter is set to `"cart"` for Cartesian coordinates or `"polar"` for polar coordinates.
- `ei_refreshview()` Redraws the current view.
- `ei_minimize()` minimizes the active electrostatics input view.
- `ei_maximize()` maximizes the active electrostatics input view.
- `ei_restore()` restores the active electrostatics input view from a minimized or maximized state.
- `ei_resize(width,height)` resizes the active electrostatics input window client area to width $\times$ height.

3.8.9 Object Properties

- `ei_getmaterial("materialname")` fetches the material specified by `materialname` from the materials library.
- `ei_addmaterial("materialname", ex, ey, qv)` adds a new material with called `"materialname"` with the material properties:

  `ex` Relative permittivity in the x- or r-direction.

  `ey` Relative permittivity in the y- or z-direction.

  `qv` Volume charge density in units of C/m^3
- `ei_addpointprop("pointpropname",Vp,qp)` adds a new point property of name `"pointpropname"` with either a specified potential `Vp` a point charge density `qp` in units of C/m.
- `ei_addboundprop("boundpropname", Vs, qs, c0, c1, BdryFormat)` adds a new boundary property with name `"boundpropname"`

  For a “Fixed Voltage” type boundary condition, set the `Vs` parameter to the desired voltage and all other parameters to zero.

  To obtain a “Mixed” type boundary condition, set `C1` and `C0` as required and `BdryFormat` to 1. Set all other parameters to zero.

  To obtain a prescribes surface charge density, set `qs` to the desired charge density in C/m^2 and set `BdryFormat` to 2.

For a "Periodic" boundary condition, set `BdryFormat` to 3 and set all other parameters to zero.

For an "Anti-Perodic" boundary condition, set `BdryFormat` to 4 set all other parameters to zero.

- `ei_addconductorprop("conductorname", Vc, qc, conductortype)` adds a new conductor property with name `"conductorname"` with either a prescribed voltage or a prescribed total charge. Set the unused property to zero. The `conductortype` parameter is 0 for prescribed charge and 1 for prescribed voltage.
- `ei_deletematerial("materialname")` deletes the material named `"materialname"`.
- `ei_deleteboundprop("boundpropname")` deletes the boundary property named `"boundpropname"`.
- `ei_deleteconductor("conductorname")` deletes the conductor named `conductorname`.
- `ei_deletepointprop("pointpropname")` deletes the point property named `"pointpropname"`
- `ei_modifymaterial("BlockName",propnum,value)` This function allows for modification of a material's properties without redefining the entire material (e.g. so that current can be modified from run to run). The material to be modified is specified by `"BlockName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	`BlockName`	Name of the material
1	`ex`	x (or r) direction relative permittivity
2	`ey`	y (or z) direction relative permittivity
3	`qs`	Volume charge

- `ei_modifyboundprop("BdryName",propnum,value)` This function allows for modification of a boundary property. The BC to be modified is specified by `"BdryName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

`propnum`	Symbol	Description
0	`BdryName`	Name of boundary property
1	`Vs`	Fixed Voltage
2	`qs`	Prescribed charge density
3	`c0`	Mixed BC parameter
4	`c1`	Mixed BC parameter
5	`BdryFormat`	Type of boundary condition: 0 = Prescribed V 1 = Mixed 2 = Surface charge density 3 = Periodic 4 = Antiperiodic

- `ei_modifypointprop("PointName",propnum,value)` This function allows for modification of a point property. The point property to be modified is specified by `"PointName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	PointName	Name of the point property
1	Vp	Prescribed nodal voltage
2	qp	Point charge density in C/m

- `ei_modifyconductorprop("ConductorName",propnum,value)` This function allows for modification of a conductor property. The conductor property to be modified is specified by `"ConductorName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	ConductorName	Name of the conductor property
1	Vc	Conductor voltage
2	qc	Total conductor charge
3	ConductorType	0 = Prescribed charge, 1 = Prescribed voltage

3.8.10 Miscellaneous

- `ei_savebitmap("filename")` saves a bitmapped screenshot of the current view to the file specified by `"filename"`. As with `ei_saveas`, a backslash in the path must be doubled.

- `ei_savemetafile("filename")` is retained for script compatibility but does nothing: femm-qt has no metafile output. Use `ei_savebitmap` to save a picture of the current view.

- `ei_refreshview()` Redraws the current view.

- `ei_close()` closes the preprocessor window and destroys the current document.

- `ei_shownames(flag)` This function allow the user to display or hide the block label names on screen. To hide the block label names, `flag` should be 0. To display the names, the parameter should be set to 1.

- `ei_readdxf("filename")` This function imports a dxf file specified by `"filename"`.

- `ei_savedxf("filename")` This function saves geometry informatoin in a dxf file specified by `"filename"`.

- `ei_defineouterspace(Zo,Ro,Ri)` defines an axisymmetric external region to be used in conjuction with the Kelvin Transformation method of modeling unbounded problems. The `Zo` parameter is the z-location of the origin of the outer region, the `Ro` parameter is the radius of the outer region, and the `Ri` parameter is the radius of the inner region (*i.e.* the region of interest). In the exterior region, the permeability varies as a function of distance from

the origin of the external region. These parameters are necessary to define the permeability variation in the external region.

- `ei_attachouterspace()` marks all selected block labels as members of the external region used for modeling unbounded axisymmetric problems via the Kelvin Transformation.
- `ei_detachouterspace()` undefines all selected block labels as members of the external region used for modeling unbounded axisymmetric problems via the Kelvin Transformation.
- `ei_attachdefault()` marks the selected block label as the default block label. This block label is applied to any region that has not been explicitly labeled.
- `ei_detachdefault()` undefines the default attribute for the selected block labels.
- `ei_makeABC(n,R,x,y,bc)` creates a series of circular shells that emulate the impedance of an unbounded domain (i.e. an Improvised Asymptotic Boundary Condition). The `n` parameter contains the number of shells to be used (should be between 1 and 12), `R` is the radius of the solution domain, and `(x,y)` denotes the center of the solution domain. The `bc` parameter should be specified as 0 for a Dirichlet outer edge or 1 for a Neumann outer edge. If the function is called without all the parameters, the function makes up reasonable values for the missing parameters.
- `ei_smartmesh(state)` Turns on or off smart meshing for the current document. Set `state` to `1` to enable smart meshing or `0` to disable it.
- `ei_setcomment("comment")` Sets the comment/description string associated with the current problem.
- `ei_getboundingbox()` Returns four values: `xmin, xmax, ymin, ymax`, the bounding box of the current geometry.
- `ei_gettitle()` Returns the file path of the current document as a string.

3.9 Electrostatics Post Processor Scripting Command Set

There are a number of scripting commands designed to operate in the postprocessor. As with the preprocessor commands, each name carries exactly one underscore, the one after the two-letter prefix.

3.9.1 Data Extraction Commands

- `eo_lineintegral(type)` Calculate the line integral for the defined contour

`type`	Integral
0	$E \cdot t$
1	$D \cdot n$
2	Contour length
3	Force from stress tensor
4	Torque from stress tensor

This integral returns either 1 or 2 values, depending on the integral type, *e.g.* :

```
Fx, Fy = eo_lineintegral(3)
```

- `eo_blockintegral(type)` Calculate a block integral for the selected blocks

type	Integral
0	Stored Energy
1	Block Cross-section
2	Block Volume
3	Average *D* over the block
4	Average *E* over the block
5	Weighted Stress Tensor Force
6	Weighted Stress Tensor Torque

Returns one or two floating point values as results, *e.g.*:

```
Fx, Fy = eo_blockintegral(4)
```

- `eo_getpointvalues(X,Y)` Get the values associated with the point at x,y The return values, in order, are:

Symbol	Definition
V	Voltage
Dx	x- or r- direction component of displacement
Dy	y- or z- direction component of displacement
Ex	x- or r- direction component of electric field intensity
Ey	y- or z- direction component of electric field intensity
ex	x- or r- direction component of permittivity
ey	y- or z- direction component of permittivity
nrg	electric field energy density

Example: To catch all values at (0.01,0) use

```
V,Dx,Dy,Ex,Ey,ex,ey,nrg= eo_getpointvalues(0.01,0)
```

- `eo_makeplot(PlotType,NumPoints,Filename,FileFormat)` Allows scripted access to the X-Y plot routines. If only PlotType or only PlotType and NumPoints are specified, the command is interpreted as a request to plot the requested plot type to the screen. If, in addition, the Filename parameter is specified, the sampled values are written to that file instead of being plotted, in exactly the format the `Export...` button on the plot window produces: comma-separated values with a header row naming the columns, then one row per point holding the position along the contour followed by one column for each plotted quantity. A fourth FileFormat argument is accepted and ignored; there is only one data format, and no metafile output. Valid entries for PlotType are:

PlotType	Definition
0	V (Voltage)
1	|D| (Magnitude of flux density)
2	D . n (Normal flux density)
3	D . t (Tangential flux density)
4	|E| (Magnitude of field intensity)
5	E . n (Normal field intensity)
6	E . t (Tangential field intensity)

For example, if one wanted to plot V to the screen with 200 points evaluated to make the graph, the command would be:
`eo_makeplot(0,200)`
To write the data to disk instead of plotting it, the command would be of the form:
`eo_makeplot(0,200,"c:temp.txt")`

- `eo_getprobleminfo()` Returns info on problem description. Returns three values: the Problem type (0 for planar and 1 for axisymmetric); the depth assumed for planar problems in units of meters; and the length unit used to draw the geometry represented in units of meters.

- `eo_getconductorproperties("conductor")` Properties are returned for the conductor property named "conductor". Two values are returned: The voltage of the specified conductor, and the charge carried on the specified conductor.

3.9.2 Selection Commands

- `eo_seteditmode(mode)` Sets the mode of the postprocessor to point, contour, or area mode. Valid entries for mode are `"point"`, `"contour"`, and `"area"`.

- `eo_selectblock(x,y)` Select the block that contains point `(x,y)`.

- `eo_groupselectblock(n)` Selects all of the blocks that are labeled by block labels that are members of group n. If no number is specified (*i.e.* `eo_groupselectblock()`), all blocks are selected.

- `eo_selectconductor("name")` Selects all nodes, segments, and arc segments that are part of the conductor specified by the string `("name")`. This command is used to select conductors for the purposes of the "weighted stress tensor" force and torque integrals, where the conductors are points or surfaces, rather than regions (*i.e.* can't be selected with `eo_selectblock`).

- `eo_addcontour(x,y)` Adds a contour point at `(x,y)`. If this is the first point then it starts a contour, if there are existing points the contour runs from the previous point to this point. The `eo_addcontour` command has the same functionality as a right-button-click contour point addition when the program is running in interactive mode.

- `eo_bendcontour(angle,anglestep)` Replaces the straight line formed by the last two points in the contour by an arc that spans angle degrees. The arc is actually composed of many straight lines, each of which is constrained to span no more than anglestep degrees. The `angle` parameter can take on values from -180 to 180 degrees. The `anglestep` parameter must be greater than zero. If there are less than two points defined in the contour, this command is ignored.

- `eo_selectpoint(x,y)` Adds a contour point at the closest input point to `(x,y)`. If the selected point and a previous selected points lie at the ends of an arcsegment, a contour is added that traces along the arcsegment. The `selectpoint` command has the same functionality as the left-button-click contour point selection when the program is running in interactive mode.

- `eo_clearcontour()` Clear a prevously defined contour

- `eo_clearblock()` Clear block selection

3.9.3 Zoom Commands

- `eo_zoomnatural()` Zoom to the natural boundaries of the geometry.

- `eo_zoomin()` Zoom in one level.

- `eo_zoomout()` Zoom out one level.

- `eo_zoom(x1,y1,x2,y2)` Zoom to the window defined by lower left corner `(x1,y1)` and upper right corner `(x2,y2)`.

3.9.4 View Commands

- `eo_showmesh()` Show the mesh.

- `eo_hidemesh()` Hide the mesh.

- `eo_showpoints()` Show the node points from the input geometry.

- `eo_hidepoints()` Hide the node points from the input geometry.

- `eo_smooth("flag")` This function controls whether or not smoothing is applied to the D and E fields which are naturally piece-wise constant over each element. Setting flag equal to `"on"` turns on smoothing, and setting flag to `"off"` turns off smoothing.

- `eo_showgrid()` Show the grid points.

- `eo_hidegrid()` Hide the grid points.

  `eo_gridsnap("flag")` Setting flag to ”on” turns on snap to grid, setting flag to "off" turns off snap to grid.

- `eo_setgrid(density,"type")` Change the grid spacing. The density parameter specifies the space between grid points, and the type parameter is set to `"cart"` for Cartesian coordinates or `"polar"` for polar coordinates.
- `eo_hidedensityplot()` hides the flux density plot.
- `eo_showdensityplot(legend,gscale,type,upper_D,lower_D)` Shows the flux density plot with options:

  `legend` Set to 0 to hide the plot legend or 1 to show the plot legend.

  `gscale` Set to 0 for a colour density plot or 1 for a grey scale density plot.

  `upper_D` Sets the upper display limit for the density plot.

  `lower_D` Sets the lower display limit for the density plot.

  `type` Sets the type of density plot. A value of 0 plots voltage, 1 plots the magnitude of D, and 2 plots the magnitude of E
- `eo_hidecontourplot()` Hides the contour plot.
- `eo_showcontourplot(numcontours,lower_V,upper_V)` shows the V contour plot with options:

  `numcontours` Number of equipotential lines to be plotted.

  `upper_V` Upper limit for contours.

  `lower_V` Lower limit for contours.

  If `numcontours` is -1 all parameters are ignored and default values are used,
  e.g. `eo_showcontourplot(-1)`
- `eo_showvectorplot(type,scalefactor)` controls the display of vectors denoting the field strength and direction. The parameters taken are the `type` of plot, which should be set to 0 for no vector plot, 1 for flux density D, and 2 for field intensity E. The `scalefactor` determines the relative length of the vectors. If the scale is set to 1, the length of the vectors are chosen so that the highest flux density corresponds to a vector that is the same length as the current grid size setting.
- `eo_minimize()` minimizes the active electrostatics output view.
- `eo_maximize()` maximizes the active electrostatics output view.
- `eo_restore()` restores the active electrostatics output view from a minimized or maximized state.
- `eo_resize(width,height)` resizes the active electrostatics output window client area to width $\times$ height.

3.9.5 Miscellaneous

- `eo_close()` close the current postprocessor window.
- `eo_refreshview()` Redraws the current view.
- `eo_reload()` Reloads the solution from disk.
- `eo_setfocus("documentname")` Switches the electrostatics output file upon which postprocessor commands are to act. If more than one electrostatics output file is open at a time, this command selects between them. `documentname` is the name of the desired document as it appears on the window's title bar; the `.res` extension may be omitted.
- `eo_savebitmap("filename")` saves a bitmapped screen shot of the current view to the file specified by `"filename"`. A backslash in the path must be doubled, as in any Python string (*e.g.* `"c:\\temp\\myfemmfile.png"`); a file name containing spaces needs nothing special.
- `eo_savemetafile("filename")` is retained for script compatibility but does nothing: femm-qt has no metafile output. Use `eo_savebitmap` to save a picture of the current view.
- `eo_shownames(flag)` This function allow the user to display or hide the block label names on screen. To hide the block label names, `flag` should be 0. To display the names, the parameter should be set to 1.
- `eo_numnodes()` Returns the number of nodes in the in focus electrostatics output mesh.
- `eo_numelements()` Returns the number of elements in the in focus electrostatics output mesh.
- `eo_getnode(n)` Returns the (x,y) or (r,z) position of the nth mesh node.
- `eo_getelement(n)` Returns the following properties for the `n`th element:
  1. Index of first element node
  2. Index of second element node
  3. Index of third element node
  4. x (or r) coordinate of the element centroid
  5. y (or z) coordinate of the element centroid
  6. element area using the length unit defined for the problem
  7. group number associated with the element

  About the bulk getters. The `*_getall...` functions return the whole mesh in one call instead of one item at a time. What you get back depends on where the script runs. Called from a language binding over the IPC connection, the values arrive as a single flat array and the binding wrapper reshapes them into the matrix described below. Called from a script running inside FEMM itself, the function returns that matrix directly, as a numpy array — there is nothing to reshape. Either way the values, their order and their units are the same.

Mesh node indices are **1-based**, matching `*_getnode` and `*_getelement`. (Note that the element records inside a solution file are 0-based; the scripting interface adds one on the way out.) Quantities that can have a phase — magnetics and current flow at a nonzero frequency — come back complex; everything else comes back real.

- `eo_getallnodes()` Returns the coordinates of *all* mesh nodes as an $N \times 2$ matrix, one row per node: x_k, y_k.
- `eo_getallelements()` Returns data for *all* mesh elements as an $N \times 7$ matrix. Each row holds the same seven fields, in the same order, as `eo_getelement`.
- `eo_getalld()` Returns the electric displacement D for *all* mesh elements as an $N \times 2$ matrix, one row per element: D_x, D_y. The value is constant over each element. Unlike `eo_getpointvalues`, no smoothing is applied and no point search is performed.
- `eo_getallv()` Returns the voltage at *all* mesh nodes as a vector of N values. Note that this is per *node*, whereas `eo_getalld` is per *element*.
- `eo_gettitle()` Returns the file path of the current output document as a string.

3.10 Heat Flow Preprocessor Scripting Command Set

A number of different commands are available in the preprocessor. Command names carry exactly one underscore, the one that follows the two-letter prefix. Original FEMM additionally accepted a longer spelling that broke the remainder of each name up with further underscores; that spelling is not supported.

3.10.1 Object Add/Remove Commands

- `hi_addnode(x,y)` Add a new node at x,y
- `hi_addsegment(x1,y1,x2,y2)` Add a new line segment from node closest to (x1,y1) to node closest to (x2,y2)
- `hi_addblocklabel(x,y)` Add a new block label at (x,y)
- `hi_addarc(x1,y1,x2,y2,angle,maxseg)` Add a new arc segment from the nearest node to (x1,y1) to the nearest node to (x2,y2) with angle 'angle' divided into 'maxseg' segments.
- `hi_drawline(x1,y1,x2,y2)` Adds nodes at (x1,y1) and (x2,y2) and adds a line segment between them. Unlike `hi_addsegment`, which connects the nearest EXISTING nodes, this creates the nodes it needs.
- `hi_drawarc(x1,y1,x2,y2,angle,maxseg)` Adds nodes at (x1,y1) and (x2,y2) and adds an arc segment of the specified angle and discretization between them.
- `hi_drawrectangle(x1,y1,x2,y2)` Adds nodes at the corners of the rectangle with opposite corners (x1,y1) and (x2,y2), then adds the four segments connecting them.

- `hi_deleteselected` Delete all selected objects.
- `hi_deleteselectednodes` Delete selected nodes.
- `hi_deleteselectedlabels` Delete selected block labels.
- `hi_deleteselectedsegments` Delete selected segments.
- `hi_deleteselectedarcsegments` Delete selects arcs.

3.10.2 Geometry Selection Commands

- `hi_clearselected()` Clear all selected nodes, blocks, segments and arc segments.
- `hi_selectsegment(x,y)` Select the line segment closest to (x,y)
- `hi_selectnode(x,y)` Select the node closest to (x,y). Returns the coordinates of the selected node.
- `hi_selectlabel(x,y)` Select the label closet to (x,y). Returns the coordinates of the selected label.
- `hi_selectarcsegment(x,y)` Select the arc segment closest to (x,y)
- `hi_selectgroup(n)` Select the nth group of nodes, segments, arc segments and block labels. This function will clear all previously selected elements and leave the edit mode in 4 (group)
- `hi_selectcircle(x,y,R,editmode)` selects objects within a circle of radius R centered at (x,y). If only x, y, and R paramters are given, the current edit mode is used. If the editmode parameter is used, 0 denotes nodes, 1 denotes segments, 2 denotes block labels, 3 denotes arcs, and 4 specifies that all entity types are to be selected.
- `hi_selectrectangle(x1,y1,x2,y2,editmode)` selects objects within a rectangle defined by points (x1,y1) and (x2,y2). If no editmode parameter is supplied, the current edit mode is used. If the editmode parameter is used, 0 denotes nodes, 1 denotes segments, 2 denotes block labels, 3 denotes arcs, and 4 specifies that all entity types are to be selected.

3.10.3 Object Labeling Commands

- `hi_setnodeprop("propname",groupno, "inconductor")` Set the selected nodes to have the nodal property `"propname"` and group number `groupno`. The `"inconductor"` string specifies which conductor the node belongs to. If the node doesn't belong to a named conductor, this parameter can be set to `"<None>"`.
- `hi_setblockprop("blockname", automesh, meshsize, group)` Set the selected block labels to have the properties:

  Block property `"blockname"`.

automesh: 0 = mesher defers to mesh size constraint defined in meshsize, 1 = mesher automatically chooses the mesh density.

meshsize: size constraint on the mesh in the block marked by this label.

A member of group number group

- hi_setsegmentprop("propname", elementsize, automesh, hide, group, "inconductor") Set the select segments to have:

  Boundary property "propname"

  Local element size along segment no greater than elementsize

  automesh: 0 = mesher defers to the element constraint defined by elementsize, 1 = mesher automatically chooses mesh size along the selected segments

  hide: 0 = not hidden in post-processor, 1 == hidden in post processor

  A member of group number group

  A member of the conductor specified by the string "inconductor". If the segment is not part of a conductor, this parameter can be specified as "<None>".

- hi_setarcsegmentprop(maxsegdeg, "propname", hide, group, "inconductor") Set the selected arc segments to:

  Meshed with elements that span at most maxsegdeg degrees per element

  Boundary property "propname"

  hide: 0 = not hidden in post-processor, 1 == hidden in post processor

  A member of group number group

  A member of the conductor specified by the string "inconductor". If the segment is not part of a conductor, this parameter can be specified as "<None>".

- hi_setgroup(n) Set the group associated of the selected items to n

3.10.4 Problem Commands

- hi_probdef(units,type,precision,(depth),(minangle),(prevsoln),(timestep)) changes the problem definition. The units parameter specifies the units used for measuring length in the problem domain. Valid "units" entries are "inches", "millimeters", "centimeters", "mils", "meters, and "micrometers". Set problemtype to "planar" for a 2-D planar problem, or to "axi" for an axisymmetric problem. The precision parameter dictates the precision required by the solver. For example, entering 1.E-8 requires the RMS of the residual to be less than 10^{-8}. A fourth parameter, representing the depth of the problem in the into-the-page direction for 2-D planar problems, can also be specified for planar problems. A fifth parameter represents the minimum angle constraint sent to the mesh generator. The sixth parameter indicates the solution from the previous time step to be used in time-transient problems. The seventh parameter is the time step assumed for time transient problems.

- `hi_analyze(flag)` runs `hsolv` to solve the problem. The `flag` parameter controls whether the hsolve window is visible or minimized. For a visible window, either specify no value for `flag` or specify 0. For a minimized window, `flag` should be set to 1.
- `hi_loadsolution()` loads and displays the solution corresponding to the current geometry.
- `hi_setfocus("documentname")` Switches the heat flow input file upon which scripting commands are to act. If more than one heat flow input file is being edited at a time, this command can be used to switch between files so that the mutiple files can be operated upon programmatically via Python. `documentname` should contain the name of the desired document as it appears on the window's title bar.
- `hi_saveas("filename")` saves the file with name `"filename"`. Note if you use a path you must use two backslashes *e.g.* `c:\\temp\\myfile.feh`

3.10.5 Mesh Commands

- `hi_createmesh()` runs the mesher to create a mesh. Note that this is not a necessary precursor of performing an analysis, as `hi_analyze()` will make sure the mesh is up to date before running an analysis. The number of elements in the mesh is pushed back as a return value.
- `hi_showmesh()` toggles the flag that shows or hides the mesh.
- `hi_purgemesh()` clears the mesh out of both the screen and memory.

3.10.6 Editing Commands

- `hi_copyrotate(bx, by, angle, copies, (editaction) )`

  `bx, by` – base point for rotation

  `angle` – angle by which the selected objects are incrementally shifted to make each copy. `angle` is measured in degrees.

  `copies` – number of copies to be produced from the selected objects.

  `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group
- `hi_copytranslate(dx, dy, copies, (editaction))`

  `dx,dy` – distance by which the selected objects are incrementally shifted.

  `copies` – number of copies to be produced from the selected objects.

  `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group
- `hi_createradius(x, y, r)` turns a corner located at `(x,y)` into a curve of radius `r`.

- `hi_moverotate(bx,by,shiftangle (editaction))`

  `bx, by` – base point for rotation

  `shiftangle` – angle in degrees by which the selected objects are rotated.

  `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- `hi_movetranslate(dx,dy,(editaction))`

  `dx,dy` – distance by which the selected objects are shifted.

  `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- `hi_scale(bx,by,scalefactor,(editaction))`

  `bx, by` – base point for scaling

  `scalefactor` – a multiplier that determines how much the selected objects are scaled

  `editaction` 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- `hi_mirror(x1,y1,x2,y2,(editaction))` mirror the selected objects about a line passing through the points `(x1,y1)` and `(x2,y2)`. Valid `editaction` entries are 0 for nodes, 1 for lines (segments), 2 for block labels, 3 for arc segments, and 4 for groups.

- `hi_seteditmode(editmode)` Sets the current editmode to:

  `"nodes"` – nodes

  `"segments"` - line segments

  `"arcsegments"` - arc segments

  `"blocks"` - block labels

  `"group"` - selected group

  This command will affect all subsequent uses of the other editing commands, if they are used WITHOUT the `editaction` parameter.

3.10.7 Zoom Commands

- `hi_zoomnatural()` zooms to a "natural" view with sensible extents.

- `hi_zoomout()` zooms out by a factor of 50%.

- `hi_zoomin()` zoom in by a factor of 200%.

- `hi_zoom(x1,y1,x2,y2)` Set the display area to be from the bottom left corner specified by `(x1,y1)` to the top right corner specified by `(x2,y2)`.

3.10.8 View Commands

- `hi_showgrid()` Show the grid points.
- `hi_hidegrid()` Hide the grid points.
- `hi_gridsnap("flag")` Setting flag to ”on” turns on snap to grid, setting flag to "off"turns off snap to grid.
- `hi_setgrid(density,"type")` Change the grid spacing. The density parameter specifies the space between grid points, and the type parameter is set to `"cart"` for Cartesian coordinates or `"polar"` for polar coordinates.
- `hi_refreshview()` Redraws the current view.
- `hi_minimize()` minimizes the active heat flow input view.
- `hi_maximize()` maximizes the active heat flow input view.
- `hi_restore()` restores the active heat flow input view from a minimized or maximized state.
- `hi_resize(width,height)` resizes the active heat flow input window client area to width $\times$ height.

3.10.9 Object Properties

- `hi_getmaterial("materialname")` fetches the material specified by `materialname` from the materials library.
- `hi_addmaterial("materialname", kx, ky, qv, kt)` adds a new material with called `"materialname"` with the material properties:

  `kx` Thermal conductivity in the x- or r-direction.

  `ky` Thermal conductivity in the y- or z-direction.

  `qv` Volume heat generation density in units of W/m^3.

  `kt` Volumetric heat capacity in units of MJ/(m^3*K).
- `hi_addpointprop("pointpropname",Tp,qp)` adds a new point property of name `"pointpropname"` with either a specified temperature `Tp` or a point heat generation density `qp` in units of W/m.
- `hi_addboundprop("boundpropname", BdryFormat, Tset, qs, Tinf, h, beta, TinfRad)` adds a new boundary property with name `"boundpropname"`.
  - For a “Fixed Temperature” type boundary condition, set the `Tset` parameter to the desired temperature and all other parameters to zero.
  - To obtain a “Heat Flux” type boundary condition, set `qs` to be the heat flux density and `BdryFormat` to 1. Set all other parameters to zero.

- To obtain a convection boundary condition, set `h` to the desired heat transfer coefficient and `Tinf` to the desired external temperature and set `BdryFormat` to 2. Heat flux `qs` is also honored with this boundary condition type, i.e. can model convection + flux.
- For a Radiation boundary condition, set `beta` equal to the desired emissivity and `TinfRad` to the desired external temperature and set `BdryFormat` to 3. Also includes a convection contribution and heat , so `h` and `Tinf` that define convection and specified heat flux `qs` are also honored with this BC type. If no `TinfRad` is specified, it is assumed that `Tinf` applies to both convection and radiation portions.
- For a "Periodic" boundary condition, set `BdryFormat` to 4 and set all other parameters to zero.
- For an "Anti-Perodic" boundary condition, set `BdryFormat` to 5 set all other parameters to zero.

- `hi_addconductorprop("conductorname", Tc, qc, conductortype)` adds a new conductor property with name `"conductorname"` with either a prescribed temperature or a prescribed total heat flux. Set the unused property to zero. The `conductortype` parameter is 0 for prescribed heat flux and 1 for prescribed temperature.
- `hi_deletematerial("materialname")` deletes the material named `"materialname"`.
- `hi_deleteboundprop("boundpropname")` deletes the boundary property named `"boundpropname"`.
- `hi_deleteconductor("conductorname")` deletes the conductor named `conductorname`.
- `hi_deletepointprop("pointpropname")` deletes the point property named `"pointpropname"`
- `hi_modifymaterial("BlockName",propnum,value)` This function allows for modification of a material's properties without redefining the entire material (e.g. so that current can be modified from run to run). The material to be modified is specified by `"BlockName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	`BlockName`	Name of the material
1	`kx`	x (or r) direction thermal conductivity
2	`ky`	y (or z) direction thermal conductivity
3	`qs`	Volume heat generation
4	`kt`	Volumetric heat capacity

- `hi_modifyboundprop("BdryName",propnum,value)` This function allows for modification of a boundary property. The BC to be modified is specified by `"BdryName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	BdryName	Name of boundary property
1	BdryFormat	Type of boundary condition: 0 = Prescribed temperature 1 = Heat Flux 2 = Convection 3 = Radiation 4 = Periodic 5 = Antiperiodic
2	Tset	Fixed Temperature
3	qs	Prescribed heat flux density
4	Tinf	External temperature for convection
5	h	Heat transfer coefficient
6	beta	Emissivity
7	TinfRad	External temperature for radiation

- hi_modifypointprop("PointName",propnum,value) This function allows for modification of a point property. The point property to be modified is specified by "PointName". The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	PointName	Name of the point property
1	Tp	Prescribed nodal temperature
2	qp	Point heat generation in W/m

- hi_modifyconductorprop("ConductorName",propnum,value) This function allows for modification of a conductor property. The conductor property to be modified is specified by "ConductorName". The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	ConductorName	Name of the conductor property
1	Tc	Conductor temperature
2	qc	Total conductor heat flux
3	ConductorType	0 = Prescribed heat flow, 1 = Prescribed temperature

- hi_addtkpoint("materialname",T,k) adds the point (T,k) to the thermal conductivity vs. temperature curve for the material specified by "materialname".

- hi_cleartkpoints("materialname") erases all of the thermal conductivity points that have been defined for the material named "materialname".

3.10.10 Miscellaneous

- hi_savebitmap("filename") saves a bitmapped screenshot of the current view to the file specified by "filename". As with hi_saveas, a backslash in the path must be doubled.

- `hi_savemetafile("filename")` is retained for script compatibility but does nothing: femm-qt has no metafile output. Use `hi_savebitmap` to save a picture of the current view.
- `hi_refreshview()` Redraws the current view.
- `hi_close()` closes the preprocessor window and destroys the current document.
- `hi_shownames(flag)` This function allow the user to display or hide the block label names on screen. To hide the block label names, `flag` should be 0. To display the names, the parameter should be set to 1.
- `hi_readdxf("filename")` This function imports a dxf file specified by `"filename"`.
- `hi_savedxf("filename")` This function saves geometry information in a dxf file specified by `"filename"`.
- `hi_defineouterspace(Zo,Ro,Ri)` defines an axisymmetric external region to be used in conjuction with the Kelvin Transformation method of modeling unbounded problems. The `Zo` parameter is the z-location of the origin of the outer region, the `Ro` parameter is the radius of the outer region, and the `Ri` parameter is the radius of the inner region (*i.e.* the region of interest). In the exterior region, the permeability varies as a function of distance from the origin of the external region. These parameters are necessary to define the permeability variation in the external region.
- `hi_attachouterspace()` marks all selected block labels as members of the external region used for modeling unbounded axisymmetric problems via the Kelvin Transformation.
- `hi_detachouterspace()` undefines all selected block labels as members of the external region used for modeling unbounded axisymmetric problems via the Kelvin Transformation.
- `hi_attachdefault()` marks the selected block label as the default block label. This block label is applied to any region that has not been explicitly labeled.
- `hi_detachdefault()` undefines the default attribute for the selected block labels.
- `hi_makeABC(n,R,x,y,bc)` creates a series of circular shells that emulate the impedance of an unbounded domain (i.e. an Improvised Asymptotic Boundary Condition). The `n` parameter contains the number of shells to be used (should be between 1 and 12), `R` is the radius of the solution domain, and `(x,y)` denotes the center of the solution domain. The `bc` parameter should be specified as 0 for a Dirichlet outer edge or 1 for a Neumann outer edge. If the function is called without all the parameters, the function makes up reasonableÂăvaluesÂăforÂătheÂămissingÂăparameters.Âă
- `hi_smartmesh(state)` Turns on or off smart meshing for the current document. Set `state` to `1` to enable smart meshing or `0` to disable it.
- `hi_setcomment("comment")` Sets the comment/description string associated with the current problem.

- `hi_getboundingbox()` Returns four values: `xmin, xmax, ymin, ymax`, the bounding box of the current geometry.
- `hi_gettitle()` Returns the file path of the current document as a string.

3.11 Heat Flow Post Processor Scripting Command Set

There are a number of scripting commands designed to operate in the postprocessor. As with the preprocessor commands, each name carries exactly one underscore, the one after the two-letter prefix.

3.11.1 Data Extraction Commands

- `ho_lineintegral(type)` Calculate the line integral for the defined contour

`type`	Integral
0	Temperature difference ($G \cdot t$)
1	Heat flux through the contour ($F \cdot n$)
2	Contour length
3	Average Temperature

This integral returns either 1 or 2 values, depending on the integral type, *e.g.* :

```
Ftot, Favg = ho_lineintegral(2)
```

- `ho_blockintegral(type)` Calculate a block integral for the selected blocks

`type`	Integral
0	Average T over the block
1	Block Cross-section
2	Block Volume
3	Average F over the block
4	Average G over the block

Returns one or two floating point values as results, *e.g.*:

```
Gx, Gy = ho_blockintegral(4)
```

- `ho_getpointvalues(X,Y)` Get the values associated with the point at x,y The return values, in order, are:

`Symbol`	Definition
`V`	Temperature
`Fx`	x- or r- direction component of heat flux density
`Fy`	y- or z- direction component of heat flux density
`Gx`	x- or r- direction component of temperature gradient
`Gy`	y- or z- direction component of temperature gradient
`kx`	x- or r- direction component of thermal conductivity
`ky`	y- or z- direction component of thermal conductivity

Example: To catch all values at (0.01,0) use

```
T,Fx,Fy,Gx,Gy,kx,ky= ho_getpointvalues(0.01,0)
```

- `ho_makeplot(PlotType,NumPoints,Filename,FileFormat)` Allows scripted access to the X-Y plot routines. If only PlotType or only PlotType and NumPoints are specified, the command is interpreted as a request to plot the requested plot type to the screen. If, in addition, the Filename parameter is specified, the sampled values are written to that file instead of being plotted, in exactly the format the `Export...` button on the plot window produces: comma-separated values with a header row naming the columns, then one row per point holding the position along the contour followed by one column for each plotted quantity. A fourth FileFormat argument is accepted and ignored; there is only one data format, and no metafile output. Valid entries for PlotType are:

PlotType	Definition
0	V (Temperature)
1	|D| (Magnitude of heat flux density)
2	D . n (Normal heat flux density)
3	D . t (Tangential heat flux density)
4	|E| (Magnitude of field intensity)
5	E . n (Normal field intensity)
6	E . t (Tangential field intensity)

For example, if one wanted to plot V to the screen with 200 points evaluated to make the graph, the command would be:
`ho_makeplot(0,200)`
To write the data to disk instead of plotting it, the command would be of the form:
`ho_makeplot(0,200,"c:temp.txt")`

- `ho_getprobleminfo()` Returns info on problem description. Returns three values: the Problem type (0 for planar and 1 for axisymmetric); the depth assumed for planar problems in units of meters; and the length unit used to draw the problem in meters.

- `ho_getconductorproperties("conductor")` Properties are returned for the conductor property named "conductor". Two values are returned: The temperature of the specified conductor, and the total heat flux through the specified conductor.

3.11.2 Selection Commands

- `ho_seteditmode(mode)` Sets the mode of the postprocessor to point, contour, or area mode. Valid entries for mode are `"point"`, `"contour"`, and `"area"`.

- `ho_selectblock(x,y)` Select the block that contains point `(x,y)`.

- `ho_groupselectblock(n)` Selects all of the blocks that are labeled by block labels that are members of group n. If no number is specified (*i.e.* `ho_groupselectblock()`), all blocks are selected.

- `ho_selectconductor("name")` Selects all nodes, segments, and arc segments that are part of the conductor specified by the string `("name")`. This command is used to select conductors for the purposes of the "weighted stress tensor" force and torque integrals, where the conductors are points or surfaces, rather than regions (*i.e.* can't be selected with `ho_selectblock`).

- `ho_addcontour(x,y)` Adds a contour point at `(x,y)`. If this is the first point then it starts a contour, if there are existing points the contour runs from the previous point to this point. The `ho_addcontour` command has the same functionality as a right-button-click contour point addition when the program is running in interactive mode.

- `ho_bendcontour(angle,anglestep)` Replaces the straight line formed by the last two points in the contour by an arc that spans angle degrees. The arc is actually composed of many straight lines, each of which is constrained to span no more than anglestep degrees. The `angle` parameter can take on values from -180 to 180 degrees. The `anglestep` parameter must be greater than zero. If there are less than two points defined in the contour, this command is ignored.

- `ho_selectpoint(x,y)` Adds a contour point at the closest input point to `(x,y)`. If the selected point and a previous selected points lie at the ends of an arcsegment, a contour is added that traces along the arcsegment. The `selectpoint` command has the same functionality as the left-button-click contour point selection when the program is running in interactive mode.

- `ho_clearcontour()` Clear a prevously defined contour

- `ho_clearblock()` Clear block selection

3.11.3 Zoom Commands

- `ho_zoomnatural()` Zoom to the natural boundaries of the geometry.

- `ho_zoomin()` Zoom in one level.

- `ho_zoomout()` Zoom out one level.

- `ho_zoom(x1,y1,x2,y2)` Zoom to the window defined by lower left corner `(x1,y1)` and upper right corner `(x2,y2)`.

3.11.4 View Commands

- `ho_showmesh()` Show the mesh.

- `ho_hidemesh()` Hide the mesh.

- `ho_showpoints()` Show the node points from the input geometry.

- `ho_hidepoints()` Hide the node points from the input geometry.

- `ho_smooth("flag")` This function controls whether or not smoothing is applied to the F and G fields which are naturally piece-wise constant over each element. Setting flag equal to `"on"` turns on smoothing, and setting flag to `"off"` turns off smoothing.
- `ho_showgrid()` Show the grid points.
- `ho_hidegrid()` Hide the grid points.

  `ho_gridsnap("flag")` Setting flag to ”on” turns on snap to grid, setting flag to "off" turns off snap to grid.
- `ho_setgrid(density,"type")` Change the grid spacing. The density parameter specifies the space between grid points, and the type parameter is set to `"cart"` for Cartesian coordinates or `"polar"` for polar coordinates.
- `ho_hidedensityplot()` hides the heat flux density plot.
- `ho_showdensityplot(legend,gscale,type,upper,lower)` Shows the heat flux density plot with options:

  `legend` Set to 0 to hide the plot legend or 1 to show the plot legend.

  `gscale` Set to 0 for a colour density plot or 1 for a grey scale density plot.

  `upper` Sets the upper display limit for the density plot.

  `lower` Sets the lower display limit for the density plot.

  `type` Sets the type of density plot. A value of 0 plots temperature, 1 plots the magnitude of F, and 2 plots the magnitude of G
- `ho_hidecontourplot()` Hides the contour plot.
- `ho_showcontourplot(numcontours,lower_V,upper_V)` shows the V contour plot with options:

  `numcontours` Number of equipotential lines to be plotted.

  `upper_V` Upper limit for contours.

  `lower_V` Lower limit for contours.

  If `numcontours` is -1 all parameters are ignored and default values are used,
  e.g. `ho_showcontourplot(-1)`
- `ho_showvectorplot(type,scalefactor)` controls the display of vectors denoting the field strength and direction. The parameters taken are the `type` of plot, which should be set to 0 for no vector plot, 1 for heat flux density F, and 2 for temperature gradient G. The `scalefactor` determines the relative length of the vectors. If the scale is set to 1, the length of the vectors are chosen so that the highest flux density corresponds to a vector that is the same length as the current grid size setting.
- `ho_minimize()` minimizes the active heat flow input view.
- `ho_maximize()` maximizes the active heat flow input view.

- `ho_restore()` restores the active heat flow input view from a minimized or maximized state.
- `ho_resize(width,height)` resizes the active heat flow input window client area to width $\times$ height.

3.11.5 Miscellaneous

- `ho_close()` close the current postprocessor window.
- `ho_refreshview()` Redraws the current view.
- `ho_reload()` Reloads the solution from disk.
- `ho_setfocus("documentname")` Switches the heat flow output file upon which post-processor commands are to act. If more than one heat flow output file is open at a time, this command selects between them. `documentname` is the name of the desired document as it appears on the window's title bar; the `.anh` extension may be omitted.
- `ho_savebitmap("filename")` saves a bitmapped screen shot of the current view to the file specified by `"filename"`. A backslash in the path must be doubled, as in any Python string (*e.g.* `"c:\\temp\\myfemmfile.png"`); a file name containing spaces needs nothing special.
- `ho_savemetafile("filename")` is retained for script compatibility but does nothing: femm-qt has no metafile output. Use `ho_savebitmap` to save a picture of the current view.
- `ho_shownames(flag)` This function allow the user to display or hide the block label names on screen. To hide the block label names, `flag` should be 0. To display the names, the parameter should be set to 1.
- `ho_numnodes()` Returns the number of nodes in the in focus heat flow output mesh.
- `ho_numelements()` Returns the number of elements in the in focus heat flow output mesh.
- `ho_getnode(n)` Returns the (x,y) or (r,z) position of the nth mesh node.
- `ho_getelement(n)` Returns the following properties for the `n`th element:
  1. Index of first element node
  2. Index of second element node
  3. Index of third element node
  4. x (or r) coordinate of the element centroid
  5. y (or z) coordinate of the element centroid
  6. element area using the length unit defined for the problem
  7. group number associated with the element

About the bulk getters. The `*_getall...` functions return the whole mesh in one call instead of one item at a time. What you get back depends on where the script runs. Called from a language binding over the IPC connection, the values arrive as a single flat array and the binding wrapper reshapes them into the matrix described below. Called from a script running inside FEMM itself, the function returns that matrix directly, as a numpy array — there is nothing to reshape. Either way the values, their order and their units are the same. Mesh node indices are **1-based**, matching `*_getnode` and `*_getelement`. (Note that the element records inside a solution file are 0-based; the scripting interface adds one on the way out.) Quantities that can have a phase — magnetics and current flow at a nonzero frequency — come back complex; everything else comes back real.

- `ho_getallnodes()` Returns the coordinates of *all* mesh nodes as an $N \times 2$ matrix, one row per node: x_k, y_k. Much faster than calling `ho_getnode` in a loop.
- `ho_getallelements()` Returns data for *all* mesh elements as an $N \times 7$ matrix. Each row holds the same seven fields, in the same order, as `ho_getelement`: the three node indices, the centroid coordinates, the element area and the group number.
- `ho_getallf()` Returns the heat flux density F for *all* mesh elements as an $N \times 2$ matrix, one row per element: F_x, F_y. The value is constant over each element. Unlike `ho_getpointvalues`, no smoothing is applied and no point search is performed.
- `ho_getallt()` Returns the temperature at *all* mesh nodes as a vector of N values. Note that this is per *node*, whereas `ho_getallf` is per *element*.
- `ho_gettitle()` Returns the file path of the current output document as a string.

3.12 Current Flow Preprocessor Scripting Command Set

A number of different commands are available in the preprocessor. Command names carry exactly one underscore, the one that follows the two-letter prefix. Original FEMM additionally accepted a longer spelling that broke the remainder of each name up with further underscores; that spelling is not supported.

3.12.1 Object Add/Remove Commands

- `ci_addnode(x,y)` Add a new node at x,y
- `ci_addsegment(x1,y1,x2,y2)` Add a new line segment from node closest to (x1,y1) to node closest to (x2,y2)
- `ci_addblocklabel(x,y)` Add a new block label at (x,y)
- `ci_addarc(x1,y1,x2,y2,angle,maxseg)` Add a new arc segment from the nearest node to (x1,y1) to the nearest node to (x2,y2) with angle 'angle' divided into 'maxseg' segments.

- `ci_drawline(x1,y1,x2,y2)` Adds nodes at (x1,y1) and (x2,y2) and adds a line segment between them. Unlike `ci_addsegment`, which connects the nearest EXISTING nodes, this creates the nodes it needs.
- `ci_drawarc(x1,y1,x2,y2,angle,maxseg)` Adds nodes at (x1,y1) and (x2,y2) and adds an arc segment of the specified angle and discretization between them.
- `ci_drawrectangle(x1,y1,x2,y2)` Adds nodes at the corners of the rectangle with opposite corners (x1,y1) and (x2,y2), then adds the four segments connecting them.
- `ci_deleteselected` Delete all selected objects.
- `ci_deleteselectednodes` Delete selected nodes.
- `ci_deleteselectedlabels` Delete selected block labels.
- `ci_deleteselectedsegments` Delete selected segments.
- `ci_deleteselectedarcsegments` Delete selects arcs.

3.12.2 Geometry Selection Commands

- `ci_clearselected()` Clear all selected nodes, blocks, segments and arc segments.
- `ci_selectsegment(x,y)` Select the line segment closest to (x,y)
- `ci_selectnode(x,y)` Select the node closest to (x,y). Returns the coordinates of the selected node.
- `ci_selectlabel(x,y)` Select the label closet to (x,y). Returns the coordinates of the selected label.
- `ci_selectarcsegment(x,y)` Select the arc segment closest to (x,y)
- `ci_selectgroup(n)` Select the nth group of nodes, segments, arc segments and block labels. This function will clear all previously selected elements and leave the edit mode in 4 (group)
- `ci_selectcircle(x,y,R,editmode)` selects objects within a circle of radius R centered at (x,y). If only x, y, and R paramters are given, the current edit mode is used. If the editmode parameter is used, 0 denotes nodes, 1 denotes segments, 2 denotes block labels, 3 denotes arcs, and 4 specifies that all entity types are to be selected.
- `ci_selectrectangle(x1,y1,x2,y2,editmode)` selects objects within a rectangle defined by points (x1,y1) and (x2,y2). If no editmode parameter is supplied, the current edit mode is used. If the editmode parameter is used, 0 denotes nodes, 1 denotes segments, 2 denotes block labels, 3 denotes arcs, and 4 specifies that all entity types are to be selected.

3.12.3 Object Labeling Commands

- `ci_setnodeprop("propname",groupno, "inconductor")` Set the selected nodes to have the nodal property `"propname"` and group number `groupno`. The `"inconductor"` string specifies which conductor the node belongs to. If the node doesn't belong to a named conductor, this parameter can be set to `"<None>"`.

- `ci_setblockprop("blockname", automesh, meshsize, group)` Set the selected block labels to have the properties:

  Block property `"blockname"`.

  `automesh`: 0 = mesher defers to mesh size constraint defined in `meshsize`, 1 = mesher automatically chooses the mesh density.

  `meshsize`: size constraint on the mesh in the block marked by this label.

  A member of group number `group`

- `ci_setsegmentprop("propname", elementsize, automesh, hide, group, "inconductor",)` Set the select segments to have:

  Boundary property `"propname"`

  Local element size along segment no greater than `elementsize`

  `automesh`: 0 = mesher defers to the element constraint defined by `elementsize`, 1 = mesher automatically chooses mesh size along the selected segments

  `hide`: 0 = not hidden in post-processor, 1 == hidden in post processor

  A member of group number `group`

  A member of the conductor specified by the string `"inconductor"`. If the segment is not part of a conductor, this parameter can be specified as `"<None>"`.

- `ci_setarcsegmentprop(maxsegdeg, "propname", hide, group, "inconductor")` Set the selected arc segments to:

  Meshed with elements that span at most `maxsegdeg` degrees per element

  Boundary property `"propname"`

  `hide`: 0 = not hidden in post-processor, 1 == hidden in post processor

  A member of group number `group`

  A member of the conductor specified by the string `"inconductor"`. If the segment is not part of a conductor, this parameter can be specified as `"<None>"`.

- `ci_setgroup(n)` Set the group associated of the selected items to n

3.12.4 Problem Commands

- `ci_probdef(units,type,frequency,precision,(depth),(minangle))` changes the problem definition. The `units` parameter specifies the units used for measuring length in the problem domain. Valid `"units"` entries are `"inches"`, `"millimeters"`, `"centimeters"`, `"mils"`, `"meters`, and `"micrometers"`. Set `problemtype` to `"planar"` for a 2-D planar problem, or to `"axi"` for an axisymmetric problem. The `frequency` parameter specifies the frequency in Hz at which the analysis ois to be performed. The `precision` parameter dictates the precision required by the solver. For example, entering `1.E-8` requires the RMS of the residual to be less than 10^{-8}. A fourth parameter, representing the depth of the problem in the into-the-page direction for 2-D planar problems, can also be specified for planar problems. A sixth parameter represents the minimum angle constraint sent to the mesh generator.

- `ci_analyze(flag)` runs `belasolv` to solve the problem. The `flag` parameter controls whether the Belasolve window is visible or minimized. For a visible window, either specify no value for `flag` or specify 0. For a minimized window, `flag` should be set to 1.

- `ci_loadsolution()` loads and displays the solution corresponding to the current geometry.

- `ci_setfocus("documentname")` Switches the current flow input file upon which scripting commands are to act. If more than one current flow input file is being edited at a time, this command can be used to switch between files so that the multiple files can be operated upon programmatically via Python. `documentname` should contain the name of the desired document as it appears on the window's title bar.

- `ci_saveas("filename")` saves the file with name `"filename"`. Note if you use a path you must use two backslashes *e.g.* `c:\\temp\\myfemmfile.fec`

3.12.5 Mesh Commands

- `ci_createmesh()` runs the mesher to create a mesh. Note that this is not a necessary precursor of performing an analysis, as `ci_analyze()` will make sure the mesh is up to date before running an analysis. The number of elements in the mesh is pushed back as a return value.

- `ci_showmesh()` toggles the flag that shows or hides the mesh.

- `ci_purgemesh()` clears the mesh out of both the screen and memory.

3.12.6 Editing Commands

- `ci_copyrotate(bx, by, angle, copies, (editaction) )`

  `bx, by` – base point for rotation

  `angle` – angle by which the selected objects are incrementally shifted to make each copy. `angle` is measured in degrees.

copies – number of copies to be produced from the selected objects.

editaction 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- ci_copytranslate(dx, dy, copies, (editaction))

  dx,dy – distance by which the selected objects are incrementally shifted.

  copies – number of copies to be produced from the selected objects.

  editaction 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- ci_createradius(x, y, r) turns a corner located at (x,y) into a curve of radius r.

- ci_moverotate(bx,by,shiftangle (editaction))

  bx, by – base point for rotation

  shiftangle – angle in degrees by which the selected objects are rotated.

  editaction 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- ci_movetranslate(dx,dy,(editaction))

  dx,dy – distance by which the selected objects are shifted.

  editaction 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- ci_scale(bx,by,scalefactor,(editaction))

  bx, by – base point for scaling

  scalefactor – a multiplier that determines how much the selected objects are scaled

  editaction 0 –nodes, 1 – lines (segments), 2 –block labels, 3 – arc segments, 4- group

- ci_mirror(x1,y1,x2,y2,(editaction)) mirror the selected objects about a line passing through the points (x1,y1) and (x2,y2). Valid editaction entries are 0 for nodes, 1 for lines (segments), 2 for block labels, 3 for arc segments, and 4 for groups.

- ci_seteditmode(editmode) Sets the current editmode to:

  "nodes" – nodes

  "segments" - line segments

  "arcsegments" - arc segments

  "blocks" - block labels

  "group" - selected group

  This command will affect all subsequent uses of the other editing commands, if they are used WITHOUT the editaction parameter.

3.12.7 Zoom Commands

- `ci_zoomnatural()` zooms to a “natural” view with sensible extents.
- `ci_zoomout()` zooms out by a factor of 50%.
- `ci_zoomin()` zoom in by a factor of 200%.
- `ci_zoom(x1,y1,x2,y2)` Set the display area to be from the bottom left corner specified by `(x1,y1)` to the top right corner specified by `(x2,y2)`.

3.12.8 View Commands

- `ci_showgrid()` Show the grid points.
- `ci_hidegrid()` Hide the grid points.
- `ci_gridsnap("flag")` Setting flag to ”on” turns on snap to grid, setting flag to "off"turns off snap to grid.
- `ci_setgrid(density,"type")` Change the grid spacing. The density parameter specifies the space between grid points, and the type parameter is set to `"cart"` for Cartesian coordinates or `"polar"` for polar coordinates.
- `ci_refreshview()` Redraws the current view.
- `ci_minimize()` minimizes the active current flow input view.
- `ci_maximize()` maximizes the active current flow input view.
- `ci_restore()` restores the active current flow input view from a minimized or maximized state.
- `ci_resize(width,height)` resizes the active current flow input window client area to width $\times$ height.

3.12.9 Object Properties

- `ci_getmaterial("materialname")` fetches the material specified by `materialname` from the materials library.
- `ci_addmaterial("materialname", ox, oy, ex, ey, ltx, lty)` adds a new material with called `"materialname"` with the material properties:

  `ox` Electrical conductivity in the x- or r-direction in units of S/m.

  `oy` Electrical conductivity in the y- or z-direction in units of S/m.

  `ex` Relative permittivity in the x- or r-direction.

  `ey` Relative permittivity in the y- or z-direction.

`ltx` Dielectric loss tangent in the x- or r-direction.

`lty` Dielectric loss tangent in the y- or z-direction.

- `ci_addpointprop("pointpropname",Vp,qp)` adds a new point property of name `"pointpropname"` with either a specified potential `Vp` a point current density `qp` in units of A/m.
- `ci_addboundprop("boundpropname", Vs, qs, c0, c1, BdryFormat)` adds a new boundary property with name `"boundpropname"`

  For a "Fixed Voltage" type boundary condition, set the `Vs` parameter to the desired voltage and all other parameters to zero.

  To obtain a "Mixed" type boundary condition, set `C1` and `C0` as required and `BdryFormat` to 1. Set all other parameters to zero.

  To obtain a prescribes surface current density, set `qs` to the desired current density in A/m^2 and set `BdryFormat` to 2.

  For a "Periodic" boundary condition, set `BdryFormat` to 3 and set all other parameters to zero.

  For an "Anti-Perodic" boundary condition, set `BdryFormat` to 4 set all other parameters to zero.

- `ci_addconductorprop("conductorname", Vc, qc, conductortype)` adds a new conductor property with name `"conductorname"` with either a prescribed voltage or a prescribed total current. Set the unused property to zero. The `conductortype` parameter is 0 for prescribed current and 1 for prescribed voltage.
- `ci_deletematerial("materialname")` deletes the material named `"materialname"`.
- `ci_deleteboundprop("boundpropname")` deletes the boundary property named `"boundpropname"`.
- `ci_deleteconductor("conductorname")` deletes the conductor named `conductorname`.
- `ci_deletepointprop("pointpropname")` deletes the point property named `"pointpropname"`
- `ci_modifymaterial("BlockName",propnum,value)` This function allows for modification of a material's properties without redefining the entire material (e.g. so that current can be modified from run to run). The material to be modified is specified by `"BlockName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	`BlockName`	Name of the material
1	`ox`	x (or r) direction conductivity
2	`oy`	y (or z) direction conductivity
3	`ex`	x (or r) direction relative permittivity
4	`ey`	y (or z) direction relative permittivity
5	`ltx`	x (or r) direction dielectric loss tangent
6	`lty`	y (or z) direction dielectric loss tangent

- `ci_modifyboundprop("BdryName",propnum,value)` This function allows for modification of a boundary property. The BC to be modified is specified by `"BdryName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	BdryName	Name of boundary property
1	Vs	Fixed Voltage
2	qs	Prescribed current density
3	c0	Mixed BC parameter
4	c1	Mixed BC parameter
5	BdryFormat	Type of boundary condition: 0 = Prescribed V 1 = Mixed 2 = Surface current density 3 = Periodic 4 = Antiperiodic

- `ci_modifypointprop("PointName",propnum,value)` This function allows for modification of a point property. The point property to be modified is specified by `"PointName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	PointName	Name of the point property
1	Vp	Prescribed nodal voltage
2	qp	Point current density in A/m

- `ci_modifyconductorprop("ConductorName",propnum,value)` This function allows for modification of a conductor property. The conductor property to be modified is specified by `"ConductorName"`. The next parameter is the number of the property to be set. The last number is the value to be applied to the specified property. The various properties that can be modified are listed below:

propnum	Symbol	Description
0	ConductorName	Name of the conductor property
1	Vc	Conductor voltage
2	qc	Total conductor current
3	ConductorType	0 = Prescribed current, 1 = Prescribed voltage

3.12.10 Miscellaneous

- `ci_savebitmap("filename")` saves a bitmapped screenshot of the current view to the file specified by `"filename"`. As with `ci_saveas`, a backslash in the path must be doubled.

- `ci_savemetafile("filename")` is retained for script compatibility but does nothing: femm-qt has no metafile output. Use `ci_savebitmap` to save a picture of the current view.

- `ci_refreshview()` Redraws the current view.
- `ci_close()` closes the preprocessor window and destroys the current document.
- `ci_shownames(flag)` This function allow the user to display or hide the block label names on screen. To hide the block label names, `flag` should be 0. To display the names, the parameter should be set to 1.
- `ci_readdxf("filename")` This function imports a dxf file specified by `"filename"`.
- `ci_savedxf("filename")` This function saves geometry information in a dxf file specified by `"filename"`.
- `ci_defineouterspace(Zo,Ro,Ri)` defines an axisymmetric external region to be used in conjuction with the Kelvin Transformation method of modeling unbounded problems. The `Zo` parameter is the z-location of the origin of the outer region, the `Ro` parameter is the radius of the outer region, and the `Ri` parameter is the radius of the inner region (*i.e.* the region of interest). In the exterior region, the permeability varies as a function of distance from the origin of the external region. These parameters are necessary to define the permeability variation in the external region.
- `ci_attachouterspace()` marks all selected block labels as members of the external region used for modeling unbounded axisymmetric problems via the Kelvin Transformation.
- `ci_detachouterspace()` undefines all selected block labels as members of the external region used for modeling unbounded axisymmetric problems via the Kelvin Transformation.
- `ci_attachdefault()` marks the selected block label as the default block label. This block label is applied to any region that has not been explicitly labeled.
- `ci_detachdefault()` undefines the default attribute for the selected block labels.
- `ci_makeABC(n,R,x,y,bc)` creates a series of circular shells that emulate the impedance of an unbounded domain (i.e. an Improvised Asymptotic Boundary Condition). The `n` parameter contains the number of shells to be used (should be between 1 and 12), `R` is the radius of the solution domain, and `(x,y)` denotes the center of the solution domain. The `bc` parameter should be specified as 0 for a Dirichlet outer edge or 1 for a Neumann outer edge. If the function is called without all the parameters, the function makes up reasonable values for the missing parameters.
- `ci_smartmesh(state)` Turns on or off smart meshing for the current document. Set `state` to `1` to enable smart meshing or `0` to disable it.
- `ci_setcomment("comment")` Sets the comment/description string associated with the current problem.
- `ci_getboundingbox()` Returns four values: `xmin, xmax, ymin, ymax`, the bounding box of the current geometry.
- `ci_gettitle()` Returns the file path of the current document as a string.

3.13 Current Flow Post Processor Scripting Command Set

There are a number of scripting commands designed to operate in the postprocessor. As with the preprocessor commands, each name carries exactly one underscore, the one after the two-letter prefix.

3.13.1 Data Extraction Commands

- `co_lineintegral(type)` Calculate the line integral for the defined contour

`type`	Integral
0	$E \cdot t$
1	$J \cdot n$
2	Contour length
3	Average voltage over contour
4	Force from stress tensor
5	Torque from stress tensor

  This integral returns either 1 or 2 values, depending on the integral type, *e.g.* :

```
Fx, Fy = co_lineintegral(4)
```

- `co_blockintegral(type)` Calculate a block integral for the selected blocks

`type`	Integral
0	Real Power
1	Reactive Power
2	Apparent Power
3	Time-Average Stored Energy
4	Block cross-section area
5	Block volume
6	x (or r) direction Weighted Stress Tensor force, DC component
7	y (or z) direction Weighted Stress Tensor force, DC component
8	x (or r) direction Weighted Stress Tensor force, 2x frequency component
9	y (or z) direction Weighted Stress Tensor force, 2x frequency component
10	Weighted Stress Tensor torque, DC component
11	Weighted Stress Tensor torque, 2x frequency component

  Returns a value that can be complex, as necessary.

- `co_getpointvalues(X,Y)` Get the values associated with the point at x,y The return values, in order, are:

Symbol	Definition
V	Voltage
Jx	x- or r- direction component of current density
Jy	y- or z- direction component of current density
Kx	x- or r- direction component of AC conductivity
Ky	y- or z- direction component of AC conductivity
Ex	x- or r- direction component of electric field intensity
Ey	y- or z- direction component of electric field intensity
ex	x- or r- direction component of permittivity
ey	y- or z- direction component of permittivity
Jdx	x- or r- direction component of displacement current density
Jdy	y- or z- direction component of displacement current density
ox	x- or r- direction component of permittivity
oy	y- or z- direction component of permittivity
Jcx	x- or r- direction component of conduction current density
Jcy	y- or z- direction component of conduction current density

- co_makeplot(PlotType,NumPoints,Filename,FileFormat) Allows scripted access to the X-Y plot routines. If only PlotType or only PlotType and NumPoints are specified, the command is interpreted as a request to plot the requested plot type to the screen. If, in addition, the Filename parameter is specified, the sampled values are written to that file instead of being plotted, in exactly the format the Export... button on the plot window produces: comma-separated values with a header row naming the columns, then one row per point holding the position along the contour followed by one column for each plotted quantity. A fourth FileFormat argument is accepted and ignored; there is only one data format, and no metafile output. Valid entries for PlotType are:

PlotType	Definition
0	V (Voltage)
1	|J| (Magnitude of current density)
2	J . n (Normal current density)
3	J . t (Tangential current density)
4	|E| (Magnitude of field intensity)
5	E . n (Normal field intensity)
6	E . t (Tangential field intensity)
7	|Jc| (Magnitude of conduction current density)
8	Jc . n (Normal conduction current density)
9	Jc . t (Tangential conduction current density)
10	|Jd| (Magnitude of displacement current density)
11	Jd . n (Normal displacement current density)
12	Jd . t (Tangential displacement current density)

For example, if one wanted to plot V to the screen with 200 points evaluated to make the graph, the command would be:

```
co_makeplot(0,200)
```

To write the data to disk instead of plotting it, the command would be of the form:

```
co_makeplot(0,200,"c:temp.txt")
```

- `co_getprobleminfo()` Returns info on problem description. Returns four values:

Return value	Definition
1	problem type
2	frequency in Hz
3	depth assumed for planar problems in meters.
4	length unit used to draw the problem, represented in meters

- `co_getconductorproperties("conductor")` Properties are returned for the conductor property named "conductor". Two values are returned: The voltage of the specified conductor, and the current on the specified conductor.

3.13.2 Selection Commands

- `co_seteditmode(mode)` Sets the mode of the postprocessor to point, contour, or area mode. Valid entries for mode are `"point"`, `"contour"`, and `"area"`.

- `co_selectblock(x,y)` Select the block that contains point `(x,y)`.

- `co_groupselectblock(n)` Selects all of the blocks that are labeled by block labels that are members of group n. If no number is specified (*i.e.* `co_groupselectblock()`), all blocks are selected.

- `co_selectconductor("name")` Selects all nodes, segments, and arc segments that are part of the conductor specified by the string `("name")`. This command is used to select conductors for the purposes of the "weighted stress tensor" force and torque integrals, where the conductors are points or surfaces, rather than regions (*i.e.* can't be selected with `co_selectblock`).

- `co_addcontour(x,y)` Adds a contour point at `(x,y)`. If this is the first point then it starts a contour, if there are existing points the contour runs from the previous point to this point. The `co_addcontour` command has the same functionality as a right-button-click contour point addition when the program is running in interactive mode.

- `co_bendcontour(angle,anglestep)` Replaces the straight line formed by the last two points in the contour by an arc that spans angle degrees. The arc is actually composed of many straight lines, each of which is constrained to span no more than anglestep degrees. The `angle` parameter can take on values from -180 to 180 degrees. The `anglestep` parameter must be greater than zero. If there are less than two points defined in the contour, this command is ignored.

- `co_selectpoint(x,y)` Adds a contour point at the closest input point to `(x,y)`. If the selected point and a previous selected points lie at the ends of an arcsegment, a contour is added that traces along the arcsegment. The `selectpoint` command has the same functionality as the left-button-click contour point selection when the program is running in interactive mode.

- `co_clearcontour()` Clear a prevously defined contour

- co_clearblock() Clear block selection

3.13.3 Zoom Commands

- co_zoomnatural() Zoom to the natural boundaries of the geometry.
- co_zoomin() Zoom in one level.
- co_zoomout() Zoom out one level.
- co_zoom(x1,y1,x2,y2) Zoom to the window defined by lower left corner (x1,y1) and upper right corner (x2,y2).

3.13.4 View Commands

- co_showmesh() Show the mesh.
- co_hidemesh() Hide the mesh.
- co_showpoints() Show the node points from the input geometry.
- co_hidepoints() Hide the node points from the input geometry.
- co_smooth("flag") This function controls whether or not smoothing is applied to the D and E fields which are naturally piece-wise constant over each element. Setting flag equal to "on" turns on smoothing, and setting flag to "off" turns off smoothing.
- co_showgrid() Show the grid points.
- co_hidegrid() Hide the grid points.

  co_gridsnap("flag") Setting flag to ”on” turns on snap to grid, setting flag to "off" turns off snap to grid.
- co_setgrid(density,"type") Change the grid spacing. The density parameter specifies the space between grid points, and the type parameter is set to "cart" for Cartesian coordinates or "polar" for polar coordinates.
- co_hidedensityplot() hides the current density plot.
- co_showdensityplot(legend,gscale,type,upper,lower) Shows the current density plot with options:

  legend Set to 0 to hide the plot legend or 1 to show the plot legend.

  gscale Set to 0 for a colour density plot or 1 for a grey scale density plot.

  upper Sets the upper display limit for the density plot.

  lower Sets the lower display limit for the density plot.

  type Sets the type of density plot. Specific choices for the type of density plot include:

type	Description
0	$\|V\|$
1	$\|\mathrm{Re}(V)\|$
2	$\|\mathrm{Im}(V)\|$
3	$\|J\|$
4	$\|\mathrm{Re}(J)\|$
5	$\|\mathrm{Im}(J)\|$
6	$\|E\|$
7	$\|\mathrm{Re}(E)\|$
8	$\|\mathrm{Im}(E)\|$

- `co_hidecontourplot()` Hides the contour plot.
- `co_showcontourplot(numcontours,lower_V,upper_V),type` shows the V contour plot with options:

  `numcontours` Number of equipotential lines to be plotted;

  `upper_V` Upper limit for contours;

  `lower_V` Lower limit for contours;

  `type` the type of contour plot to be rendered.

  If `numcontours` is -1 all parameters are ignored and default values are used,
  e.g. `co_showcontourplot(-1)`

  The type can take on the values of `"real"`, `"imag"`, or `"both"`, denoting the real part of voltage, the imaginary part of voltage, or both components of voltage.
- `co_showvectorplot(type,scalefactor)` controls the display of vectors denoting the field strength and direction. The `type` parameter can take on the following values:

type	Description
0	No vector plot
1	$\mathrm{Re}(J)$
2	$\mathrm{Re}(E)$
3	$\mathrm{Im}(J)$
4	$\mathrm{Im}(E)$
5	$\mathrm{Re}(J)$ and $Im(J)$
6	$\mathrm{Re}(E)$ and $Im(E)$

  The `scalefactor` determines the relative length of the vectors.
  If the scale is set to 1, the length of the vectors are chosen so that the highest field magnitude corresponds to a vector that is the same length as the current grid size setting.
- `co_minimize()` minimizes the active current flow output view.
- `co_maximize()` maximizes the active current flow output view.
- `co_restore()` restores the active current flow output view from a minimized or maximized state.
- `co_resize(width,height)` resizes the active current flow output window client area to width $\times$ height.

3.13.5 Miscellaneous

- `co_close()` close the current postprocessor window.
- `co_refreshview()` Redraws the current view.
- `co_reload()` Reloads the solution from disk.
- `co_setfocus("documentname")` Switches the current flow output file upon which postprocessor commands are to act. If more than one current flow output file is open at a time, this command selects between them. `documentname` is the name of the desired document as it appears on the window's title bar; the `.anc` extension may be omitted.
- `co_savebitmap("filename")` saves a bitmapped screen shot of the current view to the file specified by `"filename"`. A backslash in the path must be doubled, as in any Python string (*e.g.* `"c:\\temp\\myfemmfile.png"`); a file name containing spaces needs nothing special.
- `co_savemetafile("filename")` is retained for script compatibility but does nothing: femm-qt has no metafile output. Use `co_savebitmap` to save a picture of the current view.
- `co_shownames(flag)` This function allow the user to display or hide the block label names on screen. To hide the block label names, `flag` should be 0. To display the names, the parameter should be set to 1.
- `co_numnodes()` Returns the number of nodes in the in focus current flow output mesh.
- `co_numelements()` Returns the number of elements in the in focus current flow output mesh.
- `co_getnode(n)` Returns the (x,y) or (r,z) position of the nth mesh node.
- `co_getelement(n)` Returns the following properties for the `n`th element:
  1. Index of first element node
  2. Index of second element node
  3. Index of third element node
  4. x (or r) coordinate of the element centroid
  5. y (or z) coordinate of the element centroid
  6. element area using the length unit defined for the problem
  7. group number associated with the element

  About the bulk getters. The `*_getall...` functions return the whole mesh in one call instead of one item at a time. What you get back depends on where the script runs. Called from a language binding over the IPC connection, the values arrive as a single flat array and the binding wrapper reshapes them into the matrix described below. Called from a script running inside FEMM itself, the function returns that matrix directly, as a numpy array — there is nothing to reshape. Either way the values, their order and their units are the same.

Mesh node indices are **1-based**, matching `*_getnode` and `*_getelement`. (Note that the element records inside a solution file are 0-based; the scripting interface adds one on the way out.) Quantities that can have a phase — magnetics and current flow at a nonzero frequency — come back complex; everything else comes back real.

- `co_getallnodes()` Returns the coordinates of *all* mesh nodes as an $N \times 2$ matrix, one row per node: x_k, y_k. Much faster than calling `co_getnode` in a loop.
- `co_getallelements()` Returns data for *all* mesh elements as an $N \times 7$ matrix. Each row holds the same seven fields, in the same order, as `co_getelement`: the three node indices, the centroid coordinates, the element area and the group number.
- `co_getallj()` Returns the current density J for *all* mesh elements as an $N \times 2$ matrix, one row per element: J_x, J_y. The value is constant over each element. At a nonzero frequency the entries are complex. Unlike `co_getpointvalues`, no smoothing is applied and no point search is performed.
- `co_getallv()` Returns the voltage at *all* mesh nodes as a vector of N values, complex at a nonzero frequency. Note that this is per *node*, whereas `co_getallj` is per *element*.
- `co_gettitle()` Returns the file path of the current output document as a string.

Chapter 4

Plotting with matplotlib

FEMM's scripting engine can drive `matplotlib`, so a script can produce a publication-quality figure directly from a solution:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.plot([0, 1, 2], [0, 1, 4], 'r-')
ax.set_xlabel("x")
plt.show()
```

A real, interactive window appears — with the usual zoom, pan and save toolbar — and it stays alive after the script ends, so you can keep working with it.

FEMM does not contain `matplotlib` and does not install it. The figure is drawn by whatever CPython you already have on the machine, running as a separate process that FEMM starts on demand and talks to over a local socket. Nothing is sent anywhere off the machine, and nothing is installed by using this feature.

4.1 What you need

A CPython 3.9 or newer with `matplotlib`, and a window toolkit for it to draw with. On Windows and macOS that usually means just

```
pip install matplotlib
```

because the installers from `python.org` already include Tk, which is the toolkit `matplotlib` falls back on. Nothing else is needed.

If Tk is absent — Homebrew's Python omits it, and most Linux distributions package it separately — install it alongside: `brew install python-tk` on macOS, `python3-tk` on Debian and Ubuntu, `py311-tkinter` on FreeBSD. On FreeBSD use `pkg` rather than `pip` for `matplotlib` as well, since there are no prebuilt wheels there and `pip` would try to compile it from source.

`PyQt6` or `PySide6` are *optional*. If one of them is installed FEMM prefers it, giving the Qt window style instead of the Tk one; the plots are identical either way.

FEMM finds a suitable interpreter by itself the first time a script plots something. If none of the candidates will do, the error says which were tried and what was wrong with each:

```
BridgeError: no usable Python found for the matplotlib bridge.
Tried: python.exe (has no interactive matplotlib backend)
       python3.exe (could not be launched)
Install it with: pip install matplotlib. The python.org installer
already provides the Tk toolkit it needs; PyQt6 is optional and only
changes the window style
```

To point FEMM at a particular interpreter — if you keep several, or use a virtual environment — set it under **Edit** → **Scripting Preferences**, or from a script:

```
import pybridge
pybridge.set_interpreter("C:/Users/me/venv/Scripts/python.exe")
```

The setting is remembered. The `FEMMQT_PYTHON` environment variable does the same thing for a single run.

4.2 Which plot command to use

FEMM has a built-in `plot()` for quick *x*–*y* curves — it takes an *x* array and up to seven *y* arrays, with optional labels, and opens a plain plot window. It needs nothing installed, appears instantly, and is the right tool for a look at a sweep while you are working.

Reach for `matplotlib` when you want what `plot()` deliberately does not do: contour and density plots, subplots, log axes, annotations, colour bars, control over fonts and sizes, and vector output for a paper.

4.3 Plotting a solution on its own mesh

The example `examples/plot-mesh-matplotlib.py` solves a model, reads its finite-element mesh back, and draws the vector potential as a filled contour on the actual triangulation rather than on a resampled grid. The core of it is:

```
import numpy as np, pybridge

nn, ne = int(mo_numnodes()), int(mo_numelements())

x = np.zeros(nn)
y = np.zeros(nn)
a = np.zeros(nn)
for k in range(1, nn + 1):
    nx, ny = mo_getnode(k)
    x[k-1], y[k-1] = nx, ny
```

```
    a[k-1] = mo_getpointvalues(nx, ny)[0]

tri = np.zeros((ne, 3), dtype=np.int32)
for k in range(1, ne + 1):
    e = mo_getelement(k)
    tri[k-1][0] = int(e[0]) - 1     # FEMM numbers nodes from 1
    tri[k-1][1] = int(e[1]) - 1
    tri[k-1][2] = int(e[2]) - 1

plt = pybridge.import_module("matplotlib.pyplot")
fig, ax = plt.subplots()
ax.tricontourf(x, y, tri, a, levels=40)
ax.set_aspect("equal")
plt.show()
```

On a 73,000-node, 146,000-element model that reads back in about 1.4 seconds and draws in about 0.6.

Two details in that fragment matter more than they look.

Keep bulk numbers in arrays, not lists

`x`, `y` and `a` are `numpy` arrays that are allocated once and filled in place. The obvious alternative — appending to a Python list — is much slower, and not only for the list itself. A list of 73,000 floats is 73,000 separate objects on the interpreter's heap, and the garbage collector walks all of them every time anything else allocates. Measured on the mesh above, a loop that reads 20,000 elements takes

with the node data in a `numpy` array	1.9 ms
with the same data in a Python list	69.7 ms

and the gap widens as the mesh grows. The array also crosses to `matplotlib` as raw bytes rather than element by element, so the transfer costs about 60 ms for eight megabytes.

Use `int32` for indices

The triangle array is declared `dtype=np.int32`. FEMM's `numpy` is `ulab`, whose next-widest integer is `int16`, and a mesh passes its 32,767 limit almost immediately — silently, with the index wrapping to a negative number. `int32` holds a little over two billion, which is enough for any mesh, and it halves the bytes compared with storing indices as floats.

4.4 Other packages

The bridge is not specific to `matplotlib`, and it does not insist on it either: if the Python it finds has no plotting backend, FEMM will still use it for everything else, and only a request for a figure fails. Anything importable by the CPython it found can be used:

```
import pybridge
csv = pybridge.import_module("csv")

with pybridge.open("results.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["angle", "torque"])
    w.writerows(data)
```

Note `pybridge.open()`: the file is opened by the *other* process, so that a CPython reader or writer gets a file object it can actually use. Both processes see the same filesystem, so the path means the same thing to both.

Spreadsheets are the other common want, and `xlsxwriter` is close enough to universal that it gets the same treatment as `matplotlib`: just import it. Formats, charts and all.

```
import xlsxwriter
wb = xlsxwriter.Workbook("sweep.xlsx")
ws = wb.add_worksheet("torque")
bold = wb.add_format({"bold": True})

ws.write_row(0, 0, ["angle", "torque"], bold)
ws.write_column(1, 0, angle)          # a whole column in one call
ws.write_column(1, 1, torque)
wb.close()
```

Note `write_column` rather than a loop over `write`. Every call is a round trip, so writing 720 cells one at a time costs about 85 ms where two column writes cost one — the same rule as everywhere else here, which is to cross in bulk rather than per item.

Archives are the third: `zipfile` is forwarded too, so bundling up a run's outputs is the code you would write anywhere.

```
import zipfile
with zipfile.ZipFile("run.zip", "w", zipfile.ZIP_DEFLATED) as z:
    z.write("coil.fem")
    z.write("coil.ans")
    z.writestr("notes.txt", "swept 0 to 45 degrees")
```

Note that these are *paths*: both processes see the same filesystem, so the worker opens the files itself and nothing large crosses between them.

Finally `pandas`, for anyone who would rather group and pivot a sweep than index it by hand. It is forwarded like the others, but unlike them it is not usually installed — if it is missing, the error names the interpreter to install it into.

```
import pandas as pd
df = pd.DataFrame({"angle": angle, "torque": torque})
print(df["torque"].describe())
peak = df[df.torque > 0.9 * df.torque.max()]
```

A data frame lives in the other process, so a single number comes back as a number — `float(df["torque"].mean())` is an ordinary float — but a whole column stays put. To work on one with FEMM's own `numpy`, fetch it:

```
arr = pybridge.fetch(df["torque"].to_numpy())   # a numpy array, here
np.mean(arr)
```

Going the other way is automatic: a FEMM array handed to `pd.DataFrame` crosses in bulk. The usual rule applies with force — summing a 50,000-row column in the worker takes under a millisecond and fetching those 50,000 numbers takes two, but reading them one at a time with `.iloc[i]` takes the better part of a second.

What crosses, and what does not

Numbers, strings, arrays and containers of those cross by value; anything else crosses as a proxy, and calling a method on a proxy just works. There are two things that cannot cross at all, and both are worth knowing before you meet them:

- **An open file** belongs to the process that opened it. `Workbook("sweep.xlsx")` above is fine, because that is a *path* — the worker opens the file itself. But handing a file *object* to a CPython library cannot work:

  ```
  f = open("sweep.xlsx", "wb")        # FEMM's file
  wb = xlsxwriter.Workbook(f)         # error: a FileIO cannot cross
  ```

  Use `pybridge.open(path, mode)` instead, which opens the file in the worker and hands back something CPython can write to. The reverse direction works as well: FEMM's own `csv` module will happily write to a `pybridge.open()` file.

- **A function** defined in your script cannot be passed as a callback. Define it in the worker with `pybridge.exec()` and refer to it by name.

Both raise immediately and say which of the two remedies applies, so this is something you find out at the call rather than in the output.

Useful entry points, all in the `pybridge` module:

- `pybridge.import_module("name")` — any CPython module, as a proxy.
- `pybridge.eval("expr")`, `pybridge.exec("statements")` — evaluate or run code in the other process. The namespace persists between calls and starts with `np` (CPython's real `numpy`) and `plt` already imported.
- `pybridge.put("name", value)` — send an array or list across and bind it to a name there. This is the efficient way to move a lot of data.
- `pybridge.fetch(obj)` — bring a value back by value; an array returns as a `numpy` array here.

- `pybridge.info()` — which interpreter, which versions, which plotting backend is in use.
- `pybridge.restart()`, `pybridge.shutdown()` — discard the helper process and its windows, or stop it. It restarts by itself on next use.

4.5 Which `numpy`, which `scipy`

Inside FEMM, `numpy` and `scipy` are built in. They run in the same process, with no data copied anywhere, and they are what `import numpy` gives you — not the CPython packages of the same name. For almost all post-processing they are the right answer, and reaching across the bridge for them would be a straight loss.

They cover more than the word "built-in" suggests: some 115 `numpy` names, including a complete `numpy.linalg` (`solve`, `inv`, `det`, `eig`, `qr`, `svd`, `pinv`, `cond`, `rank`, `cholesky`, `norm`, all of them working on complex matrices), `numpy.fft`, `polyfit` and `polyval`, `interp`, `convolve`, `trapz`, `loadtxt` and `savetxt`; and from `scipy`, `integrate.quad`, `optimize.bisect`, `optimize.newton`, `optimize.fmin`, `signal.sosfilt` and the `special` functions.

What they do not have, and where the bridge earns its keep:

- **Curve fitting** beyond polynomials — there is no `curve_fit` or `least_squares`.
- **Spline interpolation** — `interp` is linear only.
- **Differential equations** — no `odeint` or `solve_ivp`.
- **Spectral estimation and filter design** — `sosfilt` applies a filter but nothing designs one, and there is no `welch` or `periodogram`.
- **Statistics** — no `scipy.stats` at all.

For those, ask for the CPython package by name:

```
import pybridge

pybridge.put("f", frequency)          # arrays computed in FEMM
pybridge.put("z", impedance)
pybridge.exec("from scipy.interpolate import CubicSpline")
pybridge.exec("s = CubicSpline(f, z)")
fine = pybridge.fetch(pybridge.eval("s(np.linspace(0, 1000, 2000))"))
```

Note that there is no `import scipy.interpolate` shim, deliberately. The longer spelling is a reminder that the call is leaving FEMM: each one is a round trip to the other process, which is nothing when you call it once on a finished array and a great deal if you call it inside a loop over elements. Doing the arithmetic with the built-in `numpy` and crossing once, at the end, is almost always the right shape.

The other reason to keep the two straight is that a FEMM-side function cannot cross. This does not work:

```
def residual(x):                        # a function defined here
    return measured - model(x)
opt.least_squares(residual, x0)         # BridgeError
```

because the optimiser would have to call back into FEMM. Define the function on the far side with `pybridge.exec` and it works normally; or keep the loop here, using the built-in `scipy.optimize.fmin` or `newton`, and call FEMM from inside it. The second is usually what you want anyway when each evaluation is a finite-element solve.

4.6 Limitations

- **Functions cannot cross.** A FEMM-side function cannot be passed to CPython, so there is no `key=` argument to `sorted`, no callback to `FuncAnimation`, and no interactive widget or pick event. Define such a function in the other process with `pybridge.exec` instead.
- `numpy` **and** `scipy` **stay local.** Those names refer to FEMM's own built-in array library, which runs in-process and is much faster for ordinary work than sending each operation across. When you specifically want CPython's `numpy`, ask for it: `pybridge.import_module("numpy")`, or use the `np` already present in the `eval` namespace.
- **Figures belong to the other process.** They appear in their own windows and cannot be embedded in a FEMM window.
- **One call at a time.** While a plotting call is in progress FEMM will refuse a second one rather than let two overlap. This can only arise if a script is somehow re-entered while it waits.
- Pressing **Esc** during a plotting call that has hung will abandon it. The helper process is discarded, and the next plotting command starts a fresh one; any figures it was holding are lost.

Chapter 5

Mathematica Interface

FEMM can be driven from Mathematica using the `MathFEMM` package, which controls a FEMM process over a TCP/IP socket. (Earlier versions used Mathematica's MathLink API; the current package uses the same TCP interface as all of the other language bindings — see the *TCP/IP Interface* chapter.) The package takes care of launching FEMM, establishing the connection, sending commands, and parsing the results, so that FEMM is controlled directly through a set of Mathematica functions.

To use the package, load it and open a connection:

```
Get["mathfemm.m"]
OpenFEMM[]
```

`OpenFEMM[]` connects to a FEMM process, launching one if necessary. By default it starts the interactive shell (`femm_gui`) and uses port 48015. Pass `OpenFEMM[1]` to launch the headless engine (`femm_headless`) instead, with no window, or `OpenFEMM[bHide, port]` to choose both the visibility and the port.

If a FEMM process is already listening on that port, `OpenFEMM` attaches to it rather than starting a second one. That is what allows a notebook to be re-run after an aborted script without leaving a trail of orphaned FEMM windows, and it means the `bHide` argument has no effect when an instance is already running.

Once connected, FEMM is controlled through the Mathematica functions provided by the package, which mirror the Python command set documented in the scripting chapters. For example, the magnetics preprocessor commands appear as `MIAddNode[x,y]`, `MIAddSegment[x1,y1,x2,y2]`, `MIAddBlockLabel[x,y]`, and so on. Each function sends the corresponding command to FEMM's Python interpreter and returns the result as a Mathematica expression — a number, a string, or a list, as appropriate — so values can be used directly in further Mathematica computation.

When finished, close the connection with:

```
CloseFEMM[]
```

Chapter 6

TCP/IP Interface

Beyond the supplied language bindings, FEMM can be controlled by any program that can open a TCP/IP socket. A running FEMM process exposes its Python interpreter over a socket, so an external client can send commands and read back results programmatically. (This replaces the ActiveX/COM automation server used by older, Windows-only versions of FEMM.)

A FEMM process listens for a connection on a TCP port; the default is 48015. The headless engine is started for this purpose with, for example:

```
femm_headless --port 48015
```

The interactive shell `femm_gui` also serves the same protocol while displaying its window. The connection is single-client: one program controls a given FEMM process at a time, and a second program that connects while the first is still attached is refused with an error rather than queued.

The process exits when its client disconnects, so that a binding which is killed does not leave an orphaned FEMM behind. A consequence worth knowing: a script that wants to leave the window up for inspection when it finishes must keep its connection open, rather than closing it or calling `closefemm`.

The protocol is a simple line-based text exchange. The client sends a command — any statement or expression accepted by FEMM's Python interpreter — as a single newline-terminated line, and FEMM replies with exactly one newline-terminated line:

- If the command is an expression, the reply is its value rendered as text. Real numbers are reported in full precision; a complex number is returned in the form `a+I*b`; and a tuple of results is returned with the individual items separated by tab characters.
- A statement that produces no value (an assignment, a drawing command, and the like) replies with an empty line.
- If the command raises an error, the reply is a single line beginning with `error:` followed by the error message.

Two details matter when writing a client. Values are rendered for the wire rather than as Python would print them: a boolean is `true` or `false` rather than `True` or `False`, and a string is returned unquoted. And only the value of the command reaches the socket — anything the command prints,

and the traceback of an error, go to FEMM's own output instead, so a client cannot recover them from the reply.

All of the supplied toolboxes are thin wrappers over this protocol, each presenting it as native function calls in its host language: `OctaveFEMM` (MATLAB and Octave), `MathFEMM` (Mathematica), `pyfemm` (Python), and equivalent packages for Julia, Scilab, and Lua. Those packages serve both as ready-to-use interfaces and as worked examples of how to speak the protocol from a new client.

Chapter 7

Numerical Methods

For those of you interested in what's going on behind the scenes in the solvers, this section is meant as a brief description of the methods and techniques used by FEMM. References are cited as applicable.

7.1 Finite Element Formulation

All elements were derived using variational formulations (based on minimizing energy, as opposed to Galerkin, least squares residual, and so on). Explanations of the variational approach for 2-D planar problems with first-order triangle elements are widely available in the literature ([10] in particular was referred to during the creation of FEMM). The axisymmetric electrostatics solver also uses the approach laid out in [10].

The axisymmetric case for magnetics, however, is oddly less well addressed. Hoole [2] and Silvester [11] promote solving axisymmetric problems in terms of a modified vector potential. The advantage of the modified vector potential is that closed-form expressions for each term in the element matrices can be formed. An early version of FEMM used this technique, but it is observed to yield relatively larger errors near $r = 0$. With the modified potential formulation, it is also nontrivial to compute the average flux density associated with each element. However, the formulation suggested by Henrotte in [12] gives good performance close to $r = 0$. FEMM uses a formulation that is very similar to Henrotte for axisymmetric magnetic problems.

7.2 Linear Solvers

For all problems, Krylov subspace methods are used. These techniques are appropriate for the sort of problem that FEMM solves, because the matrices are symmetric and very sparse. During assembly, only the diagonal and the upper triangle are stored, as an array of sparse rows, so that late structural edits—circuit equations, periodic boundary merges—stay cheap. Before the first solve, that is flattened into a compressed-sparse-row copy of the full matrix, and it is the flat copy that the matrix-vector product reads.

For magnetostatic, electrostatic, heat flow and current flow problems, the preconditioned conjugate gradient (PCG) algorithm is used, based on the discussion in [11]. Minor modifications are made to this algorithm to avoid computing certain quantities more than once per iteration. Circuit

constraints are the exception: a prescribed flux linkage or a prescribed total current adds a Lagrange multiplier row, which leaves the matrix symmetric but no longer positive definite. In that case the quasi-minimum residual (QMR) algorithm is used in place of PCG.

Earlier versions of FEMM preconditioned with Symmetric Successive Over-Relaxation (SSOR), as described in [13], chosen because it can be built on the fly from matrix elements that are already in storage. The alternative that Silvester promotes, an incomplete Cholesky factorization, was avoided because a factorization with the same sparsity pattern as the matrix nearly doubles the storage requirement.

FEMM now uses an incomplete LDL^T factorization with threshold-based fill. Each column of the factor keeps only those entries larger than a fixed fraction of the largest entry in the corresponding column of the original matrix, subject to a cap on how many are kept, so the storage is bounded and modest rather than a duplicate of the matrix. The factorization is root-free—L has a unit diagonal and D carries the sign—so one implementation serves both the positive definite case and the indefinite circuit-constrained case, and no square roots are taken. It is built once per assembly and reused by every iteration solved against it.

That the fill is what matters, rather than the incomplete factorization in itself, is worth stating plainly: a zero-fill incomplete factorization was measured to perform about as well as the SSOR sweep it would have replaced, which is the classically expected result, since the two are closely related. Allowing a bounded amount of fill is what pays for itself, cutting the iteration count by roughly an order of magnitude — far more than enough to cover the cost of the triangular solves.

For harmonic problems, the regular PCG algorithm cannot be used; the matrix that arises in the formulation of harmonic problems is Complex Symmetric (*i.e.* $A = A^T$), rather than Hermitian (*i.e.* $A = A^*$). Curiously, there is not much literature available on iterative solvers for complex symmetric problems, given the number of diverse applications in which these problems arise. However, there is a very good paper on the solution of linear problems with complex symmetric matrices via various flavors of Conjugate Gradient by Freund [14]. The techniques discussed by Freund allow one to operate directly on the complex symmetric matrix and take advantage of the symmetric structure to minimize the number of computations that must be performed per iteration. Although Freund supports the Quasi-Minimum Residual approach, FEMM uses the complex symmetric version of biconjugate gradient also described in [14]. After coding and comparing the speed of both BCG and QMR, it was found that BCG is somewhat faster due to a relatively smaller number of computations that must be performed per iteration (even though QMR has better convergence properties than BCG). The complex symmetric BCG is preconditioned with the same incomplete LDL^T factorization used for the real-valued problems, built in exactly the same way. Nonlinear harmonic problems add a Newton iteration on top of this solver.

In all problems, the unknowns are numbered so that the nonzero entries cluster near the diagonal. A good ordering reduces the difference between the incomplete factorization and the exact inverse, and so speeds convergence. FEMM uses the reverse Cuthill-McKee ordering described in [2]. Unlike earlier versions, which renumbered inside the solver as a hold-over from a banded Gauss elimination solver, the ordering is now produced by the mesh generator: the solver receives a mesh that is already numbered and does no renumbering of its own. An interesting paper on the effect of the ordering of the unknowns on convergence in conjugate gradient methods is [15].

The solvers run on a single thread by design. Multi-threading was implemented and then removed once the incomplete factorization was adopted: measured across problems ranging from ten thousand to about a hundred and fifty thousand nodes, four threads changed run time by be-

tween -11% and $+4\%$. The factorization and the triangular solves are inherently sequential, and the better preconditioner had already cut the iteration count by about a factor of ten, which left too little vector work to amortize the cost of starting and synchronizing threads. Running sequentially has the additional benefit that results are bit-for-bit reproducible from one run to the next, which is worth a good deal when regression testing.

7.3 Field Smoothing

Since first-order triangles are used by FEMM, the resulting solution for B and H obtained by differentiating A is constant over each element. If the raw B and H are used by the postprocessor, density plots of B and 2-D plots of field quantities along user-defined contours look terrible. The values of B and H also are not so accurate at points in an element away from the element's centroid.

The use of smoothing to recover the accuracy lost by differentiating A is known as *superconvergence*. Of the greatest interest to FEMM are so-called "patch recovery" techniques. The basic idea is that the solutions for B are most accurate at the centroid of the triangular element (known as its *Gauss Point*). One desires a continuous profile of B that can be interpolated from nodal values, in the same way that vector potential A can be represented. The problem is, the "raw" solution of B is multivalued at any node point, those values being the different constant values of B in each element surrounding the node point. The general approach to estimating the "true" value of B at any node point is to fit a least-squares plane through the values of B at the Gauss points of all elements that surround a node of interest, and to take the value of the plane at the node point's location as its smoothed value of B [16].

However, this approach to patch recovery has a lot of shortcomings. For the rather irregular meshes that can arise in finite elements, the least-squares fit problem can be ill-conditioned, or even singular, at some nodes in the finite element mesh. Furthermore, the superconvergence solution can actually be less accurate than the piece-wise constant solution in the neighborhood of boundaries and interfaces.

One can note that the patch recovery method is merely a weighted average of the flux densities in all of the elements surrounding a given node. Instead of a least-squares fit, FEMM simply weights the values of flux density in each adjacent element's Gauss point with a value inversely proportional to the distance from the Gauss point to the node point of interest. Away from boundaries, the results seem to be nearly as good as a least-squares fit. At boundaries and interfaces, the smoothed solution is no worse than the unsmoothed solution.

A related approach is used for smoothing D and E in electrostatics problems. In electrostatics problems, however, nodal values of D are found by taking the gradient of a best-fit plane through the voltage of the neighboring nodes. A number of special cases must be caught so that reasonable results are obtained at various boundaries and surfaces.

Bibliography

[1] M. Plonus, *Applied electromagnetics*. McGraw-Hill, 1978.

[2] S. R. Hoole, *Computer-aided analysis and design of electromagnetic devices*, Elsevier, 1989.

[3] J. D. Jackson, *Classical electrodynamics,* 2nd *ed*, Wiley, 1975.

[4] F. M. White, *Heat and mass transfer*, Addison-Wesley, 1988.

[5] R. Haberman, *Elementary applied partial differential equations*, Prentice-Hall, 1987.

[6] R. L. Stoll, *The analysis of eddy currents*, Oxford University Press, 1974.

[7] S. McFee, J. P. Webb, and D. A. Lowther, "A tunable volume integration formulation for force calculation in finite-element based computational magnetostatics," IEEE Transactions on Magnetics, 24(1):439-442, January 1988.

[8] F. Henrotte, G. Deliege, and K. Hameyer, "The eggshell method for the computation of electromagnetic forces on rigid bodies in 2D and 3D," CEFC 2002, Perugia, Italy, April 16-18, 2002. (pdf version)

[9] MicroPython documentation, `https://docs.micropython.org`

[10] P. E. Allaire, *Basics of the finite element method*, 1985.

[11] P. P. Silvester, *Finite elements for electrical engineers*, Cambridge University Press, 1990.

[12] F. Henrotte *et al*, "A new method for axisymmetric linear and nonlinear problems," *IEEE Transactions on Magnetics*, MAG-29(2):1352-1355, March 1993.

[13] C. A. Fletcher, *Computational techniques for fluid dynamics*, Springer-Verlag, 1988.

[14] R. W. Freund, "Conjugate gradient-type methods for linear systems with complex symmetric coefficient matrices," SIAM Journal of Scientific and Statistical Computing, 13(1):425-448, January 1992.

[15] E. F. D'Azevedo, P. A. Forsyth, and W. Tang, "Ordering methods for preconditioned conjugate gradient methods applied to unstructured grid problems," SIAM J. Matrix Anal. Appl., 12(4), July 1992.

[16] O. C. Zienkiewicz and J. Z. Zhu, " The superconvergent patch recovery and *a posteriori* estimates, part 1: the recovery technique," International Journal for Numerical Methods in Engineering, 33:1331-1364, 1992.

[17] Q. Chen and A. Konrad, "A review of finite element open boundary techniques for static and quasistatic electromagnetic field problems," *IEEE Transactions on Magnetics*, 33(1):663-676, January 1997.

[18] D. Meeker, "Improvised open boundary conditions for magnetic finite elements," *IEEE Transactions on Magnetics*, 49(10):5243-5247, October 2013. DOI 10.1109/TMAG.2013.2260348.

[19] D. Meeker, "Improvised asymptotic boundary conditions for electrostatic finite elements," *IEEE Transactions on Magnetics*, 50(6), article 7400609, June 2014. DOI 10.1109/TMAG.2014.2300196.

[20] E. M. Freeman and D. A. Lowther, "A novel mapping technique for open boundary finite element solutions to Poissons equation," *IEEE Transactions on Magnetics*, 24(6):2934-2936, November 1988.

[21] D. A. Lowther, E. M. Freeman, and B. Forghani, "A sparse matrix open boundary method for finite element analysis," *IEEE Transactions on Magnetics*, 25(4)2810-2812, July 1989.

[22] E. M. Freeman and D. A. Lowther, "An open boundary technique for axisymmetric and three dimensional magnetic and electric field problems," *IEEE Transactions on Magnetics*, 25(5):4135-4137, September 1989.

[23] A. G. Jack and B. C. Mecrow, "Methods for magnetically nonlinear problems involving significant hysteresis and eddy currents," IEEE Transactions on Magnetics, 26(2):424-429, March 1990.

[24] D. O'Kelly, "Hysteresis and eddy current losses in steel plates with nonlinear magnetization characteristics," Proceedings of the IEE, 119(11):1675-1676, November 1972.

Appendix A

Appendix

A.1 Modeling Permanent Magnets

FEMM accommodates permanent magnets, but there are some special rules associated with properly modeling them. This appendix will explain how to distill enough information from a manufacturer's literature to properly define the material in FEMM.

The manufacturer provides information about their material in the form of a demagnetization curve. A sample curve for Alnico 5 is pictured in Figure A.1. The task is to get the appropriate information out of the curve put in a FEMM Material Properties model.

Magnets can be modeled from several different, but equally valid, points of view. From the perspective finite element analysis, the most useful model is to think of the magnet as a volume of ferromagnetic material surrounded by a thin sheet of current, as shown in Figure A.2. From this

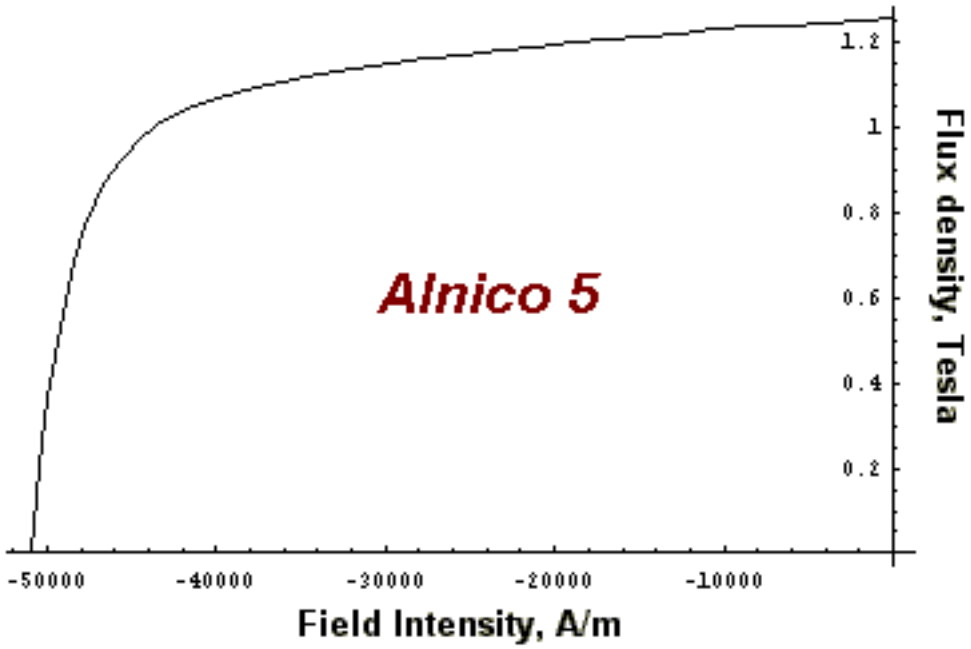


Figure A.1: Sample demagnetization curve for Alnico 5

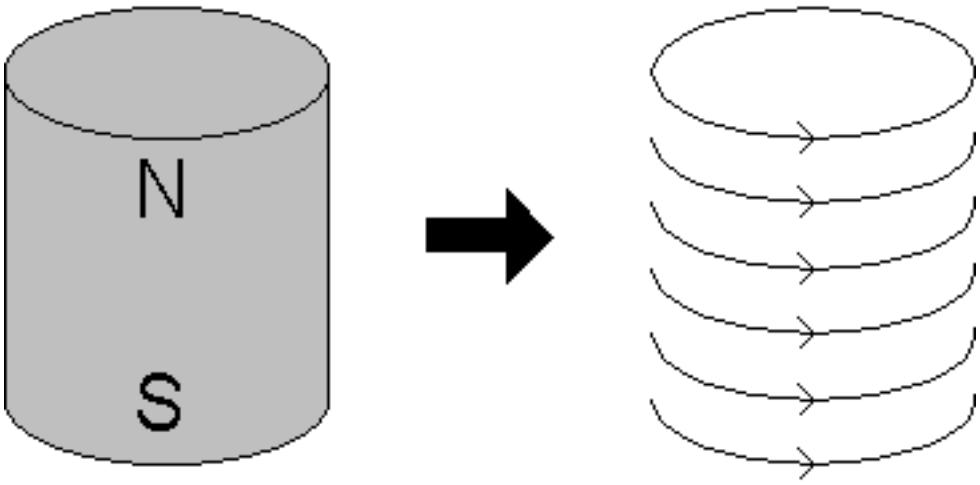


Figure A.2: Magnet as an equivalent current sheet.

point of view, the demagnetization curve is what occurs when different amounts of magnetomotive force are applied to a long magnet, acting in the direction opposing the field of the magnet. When enough MMF is applied so that the field is exactly cancelled out, the applied MMF must be exactly the same as the MMF that is driving the magnet. The B-H profile that is traversed on the way to the $B = 0$ point is just the B-H curve of the material inside the magnet.

Using these insights, the permanent magnet can be modeled. The *coercivity* (denoted H_c) of the magnet is the absolute value of the MMF that it takes to bring the field in the magnet to zero. This value (in units of Amps/Meter) is entered in the `H_C` box in the Material Properties dialog (see Figure 2.9). If the magnet material is nonlinear, the appropriate values to enter in the B-H data dialog can be obtained by shifting the curve to the right by exactly H_c, so that the $B = 0$ point lines up with the origin. For example, the shifted demagnetization curve corresponding to Alnico 5 is pictured in Figure A.3. If the demagnetization curve is straight enough to be considered linear, one can obtain the appropriate permeability by taking the slope of the demagnetization curve.

Strong rare-earth materials at room temperature have a very linear demagnetization curve. Usually, a linear model is sufficient for these materials. In addition, these materials have a relative permeability very close to 1. The modeling of these materials can be simplified (while only incurring small errors) by assuming that the permeability is exactly 1. Then, if you know the energy product of the magnet material in units of MGOe (the unit in which the energy product is almost always given), the appropriate H_c can be determined via (A.1).

$$H_c = \frac{5(10^5)\sqrt{E}}{\pi} \tag{A.1}$$

where E is the energy product in MGOe and the resulting H_c is in units of A/m (*e.g.* 40 MGOe $\approx 10^6$ A/m).

With alnico magnets, great care must be taken in interpreting the finite element results. Unlike rare-earth magnets, these magnets exhibit a great degree of hysteresis when they are demagnetized. That is, when the flux density is pushed below the "knee" in the demagnetization curve, the flux level does not recover to the previous magnitude when the opposing MMF is removed. This hysteresis is illustrated in Figure A.4. This sort of demagntization and recoil can occur when the

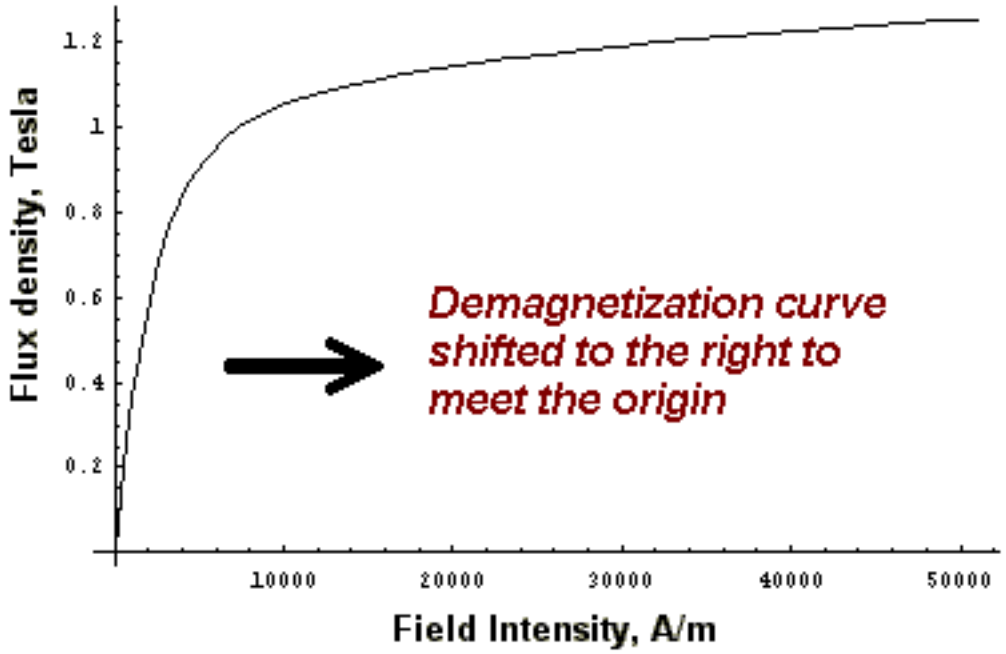


Figure A.3: Shifting the B-H curve of a permanent magnet

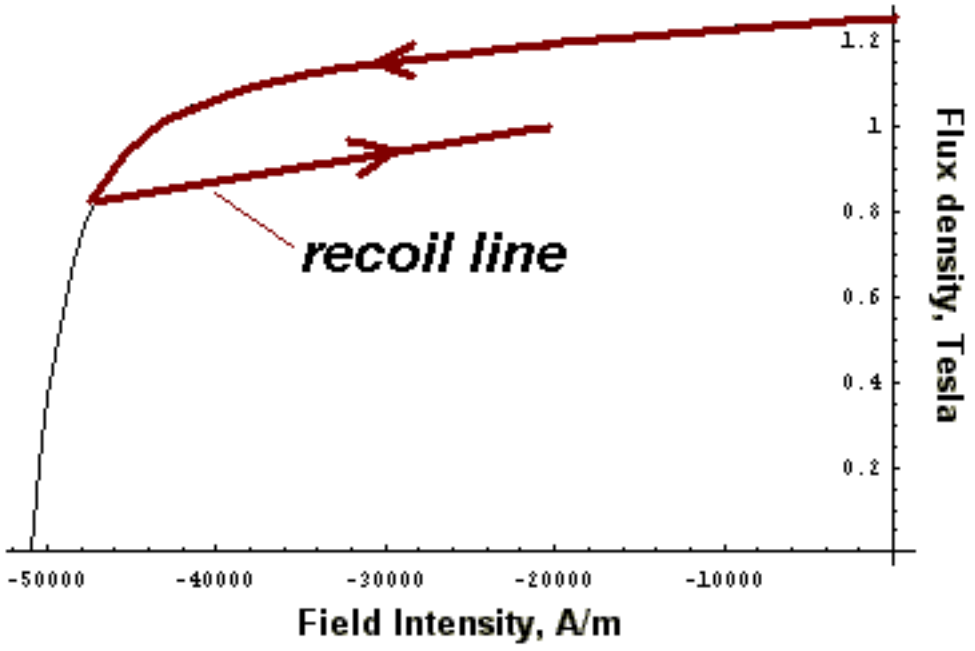


Figure A.4: Recoil in partially demagnetized Alnico 5.

magnets are being handled prior to assembly into a device. In a motor, the magnets will demagnetize somewhat when the motor is first started. They will eventually end up running back and forth along a recoil line that is below the "virgin" demagnetization curve. The point is that the modeler cannot be sure exactly where the magnets are operating–an analysis that takes this sort of hysteresis into account is beyond the scope of FEMM. Note, however, that this caution applies only for nonlinear magnets; for practical purposes, rare-earth magnets generally do not exhibit this sort of hysteresis behavior.

A.2 Bulk Lamination Modeling

A great number of magnetic devices employ cores built up out of thin laminations for the purpose of reducing eddy current effects. One way to model these materials within a finite element framework would be to model each discrete lamination (and the insulation between laminations) in the finite element geometry. An alternative is to treat the laminated material as a continuum and derive bulk properties that yield essentially the same results, while requiring a much less elaborate finite element mesh. FEMM has implemented this bulk approach to laminations.

Consider that the flux can flow through the lamination in a combination of two ways: via the "easy" direction down the laminations, or the "hard" way, across the thickness of the laminations. The hard direction is difficult for flux for two reasons. First, the rolling process makes the iron somewhat less permeable than in the easy direction. Second, and most importantly, the flux must traverse the insulation between laminations, which typically has a unit permeability.

The first assumption in deriving the bulk permeability model is that the permeability in the iron itself is isotropic. This isn't quite true, but almost all of the reluctance in the hard direction results from crossing the gap between laminations. Having a significant error in the hard direction permeability in the iron itself only results in a trivial change in the bulk reluctance in the cross-lamination direction.

Armed with this assumption, a circuit model can be produced for each direction of flux travel. For the easy direction, the circuit model is pictured in Figure A.5. There are two reluctances in parallel–one for flux that flows through the iron part of the laminations:

$$R_{ez,fe} = \frac{L}{\mu_r \mu_o c W} \tag{A.2}$$

and another reluctance for flux that flows through the air between laminations:

$$R_{ez,air} = \frac{L}{\mu_o (1-c) W} \tag{A.3}$$

where L and W are the length and width of the path traversed, and c is the fraction of the path filled with iron. Adding these two reluctances in parallel yields:

$$R_{ez} = \frac{L}{((1-c)+c\mu_r)\mu_o W} \tag{A.4}$$

Since L and W are arbitrarily chosen, the bulk permeability of the section is:

$$\mu_{ez} = ((1-c)+c\mu_r)\mu_o \tag{A.5}$$

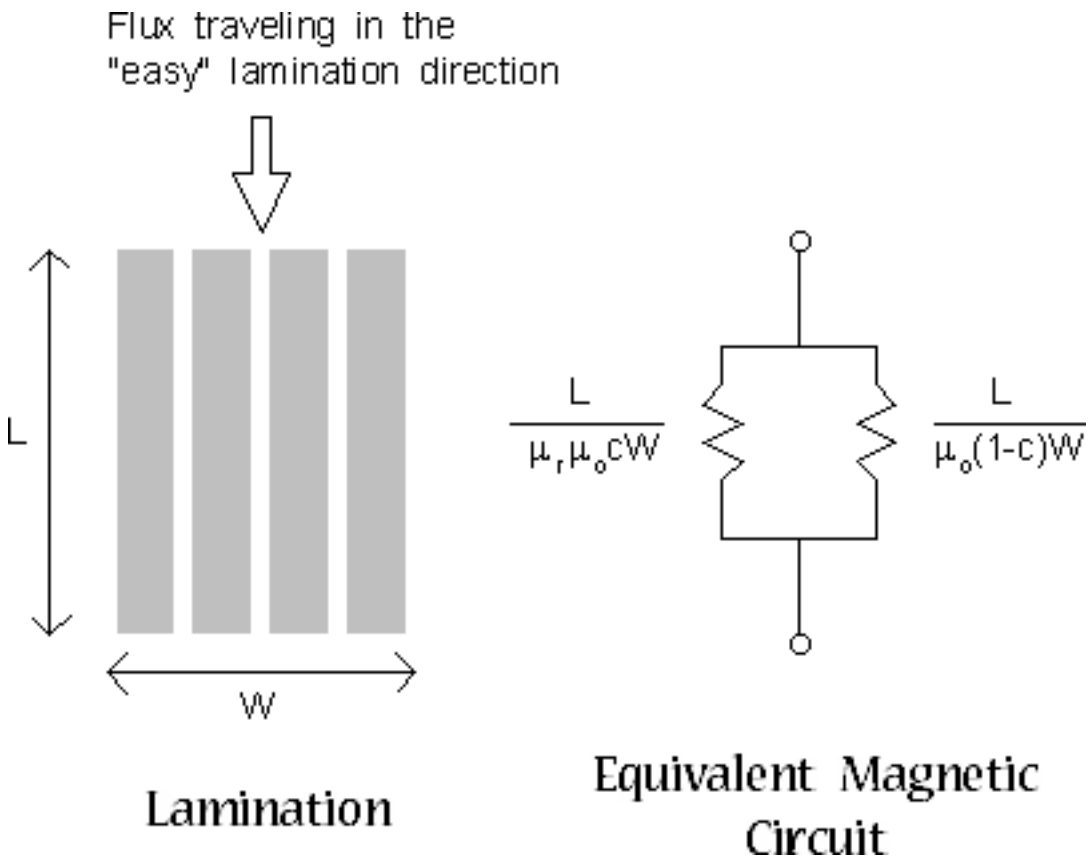


Figure A.5: Equivalent circuit for flux in the "easy" direction

For the solution of nonlinear problems, the derivation of the changes to Newton's method to accomodate the bulk lamination model are greatly simplified if it is assumed that $(1-c) << c\mu_r$. In this case, μ_{ez} can be approximated as:

$$\mu_{ez} \approx c\mu_r\mu_o \tag{A.6}$$

This approximation leads to only trivial errors until the fill factor approaches zero. For example, if $\mu_r = 1000$ with a 90% fill, the difference between (A.5) and (A.6) is only about 0.01%.

For the hard direction, a different equivalent circuit, pictured in Figure A.6 can be drawn. In this case, the circuit is two reluctances in series, as the flux has to cross the insulation and the lamination in succession. These reluctances are:

$$R_{hard,fe} = \frac{cL}{\mu_r\mu_o W} \tag{A.7}$$

$$R_{hard,air} = \frac{(1-c)L}{\mu_o W} \tag{A.8}$$

Adding these two reluctances together in series yields:

$$R_{hard} = \left(\frac{c+(1-c)\mu_r}{\mu_r\mu_o}\right)\frac{L}{W} \tag{A.9}$$

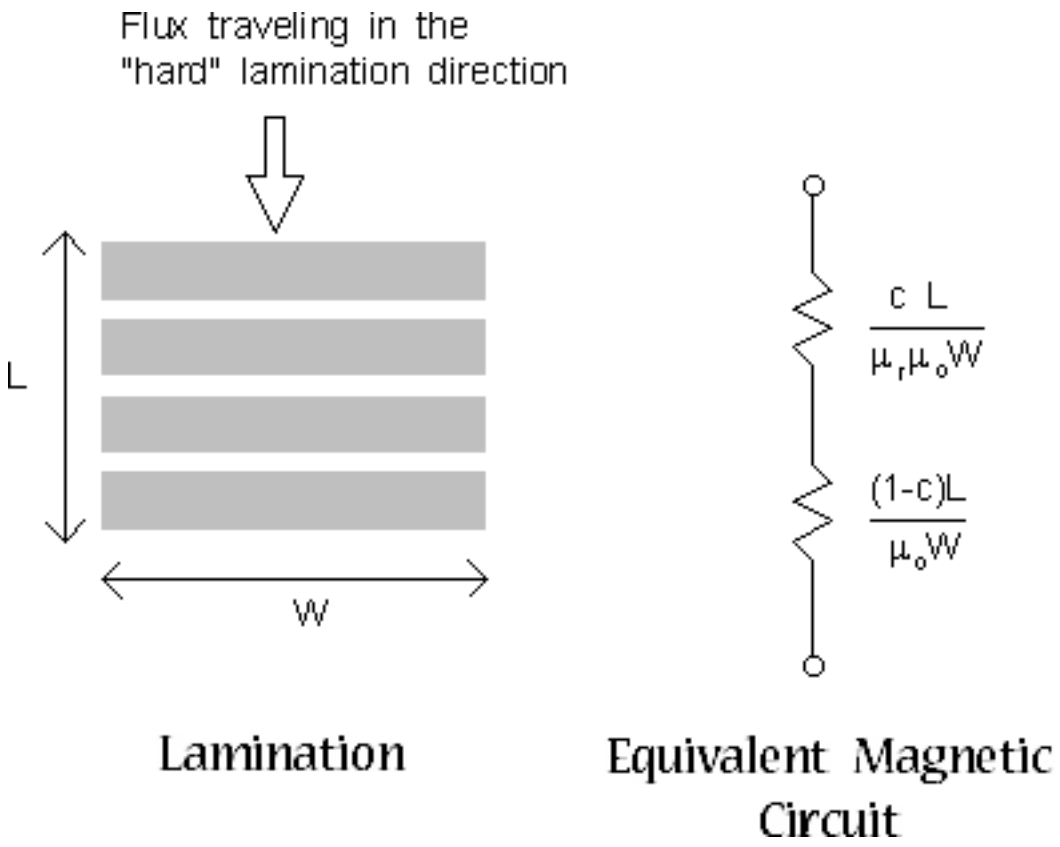


Figure A.6: Equivalent circuit for flux in the “hard” direction.

Since L and W are arbitrary, the bulk permeability in the hard direction is:

$$\mu_{hard} = \frac{\mu_r \mu_o}{c + (1-c)\mu_r} \tag{A.10}$$

If the material is laminated “in-plane,” all flux is flowing in the easy direction, and (A.6) is used as the permeability for each element. In problems that are laminated parallel to x or y, (A.6) and (A.10) are used as permeabilities in the standard fashion for elements with an anisotropic permeability.

For harmonic problems, eddy currents flow in the laminations, and hysteresis causes additional loss. If the laminations are thin compared to the other dimensions of the geometry, the effects of eddy currents and hysteresis can be encapsulated in a frequency-dependent permeability [6]. In this case, the magnetostatic permeability, μ_r is simply replaced by the frequency-dependent permeability μ_{fd} in (A.6) and (A.10):

$$\mu_{fd} = \frac{\mu_r e^{\frac{-j\phi_h}{2}} \tanh\left[e^{\frac{-j\phi_h}{2}} \sqrt{j\omega\sigma\mu_r\mu_o}\frac{d}{2}\right]}{\sqrt{j\omega\sigma\mu_r\mu_o}\frac{d}{2}} \tag{A.11}$$

In (A.11), ϕ_h represents a constant phase lag between B and H due to hysteresis, σ is the conductivity of the lamination material, d is the thickness of the iron part of the lamination, and ω is the frequency of excitation in rad/s. Note that the concept of hysteresis-induced lag can be applied to

non-laminated materials as well, simply by multiplying the magnetostatic permeability by $e^{-j\phi_h}$ for harmonic problems.

A.3 Open Boundary Problems

Typically, finite element methods are best suited to problems with well-defined, closed solution regions. However, a large number of problems that one might like to address have no natural outer boundary. A prime example is a solenoid in air. The boundary condition that one would *like* to apply is $A = 0$ at $r = \infty$. However, finite element methods, by nature, imply a finite domain. Fortunately, there are methods that can be applied to get solutions that closely approximate the "open boundary" solution using finite element methods.

A.3.1 Truncation of Outer Boundaries

The simplest, but least accurate, way to proceed is to pick an arbitrary boundary "far enough" away from the area of interest and declare either $A = 0$ or $\partial A/\partial n = 0$ on this boundary. According to [17], a rule of thumb is that the distance from the center of the problem to the outer boundary should be at least five times the distance from the center to the outside of the objects of interest. Truncation is the method employed by most magnetics finite element programs, because it requires no additional effort to implement.

The down side to truncation is that get an accurate solution in the region of interest, a volume of air much larger than the region of interest must also be modeled. Usually, this large region exterior to the area of interest can be modeled with a relatively coarse mesh to keep solution times to a minimum. However, some extra time and space is still required to solve for a region in which one has little interest.

A.3.2 Asymptotic Boundary Conditions

Pressing the `Open Boundary` button on the preprocessor toolbar builds a structure that has the impedance of an unbounded region, allowing an unbounded problem to be modeled with a finite domain. It surrounds the model with a set of nested annular shells whose material properties are chosen so that, seen from the inside, the outermost boundary behaves like a boundary at infinity—an *Improvised Asymptotic Boundary Condition*, or IABC. The theory is developed in [18] for magnetic problems and in [19] for electrostatic problems.

Pressing the button brings up a dialog asking for the number of shells (1 to 12, and 7 is a sensible default), the radius and center of the solution domain, and whether the outer edge should be Dirichlet or Neumann. Reasonable values for the radius and center are proposed from the bounding box of the geometry already drawn, so in most cases it is enough to accept what is offered. The same operation is available to scripts as `mi_makeABC`, `ei_makeABC`, `hi_makeABC` and `ci_makeABC`, which take the same parameters and invent sensible ones for any that are omitted.

Two worked examples ship with the program: 2D planar magnetic problem `IABC.fem` and axisymmetric magnetic problem `tutorial.fem`.

The shells are ordinary regions with ordinary material properties, so nothing about the solver or the postprocessor treats them specially. They will, however, appear in the field plot, and any

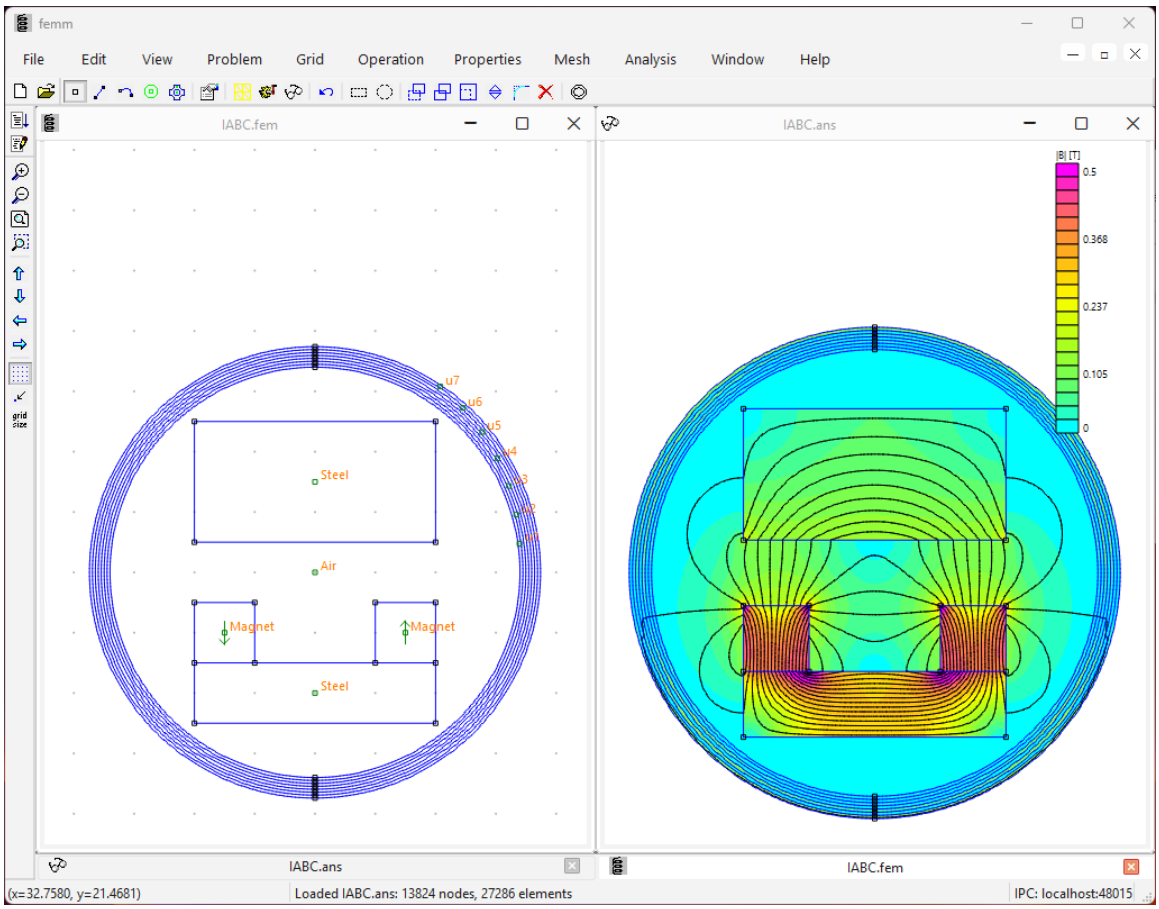


Figure A.7: 2D Planar IABC.

block integral taken over the whole model would include them—so select the regions of interest explicitly when integrating.

A.3.3 Kelvin Transformation

A particularly good approach to "open boundary" problems is the Kelvin Transformation, a technique first discussed in the context of computational magnetics in [20] and [21]. The strengths of this technique are:

- the effects of the exterior region are, in theory, exactly modeled by this approach;
- a sparse matrix representation of the problem is retained (unlike FEM-BEM methods, which give the same "exact solution" but densely couples together the boundary nodes).
- requires no "special" features in the finite element solver to implement the technique, other than the ability to apply periodic boundary conditions.

The purposes of this note are to explain what the Kelvin transformation is derived and to show how it is implemented in the context of the FEMM finite element program.

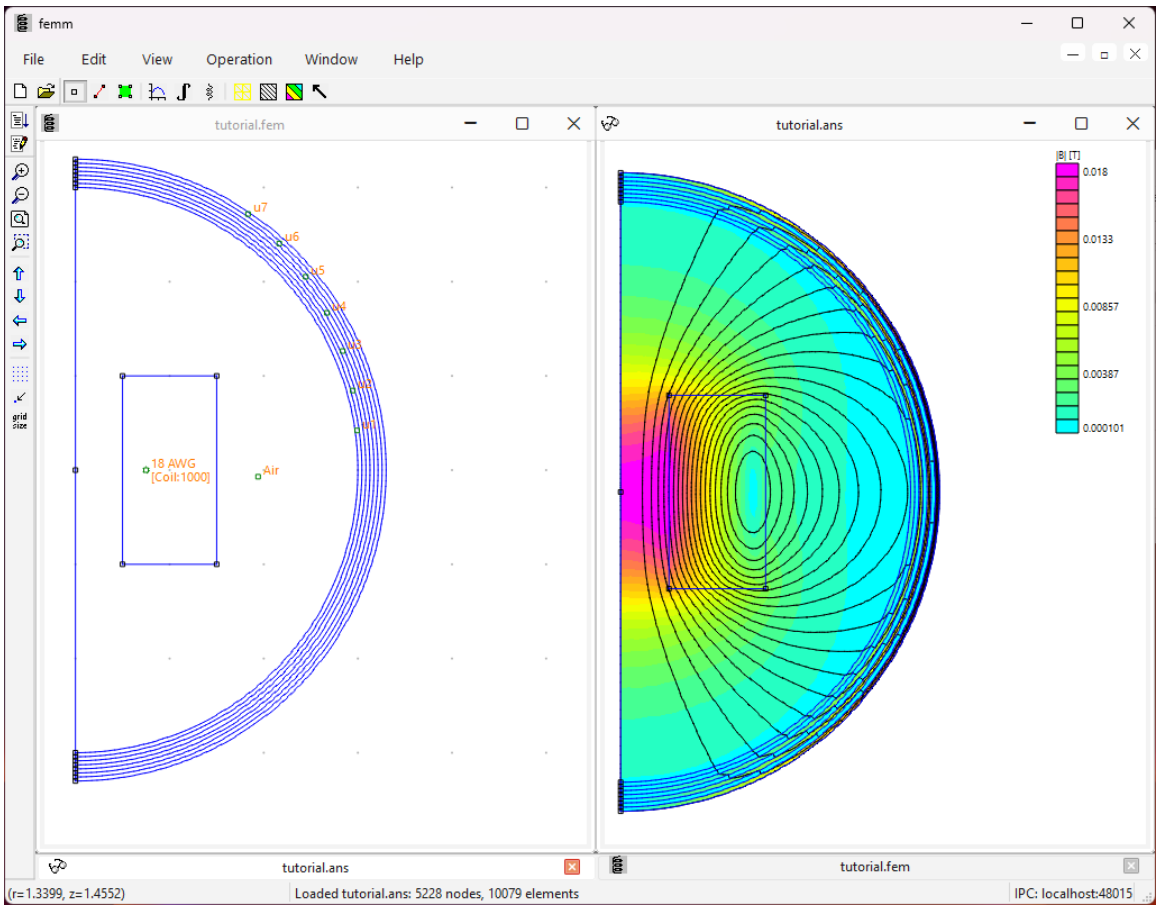


Figure A.8: Axisymmetric IABC.

Derivation

In the "far field" region, the material is typically homogeneous (*e.g. air* and free of sources. In this case, the differential equation that describes vector potential A is the Laplace equation:

$$\nabla^2 A = 0 \tag{A.12}$$

If we write (A.12) in polar notation, A is described by:

$$\frac{1}{r}\frac{\partial}{\partial r}\left(r\frac{\partial A}{\partial r}\right) + \frac{1}{r^2}\frac{\partial^2 A}{\partial \theta^2} = 0 \tag{A.13}$$

Assume that the "near field" region of the problem can be contained in a circle of radius r_o centered at the origin. The far-field region is then everything outside the circle.

One approach to unbounded problems is to attempt to map the unbounded region onto a bounded region, wherein problems can more easilby be solved. Specifically, we desire a way to transform the unbounded region outside the circle into a bounded region. One simple way to make such a mapping is to define another variable, R, that is related to r by:

$$R = \frac{r_o^2}{r} \tag{A.14}$$

By inspecting (A.14), it can be seen that this relationship maps the exterior region onto a circle of radius r_o.

The next step is to transform (A.12), the differential equation that the field must satisfy, into the mapped space. That is, (A.12) must be written in terms of R and θ rather than r and θ. We can evaluate derivatives in terms of R instead of r by employing the chain rule:

$$\frac{\partial}{\partial r} = \frac{\partial}{\partial R}\left(\frac{dR}{dr}\right) = -\frac{\partial}{\partial R}\left(\frac{R}{r_o}\right)^2 \tag{A.15}$$

Now, we can note that at $r = R = r_o$,

$$\frac{\partial A}{\partial r} = -\frac{\partial A}{\partial R} \tag{A.16}$$

and we can substitute (A.15) into (A.12) to yield, after some algebraic manipulation:

$$\frac{1}{R}\frac{\partial}{\partial R}\left(R\frac{\partial A}{\partial R}\right) + \frac{1}{R^2}\frac{\partial^2 A}{\partial \theta^2} = 0 \tag{A.17}$$

Eq. (A.17), the transformed equation for the outer region, has exactly the same form as inner region, only in terms of R rather than r. The implication is that for the 2-D planar problem, the exterior can be modeled simply by creating a problem domain consisting of two circular regions: on circular region containing the items of interest, and an additional circular region to represent the "far field." Then, periodic boundary conditions must be applied to corresponding edges of the circle to enforce the continuity of A at the edges of the two regions. The is continuity of A at the boundary between the exterior and interior regions. For a finite element formulation consisting of first-order triangles, (A.16) is enforced automatically at the boundaries of the two regions. The second circular region exactly models the infinite space solution, but does it on a bounded domain– one could always back out the field for any point in space by applying the inverse of (A.14).

Kelvin Transformation Example – `open1.fem`

As an example, consider an E-core lamination stack with a winding around it. Suppose that the objective is to determine the field around the E-core in the absence of any flux return path (*i.e.* when the magnetic circuit is open). In this case, the flux is not constrained to flow in a path that is *a priori* well defined, because the laminations that complete the flux path have been removed.

The geometry was chosen arbitrarily, the purpose here being more the procedure than the actual problem. The E-core was chosen to have a 0.5" thick center leg, 0.25" thick outer legs, and a slot depth of 0.75". The material for the core is linear with a relative permeability of 2500. The coil carries a bulk current density of 2 MA/m^2. The input geometry is picture in Figure A.9.

In Figure A.9, the core is placed within a circular region, and a second circular region is drawn next to the region containing the core. Periodic boundary conditions are applied to the arcs that define the boundaries as shown in Figure A.9. The way that periodic boundary conditions are implemented in FEMM, each periodic boundary condition defined for the problem is to be applied to two and only two corresponding entities. In this case, each boundary circle is composed of two arcs, so two periodic boundary conditions must be defined to link together each arc with in the domain with the core to its corresponding arc in the domain representing the exterior region.

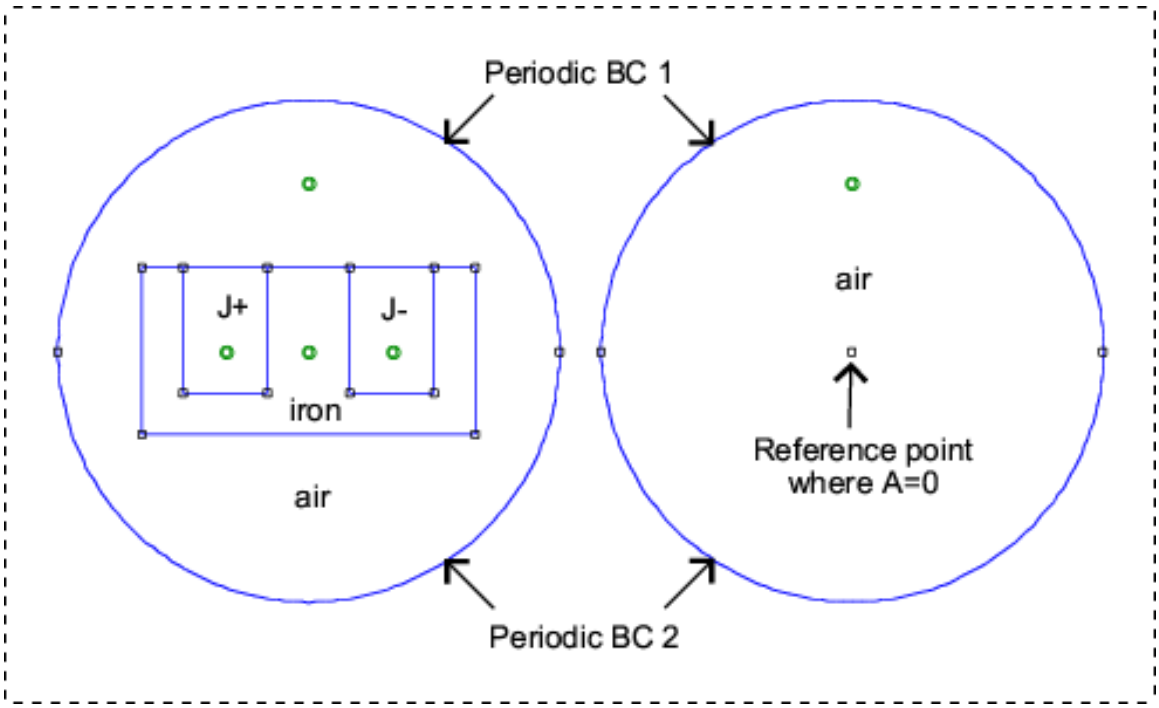


Figure A.9: Example input geometry.

Also notice that a point has been drawn in the center of the exterior region. A point property has been applied to this point that specifies that $A = 0$ at this reference point. The center of the circle maps to infinity in the analogous open problem, so it makes sense to define, in effect, $A = 0$ at infinity. If no reference point is defined, it is fairly easy to see that the solution is only unique to within a constant. The situation is analogous to a situation where Neumann boundary conditions have been defined on all boundaries, resulting in a non-unique solution for A. Due to the type of solver that FEMM employs, the problem can most likely be solved even if a reference point is not defined. However, defining a reference point eliminates the possibility of numerical difficulties due to uniqueness issues.

The resulting solution is shown in Figure A.10. As is the intention, the flux lines appear to cross out of the of the region containing the core as if unaffected by the presence of the boundary. The flux lines reappear in the domain representing the exterior region, completing their flux paths through the exterior region.

A.4 Nonlinear Time Harmonic Formulation

Starting with the 3.3 version of FEMM, the program includes a "nonlinear time harmonic" solver. In general, the notion of a "nonlinear time harmonic" analysis is something of a kludge. To obtain a purely sinusoidal response when a system is driven with a sinusoidal input, the system must, by definition, be linear. The nonlinear time harmonic analysis seeks to include the effects of nonlinearities like saturation and hysteresis on the fundamental of the response, while ignoring higher harmonic content. This is a notion similar to "describing function analysis," a widely used tool in the analysis of nonlinear control systems. There are several subtly different variations of

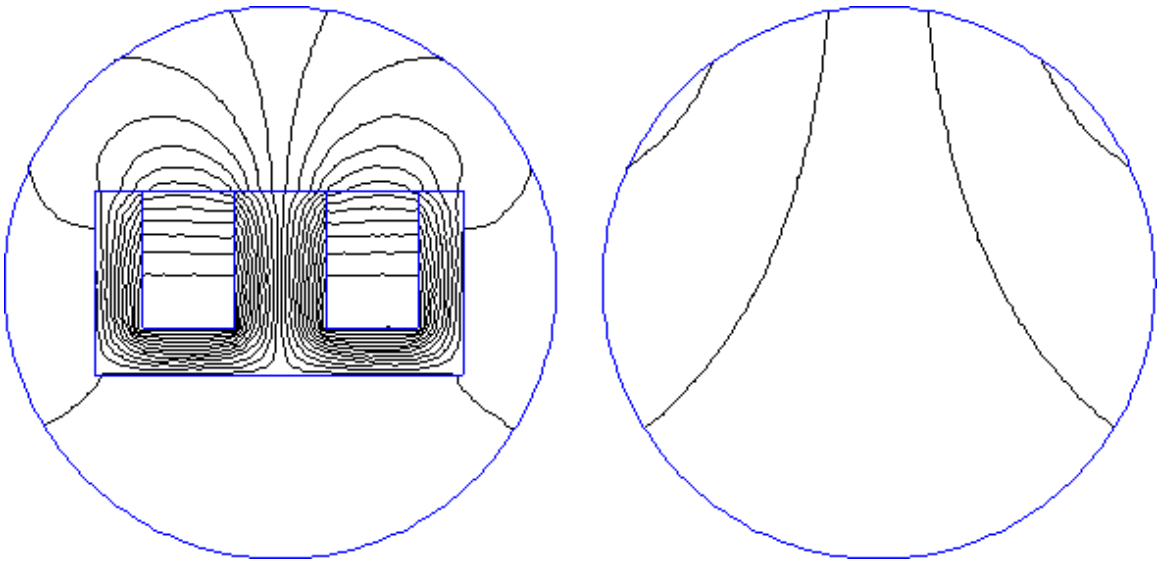


Figure A.10: Solved problem.

the formulation that can yield slightly different results, so documentation of what has actually been implement is important to the correct interpretation of the results from this solver.

An excellent description of this formulation is contained in [23]. FEMM formulates the nonlinear time harmonic problem as described in this paper. Similar to Jack and Mecrow, FEMM derives an apparent BH curve by taking H to be the sinusoidally varying quantity. The amplitude of B is obtained by taking the first coefficient in a Fourier series representation of the resulting B. For the purposes of this Fourier series computation, FEMM interpolates linearly between the user-defined points on the BH curve to get a set of points with the same H values as the input set, but with an adjusted B level. The rationale for choosing H to be the sinusoidal quantity (rather than B) is that choosing B to be sinusoidal shrinks the defined BH curve–the B values stay fixed while the H values become smaller. It then becomes hard to define a BH curve that does not get interpolated. In contrast, with H sinusoidal, the B points are typically larger than the DC flux density levels, creating a curve with an expanded range.

A "nonlinear hysteresis lag" parameter is then applied to the effective BH curve. The lag is assumed to be proportional to the permeability, which gives a hysteresis loss that is always proportional to $|B|^2$. This form was suggested by O'Kelly [24]. It has been suggested that the Steinmetz equation could be used to specify hysteresis lag, but the Steinmetz equation is badly behaved at low flux levels (i.e. one can't solve for a hysteresis lag that produces the Steinmetz $|B|^{1.6}$ form for the loss as B goes to zero.)

For nonlinear in-plane laminations, an additional step is taken to obtain an effective BH curve that also includes eddy current effects. At each H level on the user-defined BH curve, a 1D nonlinear time harmonic finite element problem is solved to obtain the total flux that flows in the lamination as a function of the H applied at the edge of the lamination. Then dividing by the lamination thickness and accounting for fill factor, and effective B that takes into account saturation, hysteresis, and eddy currents in the lamination is obtained for each H.